

基于逐时段线性规划与滚动调整的 微网购电—储能调度策略

摘 要

微网通过在外电网电价低谷或分布式光伏盈余时段为储能充电、在电价高峰时段放电，可在满足小区负载的前提下降低购电费用。本文建立统一的逐时段确定性线性规划框架，把时间分辨率、信息结构、结算制度与电价情景四个维度分别参数化，依次求解四个问题，交付 result1.xlsx、result2.xlsx、result3.xlsx、result4-2.xlsx、result4-3.xlsx 五个结果文件。

模型与口径。以 10 分钟为决策时段，决策变量为计划购电量、调整购电量、紧急购电量、充放电量与弃光量，状态量为储电量。功率平衡取等式，储能动态计入 90% 双向效率（往返 81%），容量与功率约束按题面附录。目标为购电费用最小且不乘时段长。本文给出互补性引理：往返效率小于 1 且电价为正时，“同时充放电”被“少充少放”严格支配，故线性松弛无损、无需二元变量；鉴于最优调度面可能不唯一，全部模型统一采用字典序规则选取提交解。

主要结果。问题一在附件 1 的代表性单日上得最优全天购电费 35126.948589 元、购电量 59482.698998 kWh，呈典型的分时电价套利形态，最便宜的 20% 电价时段承担 55.4% 的充电量，日周期精确闭合、全日无弃光。问题二把模型滚动到全年 365 天（52560 时段），得全期购电费 13758182.573724 元、交付期（2 月 1 日—12 月 31 日，334 天）12233050.830708 元，并证明在“完全信息 + 无购电上限”下紧急购电恒为零。问题三引入附件 3 整点光伏预报的降尺度与 6:00、12:00、18:00 三个调整时刻及 $0.5 \times / 1.5 \times$ 双向分段结算，交付期费用升至 14540616.335652 元，比问题二高 2307565.50 元（+18.863%），该差额即预报误差的代价。问题四在附件 4 的实时波动电价下重算问题二与问题三，两条链的交付期费用分别为 13006411.041148 元与 15289050.714738 元；两链差额 2282639.673590 元可完全归因为计划口径差 100230.446225 元、偏差结算 522512.788776 元与紧急购电 1659896.438588 元三项。按“0:00 时当日电价未知”的口径，决策层使用由历史价格构造的预测价、结算一律用实际价，价格预报误差的代价为 520790.839035 元与 258485.465704 元。

验证与稳健性。四问均通过产物完整性、144 时段独立回代、量纲与逐层目标复算、解析界与常识检验、跨问题一致性五级验收，等式残差不超过 4.6×10^{-12} ，交付工作簿逐格复算一致。问题一判“稳定”，问题二判“脆弱（需缩小适用范围）”，问题三与链 4-3 判“条件稳定”，链 4-2 判“稳定”；全部未通过判据与反例均如实登记，未通过调参或放宽阈值使其“通过”。

创新点。以四个维度上的参数化统一四问并形成可复用模板；用互补性引理把“是否需要二元变量”从工程判断变为数学结论；把不确定性代价拆成可计价的分项；用退化情景复现构造实现正确性锚点（六项指标逐位复现，相对差不超过 4.86×10^{-13} ）；用只改“看多远”的滚动时域对照回答结构性问题；并对“是否需要更多预报、是否需要随机优化”给出三层递进的定量回答。不足在于文献证据仅达摘要/元数据级、未显式强制充放电互补、若干物理简化，以及全部结果属确定性点预测口径，已在正文逐条披露。

关键词：微网调度；储能运行优化；线性规划；分时电价套利；偏差分段结算；滚动时域优化；价格预报误差

一、问题重述

1.1 问题背景

微网是为解决光伏发电等分布式能源的本地消纳、减少并网问题而发展起来的小型发用电系统。出于经济方面的考虑，非孤岛式微网可在外部电网（以下简称外网）电价较低或分布式能源有剩余电量的时段为储能设备充电，并在外网电价较高的时段释放电能；其调控目标是在利用小区负载、光伏发电量与储能当前储电量等数据的前提下，尽可能精准地给出微网的购电量与储能设备的充放电量，使微网提供的电力既满足小区负载、又尽可能节省购电费用。

题面附录给出储能设备参数：最大容量 12000 kWh，最大充放电功率 5000 kW，2025 年 1 月 1 日 0:00 的电量为 6000 kWh，运行过程中电量须保持在 1200–10800 kWh 之间，充放电效率为 90%。

1.2 问题提出

问题一要求建立每天 0:00 制定当天计划购电策略的数学模型：电价与小区负载逐日相同、光伏发电功率有一天预测数据；微网提供的电能不得低于小区负载；储能设备在 0:00 与 24:00 的储电量相同；目标为全天购电费用最小。需基于附件 1 给出计划购电策略与充放电安排，保存为 `result1.xlsx`。

问题二把决策期扩展到全年：电价仍逐日相同，而负载与光伏逐日变化；若微网提供的电能低于负载，需向外网紧急购电，电价为交易时刻电价的 5 倍；除紧急购电外，其余购电费用按计划购电量计算。需基于附件 1 电价与附件 2 实际负载、实际光伏，在每天 0:00 制定当天计划，给出四个指定日期（2025 年 3 月 20 日、6 月 21 日、9 月 23 日、12 月 21 日）的结果与紧急购电结果，保存为 `result2.xlsx`。

问题三引入光伏功率的滚动预报：每天 0:00、6:00、12:00、18:00 可获得未来 24 小时整点预报。可依据 0:00 预报制定当天计划，也可依据其他时刻预报调整购电策略；计划购电量高于调整购电量时，违约电价为交易时刻电价的 50%；调整购电量高于计划购电量时，超出部分电价为交易时刻电价的 1.5 倍；总费用包括计划购电费用、紧急购电费用与调整购电量的相关费用。需基于附件 1、2、3 建立模型、给出四个指定日期的结果并保存为 `result3.xlsx`，并分析是否需要引入其他时刻的预报来制定调整策略。

问题四指出外网电价本身也是实时波动的，要求在附件 4 的实时波动电价情景下，利用附件 2 的负载与光伏、附件 3 的光伏预报，重新计算问题二与问题三，并分别保存为 `result4-2.xlsx`（对应问题二）与 `result4-3.xlsx`（对应问题三）。

由题面可见，问题四在数学上等价于**两道题**。二者共用附件 4 电价，但决策结构与结算口径不同：重算问题二只有每天一次的日计划与 5 倍价紧急购电，重算问题三则另有三个调整时刻与 $0.5 \times / 1.5 \times$ 分段结算。因此本文自始至终按两条独立产物链组织计算与验收，分别记为链 4-2 与链 4-3；两链共用同一价格预测器与同一结算口径（结算一律用实际电价）。

二、问题分析

2.1 总体分析

四个问题的物理系统完全相同：一张光储微网与一个外部电网相连，储能设备的容量、功率、效率与运行区间由题面给定，光伏先供负载、盈余充入储能、不足由外网补足。因此四问的差异不在物理规律，而在**决策者知道什么、什么时候决定、按什么规则结算**三件事上。据此本文把四个问题统一抽象为四个维度上的参数化：

- 时间维度**（决策时刻集合）：问题一、二每天只有一个决策时刻；问题三与链 4-3 有 0:00、6:00、12:00、18:00 四个决策时刻，后续时刻可修改先前的决定。
- 信息维度**（各层信息集）：问题一、二的计划层被视为完全信息；问题三的计划层只知 0:00 发布的整点预报，调整层另可见当天已实现时段；链 4-2、4-3 在 0:00 时对当日实时电价一无所知，只能用历史价格。

3. **结算维度**（超额与不足如何计价）：问题一、二只有“按计划量计费”与“紧急购电按 5 倍价”两条规则；问题三与链 4-3 另有 $0.5 \times / 1.5 \times$ 双向分段结算。
4. **价格情景维度**：电价是逐日重复的单日曲线（附件 1），还是逐日波动且当日不可预知的实际序列（附件 4）。

这一抽象带来两个好处：四个问题共用一套平衡方程、储能动态、容量与功率约束和目标函数，模型规模与正确性可用同一套方法核验；同时“难点”被定位到具体维度上，“是否需要更多时刻的预报”“要不要做随机优化”“日边界为什么总贴储电量下限”这类问题都有了明确的讨论对象。技术路线如图 1。

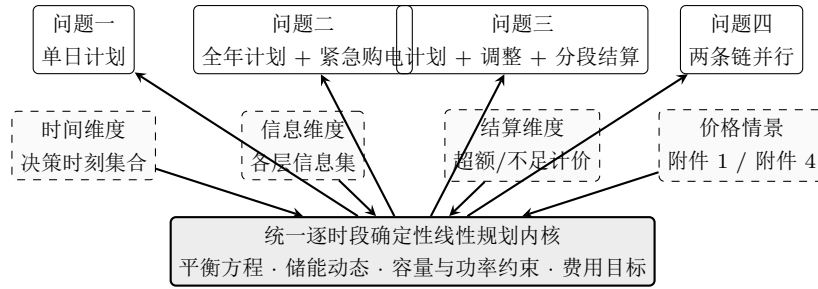


图 1：技术路线：在四个维度上参数化的统一线性规划内核

2.2 各问题的关键难点

问题一的输入（附件 1 的电价、负载与光伏预测）均视为确定值，约束中真正起作用的是储能：容量上下限、两侧功率上限、往返效率 81%，以及“0:00 与 24:00 储电量相等”的日周期条件。经济上这是典型的跨时段套利：储能把电能从低价时段搬到高价时段，搬运损耗表现为往返效率小于 1。需要论证的是两件事：是否需要二元变量强制“不同时充放电”；光伏盈余超过储能吸收能力时功率平衡如何闭合。

问题二需处理三点新结构。其一，日周期约束被**跨日状态连续性**取代——题面只在问题一要求端点储量相同，把该约束逐日施加会人为割裂跨日搬运电能的可能性，本文用逐日解耦的对照量化这一差别。其二，**终端条件必须显式声明**：题面未规定决策期末储电量，若允许自由终止，模型会把期末库存“变卖”以压低账面费用，本文保留终端自由但把该效应单独计价。其三，**紧急购电在完全信息下不会被激活**：既然 0:00 已知当天实际负载与光伏、且购电无功率上限，任何“先欠着、事后按 5 倍价补”都可事先改为按正常价买入同样电量而严格更省。该结论是模型内的推论而非实测断言，也不意味着机制可删——一旦引入购电上限，紧急购电立即被激活。

问题三的关键变化是**信息结构**：附件 3 只给整点预报，而决策与结算以 10 分钟为粒度。由此产生两个必须处理的环节：降尺度（本文取整点锚定线性插值，该口径不保证块内能量守恒，其影响可能大于调整机制本身，故作为独立消融项量化）与非预期性（0:00 计划只能用 0:00 预报，调整只能用该时刻已发布预报与已实现状态，不得使用未来实际值）。因此模型被组织为三层顺序向前递推，而非能“看到未来”的联合优化。此外，由于决策层用预报、结算层用实际光伏，预报误差必然造成缺口；调整降低了紧急购电却引入偏差违约金，其净效果只能用同一口径下的费用分解回答。

问题四的实质是**价格可观测性**的改变。“实时”意味着 0:00 制定当天计划时当天 144 个电价点尚未发生；若直接把附件 4 当作已知输入，就等于假设电价可完全预知。本文明确采用“决策层用历史价格构造的预测价、结算层一律用实际价”的口径，并把二者差额定义为“价格预报误差的代价”单独报告。由此两条链的性质不同：链 4-2 中价格不确定性只改变费用、不产生能量缺口；链 4-3 中缺口来自光伏预报误差，价格不确定性则同时影响计划层、调整层的决策与最终账单。两链费用差额可完全分解为计划口径差、偏差结算与紧急购电三项。同时，两链都采用“当天决策、终端自由”的组织方式，日边界储电量会贴住下限——这究竟是普遍性质还是特定结构的产物，需要改变前瞻深度的对照实验来回答。

三、模型假设

结合题面条件与数据的实际结构，本文作如下假设。凡属题面未规定而由本文约定的口径，均在条目中直接说明，不以隐含默认的方式使用。

1. **时间离散与对齐：**电价、负载与光伏功率均以 10 分钟为时段、每日 144 段；附件时间戳为所属时段的左端点，第 i 个数据点代表区间 $[10i, 10i + 10)$ 分钟，与结果模板的 144 个时段按位置一一对应。据此计划窗为 0:10 至 24:10，模板标称的“0:00–4:00”等标签对应物理区间整体前移 10 分钟，属模板固有相位，全文表格与图注均按此口径统一。
2. **时段内恒定：**每个时段内电价与功率视为常数，时段电量等于功率乘以 $\Delta t = 1/6$ h；反之，以电量表示的决策变量本身已含时间因子，费用表达式中不再乘以 Δt 。
3. **储能模型：**储能按常效率模型描述，充、放电各按 90% 计（往返效率 81%），静置时不发生自放电；容量运行区间为 $[1200, 10800]$ kWh，初始储电量为 6000 kWh。
4. **功率上限的作用位置：**题面“最大充放电功率 5000 kW”同时作用于并网点侧与电池侧、取更严者，故充电量上界为 $5000\Delta t = 833.3333$ kWh、放电量上界为 $5000\Delta t\eta = 750.0000$ kWh。该项为题面参数的解释约定，本文不将其表述为文献结论。
5. **弃光变量保留：**光伏盈余中未被负载与储能消纳的部分记为弃光量 s ，其目标系数为零（无上网售电收益），但变量必须保留，以便在盈余超出储能吸收能力的时段闭合功率平衡。
6. **购电无功率上限、无售电收益：**外网购电量非负且不设上界；光伏余电不上网，储能也不向电网售电。当分析中引入购电上限或售电价格时，均视为独立的结构对照情景单独报告。
7. **费用口径：**目标只计购电费用，不引入折旧、循环寿命或退化成本（题面未给系数，无法标定）。紧急购电按交易时刻电价的 5 倍计价；问题三与链 4-3 中，计划高于最终的部分按 0.5 倍价、最终高于计划的部分按 1.5 倍价计价，两段均为成本项而非退款，且以当天 0:00 计划为唯一基准、单次结算、不回溯。
8. **信息非预期性：**任一决策时刻只能使用该时刻已发布或已实现的数据。问题三与链 4-3 中，0:00 计划层只用 0:00 预报，6:00、12:00、18:00 调整层只用该时刻已发布预报加已实现状态；实际光伏只进入事后结算层，不影响任何决策量与储能轨迹。
9. **价格可观测性：**在附件 4 的波动电价情景下，0:00 制定当天计划时当天实时电价尚未发生、不可观测，决策层只能使用由历史价格构造的预测价，而费用结算一律使用实际电价；由此产生的决策—结算差异单独量化为“价格预报误差的代价”。
10. **供电可靠性由等式平衡保证：**每个时段采用等式功率平衡并显式包含紧急购电与弃光变量，由此“微网提供的电能不低于小区负载”自动成立、充电不挤占负载；模型不进一步建模切负荷。
11. **数据可用性与预报边界：**附件 2–4 在 2025 全年无缺日、无重复，附件 3 的日期列存在前向填充结构，读取时按前向填充还原；题面只提供未来 24 小时的整点光伏预报，本文不假设可获得更长时间跨度或更高时间分辨率的预报。

此外，由于最优调度面可能不唯一（储电量上下限两端均可能活跃），本文统一约定一条解的选择规则：所有模型、所有层先在主目标最优面上取解，再在其中最小化储能吞吐量（字典序规则）。验收一律以约束残差、结构恒等式、目标值与交付表格为准，不比对逐点解的唯一性。

四、符号说明

四个问题共用同一套符号。电量类变量以 kWh 为单位、本身已含时间因子；功率类输入以 kW 为单位；费用类量以元为单位。

符号	含义	单位
t, τ	时段索引，每日 $t = 1, \dots, 144$ ；全年 $\tau = 1, \dots, 52560$	—
d, i	日期索引（全年 365 天）；日内 10 分钟时段索引 $i = 1, \dots, 144$	—
m	决策时刻， $m \in \{0, 6, 12, 18\}$	h
Δt	时段长度， $\Delta t = 1/6$	h
L_t	小区负载功率	kW
PV_t	光伏发电功率； PV_t^{fc} 为预报值， PV_t^{act} 为实际值	kW
p_t	电价（问题一至三取自附件 1，逐日重复）	元/kWh
$p_{d,i}^{act}$	附件 4 的实际波动电价	元/kWh
$\hat{p}_{d,i}^{(m)}$	决策时刻 m 信息集下的预测电价； κ_m 为调整层的水平校正因子	元/kWh
b_t	计划购电量（0:00 制定）	kWh
q_t	调整后的最终购电量（替代量口径）	kWh
q_t^{em}	紧急购电量（缺口部分）	kWh
c_t, q_t^{dis}	储能充电量、放电量	kWh
s_t	弃光量（光伏盈余中未被消纳的部分）	kWh
E_t	时段末储电量（状态变量）	kWh
η	充放电效率， $\eta_{ch} = \eta_{dis} = 0.9$	—
$E_{\min}, E_{\max}$	储电量运行区间，1200 与 10800	kWh
E_{init}	初始储电量（2025-01-01 0:00），6000	kWh
$P_{\max}$	最大充放电功率，5000	kW
α_{em}	紧急购电电价倍数， $\alpha_{em} = 5$	—
$\beta_{def}, \beta_{over}$	偏差结算倍数， $\beta_{def} = 0.5$ ， $\beta_{over} = 1.5$	—
$C_{\text{plan}}, C_{\text{adj}}, C_{\text{em}}$	计划购电费、偏差结算费、紧急购电费	元
C_{total}	总费用 $C_{\text{plan}} + C_{\text{adj}} + C_{\text{em}}$	元
C^{fc}	同一组决策按决策层信念价计得的费用	元
N	净负荷电量 $\sum_t (L_t - PV_t) \Delta t$	kWh
$D_{\text{full}}, D_{\text{req}}$	全年 365 天；交付期 2025-02-01 至 12-31 共 334 天	—
T	决策期时段总数（问题二为 52560）	—

需要特别说明两处容易混用、一旦混用就无法比对的量：其一， b 与 q 是两个不同的量， b 是 0:00 制定的计划购电量， q 是该时段最终的购电量（替代量口径而非增量），问题三与链 4-3 的交付表报的是 q ；其二，“全期”指 365 天，“交付期”指 2025 年 2 月 1 日之后的 334 天，1 月作为储能预热期不计入交付——全文出现的每一个费用数字都标注其所属口径。

五、模型的建立与求解

5.1 数据预处理与特征分析

5.1.1 输入数据与口径

四个问题的输入来自附件 1-4，其用途与读取口径如表 1。需要说明的是：附件 1 中的负载与光伏预测列只服务于问题一的“单日代表”语义，附件 3 的光伏预报只服务于问题三与链 4-3，附件 4 的电价只服务于问题四。各问题的输入边界在建模时即已固定，不做跨问题借用，以保证四个问题之间互不“串答案”。

附件 4 有三个值得注意的统计事实。第一，电价存在极端低值：全期最小值为 0.0076 元/kWh，低于 0.10 元/kWh 的时段仅 28 个，因此以最小电价构造的费用下界极为宽松（不足实际费用的 1.2%），量级自检必须改用构造性上界。第二，深谷时段的 5 倍紧急购电价仅为 0.038 元/kWh，讨论惩罚机制时不能用全期均价外推。第三，附件 4 的日内形状与附件 1 的电价曲线高度一致：同时段跨日均值曲线与附件 1 曲线的相关系数为 0.9999999956（四位小数即为 1.0000）且极值位置相同，差异体现为日间水平漂移（日均价在 0.5378 至 0.9290 元/kWh 之间）叠加逐点扰动。这既解释了附件 1 电价能被逐日重复使用

表 1: 输入数据及其在各问题中的用途

数据	结构	用途与读取口径
附件 1	145 行 $\times$ 4 列, 10 分钟粒度单日	问题一至三的电价 (逐日重复 365 次); 问题一的负载与光伏预测。电价取值于 [0.3713, 1.3952] 元/kWh, 144 点之和为 110.3324 元/kWh
附件 2	365 行 $\times$ 144 列, 实际值	问题二至四的负载与光伏实际功率。负载取值于 [1995.7176, 7978.8849] kW, 光伏取值于 [0, 10216.20] kW
附件 3	365 天 $\times$ 4 个发布时刻 $\times$ 24 小时	问题三与链 4-3 的整点光伏预报, 需降尺度到 10 分钟
附件 4	366 行 $\times$ 145 列, 实时电价	问题四唯一的价格来源。取值于 [0.0076, 1.7936] 元/kWh, 全期均值 0.7661976 元/kWh
模板文件	附件 5 的 5 个结果文件	决定表格结构与填报口径; 模板中的数值不作为任何结论依据

的原因, 也说明在波动电价情景下采用“水平 $\times$ 时段”的双因子预测器有数据依据。

数据覆盖 2025 年 1 月 1 日至 12 月 31 日共 365 天、无缺日无重复; 按结果文件要求, 交付期为 2025 年 2 月 1 日至 12 月 31 日共 334 天, 1 月作为储能预热期参与滚动递推但不计入交付。全部问题均来自 2025 年 1 月 1 日 0:00 的 6000 kWh 起算, 日间通过储电量连续性衔接, 不施加逐日周期约束。

5.1.2 负载与光伏的结构特征

全年 52560 个时段中, 出现光伏功率大于负载功率 (即存在盈余) 的时段有 10130 个, 占 19.3%。这类时段是储能充电的天然窗口, 也是弃光可能发生的时段。交付期净负荷电量为 17939189.494967 kWh, 该数值只与附件 2 有关, 因此在问题二、问题三、链 4-2 与链 4-3 中**逐位相同**——它构成一个有用的交叉校验量: 若某个模型算出的净负荷与此不符, 说明取数或时间对齐出错。

由图 2 可见, 负载在午间与傍晚出现双峰, 光伏在正午达到最大, 于是净负荷在正午被光伏压得很低、在傍晚急剧抬升。这意味着低价充电窗口集中在凌晨与正午、高价放电窗口集中在傍晚与夜间, 正是分时电价套利的典型场景; 而充放电之间的能量搬运要付出 19% 的往返损耗, 因此只有当价差足以覆盖损耗时套利才发生。

5.1.3 异常与边界检查

对全部输入做了四项检查: 其一, 缺失与重复检查——附件 2-4 无缺日无重复, 附件 3 的日期列按前向填充还原为 365 天 $\times$ 4 个发布时刻; 其二, 量纲检查——功率乘以 Δt 得电量, 费用等于电价与电量之积且不乘 Δt ; 其三, 时序一致性检查——附件 1 与附件 2 的 144 点采用同一左端点口径, 若误用右端点会使全部时段整体错位 10 分钟; 其四, 净负荷一致性检查——四问共用同一净负荷数值。上述检查在每次计算后都以只读方式重新执行, 检查脚本与计算脚本相互独立。

5.2 统一优化框架

5.2.1 模型结构

四个问题共用同一组决策变量: 计划购电量 b 、最终购电量 q 、紧急购电量 q^{em} 、充电量 c 、放电量 q^{dis} 与弃光量 s , 并以时段末储电量 E 为状态变量。以问题二为例, 全年模型为

$$\min_{b, q^{em}, c, q^{dis}, s, E} C_{total} = \sum_{\tau=1}^T \left(p_{\tau} b_{\tau} + \alpha_{em} p_{\tau} q_{\tau}^{em} \right), \quad \alpha_{em} = 5, \quad T = 52560 \quad (5.1)$$

$$\text{s.t.} \quad b_{\tau} + q_{\tau}^{em} + PV_{\tau} \Delta t + q_{\tau}^{dis} = L_{\tau} \Delta t + c_{\tau} + s_{\tau}, \quad 0 \leq s_{\tau} \leq PV_{\tau} \Delta t \quad (5.2)$$

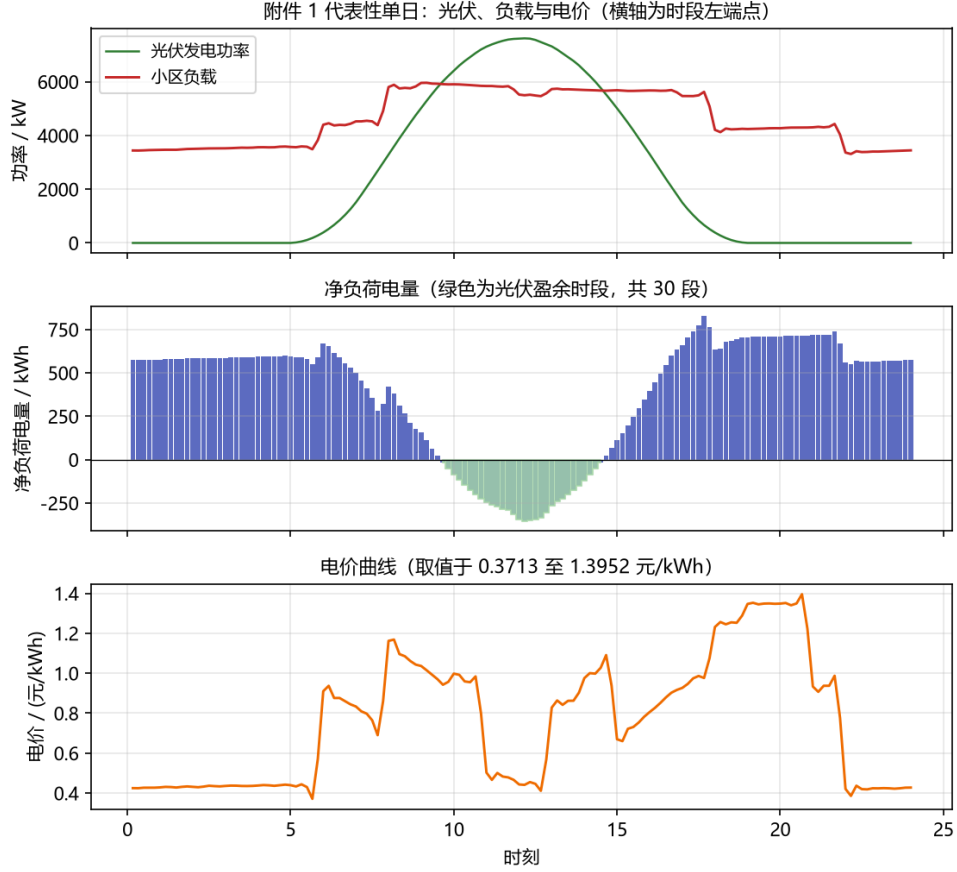


图 2: 附件 1 代表性单日的电价、小区负载与光伏预测功率, 以及净负荷电量

$$0 \leq c_\tau \leq 5000\Delta t = 833.3333, \quad 0 \leq q_\tau^{dis} \leq 5000\Delta t \eta = 750.0000 \quad (5.3)$$

$$E_\tau = E_{\tau-1} + \eta c_\tau - \frac{q_\tau^{dis}}{\eta}, \quad \eta = 0.9, \quad 1200 \leq E_\tau \leq 10800, \quad E_0 = 6000 \quad (5.4)$$

$$b_\tau \geq 0, \quad q_\tau^{em} \geq 0, \quad c_\tau \geq 0, \quad q_\tau^{dis} \geq 0, \quad s_\tau \geq 0 \quad (5.5)$$

式 (2) 的等式平衡使“供电不低于负载”自动成立, 同时充电不与负载竞争; 这一形式化与微网并网储能系统的逐时段成本最小建模传统一致 [1, 2, 3, 4]。模型规模为变量 $6T = 315360$ 个、等式约束 $2T = 105120$ 条、非零元 $9T - 1 = 473039$ 个、整数变量 0 个; 矩阵高度稀疏 (非零元占比约 1.4×10^{-5}), 实现时采用稀疏存储, 稠密存储需约 265 GB 内存并不可行。

5.2.2 三处需要论证的建模要点

其一, 是否需要二元变量强制“不同时充放电”。直观上储能不应在同一时段既充电又放电。本文不引入二元变量, 而用如下引理保证线性松弛无损。

引理 1 (互补性) 若 $\eta^2 < 1$ 且电价 $p_\tau > 0$, 则任一同时充放电的解都不是严格最优解; 模型存在满足 $c_\tau q_\tau^{dis} = 0$ 的最优解。

证明 设某时段 $c_\tau > 0$ 、 $q_\tau^{dis} > 0$, 令 $\varepsilon = \min(c_\tau, q_\tau^{dis}) > 0$, 把二者同时减去 ε : 储电量净变化为 $-\eta\varepsilon + \varepsilon/\eta > 0$ (因 $\eta^2 < 1$), 状态反而上升。因此可再把充电量减去 $\eta^2\varepsilon$, 即取 $c'_\tau = c_\tau - \varepsilon$ 、 $q_\tau^{dis'} = q_\tau^{dis} - \eta^2\varepsilon$, 此时状态净变化恰为 $-\eta\varepsilon + \eta^2\varepsilon/\eta = 0$, 状态轨迹完全不变。该修改使购电量减少 $\varepsilon(1 - \eta^2) > 0$, 费用严格下降, 与最优性矛盾。故存在最优解满足互补性。□

引理 1 说明“不需要二元变量”是精确性质而非近似；其代价是最优面可能不唯一，故本文统一采用字典序规则选取提交解。同一现象在电池调度文献中亦被讨论：当往返效率小于 1 时线性松弛的互补性可以自发满足，而强制互补会带来数值与规模上的额外代价 [5, 6]。

其二，光伏盈余如何闭合。附件 1 的光伏峰值 7612.32 kW 高于负载峰值 5958.97 kW，附件 2 中也有相当比例的时段光伏超过负载。若不加弃光变量而要求等式平衡，模型在这些时段将不可行；若改为不等式（净供给不低于负载），余电可以凭空消失，会低估可行域并扭曲费用。本文采用等式平衡加显式弃光变量（ $s \in [0, PV\Delta t]$ 、目标系数为零）的做法：盈余只能被储能吸收或以 s 弃掉，既不凭空消失也不上网售电。光伏限出力在含储能的经济调度中通常以同样的方式显式建模 [7, 8]。

其三，终端条件。题面未规定决策期末的储电量，本文保留终端自由，只要求其落在运行区间内。由于期末库存具有经济价值，终端自由会使模型在末期“放空”储能，这一效应在结果中被显式量化（问题二为 +2217.08 元），以免与价格套利混淆。

5.2.3 结构恒等式

对任意可行解，把储能状态转移式在决策期上求和可得两条与目标函数无关的恒等式，它们是比目标值更严格的正确性检验：

$$\sum_{\tau} q_{\tau}^{dis} = \eta^2 \sum_{\tau} c_{\tau} - \eta(E_T - E_0) \quad (5.6)$$

$$\sum_{\tau} b_{\tau} = N + \sum_{\tau} s_{\tau} + (1 - \eta^2) \sum_{\tau} c_{\tau} + \eta(E_T - E_0) - \sum_{\tau} q_{\tau}^{em}, \quad N = \sum_{\tau} (L_{\tau} - PV_{\tau})\Delta t \quad (5.7)$$

式 (8) 的含义是：全年购电量等于净负荷加上弃光、加上储能循环损耗，再扣除期末放空的库存。以问题二为例， $\eta(E_T - E_0) = 0.9 \times (1200 - 6000) = -4320$ kWh，即期末放空等价于少购 4320 kWh；这条账在四个问题中都被独立复核，残差为零。

5.2.4 四个维度上的参数化

四个问题不需要分别推导模型，只需在表 2 的四个维度上取不同取值。

表 2：四个问题在统一框架下的参数化

	时间维度	信息维度	结算维度	价格情景
问题一	单日 144 段	完全信息	按计划量计费	单日曲线（附件 1）
问题二	全年 52560 段	完全信息	计划量计费 + 5 倍紧急购电	逐日重复（附件 1）
问题三	全年，四个决策时刻	预报驱动（附件 3）	上述 + 0.5/1.5 倍双向分段	逐日重复（附件 1）
链 4-2	全年，每日 0:00	负载/光伏完全信息，电价仅历史	计划量计费 + 5 倍紧急购电	实时波动，当日不可知
链 4-3	全年，四个决策时刻	预报驱动，电价仅历史与已实现时段	上述 + 0.5/1.5 倍双向分段	实时波动，当日不可知

5.2.5 求解与核验

全部模型均为线性规划，采用 HiGHS 求解器在 CPU 上直接求解，不涉及 GPU：问题二的单次全年线性规划约 5.8 s，问题三与链 4-3 因分解为 1460 个单层分别约需 42 s 与 39 s，链 4-2 的逐日递推约 12 s。所有计算在隔离任务、独立输出目录下由受监督进程执行，每次运行落盘解序列、交付表格、追踪链指纹与运行清单，可独立复现。

计算完成后对每个问题执行五项只读核验：产物与追踪链完整性（输入文件指纹未变、代码哈希与登记一致）；独立回代（用原始附件与解序列重建平衡方程、状态转移与约束，不读模型自带的自检结果、不重解）；量纲与逐层目标复算（含交付表格逐格比对）；解析界与常识检验；跨问题一致性（净负荷、弃光量等价格无关量逐位一致，而费用必然不同）。四问的等式残差均不超过 4.6×10^{-12} ，容量与功率约束零越界，交付工作簿逐格复算一致。

5.3 问题一：单日计划购电策略

5.3.1 模型的建立与求解

问题一在统一框架上叠加两条题面条件：只需对一天建模（电价与负载逐日相同），以及日周期约束

$$E_0 = E_{144} = 6000 \quad (5.8)$$

模型规模为 720 个变量、288 条等式约束，是四个问题中最小的一个。由于存在日周期约束，储能只能“当天借、当当天还”，充电所用电量与放电所获电量的差额必须由购电量补足，这恰好刻画了套利的成本。这类“日前制定、逐时段成本最小”的调度问题在文献中普遍建模为线性或混合整数规划 [9, 10, 11]，而在分时电价下“低价充电、高价放电”的套利是光伏—储能经济优化的核心机制 [12, 13, 14, 15]。

最优全天购电费为 $C^* = 35126.948589$ 元，全天购电量 59482.698998 kWh，当日净负荷为 55541.972433 kWh，二者之差来自储能循环损耗。求解器返回最优状态（364 次迭代），等式残差不超过 4.5×10^{-13} ，变量界零越界。按题面表 1、表 2 的格式给出的结果见表 3、表 4。

表 3：问题一：指定时段的购电量与全天合计（单位：kWh，购电费单位：元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	486.4029	14:00–14:10	0.00
16:00–16:10	394.9315	18:00–18:10	636.9826	20:00–20:10	0.00
全天购电量 59482.698998		全天购电费 35126.948589			

表 4：问题一：储能设备在各时段的充放电量与 0:00、24:00 的储电量（单位：kWh）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4500.00	0.00	4:00–8:00	833.33	6365.84
8:00–12:00	4787.96	1703.00	12:00–16:00	5286.04	91.10
16:00–20:00	0.00	5780.13	20:00–24:00	5333.33	2859.87
0:00 储电量 6000.00		24:00 储电量 6000.00			

两张表的时段标签按结果模板的行位置给出。按本文采用的时间对齐口径，模板标称的“0:00–4:00”对应物理区间 0:10–4:10，计划窗整体前移 10 分钟，属模板固有相位；六个时段的充电量与放电量分别列示、不做冲抵。

5.3.2 策略形态与机制

最优策略呈现清晰的“低价充电、高价放电”形态（图 3、图 4）：充电时段的电价均值为 0.6425 元/kWh，放电时段的电价均值为 1.0845 元/kWh，价差 0.4420 元/kWh 明显大于往返损耗的成本；电价与充电量的相关系数为 -0.446 ，与放电量的相关系数为 $+0.592$ ；最便宜的 20% 电价时段承担了 55.4% 的充电量，最贵的 20% 电价时段承担了 63.7% 的放电量。最优解中共放电时段为 0，储电量轨迹精确闭合于 6000 kWh，弃光量为 0（当日光伏盈余全部被储能吸收）。

5.3.3 结果验证与模型族对照

验证分三个层次。数量级上，费用 35126.95 元落在独立构造的区间 [20622.73, 48052.05] 元内，其中上界 48052.046591 元是“完全不使用储能、按分时电价直接购电”的可行策略费用，本题最优解相对该策略节省 26.898%，这一比值同时验证了套利的方向与幅度。约束上，68 项独立断言全部通过：功率平衡与状态转移残差为 0，等式残差不超过 4.5×10^{-13} ， $E \in [1200, 10800]$ ， $E_{144} = E_0$ ，充电与放电峰值均不越界。填报上，写出的 result1.xlsx 与解序列逐点一致。

模型族对照的结果列于表 5。三点值得强调：其一，加入 144 个互补二元变量的混合整数模型给出的目标值与线性规划完全相同（最优间隙为 0、仅 1 个节点），说明引理 1 的“松弛无损”在本题上确实成

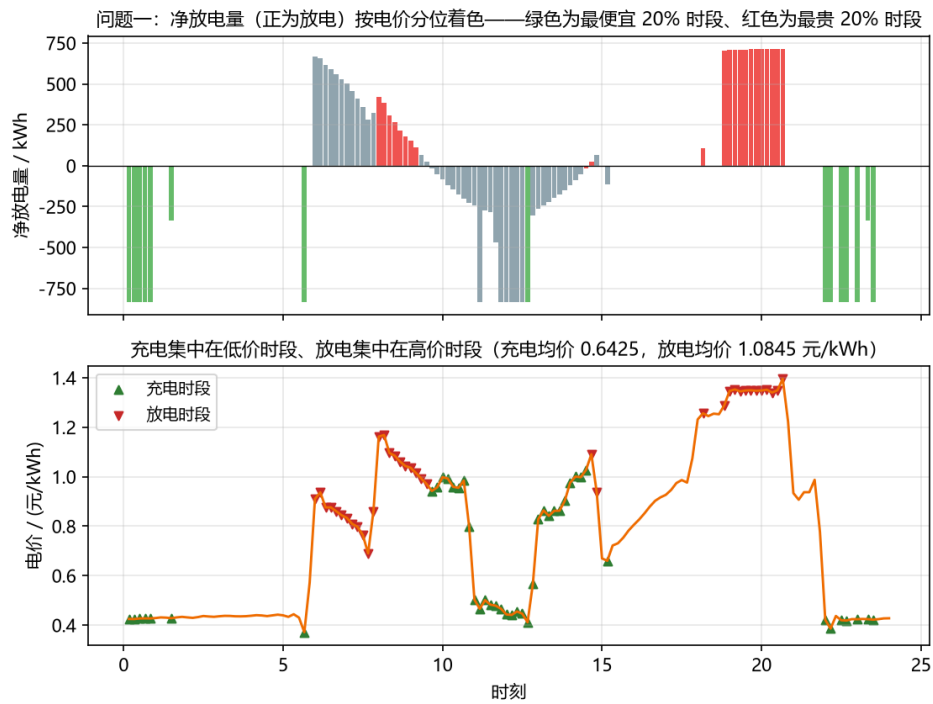


图 3：净负荷与电价分位结构：净放电量按电价分位着色，三角标记充放电时段

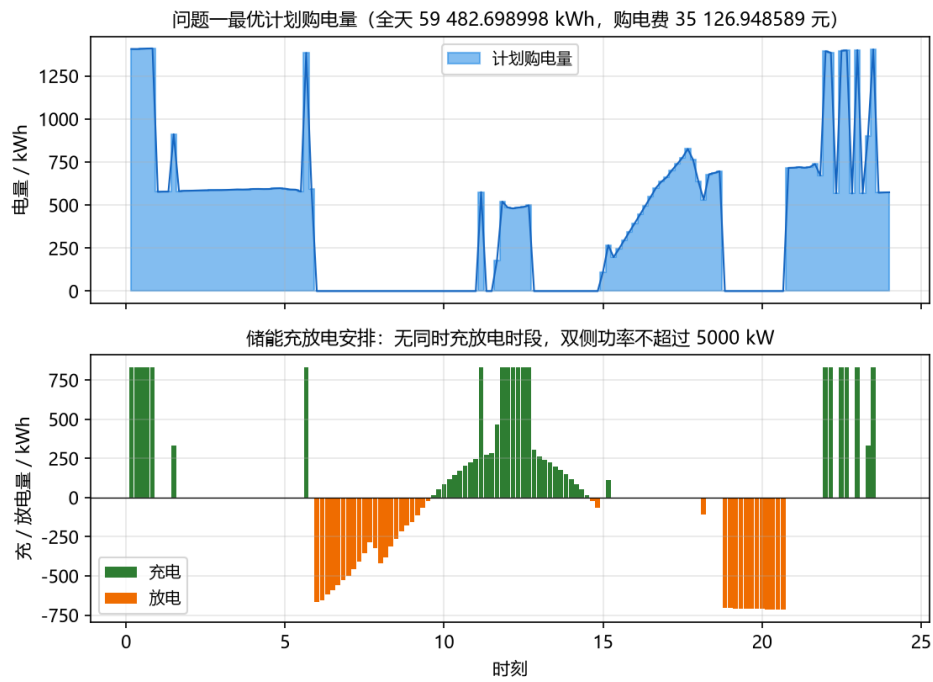


图 4：问题一最优计划购电量与储能充放电安排

立，无需付出二元变量的代价；其二，储电量离散化的动态规划在四档网格上的目标值恒不低于 C^* 且随网格加密单调收敛（最细网格偏差 +0.791%），差异纯属离散化代价而非模型结构差异；其三，日周期约束的经济价值为 2217.083333 元（占 C^* 的 6.312%），去掉该约束后模型会把期末库存放空至下限，相当于把日初存量的电能当日变现——这提醒我们“0:00 与 24:00 储电量相同”并非形式要求，而是具有实质经济含义的。

表 5：问题一的模型族与口径对照（以 $C^* = 35126.948589$ 元为基准）

对照项	费用（元）	相对 C^*
基准线性规划 / 加互补二元变量的整数模型	35126.948589	0.000%
储电量离散化动态规划（网格 400/200/100/50）	37616.52 / 36203.04 / 35688.92 / 35404.67	+7.087 至 +0.791%
去掉日周期约束 / 删除弃光变量	32909.865256 / 35126.948589	-6.312% / 0.000%
效率作用位置（仅充电/仅放电/往返整体）	33416.53 / 34168.61 / 33801.50	-4.869 至 -3.773%

需要说明，效率作用位置的三组对照会低估费用 958–1710 元，说明本文采用的“充放电两侧各按 90% 计”是几种可能口径中最保守的一种；附录 1 给定的 90% 是题面硬参数，而实际电池效率随荷电状态与老化变化，恒效率属工程简化 [16, 17]。此外，目标中不计折旧也是显式取舍——日前微网调度文献常把退化成本纳入目标 [18]，题面未给相应系数，故本文在假设中声明而不隐藏。

5.3.4 稳健性

在 948 个情景（947 个可行）的扰动实验下，问题一的结论判为“稳定”，全部判据通过。费用对往返效率最敏感且方向单调： η 由 0.9 降至 0.8 时费用上升 8.885%，升至 1.0 时下降 7.828%，弹性约为 -0.70 至 -0.80；其余参数的影响均不超过 0.7%，其中初始储电量在 [3000, 9000] kWh 内的变化对费用影响不超过 0.07%（日周期闭合使初值只影响日内可调度裕度）。噪声实验显示费用区间半宽不超过 11.6%，且均值负向偏移源于价格函数的凹性，不得解读为“预测误差能省钱”。

5.4 问题二：全年计划购电策略与紧急购电

5.4.1 模型的建立

问题二去掉日周期约束，把决策期扩展到全年 365 天，并用跨日状态连续性取代它：

$$E_{d,0} = E_{d-1,144}, \quad E_{1/1,0} = 6000, \quad E_{12/31,144} \text{ 自由} \quad (5.9)$$

式 (10) 就是状态转移式在日边界处的一次转移，不需要额外约束。与问题一相比的关键差别在于：储能在不同日期之间可以携带电能，因此“哪天充电、哪天放电”也成为决策的一部分；目标中新增紧急购电项 $\alpha_{em} p_{\tau} q_{\tau}^{em}$ ， $\alpha_{em} = 5$ 。把跨日时段统一为一个长时域优化问题，是微网与储能调度中处理日间耦合的标准做法 [19, 20, 21, 22]。

5.4.2 紧急购电恒为零的条件与意义

定理 2（紧急购电的机制性恒零） 若 电价 $p_{\tau} > 0$ ； $\alpha_{em} > 1$ ； 计划购电量 b_{τ} 无上界； 计划层对当日负载与光伏为完全信息，则模型存在满足 $q_{\tau}^{em} \equiv 0$ 的最优解。

证明 设某时段 $q_{\tau}^{em} = \delta > 0$ 。由于平衡为等式且 b_{τ} 无上界，把该时段计划购电量改为 $b'_{\tau} = b_{\tau} + \delta$ 仍可行（平衡方程左端不变），而费用变化为 $p_{\tau} \delta - \alpha_{em} p_{\tau} \delta = p_{\tau} \delta (1 - \alpha_{em}) < 0$ ，即费用严格下降，与最优性矛盾。故存在最优解使 $q_{\tau}^{em} \equiv 0$ 。□

定理 2 说明：在完全信息且购电无上限的口径下，紧急购电惩罚机制不会被激活，交付表中紧急购电量全零是模型结构的推论，而非数据巧合。这一点必须在论文中解释清楚，否则“表格全零”会被误读为漏填。同时也说明该机制不可删除：一旦引入购电功率上限，紧急购电立即被激活，本文用上限扫描给出

激活阈值——限额压到 4218.75 kW 时全年紧急购电量升至 8707.398542 kWh 并涉及 14 天，压到 2000 kW 时升至 7845369.336456 kWh 并涉及全部 365 天，总费用单调上升至 2.38 倍。因此“无上限”是必须显式声明的前提，而非可以省略的细节。文献亦普遍强调：日前决策与实时结算之间的信息落差与功率约束是应急机制被激活的直接原因 [23, 24, 25, 26, 27]。

5.4.3 主结果

全年最优购电费为 13 758 182.573724 元，其中交付期（2025 年 2 月 1 日–12 月 31 日，334 天）为 12 233 050.830708 元，1 月预热期为 1 525 131.743016 元，三者满足“交付期加预热期等于全期”的加总关系。主要总量指标列于表 6。

表 6: 问题二的主要结果

指标	数值	口径与说明
总购电费（全期 365 天）	13 758 182.573724 元	交付期 334 天为 12 233 050.830708
紧急购电费	0 元	由定理 2 保证
计划购电量 $\sum b$	22 657 938.157435 kWh	全期
充电量 $\sum c$ / 放电量 $\sum q^{dis}$	7 364 620.676347 / 5 969 662.747841 kWh	往返损耗 19%
弃光量 $\sum s$	990 168.090312 kWh	97 天出现弃光，全部落在交付期
净负荷 N	20 272 812.138617（全期） / 17 939 189.494967（交付期） kWh	与价格无关，用于跨问题校验
储电量端点	$E_{2/1,0} = 7950.000000, E_{12/31,144} = 1200.000000$ kWh	跨日滚动、终端自由

量级自检方面，费用落在独立构造的区间 [7 525 691.131, 18 298 592.366] 元内；套利方向正确，充电加权均价 0.520371 元/kWh 明显低于放电加权均价 1.154986 元/kWh。储电量两端（1200 与 10800 kWh）在最优解中均曾活跃，最大购电量出现在功率上限处，峰值购电功率约 10325.33 kW。

5.4.4 交付结果

表 7 给出题面四个指定日期的六个 10 分钟时段购电量、全天购电量与全天购电费；表 8 给出这四个日期的分块充放电量。四个日期及全年的紧急购电量均为 0，按填报口径在“紧急购电量”工作表中逐日填“—/0”、每日至少一行。需特别提醒：题面表 4 中的 2025 年 3 月 1 日及三个时段是填写格式示例，不是本题答案。

表 7: 问题二：四个指定日期的指定时段购电量与全天合计（单位：kWh；购电费单位：元）

日期	10:00	12:00	14:00	16:00	18:00	20:00	全天购电量	全天购电费
2025-03-20	0.00	558.311667	0.00	507.324167	682.353333	0.00	61 658.562881	37 975.400853
2025-06-21	0.00	0.00	0.00	27.005000	385.901667	0.00	38 934.072483	20 470.622813
2025-09-23	0.00	380.021250	0.00	493.582500	752.645000	61.911667	66 265.421309	41 351.897551
2025-12-21	0.00	906.683333	0.00	795.930417	712.633333	30.268333	93 770.214940	59 717.364845

表 8: 问题二：四个指定日期的分块充放电量（单位：kWh；块顺序为 0–4、4–8、8–12、12–16、16–20、20–24 时）

日期	量	第 1 块	第 2 块	第 3 块	第 4 块	第 5 块	第 6 块	端点储电量 0:00 → 24:00
2025-03-20	充电	1666.67	833.33	6291.46	4689.68	0.00	3503.18	8550.00 → 4352.87
	放电	0.00	5862.62	2777.38	254.72	5728.06	2911.94	
2025-06-21	充电	0.00	833.33	701.60	9965.07	0.00	8166.67	2361.06 → 8550.00
	放电	0.00	1719.95	0.00	0.00	6154.10	2485.90	
2025-09-23	充电	1666.67	833.33	5245.38	5923.78	0.00	8166.67	8550.00 → 8550.00
	放电	0.00	6746.77	1896.62	403.62	5640.00	3000.00	
2025-12-21	充电	1666.67	1666.67	5833.33	7574.21	0.00	8166.67	8550.00 → 8550.00
	放电	0.00	1425.00	7890.00	2220.11	5797.47	2842.53	

这里必须区分两种端点语义：问题一的 0:00 与 24:00 是日周期约束的端点，恒为 6000 kWh；问题二

的端点是全年滚动轨迹在某一日的切片，随日期变化，两者不可直接比较。

5.4.5 策略形态与机制解释

问题二的最优策略在更长时间尺度上重复了套利逻辑，但也出现两个新现象（图 5）。

第一，储能存在明显的“回归水平”：334 个交付日中，日初或日末储电量等于 8550.00 kWh 的各有 219 天，全年日初储电量只取 96 个不同水平。这说明最优策略倾向于把储能维持在高位以待高价时段，而不是每天放空再充满；只有 12 月 31 日（决策期最后一天）日末落到下限 1200 kWh，即期末放空，对应式 (8) 中 -4320 kWh 的库存变现项。

第二，弃光只发生在少数日子：全期仅 97 天出现弃光，合计 990168.090312 kWh，占光伏总发电量的比例很小。这说明在无售电收益的设定下储能容量基本足以吸收盈余；但弃光变量仍必须保留，否则那些盈余超过储能吸收能力的时段会使模型不可行。

策略形态的统计量为：电价与充电量的相关系数 -0.3835 ，与放电量的相关系数 $+0.6132$ ；最便宜的 20% 电价时段承担 48.12% 的充电量，最贵的 20% 时段承担 68.42% 的放电量；交付期单日费用最小 11900.552329 元、最大 60014.368886 元、均值 36625.900691 元（合计与交付期费用逐位一致）。

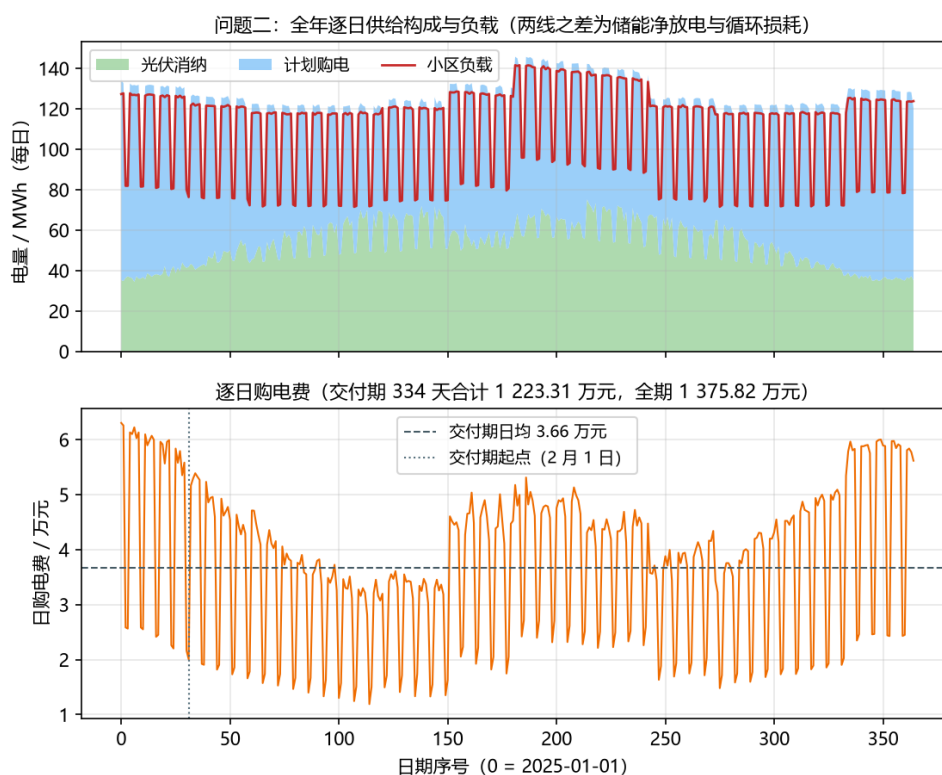


图 5：全年逐日供给构成、负载与逐日购电费

5.4.6 模型族与结构对照

本文设置了 17 组对照（表 9），结论有三点。

其一，核心结论不依赖模型族与实现路线。用不同变量排序、消去弃光变量、改用对偶单纯形法的独立第二实现给出的费用与主模型逐位相同；把全年问题精确分解为 365 个单日问题（耦合量取全年最优值）也给出相同结果。差异为零说明主结果不是实现假象。

其二，跨日与终端口径有可量化的经济价值。逐日解耦并允许日终自由递推会使费用上升 26894.74 元，这是“跨日携带电能”的价值；逐日施加日周期约束会使费用上升 17135.02 元，这是跨日衔接与逐日终端两项的合并贡献；把全年终端固定为 6000 kWh 会使费用上升 2217.08 元，对应期末放空的 4320 kWh 库存。三者对应不同的约束改动，不可互相替代或相加。

其三，购电上限是唯一能激活紧急购电的机制（图 7）。上限在 10326 kW 及以上时不产生任何影响

问题二：题面四个指定日期的日内功率构成（蓝面积为计划购电功率，右轴）

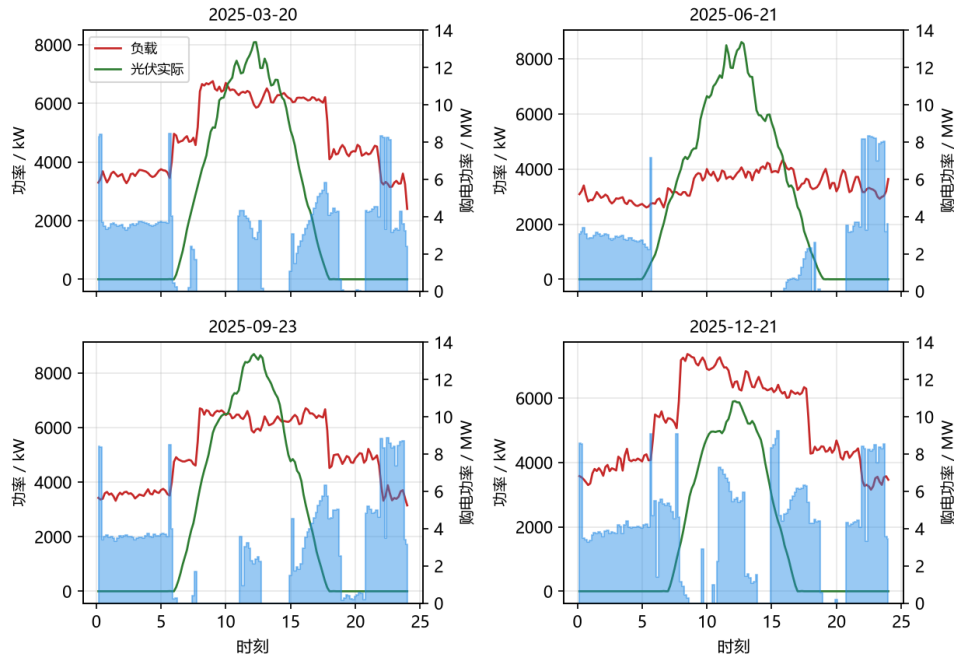


图 6：题面四个指定日期的日内功率构成

（该值只是“非绑定阈值”），而在 4218.75 kW 及以下时紧急购电被激活且费用随上限下降单调上升，代价从 +694 101 元迅速放大到 +19 011 429 元（上限 2000 kW）。这说明“紧急购电为零”的结论对上限极度敏感，必须与前提条件一并陈述。

表 9：问题二的时域组织、终端条件与购电上限对照（费用为全期，单位：元）

对照项	费用	
基准（全年单一长时域 LP）	13 758 182.573724	
独立第二实现 / 精确逐日分解	13 758 182.573724	
逐日解耦、日终自由递推	13 785 077.313742	
逐日施加日周期约束 / 终端固定为 6000 kWh	13 775 317.596266 / 13 760 399.657058	
购电上限 4375 / 4218.75 / 2000 kW	14 452 283.680247 / 14 647 566.902931 / 32 769 611.572155	
互补约束整数模型（52 560 个二元变量）	13 758 186.928903	

5.4.7 稳健性与适用边界

149 次全年线性规划的扰动实验得到两点结论（判据与阈值在跑数前冻结）。第一，年度与交付期总量**稳健**：参数扰动下费用最大偏移 14.078%（对应 $\eta = 0.72$ 的极端档），噪声情景下费用区间半宽不超过 0.96%，全部情景可行、无 NaN 或无穷值。第二，日尺度交付量对效率口径敏感： η 变化 $\pm 10\%$ 可使单日购电量与购电费移动约 $\pm 20\%$ （实测最大 19.62%），而年度总量仅移动约 $\pm 7\%$ 。因此表 7、表 8 中的单日数值须与该敏感性一并引用。

本文如实登记两条未通过的判据。其一，可行率判据的唯一失败样本来自求解器设置对照，其目标值与基线逐位相同、约束残差为 2×10^{-12} 、解可行，唯一未通过的检查是“返回解是否满足充放电互补性”——由于引理 1 只断言存在互补最优解而非任意返回解都互补，这属于退化最优面上的返回解选择问题，不构成模型不可行。其二，交付表稳定判据的失败样本来自 $\eta = 0.99$ 的情景，该情景使 2025 年 6 月 21 日全天购电量下降 19.10%，属有利方向的变化，但因判据取绝对值口径而被判失败；其反向（ $\eta = 0.81$ ）最多使单日购电费上升 12.97%，有 21 天升幅超过 10%。据此本问的稳定性分级为“脆弱（需缩小适用范围）”，判据未做事后修改。

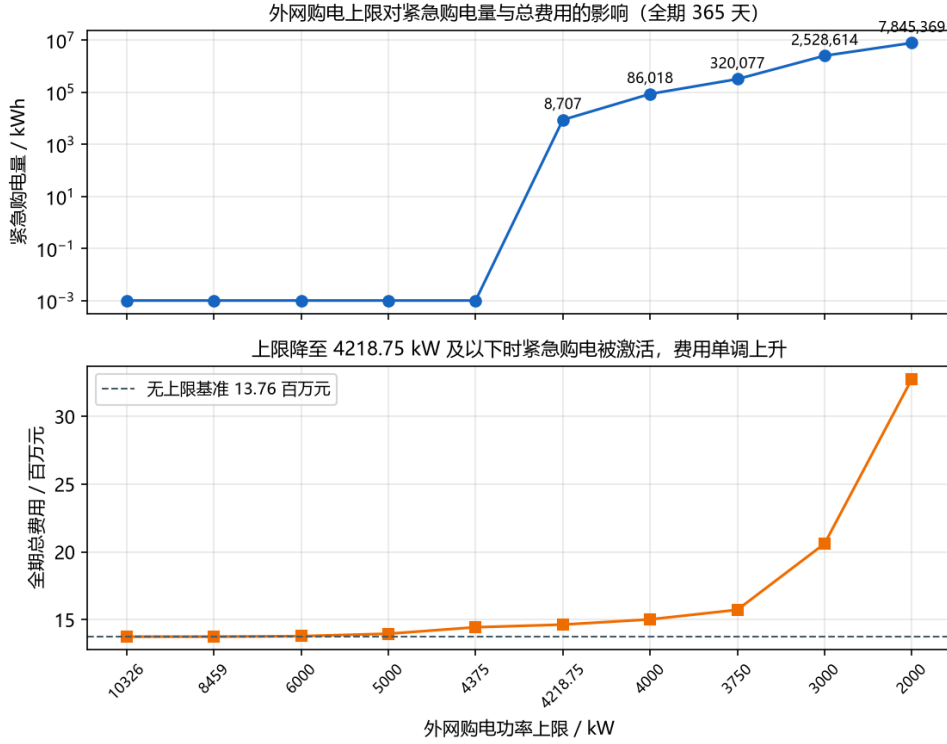


图 7: 外网购电上限对紧急购电量与总费用的影响: 上限降至 4218.75 kW 及以下时紧急购电被激活

此外登记两项属于结构性口径的发现: 一是当允许以 0.50 元/kWh 售电时, 松散的弃光上界会出现“售出电量超过物理盈余”的非物理套利 (售出 5 845 778.709901 kWh 远超物理盈余 3 120 221.003233 kWh), 须先收紧上界, 该情景收益不得当作真实售电收入; 二是当把紧急购电倍数改为 0.5 时全年紧急购电量恰等于基准的全年计划购电量, 这是同一物理计划被整体改记为紧急购电的“重标定效应”, 不得把它解读为净负荷。

5.5 问题三: 光伏预报、多时刻调整与偏差分段结算

5.5.1 预报降尺度与信息集

附件 3 给出每天 0:00、6:00、12:00、18:00 发布的未来 24 小时整点光伏功率预报, 而决策与结算以 10 分钟为粒度, 因此必须先把整点预报降尺度。本文的口径为: 把“预报 k 小时”解释为发布时刻 m 之后第 k 个整点的功率点值; 时段 i 的左端点落在小时区间 $[h, h+1)$ 时取 $P(h)$ 与 $P(h+1)$ 的线性插值; 发布时刻之后的第一小时无左端整点值, 用 $P(m+1)$ 前向保持。把小时级预报细化到分钟级的插值方法及其对调度结果的影响在可再生能源调度文献中已有系统讨论 [28, 29, 30]。

该口径有两个必须披露的性质。其一, **不强制块内能量守恒**: 整点锚定线性插值得到的块内平均功率为 $(2.5A_{k-1} + 3.5A_k)/6$, 一般不等于 A_k , 因此降尺度会系统性地改变能量。其二, **跨年项不消费**: $m = 6, 12, 18$ 时分别有 19-24、13-24、7-24 小时的预报落在次日之后, 合计 36 项被丢弃, 本文不做截断也不做外推。这两点决定了降尺度口径必须在结果中被单独量化。

信息集方面严格遵守非预期性: 0:00 计划层只使用 0:00 发布的预报; 6:00、12:00、18:00 调整层只使用该时刻已发布的预报、已实现的过去状态以及当天已知的负载与电价; 附件 2 的实际光伏只进入事后结算层。时段 i 由唯一一个决策时刻支配, 记为 $\nu(i) = 6[(i-1)/36]$, 无需引入离散选择变量, 模型保持线性。

5.5.2 三层模型的建立

模型按时间顺序组织为三层, 逐层求解、逐层提交, 不允许用一次性联合优化让前两层“看到”实际光伏。

计划层 (0:00) 使用 0:00 预报的全天光伏轨迹 $\Pi_0[PV]$, 求解

$$\min_{b, c, q^{dis}, s, E} \sum_{i=1}^{144} p_i b_i \quad \text{s.t.} \quad b_i + \Pi_0[PV]_i \Delta t + q_i^{dis} = L_i \Delta t + c_i + s_i \quad (5.10)$$

外加储能动态、容量与功率约束；储电量在日边界衔接上一日已提交的末态，全年终端自由。

调整层（6:00、12:00、18:00）以当天 0:00 的计划量 b 为参数，只决定其支配时段的最终购电量 q ，目标为该层剩余时段上的偏差惩罚：

$$\min \sum_{i \in R_m} \left[\beta_{def} p_i (b_i - q_i)^+ + \beta_{over} p_i (q_i - b_i)^+ \right], \quad \beta_{def} = 0.5, \quad \beta_{over} = 1.5 \quad (5.11)$$

其中 $(x)^+ = \max(x, 0)$ ，实现时用两个非负辅助变量分段线性化，因系数为正故线性松弛无损；调整层初值取实际已实现的储电量，已提交时段不再修改。这类“滚动调整 + 双向偏差惩罚”的多阶段结构在日内再调度与偏差结算文献中被广泛采用 [31, 32, 33, 34, 35, 36]。

实际结算层在全部决策提交后事后计算，是附件 2 实际光伏的唯一入口。设剩余量 $r = q + q^{dis} + PV^{act} \Delta t - L \Delta t - c$ ，则

$$s' = \max(0, r), \quad q^{em} = \max(0, -r) \quad (5.12)$$

$$C_{plan} = \sum p b, \quad C_{adj} = \sum \left[\beta_{def} p (b - q)^+ + \beta_{over} p (q - b)^+ \right], \quad C_{em} = \sum \alpha_{em} p q^{em} \quad (5.13)$$

其中 b 是计划购电量、 q 是最终购电量（替代量口径），二者不可混用。

5.5.3 主结果与交付结果

交付期总费用为 14540616.335652 元，构成为：计划购电费 12362860.209641 元、调整结算费 510208.940910 元、紧急购电费 1667547.185102 元（全期三项分别为 13885669.475555、563098.132533、1730408.537141 元）。费用落在独立构造的区间 [6963514.20, 16407319.63] 元内，自下界起位于 80.23% 处；交付期均价 0.697655 元/kWh，落在最小电价与 5 倍最大电价之间。交付期紧急购电量 391274.485914 kWh、弃光量 2087727.847547 kWh、最终购电量 $\sum q = 20842142.600809$ kWh（注意该值不是计划购电量 $\sum b = 20359871.873818$ kWh）。

表 10 给出四个指定日期的全天费用及其分项与紧急购电的合并区间数和合计电量。四个日期的 0:00 与 24:00 储电量均为 1200.000000 kWh，充放电电量按 4 小时块分别列示、不做冲抵。同样提醒，题面表 4 中的 2025 年 3 月 1 日及其区间为格式示例。

表 10：问题三：四个指定日期的全天购电量、购电费与费用分项（购电量单位：kWh，费用单位：元）

日期	全天购电量 $\sum q$	全天购电费	计划费	调整费	紧急购电费
2025-03-20	70040.887993	52814.209286	41095.385464	2373.895890	9344.927933
2025-06-21	33532.164963	22848.629380	18055.815827	1088.445153	3704.368400
2025-09-23	68433.174470	47936.179117	42772.756895	0.000000	5163.422222
2025-12-21	94749.308713	62365.864792	59764.834892	1182.684937	1418.344963
紧急购电合并区间数（天）		5	7	9	8
紧急购电量合计（kWh）		2062.489153	840.356133	1150.194481	352.514922

5.5.4 机制解释

调整费全部来自超发方向。式 (14) 中欠发方向费用为 0.0 元、超发方向为 563098.132533 元，即全部调整成本来自“最终购电量高于计划”的时段（上调 5810 个时段、下调仅 86 个时段）。原因是计划层只看 0:00 预报而结算层使用实际光伏：预报低估光伏时计划量偏高（对应下调），高估时偏低（对应上调），实

测以后者为主。这一不对称说明，若计划层对紧急购电有所预期本可降低违约金——本文刻意不引入这种预期，以保证计划层与题面“根据 0:00 预报制定计划”的语义一致。

紧急购电非零的成因是预报误差，而非购电上限。决策层用附件 3 预报、结算层用附件 2 实际值，二者的差异必然在结算层造成缺口，由 5 倍价的紧急购电补足，交付期合计 391 274.485914 kWh。本问不设购电功率上限，紧急购电并非“上限被顶到”所致。

问题三比问题二更贵，这是一个必须披露的结论。两问交付期净负荷逐位相同（17 939 189.494967 kWh），而交付期费用分别为 14 540 616.34 元与 12 233 050.83 元，差额为

$$2\,307\,565.50 \text{ 元 } (+18.863\%) \quad (5.14)$$

构成为紧急购电费 1 667 547.185102 元加调整费 510 208.940910 元，本质是“预报误差的代价”。本文据此明确指出：问题三并不比问题二更省。模型之所以没有抵消这一代价，是因为计划层的目标 $\min \sum p b$ 只看 0:00 预报、对紧急购电没有任何预期，模型内部缺少对冲机制；这一事实同时构成了后文讨论“是否需要引入随机优化”的必要性证据。

日边界储电量恒为下限，且这是结构性结果而非题面要求。全年 365 天的 $E_{d,144}$ 全部等于 1200 kWh，唯一取值数为 1。题面只在问题一要求“0:00 与 24:00 储电量相同”，本问没有该要求；该现象源于“计划层为单日线性规划 + 目标只含计划费用 + 终端自由”这一组合使跨日库存的边际价值为零，属结构性涌现结果，不应表述为模型缺陷，也不应表述为“题面要求每天放空”。此外，2025 年 2 月 1 日 0:00 的储电量在本问为 1200.00 kWh、在问题二为 7950.00 kWh，差异源于两问信息集不同（问题二用实际值、本问为预报驱动的三层滚动），二者不可直接比较。

问题三：光伏预报的降尺度与“预报—实际”落差（缺口由紧急购电承接）

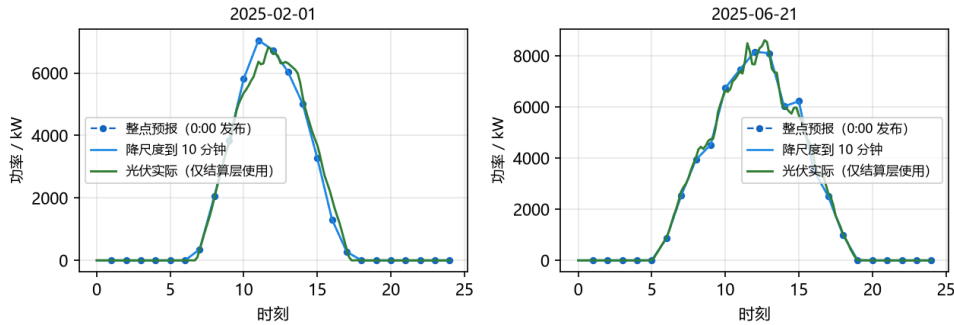


图 8：光伏预报的降尺度与“预报—实际”落差

5.5.5 模型族对照与关键口径影响

12 组对照的结果表明（表 11），主结果不依赖实现路线、求解器设置与决策密度：逐日解耦的重组实现与主模型逐位同值；两组独立第二实现的链级相对差不超过 2.17×10^{-5} （约 350 元）；把决策时刻加密到每 3 小时或每 1 小时非但没有收益，反而分别使费用上升 398.90 元与 590.07 元。本文按预注册纪律如实登记这一反例，不将其解读为“决策越密越差”。

表 11 中有三项口径的影响需要强调。第一，**降尺度口径的影响大于调整机制本身的价值**：改为分段常数前向保持使费用上升 2 561 954.057440 元（+15.83%），其中紧急购电项贡献占 94.7%，机制是该口径在主导时段低估光伏 2.68%、在 12:00 预报域低估 11.29%，从而系统性地制造缺口。第二，**完全信息上界比主模型低 14.96%**，这是信息价值的度量，且该上界与问题二的 accepted 结果逐位相同——它同时充当问题三与问题二之间的独立交叉锚点，说明两问的取数与时间对齐完全一致。第三，**调整机制的价值**为 326 656.917514 元（相对“仅 0:00 计划”），由“减少的 889 755.05 kWh 紧急购电”与“新增的 563 098.13 元违约金”共同决定。

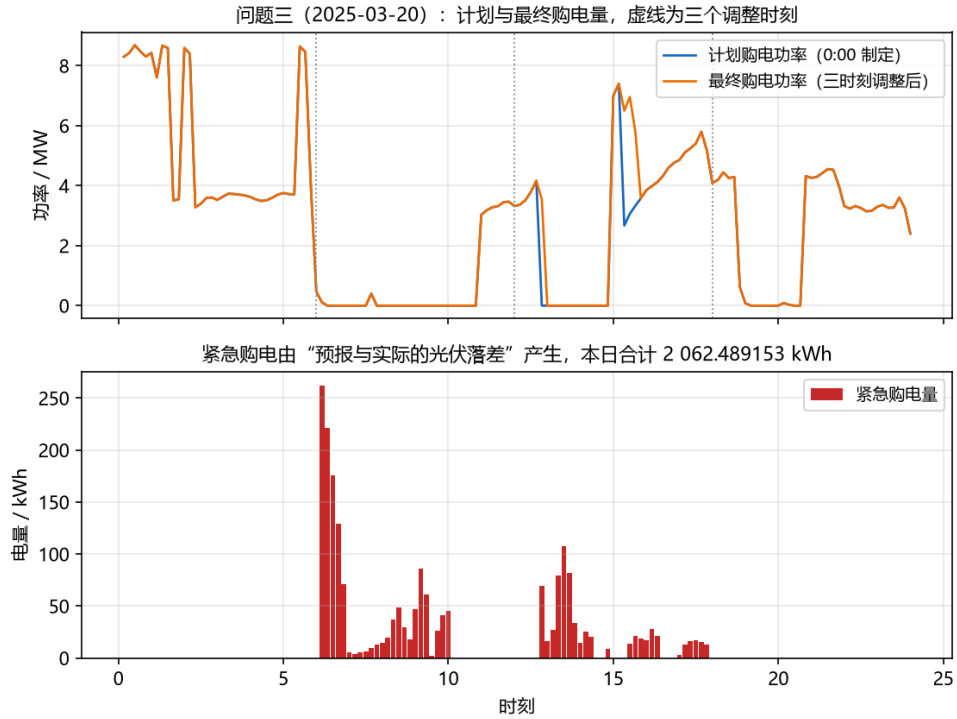


图 9：计划量、最终购电量与紧急购电量的日内剖面（虚线为三个调整时刻）

表 11：问题三的模型族与口径对照（全期 365 天，单位：元）

对照项	费用	相对主模型
主模型（三层顺序向前递推 + 闭式结算）	16 179 176.145228	—
逐日解耦重组实现 / 独立第二实现	16 179 176.145228 / 16 178 825.507754	0.000000% / $-2.17 \times 10^{-3}\%$
完全信息上界（单一全年 LP）	13 758 182.573724	-14.960%
仅 0:00 计划、无调整	16 505 833.062742	+2.019%
决策加密至每 3 小时 / 每 1 小时	16 179 575.041038 / 16 179 766.212646	+0.0025% / +0.0036%
降尺度改为分段常数（前向保持）	18 741 130.202668	+15.834%
日初复位为 6000 kWh（非物理对照）	15 360 260.582255	-5.061%

5.5.6 稳健性，以及“是否需要引入其他时刻的预报”

扰动实验表明：六组参数方向全部不反转，其中往返效率影响最大（单侧最大 10.9715%），初始储电量对交付期费用完全没有影响（六档差异均为 0.0，已由独立重解逐位复现）；噪声族内部离散度小于 1%，但期望费用随误差幅度系统性上移（ $\sigma = 5\%$ 约 +3.9%、 $\sigma = 10\%$ 约 +11.2%），原因是噪声只扰动结算层实际值而决策层仍使用预报，缺口由单侧紧急购电与非对称双向结算承接。唯一未通过的判据是“跨噪声幅度档的族间均值差 7.29% 超过 3%”，同幅度档内仅 0.06%；该判据把“族内离散度”与“跨档均值一致性”绑在一起，属口径设计问题，不构成“模型对噪声不稳定”的证据。据此本问分级为“条件稳定（需给出适用边界）”。购电上限的可行分界落在 (4375, 5000] kW，终端条件的经济价值为 3325.625002 元 (+0.0228713%)。

关于题面“请分析是否需要引入其他时刻的预报制定调整购电策略”，本文的回答限定在决策频率层面，并给出三层证据。第一层是“有没有调整”：加入 6:00、12:00、18:00 三个调整时刻可节省 326 656.92 元 (2.02%)，说明调整机制本身有效。第二层是“在四个时刻之上继续加密”：加密到每 3 小时或每 1 小时，费用反而分别上升 398.90 元与 590.07 元，即没有可测收益，而计算时间从 10.5 s 增加到 69.9 s 与 194.8 s。第三层是“新增预报信息量”：附件 3 只在 0:00、6:00、12:00、18:00 发布预报，本文不构造并不存在的输入，因此上述结论不覆盖“若能获得更频繁发布的预报是否有价值”这一问题——那属于信息供给的变化，需要新的数据来源才能回答。综合而言，在现有预报发布条件下，题面给定的四个决策时刻

已经足够，无需引入其他时刻。

5.6 问题四：波动电价下重算两问

5.6.1 价格可观测性口径与预测器

问题四的核心困难不在优化而在信息。附件 4 给出的是 2025 年逐日波动的实时电价，而“实时”意味着：当 0:00 制定当天计划时，当天 144 个时段的电价尚未发生。若把附件 4 直接当作已知输入，就等于假设电价可以被完全预知，这与题面不符，也会把问题四退化为一次“换电价重算”。因此本文明确采用如下口径：

价格可观测性约定：0:00 制定当天计划时，当天以及任何未来日的实时电价均不可观测；决策层只能使用历史电价（前一日及更早）与其他已有数据构造预测价；费用结算一律使用附件 4 的实际电价。

在此约定下，链 4-3 的调整层虽然拥有当天已实现时段的信息，6:00、12:00、18:00 时对当日剩余时段的电价仍然不可知，只能用滚动更新的预测；只有结算时才使用实际价。主预测器取“前一日同时段” $\hat{p}_{d,i}^{(0)} = p_{d-1,i}^{act}$ ：该预测器不需要估计参数，且与附件 4 的日内形状稳定性相容。为保证结论不依赖预测器选择，本文同时给出“水平 $\times$ 时段”双因子预测器（用最近 7 个可用历史日的日均价作为当日水平、用历史同时段的归一化形状作为日内结构）与“历史同时段均值”预测器作为基准对照。链 4-3 的调整层在此基础上做滚动水平校正：

$$\hat{p}_{d,i}^{(m)} = \begin{cases} p_{d,i}^{act}, & i \leq 6m \text{ (已实现, 可直接观测)} \\ \kappa_m \hat{p}_{d,i}^{(0)}, & i > 6m \end{cases}, \quad \kappa_m = \text{clip}(A_m/B_m, 0.5, 2.0), \quad \kappa_0 \equiv 1 \quad (5.15)$$

其中 A_m 为当天截至时刻 m 已实现电价的均值， B_m 为前一日同时段均价；含义是：若当天价格整体高于前一日同日同时段水平，就把剩余时段的预测按比例上调。实测 κ_m 的取值区间为 $[0.66233, 1.54939]$ ，从未触及上下界，即校正始终是内点解。实时电价预测与“用预测价做决策、用实际价结算”的分工在电力市场与储值优化文献中已有共识 [37, 38, 39, 40, 41, 42, 43]。

预测器的滚动回测结果（只用被测日之前的信息、覆盖交付期 334 天）列于表 12。

表 12：价格预测器的滚动回测（交付期 334 天，单位：元/kWh 或百分数）

预测器	MAE	MAPE	RMSE	P_{50}	P_{90}	P_{99}
前一日同时段（主口径）	0.084326	13.7801%	0.123837	0.0500	0.2294	0.3773
水平 $\times$ 时段双因子	0.093387	15.2904%	0.121566	0.0723	0.206537	0.335853
历史同时段均值	0.100231	16.3301%	0.136878	0.0710	0.241094	0.396309

表 12 提示一个必须成对报告的现象：在三个基准预测器中，主口径的**平均类误差**最小（MAE 与 MAPE 均为三者最优），而双因子预测器的**尾部类误差**最小（RMSE、 P_{90} 、 P_{99} 均优于其余两个）。两个预测器的优劣方向相反，因此只报单项指标会得出误导性结论，本文按平均类与尾部类指标成对给出；作为消融对照的自回归预测器在 MAE（0.080664）上低于主口径，本文如实披露该反例，但不据此替换主口径。另外，这两个基准预测器在决策费用上给出的结果几乎相同，说明“预测机制”在本题中只提供一条独立的信息轴，不应被计为两个独立的敏感性来源。

5.6.2 两条链的模型

链 4-2（重算问题二）。由于 0:00 不知道当天电价，链 4-2 不能用“单一全年长时域线性规划”——那需要整年的价格序列。本文采用逐日向前递推：对每个交付日 d ，用预测价与当天已知的实际负载、实际光伏求解一个单日规划

价格预测器的滚动回测（交付期 334 天，只用被测日之前信息）：两类指标优劣方向相反，须成对报告

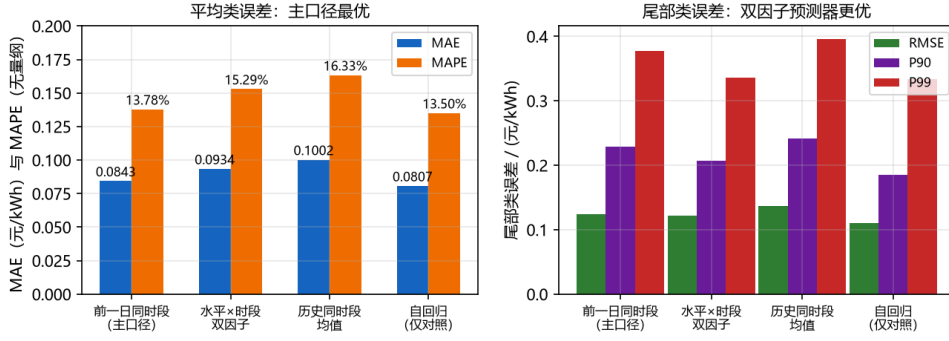


图 10：价格预测器的滚动回测：平均类与尾部类指标的最优方向相反

$$\min \sum_i \hat{p}_{d,i}^{(0)} b_{d,i} + \sum_i \alpha_{em} \hat{p}_{d,i}^{(0)} q_{d,i}^{em} \quad \text{s.t.} \quad b_i + q_i^{em} + PV_i^{act} \Delta t + q_i^{dis} = L_i \Delta t + c_i + s_i \quad (5.16)$$

提交计划量后断言 $q \equiv b$ ，储电量按日边界连续性传递、终端自由。全年 365 次单日规划共约 11.9 s。

链 4-3（重算问题三）。在问题三的结构上，把计划层的价格换成 $\hat{p}^{(0)}$ ($\kappa_0 \equiv 1$)、调整层的价格换成式 (16) 的 $\hat{p}^{(m)}$ ，其余（预报降尺度、偏差分段、结算层闭式）与问题三相同。全年共 1460 个单日线，约 38.9 s。

决策—结算分离与“价格预报误差的代价”。同一组提交的决策量可以用两套价格分别计价：按决策层信念价计得 C^{fc} ，按附件 4 实际价计得 C^{act} 。本文以 C^{act} 为唯一主结果，并把差额 $\Delta C^{price} = C^{act} - C^{fc}$ 定义为“价格预报误差的代价”单独报告。这一分解与问题三中“预报误差的代价”在结构上同构：一个是价格预测错了的代价，一个是光伏预测错了的代价。日前与实时市场的两结算与价格不确定性对储能收益的影响在文献中已被系统研究 [44, 45, 46, 47, 48]。

5.6.3 主结果与三分项归因

两条链的交付期主结果列于表 13。链 4-2 的交付期费用为 13 006 411.041148 元，链 4-3 为 15 289 050.714738 元，后者比前者高

$$2\,282\,639.673590 = 100\,230.446225 + 522\,512.788776 + 1\,659\,896.438588 \quad (\text{单位：元}) \quad (5.17)$$

即差额可以完全分解为三项：计划口径差（两链计划层的价格与信息集不同导致的计划费用差异）、偏差结算制度（链 4-3 独有的 $0.5 \times / 1.5 \times$ 双向分段费用）与紧急购电（光伏预报误差在结算层形成的缺口），恒等式残差不超过 10^{-6} 元（来自表格的六位小数舍入）。这一“三分项归因”是本文对问题四最主要的定量结论，它把一个 2.28×10^6 元的总差额变成了三条各自可解释、可单独核验的通道。

量级与上界自检：两链交付期费用均落在 $[3 \times 10^6, 5 \times 10^7]$ 元带内（日均分别为 38941.35 元与 45775.60 元）；用“完全不用储能”的可行策略构造的价格项上界为 17 026 782.858099 元与 18 884 193.220703 元，两链主值均低于上界，对应的储能价值为 4 020 371.82 元与 1 737 732.14 元。注意不能用附件 4 的最小电价构造下界——由于存在 0.0076 元/kWh 的极端低值，该下界不足实际费用的 1.2%，不具判别力。

表 14 给出两条链在题面四个指定日期的结果。链 4-2 报的是计划购电量 b 、全天购电费等于计划费；链 4-3 报的是最终购电量 q 、全天购电费为三项之和。两链的数值必然不同（价格情景与决策结构均不同）：四个日期的紧急购电在链 4-2 上全为零、在链 4-3 上分别为 5、7、9、4 个合并区间。以 2025 年 3 月 20 日为例，链 4-2 的六个 4 小时块充电量为 [8082.85, 3333.33, 5658.77, 5322.37, 0.00, 833.33] kWh、放电量为 [0, 6469.73, 2777.38, 254.72, 6478.06, 2836.94] kWh；链 4-3 的充电量为 [8082.85, 3333.33, 2856.65, 6959.71, 522.57, 833.33] kWh、放电量为 [0, 6417.07, 2313.92, 250.65, 6478.06, 2836.94] kWh。两链该日的端部储电量均为 1200.0 kWh，属逐日滚动切片，与问题一的单日周期端点（恒 6000 kWh）语义不同。

表 13: 问题四两条链的交付期主结果 (334 天; 费用单位: 元, 电量单位: kWh)

量	链 4-2	链 4-3
计划购电费 $C_{\text{plan}}^{\text{act}}$	13 006 411.041148	13 106 641.487373
调整结算费 $C_{\text{adj}}^{\text{act}}$	0 (无该项)	522 512.788776
紧急购电费 $C_{\text{em}}^{\text{act}}$	0 (机制性恒零)	1 659 896.438588
总费用 $C_{\text{total}}^{\text{act}}$	13 006 411.041148	15 289 050.714738
全期 (365 天) 总费用	14 755 884.322036	17 181 202.515506
信念价口径费用 C^{fc}	12 485 620.202113	15 030 565.249034
价格预报误差的代价 ΔC^{price}	520 790.839035	258 485.465704
计划购电量 $\sum b$ / 最终购电量 $\sum q$	20 241 516.374467 / 20 241 516.374467	20 411 764.664671 / 20 807 826.888962
紧急购电量 $\sum q^{\text{em}}$ / 弃光量 $\sum s'$	0 / 990 168.090312	394 026.040750 / 1 858 369.774422
净负荷 (两链逐位相同)	17 939 189.494967	17 939 189.494967

表 14: 问题四: 四个指定日期的指定时段购电量与全天合计 (购电量单位: kWh, 购电费单位: 元)

链	日期	10:00	12:00	14:00	16:00	18:00	20:00	全天购电量	全天购电费
4-2	2025-03-20	0.00	558.311667	0.00	507.324167	0.00	726.368333	66 622.786007	41 543.092634
	2025-06-21	0.00	0.00	0.00	27.005000	0.00	0.00	32 532.468316	15 588.839124
	2025-09-23	0.00	380.021250	0.00	493.582500	2.645000	61.911667	66 265.421275	43 331.557914
	2025-12-21	0.00	906.683333	0.00	795.930417	712.633333	30.268333	94 061.221951	70 297.812713
4-3	2025-03-20	0.00	553.563367	0.00	641.129717	0.00	726.368333	67 656.452692	52 443.510373
	2025-06-21	0.00	0.00	0.00	112.922733	0.00	0.00	34 007.160791	19 429.156761
	2025-09-23	0.00	358.668517	0.00	550.185183	2.645000	61.911667	68 433.174459	50 189.088074
	2025-12-21	0.00	833.333333	0.00	806.735183	712.633333	30.268333	94 508.735661	76 068.603362

5.6.4 机制解释

链 4-2 的价格不确定性不产生能量缺口 (图 11)。决策层使用的信念价与结算层的实际价之间存在系统偏差, 但链 4-2 中负载与光伏在 0:00 即完全已知, 价格预测错了只会让计划买贵或买便宜、不会造成供给不足, 因此紧急购电恒为零 (实测最大绝对值 2.27×10^{-13} kWh, 属求解器容差级噪声, 故表述为“机制性恒零”而非“逐位为零”)。价格误差的全部后果体现为 520 790.839035 元的费用差。

链 4-3 的缺口来自光伏预报误差。该链的紧急购电全部由“计划层与调整层用附件 3 预报、结算层用附件 2 实际”这一信息落差造成, 与价格预测器无关: 把 κ_m 冻结为 1 (即不做滚动校正) 对实际费用几乎没有影响 (相差 1.49×10^{-8} 元), 但会显著改变信念价口径下的费用差 (由 258 485.47 元升至 573 883.11 元)。这说明滚动价格校正的作用在于改善“决策者对账单的预期”, 而非降低实际支出。

两链的相对敏感性方向一致但幅度不同。在 10 个共同情景上两链的费用变化方向同号; 但在价格、负载与预测器三类扰动上, 链 4-3 反而比链 4-2 更不敏感 (分别低约 10%、17%、24%), 机制是调整层用当天已实现电价做了水平校正, 形成部分对冲; 相应地, 链 4-3 对光伏相关扰动远为敏感 (为链 4-2 的 42-65 倍)。这一“一边更钝、一边更锐”的对比正是三条归因通道的直接体现。

5.6.5 稳健性与消融

鲁棒性。链 4-2 的 162 个情景全部成功、全部判据通过, 分级为“稳定”; 其最大敏感源是题面硬参数而非价格: 往返效率的 -20% 档使费用上升 13.345%, 储电量上限收紧 20% 使费用上升 5.141%, 而紧急购电倍数在六档上给出完全相同的费用 (因紧急购电恒为零, 该参数在本链上是空参数), 初始储电量对交付期费用没有响应。链 4-3 的 198 个情景中 195 个成功, 3 个不可行情景全部是预注册的购电上限负对照 (3500、4000、4500 kW); 另有 1 条判据未通过: 光伏与负载的白噪声 $\sigma = 10\%$ 族使费用均值上升 10.5919%, 超过 5% 的预注册阈值, 机制是紧急购电项的 $\max(0, \cdot)$ 凸性系统性抬高期望紧急购电, 而所有受判样本均可行、恒等式残差为零, 属结构性而非实现缺陷。据此链 4-3 分级为“条件稳定 (需给出适用边界)”: 在 $\sigma \leq 5\%$ 的输入扰动下结论稳健, $\sigma = 10\%$ 属超设计工况; 结论对价格侧稳健、对光伏侧

问题四：0:00 制定计划时当日电价尚不可知——决策层用预测价、结算层一律用实际价

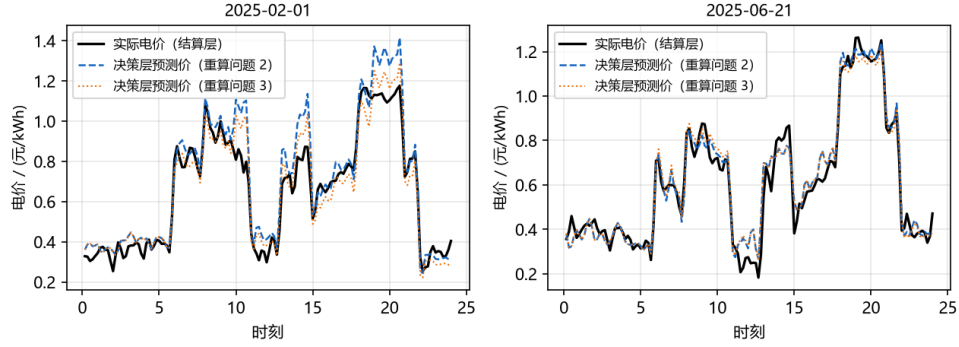


图 11：决策层信念价与结算层实际价的分离（两链共用）

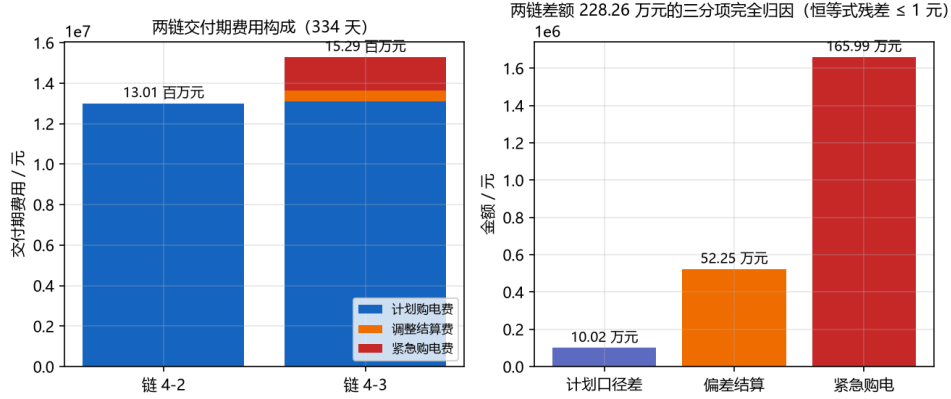


图 12：两链费用分项与交付期差额的三分项归因

不稳健。本文不通过调参或放宽判据使该判据“通过”。

消融与实现正确性锚点。三组决定性对照（13 个对照项分链展开为 23 个算例）全部执行完毕。

第一，**退化情景复现（实现正确性锚点）**：把链 4-3 的价格数据整体替换为附件 1 的单日曲线（逐日重复 365 次），其余输入与解的选择规则不变，此时“前一日同时段”预测在全年上精确成立、滚动校正因子恒为 1，模型理应退化为问题三。实测六项指标**逐位复现**问题三结果（全期 16 179 176.145228 元、交付期 14 540 616.335652 元、计划费 13 885 669.475555 元、调整费 563 098.132533 元、紧急购电费 1 730 408.537141 元、交付期紧急购电量 391 274.485914 kWh），六项相对差均不超过 4.86×10^{-13} ，远小于 10^{-9} 的判据。这是本文对“问题四的实现是否正确”给出的最硬证据：一条完全独立于价格情景的退化路径必须精确回到已知答案。

第二，**两阶段随机规划对照（是否值得引入随机优化）**。为回答“点预测加确定性优化是否足够”，本文构造两阶段随机线性规划：第一阶段选择全天共用的计划购电量，第二阶段在每个价格情景内调整并结算；情景取自附件 4 的历史实际价格（对目标日取此前同型日的 144 点实际价向量、等概率，共 24 个情景），并裁剪为 12 个代表日的确定性等价大规模。结果显示，链 4-2 上随机方案使同一窗口内的期望费用比点预测方案低 3.2860%（13 412.83 元），计划量的形状差异表现为“0-4 时与 16-20 时增购、4-8 时与 20-24 时减购”，即在低价时段多买、高价时段少买，属数量对冲。链 4-3 上随机方案报告的降幅为 19.02%，但这是因为预注册的追索结构使偏差变量恒为零、其期望费用与链 4-2 逐位相同，因此不得把该降幅表述为“对冲在链 4-3 上有 19% 的收益”。综合判断：随机优化确实有收益，但在 12 个代表日窗口上不超过 3.3%，且需要额外的场景建模与计算预算，本文据此维持“价格点预测 + 确定性线性规划”为主口径；这一判断也与“随机或鲁棒优化必须由问题结构与预算证明必要性”的方法论要求一致 [49, 50, 51]。

第三，**滚动时域对照（前瞻深度）**。在决策时刻与承诺规则完全不变的前提下，只改变“优化时看多远”，

把计划层的前瞻深度设为 $H \in \{1, 3, 7, 14\}$ 天，窗口内的未来变量只用于优化、不进入承诺。 $H = 1$ 必须逐位复现主口径，实测全期与交付期费用差均为 0、整条储电量轨迹逐点差为 0，构成基线闸门。 $H \geq 3$ 时两链交付期费用分别再降 0.1785% 与 0.1492%，且前瞻深度在 $H = 3$ 之后饱和（ H 由 3 增到 14 的费用增量仅 -3.58 元与 -5.49 元）。更重要的是，该对照直接回答了前文提出的结构性问题：主口径与 $H = 1$ 下两链的日末储电量全部贴住下限 1200 kWh（364/365 与 365/365 天），而 $H \geq 3$ 时日末储电量均值升至约 5335.9 与 5327.9 kWh、贴下限日数降至 30/365、唯一取值数增至 190 与 191。因此“日边界不携带电量”只是“逐日 0:00 决策加终端自由”这一结构下的性质，不是与模型无关的普遍规律，本文不作外推。此外如实登记一个反例：把两阶段随机与前瞻深度组合（24 个场景 $\times$ $H = 3$ 、12 个代表日）后，期望费用反而比同窗点预测方案高 2.6959%（11004.04 元），即“对冲加前瞻”在该窗口上不叠加收益；该结论只对窗口成立，本文不据此修改预注册设置。

最后，本文明确维持两条链的主口径不变：链 4-2 为“前一日同时段预测价 + 逐日确定性线性规划”，链 4-3 为“前一日同时段预测价 + 滚动水平校正 + 三层递推 + 双向分段结算”。模型族对照对费用结论的改变不超过 0.28% 与 0.15% 且无方向反转，退化情景锚点提供了实现正确性的硬证据，随机对照的收益幅度不足以证明更换主口径的必要性。

六、模型的稳健性与灵敏度分析

6.1 实验设计原则

四个问题的稳健性、灵敏度与消融分析遵循三条统一原则。第一，**判据预注册**：核心结论、判据、阈值、扰动网格与样本量在跑数前冻结，事后不得修改；凡未能通过或被证伪的项一律如实登记，不通过调参、放宽阈值或挑选指标使其“通过”。第二，**不污染主口径**：随机规划、鲁棒优化、预测器替换、决策加密、前瞻深度等对照全部只进消融，主结果始终出自“确定性线性规划 + 题面规定的信息结构”。第三，**基线闸门**：每批扰动实验必须首先逐位复现主结果，否则整批作废，以保证对照之间的可比性——四问的基线复现相对差全部为 0（或不超过 10^{-14} ）。

6.2 稳定性分级汇总

四问的扰动实验规模与结论汇总于表 15。需要强调，分级所用的判据集合在跑数前即已固定，因此“未通过”是结论而不是失败：它给出的是模型结论的适用边界。

表 15：四个问题的扰动实验规模、分级与主要敏感源

问题	情景数	未通过判据	分级	主要敏感源与关键量
问题一	948（947 可行）	无	稳定	往返效率 η （0.9 $\rightarrow$ 0.8 时 +8.885%）；初值影响 $\leq 0.07\%$
问题二	149 次全年 LP	2 条（已归因）	脆弱（需缩小适用范围）	η （ $\pm 10\%$ 使年度总量约 $\pm 7\%$ 、单日量约 $\pm 20\%$ ）
问题三	156 情景	1 条（噪声跨档均值）	条件稳定（需给出适用边界）	η （单侧最大 10.9715%）；初值对交付期无影响
链 4-2	162	无	稳定	往返效率（-20% 档 +13.345%）与储电量上限（-20% 档 +5.141%）
链 4-3	198（195 成功）	1 条（ $\sigma = 10\%$ 噪声族）	条件稳定（需给出适用边界）	光伏预报偏差（+10% 档 +23.25%）与光伏噪声

6.3 敏感性主因的识别

四个问题在敏感源上给出一致而清晰的图景，其中三点具有一般性。

第一，往返效率是唯一一阶参数。在四个问题上效率的影响都远大于其他参数：问题一 η 由 0.9 降至 0.8 时费用上升 8.885%、升至 1.0 时下降 7.828%；问题二 η 取 0.72 时费用上升 14.078%，弹性约 -0.66 至 -0.70；问题三单侧最大影响 10.9715%；链 4-2 为 13.345%。相反，初始储电量的影响在所有问题上

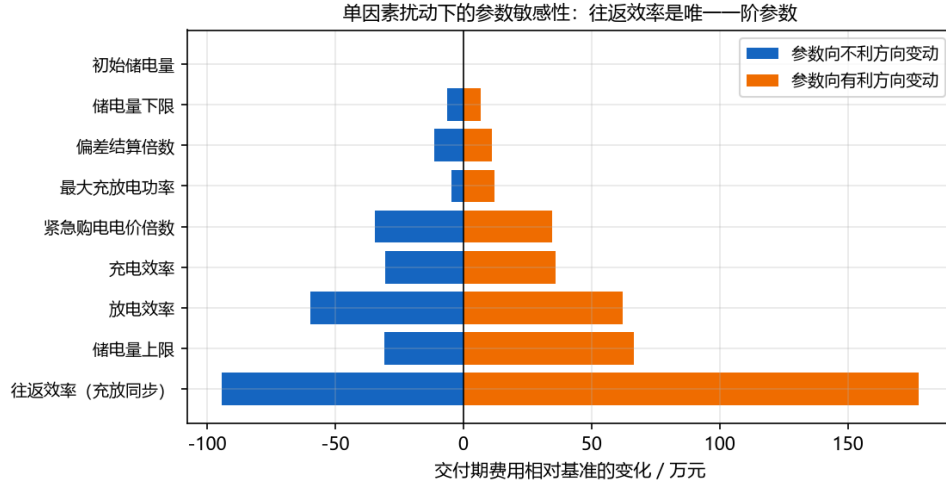


图 13: 参数敏感性的龙卷风图（问题三；其余问题为同口径图）

都可忽略（问题一不超过 0.07%，问题二不超过 0.0042%，问题三对交付期费用完全没有影响，链 4-2 无响应）。这一对比有明确的物理解释：效率直接决定储能搬运电能的成本，而初值只影响开局的少量调节裕度；文献亦指出实际效率随荷电状态与功率变化，恒效率属工程简化，因此该参数既是最敏感项、也是最需要实测标定的一项 [16, 17]。

第二，敏感性的口径依赖必须随数字一起披露。问题二的交付表稳定判据之所以未通过，正是因为日尺度量与年度总量对效率的敏感度相差约三倍： η 变化 $\pm 10\%$ 可使单日购电量与购电费移动约 $\pm 20\%$ ，而年度总量仅约 $\pm 7\%$ 。因此本文在引用表 7、表 8 的单日数值时一律附带该声明。

第三，“价格波动是第一不确定性源”在最大值口径下不成立。这是本文如实登记的一个反例：在平均口径下价格类扰动的影响确实居前，但在最大绝对值口径下价格类扰动的最大影响为 10.0%，低于效率扰动的 13.345%（链 4-2）与 11.700%（链 4-3）。真正的一阶通道在链 4-2 上是题面硬参数往返效率，在链 4-3 上是光伏预报误差。本文不使用“价格是第一不确定性源”这一未经审慎表述的结论。

此外，链 4-3 的未通过判据给出了本问题最有价值的适用边界：光伏与负载的白噪声在 $\sigma = 5\%$ 下使费用上升约 3.9%、在 $\sigma = 10\%$ 下上升 11.2%，而价格侧同类扰动影响明显更小。这说明在“预报驱动的三层决策 + 单侧紧急购电 + 非对称双向结算”这一结构下，模型对光伏侧误差的暴露远大于对价格侧误差的暴露，机制是紧急购电项的凸性把误差系统性地转化为费用。对工程实践的直接含义是：若要降低总费用，优先级应是提高光伏预报精度，而不是提高价格预报精度。

6.4 模型族与口径对照的整体结论

跨四个问题的消融给出四条一致结论。

第一，线性松弛无损，无需二元变量。问题一中加入 144 个互补二元变量的整数模型与线性模型目标值完全相同；问题二中加入 52560 个二元变量的全规模整数模型只比线性模型高 4.355178 元（相对差 3.2×10^{-7} ），代价是求解时间从 5.8 s 增到 301.6 s 且未在时限内证明最优。引理 1 给出了这一现象的解析解释，也说明在规模较大时不必按混合整数规划建模。

第二，独立第二实现与精确分解都逐位复现主结果。问题二中用不同变量排序、消去弃光变量、改用对偶单纯形法的独立实现与主模型费用逐位相同；问题三中逐日解耦的重组实现亦逐位同值。这说明主结果不是实现假象。

第三，口径影响可以大于机制价值。问题三中把光伏预报的降尺度从整点锚定线性插值改为分段常数前向保持，会使费用上升 15.83%（2561954.06 元），远大于调整机制本身 2.02% 的价值；问题四中价格预测器的选择对费用影响不超过 0.28%。这提示数据口径的选择往往比模型结构的精细化更值得审慎对待。

第四，时域口径具有可量化的经济含义。跨日状态连续、终端条件与日周期约束三项分别对应 26894.74

元、2217.08 元与 17135.02 元的费用差异；而“日边界不携带电量”这一在四条链上都出现的现象，被滚动时域对照证明只是“逐日决策加终端自由”结构的产物——把前瞻深度提高到 3 天以上，日末储电量均值即升至约 5330 kWh、贴下限日数由 364/365 降至 30/365（图 14）。

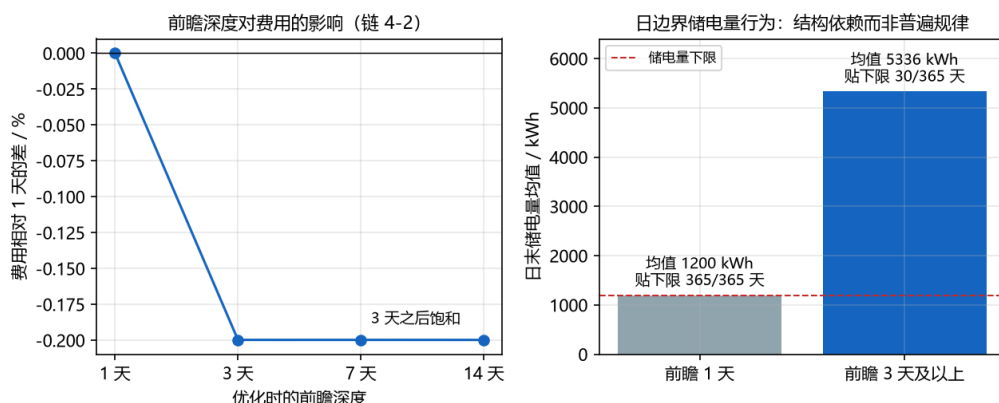


图 14: 前瞻深度对费用的影响与日边界储电量行为：日边界不携带电量只在特定结构下成立

6.5 如实登记的反例与未通过项

按“不得强凑单调性”的原则，本文登记以下六项与预期不符或未通过的结果：

1. 问题二的可行率判据未通过：唯一失败样本的目标值与基线逐位相同、解可行，未通过项是“返回解是否满足充放电互补性”，属退化最优面上的返回解选择问题。
2. 问题二的交付表稳定判据未通过： $\eta = 0.99$ 使 2025 年 6 月 21 日全天购电量下降 19.10%，属有利方向的变化，但因判据取绝对值口径而被判失败；反向 ($\eta = 0.81$) 最多使单日电费上升 12.97%。
3. 问题三的噪声判据未通过：跨噪声幅度档的族间均值差 7.29% 超过 3% 阈值，而同幅度档内仅 0.06%，属把两类统计量绑在同一判据里的口径设计问题。
4. 链 4-3 的噪声判据未通过： $\sigma = 10\%$ 族均值上升 10.5919% 超过 5% 阈值，机制为紧急购电项的凸性，属结构性而非实现缺陷。
5. 决策加密无收益：“每 3 小时”与“每 1 小时”的决策密度使费用分别上升 398.90 元与 590.07 元。本文不将其解读为“越密越差”，只登记“在题面给定的四个发布时刻下无收益”。
6. 组合对照不叠加收益：两阶段随机与前瞻深度的组合方案在同一窗口内的期望费用比点预测方案高 2.6959%。该结论只对 12 个代表日的窗口成立，本文不作外推。

6.6 对“是否需要更多预报、是否需要随机优化”的回答

题面在问题三中提出了“是否需要引入其他时刻的预报制定调整购电策略”这一问题，本文把它拆成三个递进层面回答，并以计算成本作为约束。

层面一：要不要有调整？要。加入 6:00、12:00、18:00 三个调整时刻可节省 326656.92 元（2.02%），机制是减少 889755.05 kWh 的 5 倍价紧急购电，代价是新增 563098.13 元偏差违约金，净收益为正。

层面二：在四个时刻之上是否还要加密？不需要。加密到每 3 小时或每 1 小时的费用反而分别上升 398.90 元与 590.07 元，而计算时间从 10.5 s 增到 69.9 s 与 194.8 s。由于附件 3 只在四个时刻发布预报，加密决策时刻并不增加信息量，只是把同一套预报用得更频繁。

层面三：要不要为了“看更远”而引入滚动优化？有收益但边际递减。把前瞻深度从 1 天提高到 3 天，两链交付期费用分别再降 0.1785% 与 0.1492%；提高到 7 天与 14 天几乎不再改善。因此真正的价值来自“至少看 3 天”，而不是“看得越远越好”；同时该改动会使日末储电量脱离下限，从根本上改变模型的跨日行为。

关于随机优化：本文构造的两阶段随机规划对照在链 4-2 上使同窗期望费用降低 3.2860%，说明价格不确定性确实存在可对冲的部分；但链 4-3 的随机对照因预注册的自由追索而退化，其 19.02% 的“降幅”不可作为对冲收益。综合收益幅度（不超过 3.3%）、场景建模所需的历史样本量与额外计算预算，本文维持“价格点预测 + 确定性线性规划”为主口径，并明确指出：若要改换主口径，必须先补足价格与光伏误差的实测分布证据 [49, 50]。

七、模型的评价与推广

7.1 模型的优点

第一，单一内核覆盖四个问题，口径可追溯。四个问题共用同一套 10 分钟粒度的线性规划内核，差异被完全参数化到时间、信息、结算与价格四个维度上（表 2）。变量定义、单位、参数值与时间范围在四问之间逐项一致，因此任何一个问题的实现都可以用同一套核验脚本检查，也使得“净负荷”“弃光量”这类与价格无关的量成为跨问题的硬校验点：四问的交付期净负荷均为 17 939 189.494967 kWh，问题二与链 4-2 的弃光量均为 990 168.090312 kWh，而费用必然不同——这既证明价格情景真实生效，也排除了“把上一问结果填到下一问”的可能。

第二，实现正确性被当作一等目标，并且给出了硬证据。本文构造了三类相互独立的验证手段：一是**解析界**，为每个问题独立构造可行上界与下界（如问题一的“不用储能”策略给出 48052.046591 元），并明确指出因附件 4 存在 0.0076 元/kWh 的极端低值、以最小电价构造的下界不具判别力；二是**独立第二实现**，用不同的变量排序、变量消去与求解器设置重算，问题二中与主模型逐位相同；三是**退化情景锚点**，把链 4-3 的价格换成附件 1 的单日曲线后必须逐位复现问题三的六个数值，实测相对差不超过 4.86×10^{-13} （判据为 10^{-9} ）。这三类手段把“实现是否正确”与“模型是否合适”两个问题彻底分开。

第三，量纲与索引口径被显式固定，避免了最隐蔽的一类错误。本文明确规定：决策变量以 kWh 为单位、本身已含时间因子，费用表达式中不再乘 Δt ；附件时间戳为所属时段的左端点，据此计划窗为 0:10 至 24:10。这两项约定看似细节，但一旦出错，费用会整体偏差 6 倍、全部时段整体错位，而结果“看起来仍然合理”。核验中对每一项都做了独立复算：若误乘 Δt ，问题二交付期费用会落到约 2.04×10^6 元，与正确值相差 6 倍。

第四，对“未通过”与“不利结果”的处理方式本身构成结论的一部分。四个问题共登记六项未通过判据与反例（表 15 及相关正文），全部如实保留并逐条给出归因与适用边界。例如问题二的“脆弱”分级并非模型失败，而是准确指出了日尺度量对效率口径的高敏感性；链 4-3 的“条件稳定”则准确指出了模型对光伏侧误差的暴露远大于价格侧——这一结论对实际运行有直接指导意义。

第五，计算预算可控且不依赖特殊硬件。最大规模实例为问题二的 52 560 个时段、315 360 个变量、105 120 条等式约束，在普通 CPU 上以稀疏矩阵求解约 5.8 s；问题三与链 4-3 因分解为 1460 个单日层分别约需 42 s 与 39 s；全部鲁棒性与消融实验（合计上千次线性规划）在数十分钟内完成。这意味着模型可以被反复重算，适合作为决策支持工具 [52]。

7.2 模型的不足与适用边界

第一，确定性口径，预报误差的代价只能事后量化。主结果不含误差分布，也不含随机或鲁棒优化。问题三比问题二贵 2 307 565.50 元（+18.863%），同净负荷下这一差额完全由预报误差造成；链 4-3 比链 4-2 贵 2 282 639.673590 元。本文用“价格预报误差的代价”（520 790.839035 元与 258 485.465704 元）与“光伏预报误差的代价”（紧急购电费 1 659 896.438588 元）分别量化了这两类误差，但都属事后度量；消融中的随机对照说明改进空间在 3% 量级，不足以为更换主口径提供充分理由。

第二，光伏降尺度不保块内能量。附件 3 只给整点预报，本文按整点锚定线性插值降到 10 分钟，块内平均功率为 $(2.5A_{k-1} + 3.5A_k)/6$ ，一般不等于整点值。若改用分段常数前向保持，费用会上升 15.83%，说明该项口径影响很大。本文的做法是保留双端整点信息并显式披露其能量性质，但这是本问题最需审慎的一项建模选择。

第三，若干物理简化与题面参数的解释约定。模型假设常效率（不随荷电状态、功率与温度变化）、无自放电、购电无功率上限、无售电收益、不计储能折旧；其中“5000 kW 同时作用于并网点侧与电池侧、取更严者”是本文对题面参数的解释约定，不构成文献支持的结论。这些简化的影响方向已被量化：效率是唯一一阶敏感参数；若存在 3000 kW 的隐含购电上限，问题一的计划将不可行；在允许售电的情景下，松散的弃光上界会出现售出电量超过物理盈余的非物理套利。

第四，文献证据等级不足。全文引用的文献均只达到摘要或元数据级，没有一条经过全文精读，因此引用只用于支撑框架级与机制级主张（“这类问题可以用线性规划建模”“分时电价下存在充放电套利动机”“偏差结算制度会影响储能策略”），不用于支撑任何公式级或定量级结论。题面给定的硬参数（紧急购电 5 倍价、偏差结算 0.5 倍与 1.5 倍）与本文的模型内推论（紧急购电恒零）同样不作为文献结论陈述。

第五，最优面非唯一带来呈现上的技术限制。线性规划的最优调度面可能不唯一（储电量上下限两端均可能活跃），因此验收以约束残差、结构恒等式、目标值与交付表格为准，不比对逐点解的唯一性；本文统一采用字典序规则选取提交解，并量化了该规则的影响（问题三约 10^{-4} 量级、链 4-3 为 0.110739 元），说明它不改变任何结论，但读者不应把不同求解器返回的轨迹差异理解为模型不稳定。

第六，前瞻深度与日边界行为的结构依赖。在“逐日 0:00 决策加终端自由”的组织方式下，四条链的日末储电量都贴住下限 1200 kWh。滚动时域对照证明这只是该结构的产物：把前瞻深度提高到 3 天以上，日末储电量均值即升至约 5330 kWh。因此本文不把“日边界不携带电量”写成普遍规律——这既是结论，也是对模型适用边界的提示。

7.3 模型的改进方向

其一，**提升证据等级：**取得关键文献全文，把引用从摘要级提升到全文级，重新核验公式级与定量级主张，尤其是偏差结算制度与价格预测方法相关的主张。

其二，**补充误差分布的实测证据，**在此基线上重新评估随机规划与分布鲁棒优化是否应作为主口径。本文已给出评估框架（两阶段随机、滚动时域、二者组合）与预注册判据，但结论依赖场景构造，需要价格与光伏误差的真实分布数据支撑。

其三，**细化储能物理模型：**引入与荷电状态、充放电功率相关的动态效率与自放电项，并在此基线上重估费用与稳定性分级；本文已量化效率的敏感性，可作为该改进价值的前置估计。

其四，**引入极短的实时平衡层：**题面允许在“其他时刻”获得预报，本文结论限于现有发布时刻下的决策频率；若能获得更频繁的预报发布，则需重新评估“信息供给增加”的价值。

7.4 本文的创新点

创新点一：以“四个维度上的参数化”统一四个问题，并由此得到一个可复用的建模模板。本文没有为四个问题分别建模，而是先证明它们共享同一物理系统与同一数学结构，再把差异定位到时间、信息、结算、价格四个维度。这既减少了重复推导，更让“什么是这道题的难点”变得可判定：问题一、二在时域与终端口径，问题三在信息结构与降尺度，问题四在价格可观测性。

创新点二：给出互补性引理，把“是否需要二元变量”从工程判断变成数学结论。通过对往返效率小于 1 的利用，本文证明同时充放电被严格支配，从而线性松弛无损；该结论被两个不同规模的整数模型对照独立验证（问题一中 144 个二元变量模型目标值完全相同，问题二中 52560 个二元变量模型仅差 3.2×10^{-7} 而耗时增加 50 倍）。这节省的不只是计算量，也避免了因引入大 M 常数而带来的数值脆弱性。

创新点三：把“代价”做成可分解的量，而不是笼统的“不确定性影响”。本文对两类误差分别给出价格与金额，并在问题四中把两条链 2282639.673590 元的总差额完全分解为计划口径差、偏差结算与紧急购电三项（恒等式残差不超过 10^{-6} 元）。这种分解使“哪一条通道更值得改进”有了定量答案：对链 4-3 而言，降低光伏预报误差的收益远大于提高价格预测精度。

创新点四：用“退化情景复现”构造实现正确性锚点。把链 4-3 的价格换成附件 1 的单日曲线后，模型应当精确退化为问题三；实测六项指标逐位复现，相对差不超过 4.86×10^{-13} 。这一手法把“实现是否

正确”变成一个可以客观判定的问题，且适用于任何“旧问题加新情景”的题目结构。

创新点五：用滚动时域对照回答“结构性问题”，而非仅作灵敏度扫描。“日边界储电量为什么总贴下限”无法通过参数扰动回答，因为它是结构性的。本文设计了对决策时刻与承诺规则完全不变、只改变“优化时看多远”的对照实验：前瞻深度为1天时逐位复现主口径，提高到3天以上时日末储电量均值升至约5330 kWh、贴下限日数由364/365降至30/365。这一“只改一个维度、其余严格不变”的设计使结论具有因果含义。

创新点六：对不确定性与前瞻给出“三层递进回答”，把建议落到可执行的决策上。本文对“是否需要更多预报”给出层次分明的答案：需要调整机制、不需要加密决策时刻；需要一定前瞻（至少3天）、不需要无限前瞻；随机优化有收益（3.29%）但幅度不足以替代主口径。同时指出优先级：由于模型对光伏侧误差的暴露远大于价格侧，提高光伏预报精度应是首要投入。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于文献检索与元数据核验、模型与公式的推导辅助、代码实现与调试、结果一致性检查以及语言润色。全部模型的选型、参数取值、口径裁定与最终结论均由参赛队员判断与复核，AI 工具未参与任何结果的直接生成；详细使用情况见支撑材料《AI 工具使用详情》。

参考文献

- [1] (CSEE Journal of Power and Energy Systems, Crossref 未返回作者字段). Microgrid Energy Management with Energy Storage Systems: A Review[J]. CSEE Journal of Power and Energy Systems, 2023.
- [2] Moradi Hadis, Esfahanian Mahdi, Abtahi Amir, Zilouchian Ali. Optimization and energy management of a standalone hybrid microgrid in the presence of battery storage system[J]. Energy, 2018.
- [3] Alguhi Ahmed A., Alotaibi Majed A. Optimal Operation of Battery Energy Storage Systems in Microgrid-Connected Distribution Networks for Economic Efficiency and Grid Security[J]. Energies, 2025.
- [4] Valibeygi Amir, de Callafon Raymond A. Optimal Battery Dispatch and Real-Time State of Charge Tracking for Microgrid Applications[C]. 2020 American Control Conference (ACC), 2020.
- [5] Wang Qi, Wu Wenchuan, Lin Chenhui, Xu Shuwei, Wang Siyuan, Tian Jian. An exact relaxation method for complementarity constraints of energy storages in power grid optimization problems[J]. Applied Energy, 2024.
- [6] Nazir Nawaf, Almassalkhi Mads. Guaranteeing a Physically Realizable Battery Dispatch Without Charge-Discharge Complementarity Constraints[J]. IEEE Transactions on Smart Grid, 2023.
- [7] Grimaldi Alberto, Minuto Francesco, Demetrio, Perol Alessandro, Casagrande Silvia, Lanzini Andrea. Techno-economic optimization of utility-scale battery storage integration with a wind farm for wholesale energy arbitrage considering wind curtailment and battery degradation[J]. Journal of Energy Storage, 2025.
- [8] Qing Jiayuan, Jing Shi, Zhao Yincheng, Zhang Sen, Wan Hao, Cao Yihang. An Optimization for Capacity Configuration of Photovoltaic - Energy Storage Microgrid Considering the Solar Power Curtailment Rate[C]. 2025 7th Asia Energy and Electrical Engineering Symposium (AEEES), 2025.
- [9] Carpinelli G., Bracale A., Caramia P. The GREAT project: Integer linear programming-based day-ahead optimal scheduling of a DC microgrid[C]. 2013 12th International Conference on Environment and Electrical Engineering, 2013.
- [10] Alamir Nehmedo, Kamel Salah. Optimized Day-Ahead Scheduling of Cooperative Multi-Microgrid with Demand Response and Shapley Value-Based Cost Allocation[C]. 2025 26th International Middle East Power Systems Conference (MEPCON), 2025.

- [11] Lee Lae Yeop, Choi Seong Gon. Power Purchase Cost Minimization in Radial Microgrid Network with Linear Programming[C]. 2020 International Conference on Information and Communication Technology Convergence (ICTC), 2020.
- [12] Lin Yangshu, Shao Yuhao, Yang Chao, Guo Wenxuan, Liu Weijie, Zheng Chenghang, Gao Xiang. Evaluation and optimization for integrated photo-voltaic and battery energy storage systems under time-of-use pricing in the industrial park[J]. Journal of Energy Storage, 2024.
- [13] Baloyi T., Chowdhury S. Sizing and Selection of Battery Energy Storage System for Time of Use Arbitrage in a Commercial Building in South Africa[J]. 2021 IEEE PES/IAS PowerAfrica, 2021.
- [14] Olivieri Zachary T., McConky Katie. Optimization of residential battery energy storage system scheduling for cost and emissions reductions[J]. Energy and Buildings, 2020.
- [15] Shen Fangfei, Luo Quanming. A Multi-Objective Optimization Framework for Optimal Configuration of Battery Energy Storage System in Peak Shaving and Valley Filling Scenarios[J]. Applied Sciences, 2026.
- [16] Vykhodtsev Anton V., Jang Darren, Wang Qianpu, Rosehart William, Zareipour Hamidreza. A review of modelling approaches to characterize lithium-ion battery energy storage systems in techno-economic analyses of power systems[J]. Renewable and Sustainable Energy Reviews, 2022.
- [17] Huang Hongbo, Wang Hui, Cai Yang, Chen Xiwei, Li Tingting. Optimal Configuration of Fire-Storage Capacity Considering Dynamic Charge-Discharge Efficiency of Hybrid Energy Storage[J]. Frontiers in Energy Research, 2022.
- [18] Zhang Zhong, Wang Jianxue, Wang Xiuli. An improved charging/discharging strategy of lithium batteries considering depreciation cost in day-ahead microgrid scheduling[J]. Energy Conversion and Management, 2015.
- [19] Silvente Javier, Kopanos Georgios M., Pistikopoulos Efstratios N., Espuña Antonio. A rolling horizon optimization framework for the simultaneous energy supply and demand planning in microgrids[J]. Applied Energy, 2015.
- [20] Holjevac Ninoslav, Capuder Tomislav, Zhang Ning, Kuzle Igor, Kang Chongqing. Corrective receding horizon scheduling of flexible distributed multi-energy microgrids[J]. Applied Energy, 2017.
- [21] Hu Jiefeng, Shan Yinghao, Guerrero Josep M., Ioinovici Adrian, Chan Ka Wing, Rodríguez José. Model predictive control of microgrids –An overview[J]. Renewable and Sustainable Energy Reviews, 2020.
- [22] Finnah Benedikt, Gönsch Jochen, Ziel Florian. Integrated day-ahead and intraday self-schedule bidding for energy storage systems using approximate dynamic programming[J]. European Journal of Operational Research, 2021.
- [23] Chazarra Manuel, Pérez-Díaz Juan I., García-González Javier. Value of perfect information of spot prices in the joint energy and reserve hourly scheduling of pumped storage plants[J]. Electric Power Systems Research, 2017.
- [24] Toubreau Jean-François, Bottieau Jérémie, De Grève Zacharie, Vallée François, Bruninx Kenneth. Data-Driven Scheduling of Energy Storage in Day-Ahead Energy and Reserve Markets With Probabilistic Guarantees on Real-Time Delivery[J]. IEEE Transactions on Power Systems, 2020.
- [25] van der Veen Reinier A.C., Hakvoort Rudi A. The electricity balancing market: Exploring the design challenge[J]. Utilities Policy, 2016.
- [26] Herre Lars. Impact of Imbalance Settlement System Design on Risk-Averse Energy Storage[C]. 2020 17th International Conference on the European Energy Market (EEM), 2020.

- [27] Joos Michael, Staffell Iain. Short-term integration costs of variable renewable energy: Wind curtailment and balancing in Britain and Germany[J]. *Renewable and Sustainable Energy Reviews*, 2018.
- [28] Diamantis Almpantis, Henrik Davidsson, Martin Andersson. A minute-resolution downscaling algorithm for high-resolution global irradiance time series[J]. *Solar Energy Advances*, 2024.
- [29] Florian Barbieri, Sumedha Rajakaruna, Arindam Ghosh. Very short-term photovoltaic power forecasting with cloud modeling: A review[J]. *Renewable and Sustainable Energy Reviews*, 2017.
- [30] Peter Stenzel, Jochen Linssen, Johannes Fler, Florian Busch. Impact of temporal resolution of supply and demand profiles on the design of photovoltaic battery systems for increased self-consumption[C]. 2016 IEEE International Energy Conference (ENERGYCON), 2016.
- [31] Zhongwen Li, Chuanzhi Zang, Peng Zeng, Haibin Yu. Combined Two-Stage Stochastic Programming and Receding Horizon Control Strategy for Microgrid Energy Management Considering Uncertainty[J]. *Energies*, 2016.
- [32] Arnab Bhattacharya, Jeffrey P. Kharoufeh, Bo Zeng. Managing Energy Storage in Microgrids: A Multistage Stochastic Programming Approach[J]. *IEEE Transactions on Smart Grid*, 2018.
- [33] Mahmoud Elkazaz, Mark Sumner, David Thomas. Energy management system for hybrid PV-wind-battery microgrid using convex programming, model predictive and rolling horizon predictive control with experimental validation[J]. *International Journal of Electrical Power & Energy Systems*, 2020.
- [34] Irene Danti Lopez, Damian Flynn, Maël Desmartin, Marcelo Saguan. Intraday dispatch, energy storage and the value of re-scheduling in systems with high wind shares[C]. 2019 IEEE Power & Energy Society General Meeting (PESGM), 2019.
- [35] Takuji Matsumoto, Derek Bunn, Yuji Yamada. Mitigation of the Inefficiency in Imbalance Settlement Designs Using Day-Ahead Prices[J]. *IEEE Transactions on Power Systems*, 2022.
- [36] S. Asgarpour, S.K. Panarelli. Expected Cost Penalty due to Deviation from Economic Dispatch for interconnected power systems[J]. *IEEE Transactions on Power Systems*, 1995.
- [37] Jesus Lago, Fjo De Ridder, Bart De Schutter. Forecasting spot electricity prices: Deep learning approaches and empirical comparison of traditional algorithms[J]. *Applied Energy*, 2018.
- [38] Arnaud de Myttenaere, Boris Golden, Bénédicte Le Grand, Fabrice Rossi. Mean Absolute Percentage Error for regression models[J]. *Neurocomputing*, 2016.
- [39] Hamed Chitsaz, Payam Zamani-Dehkordi, Hamidreza Zareipour, Palak P. Parikh. Electricity Price Forecasting for Operational Scheduling of Behind-the-Meter Storage Systems[J]. *IEEE Transactions on Smart Grid*, 2018.
- [40] Hadi Khani, Mohammad R. Dadash Zadeh. Online Adaptive Real-Time Optimal Dispatch of Privately Owned Energy Storage Systems Using Public-Domain Electricity Market Prices[J]. *IEEE Transactions on Power Systems*, 2015.
- [41] Iain Staffell, Mazda Rustomji. Maximising the value of electricity storage[J]. *Journal of Energy Storage*, 2016.
- [42] Tebello Mathaba, Xiaohua Xia, Jiangfeng Zhang. Analysing the economic benefit of electricity price forecast in industrial load scheduling[J]. *Electric Power Systems Research*, 2014.
- [43] Bri-Mathias Hodge, Carlo Brancucci Martinez-Anido, Qin Wang, Erol Chartan, Anthony Florita, Juha Kiviluoma. The combined value of wind and solar power forecasting improvements and electricity storage[J]. *Applied Energy*, 2018.

- [44] Dheepak Krishnamurthy, Canan Uckun, Zhi Zhou, Prakash R. Thimmapuram, Audun Botterud. Energy Storage Arbitrage Under Day-Ahead and Real-Time Price Uncertainty[J]. IEEE Transactions on Power Systems, 2018.
- [45] Xiyun Yang, Liwei Fan, Xiangjun Li, Lingzhuochao Meng. Day-ahead and real-time market bidding and scheduling strategy for wind power participation based on shared energy storage[J]. Electric Power Systems Research, 2023.
- [46] Jeremie Bottieau, Louis Hubert, Zacharie De Greve, Francois Vallee, Jean-Francois Toubreau. Very-Short-Term Probabilistic Forecasting for a Risk-Aware Participation in the Single Price Imbalance Settlement[J]. IEEE Transactions on Power Systems, 2020.
- [47] Jiajia Yang, Zhao Yang Dong, Fushuan Wen, Qixin Chen, Fengji Luo, Weijia Liu, Junpeng Zhan. A Penalty Scheme for Mitigating Uninstructed Deviation of Generation Outputs From Variable Renewables in a Distribution Market[J]. IEEE Transactions on Smart Grid, 2020.
- [48] Nerijus Paulauskas, Vsevolod Kapustin. Battery Scheduling Optimization and Potential Revenue for Residential Storage Price Arbitrage[J]. Batteries, 2024.
- [49] Alireza Akbari-Dibavar, Kazem Zare, Sayyad Nojavan. A hybrid stochastic-robust optimization approach for energy storage arbitrage in day-ahead and real-time markets[J]. Sustainable Cities and Society, 2019.
- [50] Ciaran O'Connor, Mohamed Bahloul, Roberto Rossi, Steven Prestwich, Andrea Visentin. Conformal Prediction for electricity price forecasting in the day-ahead and real-time balancing market[J]. Energy and AI, 2025.
- [51] Rawa Muhyaddin, Al-Turki Yusuf, Sedraoui Khaled, Dadfar Sajjad, Khaki Mehrdad. Optimal operation and stochastic scheduling of renewable energy of a microgrid with optimal sizing of battery energy storage considering cost reduction[J]. Journal of Energy Storage, 2023.
- [52] Tao Hai, Narinderjit Singh Sawaran Singh, Farah Jamal. Energy management of a microgrid with integration of renewable energy sources considering energy storage systems with electricity price[J]. Journal of Energy Storage, 2025.

附录

附录 A 支撑材料文件列表

支撑材料包含五个结果文件、四个问题的完整可运行源程序，以及 AI 工具使用记录，清单如下。

类别	文件	说明
结果文件	result1.xlsx	问题一的计划购电量与储能充放电量
结果文件	result2.xlsx	问题二的计划购电量、充放电量与紧急购电量
结果文件	result3.xlsx	问题三的计划购电量、调整购电量、充放电量与紧急购电量
结果文件	result4-2.xlsx	波动电价下重算问题二的完整结果
结果文件	result4-3.xlsx	波动电价下重算问题三的完整结果
源程序	prob01_io.py、prob01_model.py、run_prob01.py	问题一：输入输出、模型构造、求解主程序
源程序	prob02_io.py、prob02_model.py、run_prob02.py	问题二：输入输出、模型构造、求解主程序
源程序	prob03_io.py、prob03_model.py、run_prob03.py	问题三：输入输出、三层模型、求解主程序
源程序	prob04_io.py、prob04_model.py、prob04_predict.py、run_prob04.py	问题四：输入输出、两链模型、价格预测器、求解主程序
其他	《AI 工具使用详情》	AI 工具使用记录与人工复核说明

四个问题的求解程序共用同一套接口约定：输入为附件 `xlsx` 文件路径与结果模板路径，输出为解序列、交付表格、运行清单与交付工作簿。复现命令形如

```
python run_prob01.py --data 附件 1.xlsx --template result1.xlsx --output < 输出目录 >
```

所有程序均为 Python 3.14 编写，仅依赖 `numpy`、`scipy`、`openpyxl` 三个第三方库；线性规划统一调用 `scipy.optimize.linprog` 的 `highs` 方法在 CPU 上求解，不需要 GPU，也不依赖任何商业求解器。

附录 B 求解程序清单

B.1 问题一：输入输出模块

```

1  # -*- coding: utf-8 -*-
2  """prob01 输入读取与 ``result1.xlsx`` 填报。
3
4  只读 ``data/附件1.xlsx`` 与模板 ``data/附件5/result1.xlsx``；不修改任何原始数据。
5  口径来源：assumption_v003（AS01/AS02/AS09/AS13）与团队勘误 E1/E2/E3。
6  """
7  from __future__ import annotations
8
9  import hashlib
10 import json
11 from dataclasses import dataclass
12 from pathlib import Path
13 from typing import Any, Sequence
14
15 import numpy as np
16 import openpyxl
17
18 EXPECTED_HEADER = ("时间", "电价", "小区负载", "光伏发电预测功率")
19 PLAN_SHEET = "计划购电量"
20 STORAGE_SHEET = "充放电量"
21 BLOCK_ROWS = 24
22
23
24 class InputValidationError(RuntimeError):
25     """输入结构与 accepted 假设不一致时严格失败。"""
26
27
28 @dataclass(frozen=True)
29 class Attachment1:
30     """附件 1 的 144 时段电价 / 负载 / 光伏预测（功率单位 kW）。"""
31
32     price: np.ndarray
33     load_kw: np.ndarray

```

```

34     pv_kw: np.ndarray
35     time_labels: list[str]
36     md5: str
37     header: list[str]
38
39     @property
40     def periods(self) -> int:
41         return int(self.price.shape[0])
42
43
44 def md5_of(path: Path) -> str:
45     digest = hashlib.md5()
46     with path.open("rb") as handle:
47         for chunk in iter(lambda: handle.read(1 << 20), b""):
48             digest.update(chunk)
49     return digest.hexdigest()
50
51
52 def _label(value: Any) -> str:
53     if hasattr(value, "strftime"):
54         return value.strftime("%H:%M")
55     return str(value)
56
57
58 def read_attachment1(path: Path, expected_rows: int = 144) -> Attachment1:
59     """读取附件 1 第 1 个工作表并做结构校验。"""
60     if not path.is_file():
61         raise InputValidationError(f"附件 1 不存在: {path}")
62     workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
63     try:
64         sheet = workbook.worksheets[0]
65         rows = [row for row in sheet.iter_rows(min_row=1, max_col=4, values_only=True)]
66     finally:
67         workbook.close()
68     if len(rows) < 2:
69         raise InputValidationError("附件 1 没有数据行")
70     header = [str(item).strip() for item in rows[0]]
71     if tuple(header) != EXPECTED_HEADER:
72         raise InputValidationError(f"附件 1 表头不匹配: {header} != {list(EXPECTED_HEADER)}")
73     body = rows[1:]
74     if len(body) != expected_rows:
75         raise InputValidationError(f"附件 1 数据行数 {len(body)} != 期望 {expected_rows}")
76     labels = [_label(row[0]) for row in body]
77     try:
78         price = np.array([float(row[1]) for row in body], dtype=float)
79         load_kw = np.array([float(row[2]) for row in body], dtype=float)
80         pv_kw = np.array([float(row[3]) for row in body], dtype=float)
81     except (TypeError, ValueError) as exc:
82         raise InputValidationError(f"附件 1 存在非数值单元格: {exc}") from exc
83     for name, values in (("电价", price), ("负载", load_kw), ("光伏", pv_kw)):
84         if not np.all(np.isfinite(values)):
85             raise InputValidationError(f"附件 1 的{name}列含 NaN/Inf")
86     if np.any(price <= 0):
87         raise InputValidationError("附件 1 存在非正电价, LP 对偶与互补性引理的前提被破坏")
88     if np.any(load_kw < 0):
89         raise InputValidationError("附件 1 存在负负载")
90     if np.any(pv_kw < 0):
91         raise InputValidationError("附件 1 存在负光伏功率")
92     return Attachment1(
93         price=price,
94         load_kw=load_kw,
95         pv_kw=pv_kw,
96         time_labels=labels,
97         md5=md5_of(path),
98         header=header,
99     )
100
101
102 def inspect_template(path: Path) -> dict[str, Any]:
103     """读取模板的逐行标签, 用于 AS01/AS13 的对齐校验 (不写入)。"""
104     if not path.is_file():
105         raise InputValidationError(f"模板不存在: {path}")
106     workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
107     try:
108         if PLAN_SHEET not in workbook.sheetnames or STORAGE_SHEET not in workbook.sheetnames:
109             raise InputValidationError(f"模板工作表名不符合要求: {workbook.sheetnames}")
110         plan = workbook[PLAN_SHEET]
111         plan_labels = [str(row[0]) for row in plan.iter_rows(min_row=2, max_col=1, values_only=True)]
112         storage = workbook[STORAGE_SHEET]
113         storage_rows = [row for row in storage.iter_rows(min_row=2, max_col=5, values_only=True)]
114     finally:

```

```

115     workbook.close()
116     return {
117         "plan_labels": plan_labels,
118         "storage_labels": [str(row[0]) for row in storage_rows],
119         "storage_endpoint_labels": [str(row[3]) if row[3] is not None else None for row in storage_rows[:2]],
120         "plan_periods": len(plan_labels),
121     }
122
123
124 def fill_result_workbook(
125     *,
126     template: Path,
127     dest: Path,
128     purchase_kwh: Sequence[float],
129     charge_kwh: Sequence[float],
130     discharge_kwh: Sequence[float],
131     e_init: float,
132     e_final: float,
133     periods: int,
134 ) -> dict[str, Any]:
135     """把解填入 ``result1.xlsx``; 保留模板标签与样式, 只写数值单元格。"""
136     if not template.is_file():
137         raise InputValidationError(f"模板不存在: {template}")
138     workbook = openpyxl.load_workbook(template)
139     try:
140         if PLAN_SHEET not in workbook.sheetnames or STORAGE_SHEET not in workbook.sheetnames:
141             raise InputValidationError(f"模板工作表名不符合要求: {workbook.sheetnames}")
142         plan = workbook[PLAN_SHEET]
143         plan_rows = plan.max_row - 1
144         if plan_rows < periods:
145             raise InputValidationError(f"模板 [{PLAN_SHEET}] 只有 {plan_rows} 行, 少于 {periods} 个时段")
146         for index in range(periods):
147             plan.cell(row=2 + index, column=2, value=float(purchase_kwh[index]))
148
149         storage = workbook[STORAGE_SHEET]
150         blocks = storage.max_row - 1
151         filled_blocks = 0
152         for block in range(blocks):
153             start = block * BLOCK_ROWS
154             stop = min(start + BLOCK_ROWS, periods)
155             if stop <= start:
156                 break
157             storage.cell(row=2 + block, column=2, value=float(sum(charge_kwh[start:stop])))
158             storage.cell(row=2 + block, column=3, value=float(sum(discharge_kwh[start:stop])))
159             if stop - start == BLOCK_ROWS:
160                 filled_blocks += 1
161         if periods == blocks * BLOCK_ROWS:
162             storage.cell(row=2, column=5, value=float(e_init))
163             storage.cell(row=3, column=5, value=float(e_final))
164         dest.parent.mkdir(parents=True, exist_ok=True)
165         workbook.save(dest)
166     finally:
167         workbook.close()
168     return {
169         "plan_rows_filled": periods,
170         "storage_blocks_filled": filled_blocks,
171         "storage_endpoints_filled": periods == 144,
172     }
173
174
175 def write_json(path: Path, payload: Any) -> None:
176     """写 JSON; 显式拒绝 NaN/Inf, 避免把非有限值伪装成合法结果。"""
177     path.parent.mkdir(parents=True, exist_ok=True)
178     text = json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False)
179     path.write_text(text + "\n", encoding="utf-8")

```

B.2 问题一：模型构造模块

```

1  # -*- coding: utf-8 -*-
2  """prob01 计划购电 LP 的建模、求解与结构自检。
3
4  模型 (P1) 严格对应 formulation_v001 / assumption_v003, 并按团队勘误 E1 取
5  ``q_dis_t <= 750.00`` (D10 口径丙)。求解器: CPU HIGHS (``scipy.optimize.linprog``),
6  本问为确定性 LP, 不使用 GPU、不依赖随机种子。
7  """
8  from __future__ import annotations
9
10 from dataclasses import dataclass
11 from typing import Any

```

```

12
13 import numpy as np
14 from scipy.optimize import linprog
15
16 DELTA_T = 1.0 / 6.0
17 ETA_CH = 0.9
18 ETA_DIS = 0.9
19 ETA_ROUND_TRIP = ETA_CH * ETA_DIS
20 E_INIT = 6000.0
21 E_MIN = 1200.0
22 E_MAX = 10800.0
23 E_CAP_MAX = 12000.0
24 P_MAX = 5000.0
25 C_CAP = P_MAX * DELTA_T           # 833.3333 kWh (并网点侧为紧侧)
26 Q_CAP = P_MAX * DELTA_T * ETA_DIS # 750.0000 kWh (电池侧为紧侧, D10 口径丙)
27 TOL = 1e-6
28 KINDS = ("b", "c", "q", "s", "E")
29
30
31 @dataclass(frozen=True)
32 class LpData:
33     """已按 ``periods`` 截断的 LP 输入 (仅在探针模式下短于 144)。"""
34
35     price: np.ndarray
36     load_kw: np.ndarray
37     pv_kw: np.ndarray
38     periods: int
39     time_labels: tuple[str, ...] = ()
40
41     @property
42     def load_energy(self) -> np.ndarray:
43         return self.load_kw * DELTA_T
44
45     @property
46     def pv_energy(self) -> np.ndarray:
47         return self.pv_kw * DELTA_T
48
49     @property
50     def net_load_energy(self) -> float:
51         return float(np.sum(self.load_energy - self.pv_energy))
52
53
54 @dataclass
55 class LpSolution:
56     data: LpData
57     status: int
58     message: str
59     iterations: int | None
60     objective: float
61     x: np.ndarray
62     equality_residual_max: float
63     bound_violation_max: float
64
65     def column(self, kind: str) -> np.ndarray:
66         if kind not in KINDS:
67             raise ValueError(f"未知变量种类: {kind}")
68         offset = KINDS.index(kind) * self.data.periods
69         return self.x[offset : offset + self.data.periods]
70
71
72 def _index(kind: str, period: int, periods: int) -> int:
73     return KINDS.index(kind) * periods + period
74
75
76 def build_lp(data: LpData) -> tuple[np.ndarray, np.ndarray, np.ndarray, list[tuple[float, float | None]]]:
77     """构造 ``min c^T x s.t. A_eq x = b_eq, l <= x <= u`` (与 formulation_v001 §3.8 逐项对应)。"""
78     periods = data.periods
79     size = len(KINDS) * periods
80     objective = np.zeros(size)
81     for period in range(periods):
82         objective[_index("b", period, periods)] = data.price[period]
83
84     rows: list[np.ndarray] = []
85     rhs: list[float] = []
86     for period in range(periods):
87         row = np.zeros(size)
88         row[_index("b", period, periods)] = 1.0
89         row[_index("q", period, periods)] = 1.0
90         row[_index("c", period, periods)] = -1.0
91         row[_index("s", period, periods)] = -1.0
92         rows.append(row)

```

```

93     rhs.append(data.load_energy[period] - data.pv_energy[period])
94 for period in range(periods):
95     row = np.zeros(size)
96     row[_index("E", period, periods)] = 1.0
97     if period > 0:
98         row[_index("E", period - 1, periods)] = -1.0
99     row[_index("c", period, periods)] = -ETA_CH
100    row[_index("q", period, periods)] = 1.0 / ETA_DIS
101    rows.append(row)
102    rhs.append(E_INIT if period == 0 else 0.0)
103
104    bounds: list[tuple[float, float | None]] = []
105    for kind in KINDS:
106        for period in range(periods):
107            if kind == "b":
108                bounds.append((0.0, None))
109            elif kind == "c":
110                bounds.append((0.0, C_CAP))
111            elif kind == "q":
112                bounds.append((0.0, Q_CAP))
113            elif kind == "s":
114                bounds.append((0.0, data.pv_energy[period]))
115            else:
116                # (R4)+(R6): 末时段固定为 E_INIT, 其余落在运行区间内; E 下标 t 表示第 t 时段末状态。
117                bounds.append((E_INIT, E_INIT) if period == periods - 1 else (E_MIN, E_MAX))
118    return objective, np.array(rows, dtype=float), np.array(rhs, dtype=float), bounds
119
120
121 def solve_lp(data: LpData, *, time_limit_seconds: float = 60.0) -> LpSolution:
122     """求解 LP, 并返回含约束残差的解。"""
123     objective, a_eq, b_eq, bounds = build_lp(data)
124     result = linprog(
125         objective,
126         A_eq=a_eq,
127         b_eq=b_eq,
128         bounds=bounds,
129         method="highs",
130         options={"time_limit": float(time_limit_seconds), "presolve": True},
131     )
132     if result.x is None:
133         x = np.full(objective.shape[0], np.nan)
134         equality_residual = float("nan")
135         bound_violation = float("nan")
136     else:
137         x = np.asarray(result.x, dtype=float)
138         equality_residual = float(np.max(np.abs(a_eq @ x - b_eq))) if a_eq.size else 0.0
139         lower = np.array([item[0] for item in bounds], dtype=float)
140         upper = np.array([item[1] if item[1] is not None else item[0] for item in bounds], dtype=float)
141         bound_violation = float(max(np.max(lower - x), np.max(x - upper)))
142     return LpSolution(
143         data=data,
144         status=int(result.status),
145         message=str(result.message),
146         iterations=int(result.nit) if result.nit is not None else None,
147         objective=float(result.fun) if result.fun is not None else float("nan"),
148         x=x,
149         equality_residual_max=equality_residual,
150         bound_violation_max=bound_violation,
151     )
152
153
154 def evaluate(solution: LpSolution, *, full_horizon: bool = True) -> dict[str, Any]:
155     """计算交付指标、结构恒等式 (I1)/(I2) 与全部硬约束检查。"""
156     data = solution.data
157     periods = data.periods
158     purchase = solution.column("b")
159     charge = solution.column("c")
160     discharge = solution.column("q")
161     spill = solution.column("s")
162     storage = solution.column("E")
163
164     checks: list[dict[str, Any]] = []
165
166     def record(name: str, value: float, threshold: float, passed: bool, note: str = "") -> None:
167         checks.append(
168             {"name": name, "value": float(value), "threshold": float(threshold), "passed": bool(passed), "note": note}
169         )
170
171     total_purchase = float(np.sum(purchase))
172     total_charge = float(np.sum(charge))
173     total_discharge = float(np.sum(discharge))

```

```

174 total_spill = float(np.sum(spill))
175 cost = float(np.sum(data.price * purchase))
176 net_load = data.net_load_energy
177
178 identity_i1 = total_discharge - ETA_ROUND_TRIP * total_charge
179 identity_i2 = total_purchase - net_load - total_spill - (1.0 - ETA_ROUND_TRIP) * total_charge
180
181 worst_bound = solution.bound_violation_max
182 endpoint_gap = abs(float(storage[-1]) - E_INIT)
183 storage_upper_gap = max(float(np.max(storage)) - E_MAX, 0.0)
184 storage_lower_gap = max(E_MIN - float(np.min(storage)), 0.0)
185 charge_gap = max(float(np.max(charge)) - C_CAP, 0.0)
186 discharge_gap = max(float(np.max(discharge)) - Q_CAP, 0.0)
187 spill_gap = max(float(np.max(spill - data.pv_energy)), 0.0)
188 purchase_gap = max(-float(np.min(purchase)), 0.0)
189 complementarity = float(np.sum(charge * discharge))
190
191 record("equality_residual_max", solution.equality_residual_max, TOL,
192       solution.equality_residual_max <= TOL, "R1/R2 等式残差")
193 record("bound_violation_max", worst_bound, TOL, worst_bound <= TOL, "变量界越界量")
194 record("periodic_endpoint_gap", endpoint_gap, TOL, endpoint_gap <= TOL, "E_144 = E_0 = 6000")
195 record("storage_upper_violation", storage_upper_gap, TOL,
196       float(np.max(storage)) <= E_MAX + TOL, "E_t <= 10800")
197 record("storage_lower_violation", storage_lower_gap, TOL,
198       float(np.min(storage)) >= E_MIN - TOL, "E_t >= 1200")
199 record("charge_cap_violation", charge_gap, TOL, float(np.max(charge)) <= C_CAP + TOL,
200       "c_t <= 833.3333 (并网侧侧紧)")
201 record("discharge_cap_violation", discharge_gap, TOL, float(np.max(discharge)) <= Q_CAP + TOL,
202       "q_dis_t <= 750.0000 (电池侧紧, D10 口径丙)")
203 record("spill_bound_violation", spill_gap, TOL, bool(np.all(spill <= data.pv_energy + TOL)),
204       "0 <= s_t <= PV_t*tdt")
205 record("purchase_nonneg_violation", purchase_gap, TOL, float(np.min(purchase)) >= -TOL, "b_t >= 0")
206 record("complementarity_sum", complementarity, TOL, complementarity <= TOL,
207       "AS07 同时充放电残差")
208 record("identity_I1_residual", identity_i1, TOL, abs(identity_i1) <= TOL, "sum q = 0.81 * sum c")
209 record("identity_I2_residual", identity_i2, TOL, abs(identity_i2) <= TOL, "sum b = N + sum s + 0.19 * sum c")
210
211 charge_power_ac = charge / DELTA_T
212 charge_power_dc = ETA_CH * charge / DELTA_T
213 discharge_power_ac = discharge / DELTA_T
214 discharge_power_dc = discharge / (ETA_DIS * DELTA_T)
215 worst_power = float(
216     max(
217         np.max(charge_power_ac),
218         np.max(charge_power_dc),
219         np.max(discharge_power_ac),
220         np.max(discharge_power_dc),
221     )
222 )
223 record("max_side_power_kw", worst_power, P_MAX, worst_power <= P_MAX + 1e-3,
224       "并网侧侧与电池侧侧换算功率均 <= 5000 kW")
225
226 if full_horizon:
227     lower_bound = float(np.min(data.price) * net_load)
228     no_storage_cost = float(np.sum(data.price * np.maximum(data.load_energy - data.pv_energy, 0.0)))
229     record("analytic_lower_bound", lower_bound, cost, cost >= lower_bound - 1e-6, "C >= min(p) * N")
230     record("analytic_upper_bound", no_storage_cost, cost, cost <= no_storage_cost + 1e-6, "无储能可行解上界")
231     record("magnitude_10k", cost, 1e3, cost >= 1e4, "D9 量纲自检: 费用须为 10^4 元量级")
232
233 block_charge = [round(float(np.sum(charge[k * 24 : (k + 1) * 24])), 6) for k in range(periods // 24)]
234 block_discharge = [round(float(np.sum(discharge[k * 24 : (k + 1) * 24])), 6) for k in range(periods // 24)]
235
236 return {
237     "objective_yuan": round(cost, 6),
238     "total_purchase_kwh": round(total_purchase, 6),
239     "total_charge_kwh": round(total_charge, 6),
240     "total_discharge_kwh": round(total_discharge, 6),
241     "total_spill_kwh": round(total_spill, 6),
242     "net_load_kwh": round(net_load, 6),
243     "max_charge_kwh": round(float(np.max(charge)), 6),
244     "max_discharge_kwh": round(float(np.max(discharge)), 6),
245     "storage_range_kwh": [round(float(np.min(storage)), 6), round(float(np.max(storage)), 6)],
246     "storage_initial_kwh": E_INIT,
247     "storage_final_kwh": round(float(storage[-1]), 6),
248     "max_side_power_kw": round(worst_power, 6),
249     "simultaneous_charge_discharge_periods": int(np.sum((charge > TOL) & (discharge > TOL))),
250     "identity_I1_residual": float(identity_i1),
251     "identity_I2_residual": float(identity_i2),
252     "block_charge_kwh": block_charge,
253     "block_discharge_kwh": block_discharge,
254     "checks": checks,

```

```

255     "checks_failed": [item["name"] for item in checks if not item["passed"]],
256     "series": {
257         "period_index": list(range(1, periods + 1)),
258         "time_label": list(data.time_labels),
259         "price_yuan_per_kwh": [float(v) for v in data.price],
260         "load_energy_kwh": [float(v) for v in data.load_energy],
261         "pv_energy_kwh": [float(v) for v in data.pv_energy],
262         "purchase_kwh": [float(v) for v in purchase],
263         "charge_kwh": [float(v) for v in charge],
264         "discharge_kwh": [float(v) for v in discharge],
265         "spill_kwh": [float(v) for v in spill],
266         "storage_kwh": [float(v) for v in storage],
267     },
268 }

```

B.3 问题一：求解主程序

```

1  # -*- coding: utf-8 -*-
2  """prob01 计算入口：读取附件 1，求解计划购电 LP，写出 result1.xlsx 与全部追踪产物。
3
4  用法（正式任务，由 computation 阶段的 resource-manager 通过 tasks.py 提交）::
5
6      <venv>/python run_prob01.py --data data/附件1.xlsx --template data/附件5/result1.xlsx \
7          --output problems/microgrid_2025/prob01/versions/assumption_v003/results/<run> \
8          --periods 144 --time-limit 60
9
10  探针模式（`--periods` 小于 144）只用于接口自检：不写 `result1.xlsx`，改写
11  `probe_result1.xlsx`，并跳过解析界与量级检查。完整计算必须由 Runner/tasks.py 创建隔离 task 执行。
12  """
13  from __future__ import annotations
14
15  import argparse
16  import hashlib
17  import os
18  import platform
19  import sys
20  from datetime import datetime, timezone
21  from pathlib import Path
22  from typing import Any
23
24  import numpy as np
25  from prob01_io import (
26      Attachment1,
27      InputValidationError,
28      fill_result_workbook,
29      inspect_template,
30      read_attachment1,
31      write_json,
32  )
33  from prob01_model import (
34      C_CAP,
35      DELTA_T,
36      E_INIT,
37      ETA_CH,
38      ETA_DIS,
39      ETA_ROUND_TRIP,
40      P_MAX,
41      Q_CAP,
42      LpData,
43      evaluate,
44      solve_lp,
45  )
46
47  PROJECT_MARKERS = ("pyproject.toml", "AGENTS.md")
48  TABLE1_SLOTS: tuple[tuple[str, int], ...] = (
49      ("10:00-10:10", 60),
50      ("12:00-12:10", 72),
51      ("14:00-14:10", 84),
52      ("16:00-16:10", 96),
53      ("18:00-18:10", 108),
54      ("20:00-20:10", 120),
55  )
56  EXIT_OK = 0
57  EXIT_SOLVER_NOT_OPTIMAL = 2
58  EXIT_INPUT_INVALID = 3
59  EXIT_HARD_CHECK_FAILED = 4
60
61
62  def project_root() -> Path:

```

```

63     current = Path(__file__).resolve()
64     for candidate in (current, *current.parents):
65         if all((candidate / marker).exists() for marker in PROJECT_MARKERS):
66             return candidate
67     raise RuntimeError("无法从脚本位置定位项目根目录")
68
69
70 def resolve(path_text: str) -> Path:
71     candidate = Path(path_text)
72     return candidate if candidate.is_absolute() else (project_root() / candidate)
73
74
75 def code_fingerprint(code_dir: Path) -> dict[str, str]:
76     digest = hashlib.sha256()
77     files: dict[str, str] = {}
78     for path in sorted(code_dir.glob("*.py")):
79         payload = path.read_bytes()
80         files[path.name] = hashlib.sha256(payload).hexdigest()
81         digest.update(path.name.encode("utf-8"))
82         digest.update(payload)
83     files["__code_dir__"] = digest.hexdigest()
84     return files
85
86
87 def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
88     parser = argparse.ArgumentParser(description="prob01 计划购电 LP 求解器 (assumption_v003 / formulation_v001) ")
89     parser.add_argument("--data", default="data/附件1.xlsx", help="附件 1 路径 (只读)")
90     parser.add_argument("--template", default="data/附件5/result1.xlsx", help="result1.xlsx 模板路径 (只读)")
91     parser.add_argument("--output", required=True, help="输出目录 (必须在 assumption_v003 版本目录内)")
92     parser.add_argument("--periods", type=int, default=144, help="时段数; <144 为探针模式")
93     parser.add_argument("--time-limit", type=float, default=60.0, help="HiGHS 求解时限 (秒)")
94     return parser.parse_args(argv)
95
96
97 def build_tables(
98     metrics: dict[str, Any],
99     attachment: Attachment1,
100     template_info: dict[str, Any],
101     periods: int,
102     probe_mode: bool,
103 ) -> dict[str, Any]:
104     """构造表 1/表 2 的行映射, 并按 AS01 校验位置口径。
105
106     AS01 的位置口径要求**两个**标签同时在位:
107     * 附件 1 第 i 行时间戳 = 区间左端点 (如 ``10:00``);
108     * ``result1.xlsx`` 第 i 行标签 = 完整区间 (如 ``10:00-10:10``)。
109     只比对其中的一个都会把「时间戳 vs 区间标签」的语义混淆 (v1 实现的缺陷)。
110     """
111     labels = attachment.time_labels
112     plan_labels = template_info["plan_labels"]
113     purchase = metrics["series"]["purchase_kwh"]
114     rows: list[dict[str, Any]] = []
115     for name, position in TABLE1_SLOTS:
116         if position <= periods:
117             attachment_label = labels[position - 1] if position <= len(labels) else None
118             template_label = plan_labels[position - 1] if position <= len(plan_labels) else None
119             slot_start = name.split("-")[0]
120             rows.append(
121                 {
122                     "slot": name,
123                     "position": position,
124                     "attachment_timestamp": attachment_label,
125                     "template_label": template_label,
126                     "label_matches": attachment_label == slot_start and template_label == name,
127                     "purchase_kwh": round(float(purchase[position - 1]), 6),
128                 }
129             )
130     return {
131         "table1": {
132             "slots": rows,
133             "all_day_energy_kwh": metrics["total_purchase_kwh"],
134             "all_day_cost_yuan": metrics["objective_yuan"],
135         },
136         "table2": {
137             "block_charge_kwh": metrics["block_charge_kwh"],
138             "block_discharge_kwh": metrics["block_discharge_kwh"],
139             "storage_0_00_kwh": E_INIT,
140             "storage_24_00_kwh": metrics["storage_final_kwh"],
141             "clock_semantics": not probe_mode,
142             "note": "0:00/24:00 为计划窗首/末状态 (AS01 左端点口径); 探针模式下仅为截断窗端点",
143     },

```

```

144     }
145
146
147 def main(argv: list[str] | None = None) -> int:
148     args = parse_args(argv)
149     data_path = resolve(args.data)
150     template_path = resolve(args.template)
151     output_dir = resolve(args.output)
152     code_dir = Path(__file__).resolve().parent
153     probe_mode = args.periods < 144
154
155     if not 2 <= args.periods <= 144:
156         print(f"[prob01] --periods 必须在 [2, 144], 收到 {args.periods}", file=sys.stderr)
157         return EXIT_INPUT_INVALID
158
159     try:
160         template_info = inspect_template(template_path)
161         attachment = read_attachment1(data_path, expected_rows=144)
162     except InputValidationError as exc:
163         print(f"[prob01] 输入校验失败: {exc}", file=sys.stderr)
164         return EXIT_INPUT_INVALID
165
166     periods = args.periods
167     lp_data = LpData(
168         price=attachment.price[:periods].copy(),
169         load_kw=attachment.load_kw[:periods].copy(),
170         pv_kw=attachment.pv_kw[:periods].copy(),
171         periods=periods,
172         time_labels=tuple(attachment.time_labels[:periods]),
173     )
174     solution = solve_lp(lp_data, time_limit_seconds=args.time_limit)
175     run_meta = {
176         "problem_id": "microgrid_2025",
177         "question_id": "prob01",
178         "assumption_version": "assumption_v003",
179         "formulation_version": "formulation_v001",
180         "stage": "computation",
181         "probe_mode": probe_mode,
182         "periods": periods,
183         "created_at": datetime.now(timezone.utc).isoformat(),
184         "task_id": os.environ.get("AUTOMM_TASK_ID"),
185         "output_directory": str(output_dir),
186         "device": "cpu",
187         "gpu_required": False,
188         "seed": None,
189         "deterministic": True,
190         "solver": "scipy.optimize.linprog(method='highs')",
191         "time_limit_seconds": args.time_limit,
192         "input": {
193             "attachment1": str(data_path),
194             "attachment1_md5": attachment.md5,
195             "attachment1_rows": attachment.periods,
196             "template": str(template_path),
197             "template_plan_periods": template_info["plan_periods"],
198         },
199         "parameters": {
200             "delta_t_h": DELTA_T,
201             "eta_ch": ETA_CH,
202             "eta_dis": ETA_DIS,
203             "eta_round_trip": ETA_ROUND_TRIP,
204             "e_init_kwh": E_INIT,
205             "p_max_kw": P_MAX,
206             "c_cap_kwh": C_CAP,
207             "q_dis_cap_kwh": Q_CAP,
208         },
209         "code_sha256": code_fingerprint(code_dir),
210         "environment": {
211             "python": sys.version.split()[0],
212             "numpy": np.__version__,
213             "platform": platform.platform(),
214         },
215     }
216
217     if solution.status != 0:
218         status_payload = {
219             "status": solution.status,
220             "message": solution.message,
221             "feasible_incumbent": False,
222             "objective_yuan": None,
223             "iterations": solution.iterations,
224         }
225         write_json(output_dir / "solver_status.json", status_payload)

```

```

225     write_json(output_dir / "run_manifest.json", {**run_meta, "outcome": "solver_not_optimal"})
226     print(f"[prob01] 求解未达最优: status={solution.status} {solution.message}", file=sys.stderr)
227     return EXIT_SOLVER_NOT_OPTIMAL
228
229 metrics = evaluate(solution, full_horizon=not probe_mode)
230 tables = build_tables(metrics, attachment, template_info, periods, probe_mode)
231 label_checks = [item["label_matches"] for item in tables["table1"]["slots"]]
232 if not all(label_checks):
233     bad = [item for item in tables["table1"]["slots"] if not item["label_matches"]]
234     print(f"[prob01] 模板/附件时段标签与 AS01 位置口径不一致: {bad}", file=sys.stderr)
235     return EXIT_HARD_CHECK_FAILED
236
237 workbook_name = "probe_result1.xlsx" if probe_mode else "result1.xlsx"
238 workbook_info = fill_result_workbook(
239     template=template_path,
240     dest=output_dir / workbook_name,
241     purchase_kwh=metrics["series"]["purchase_kwh"],
242     charge_kwh=metrics["series"]["charge_kwh"],
243     discharge_kwh=metrics["series"]["discharge_kwh"],
244     e_init=E_INIT,
245     e_final=metrics["storage_final_kwh"],
246     periods=periods,
247 )
248
249 failed = list(metrics["checks_failed"])
250 hard_failed = list(failed)
251 write_json(output_dir / "solver_status.json", {
252     "status": solution.status,
253     "message": solution.message,
254     "solver": "highs",
255     "iterations": solution.iterations,
256     "objective_yuan": metrics["objective_yuan"],
257     "feasible_incumbent": True,
258     "mip_gap": None,
259     "device": "cpu",
260     "gpu_required": False,
261     "seed": None,
262     "equality_residual_max": solution.equality_residual_max,
263     "bound_violation_max": solution.bound_violation_max,
264     "deterministic": True,
265 })
266 write_json(output_dir / "solution.json", {
267     "objective_yuan": metrics["objective_yuan"],
268     "totals": {key: metrics[key] for key in (
269         "total_purchase_kwh",
270         "total_charge_kwh",
271         "total_discharge_kwh",
272         "total_spill_kwh",
273         "net_load_kwh",
274         "max_charge_kwh",
275         "max_discharge_kwh",
276         "max_side_power_kw",
277         "storage_initial_kwh",
278         "storage_final_kwh",
279     )},
280     "storage_range_kwh": metrics["storage_range_kwh"],
281     "simultaneous_charge_discharge_periods": metrics["simultaneous_charge_discharge_periods"],
282     "identities": {
283         "I1_residual": metrics["identity_I1_residual"],
284         "I2_residual": metrics["identity_I2_residual"],
285     },
286     "checks": metrics["checks"],
287     "checks_failed": failed,
288     "series": metrics["series"],
289 })
290 write_json(output_dir / "tables.json", tables)
291 write_json(output_dir / "run_manifest.json", {
292     **run_meta,
293     "outcome": "hard_check_failed" if hard_failed else "completed",
294     "workbook": workbook_name,
295     "workbook_info": workbook_info,
296     "checks_failed": failed,
297     "objective_yuan": metrics["objective_yuan"],
298     "total_purchase_kwh": metrics["total_purchase_kwh"],
299     "total_spill_kwh": metrics["total_spill_kwh"],
300 })
301
302 print(
303     f"[prob01] {'探针' if probe_mode else '正式'}运行完成: periods={periods} "
304     f"C*={metrics['objective_yuan']:.4f} 元, sum_b={metrics['total_purchase_kwh']:.4f} kWh, "
305     f"sum_spill={metrics['total_spill_kwh']:.4f} kWh, 失败检查={failed}"

```

```

306     )
307     if hard_failed:
308         print(f"[prob01] 硬检查未通过: {hard_failed}", file=sys.stderr)
309         return EXIT_HARD_CHECK_FAILED
310     return EXIT_OK
311
312
313 if __name__ == "__main__":
314     raise SystemExit(main())

```

B.4 问题二：输入输出模块

```

1  # -*- coding: utf-8 -*-
2  """prob02 输入读取与 ``result2.xlsx`` 填报。
3
4  只读 ``data/附件1.xlsx``、``data/附件2.xlsx`` 与模板 ``data/附件5/result2.xlsx``；不修改任何原始数据。
5  口径来源: prob02 assumption_v001 (AS01/AS02/AS12 + 团队勘误 R1/R3/R4) 与 formulation_v001 S7。
6
7  纪律 (AS02 / B4): 附件 1 **只使用电价列**；其负载/光伏预测列、附件 3、附件 4 均不进入模型。
8  本模块读取附件 1 时只为「表头结构校验」读取第 3/4 列表头文本，**不把这两列的数值传入模型**。
9  """
10 from __future__ import annotations
11
12 import hashlib
13 import json
14 from dataclasses import dataclass
15 from datetime import datetime
16 from pathlib import Path
17 from typing import Any, Sequence
18
19 import numpy as np
20 import openpyxl
21 from openpyxl.utils import get_column_letter
22
23 EXPECTED_HEADER_A1 = ("时间", "电价", "小区负载", "光伏发电预测功率")
24 LOAD_SHEET = "小区负载"
25 PV_SHEET = "光伏发电实际功率"
26 PLAN_SHEET = "计划购电量"
27 STORAGE_SHEET = "充放电量"
28 EMERGENCY_SHEET = "紧急购电量"
29 PLAN_HEADER_FIRST = "日期\\时间"
30 PLAN_HEADER_ENERGY = "全天购电量"
31 PLAN_HEADER_COST = "全天购电费"
32 STORAGE_HEADER = ("日期", "时间段", "充电量", "放电量", "时刻", "储电量")
33 EMERGENCY_HEADER = ("日期", "购电时间段", "购电量")
34 BLOCK_ROWS = 24
35 BLOCKS_PER_DAY = 6
36 PERIODS_PER_DAY = 144
37 ZERO_INTERVAL_TEXT = "---"
38 A2_DATE_HEADER = "日期\\时间"
39
40
41 class InputValidationError(RuntimeError):
42     """输入结构与 accepted 假设不一致时严格失败。"""
43
44
45 @dataclass(frozen=True)
46 class Attachment1:
47     """附件 1 的 144 个时段电价 (元/kWh) 与时间标签 (只含电价列)。"""
48
49     price: np.ndarray
50     time_labels: list[str]
51     header: list[str]
52     md5: str
53
54     @property
55     def periods(self) -> int:
56         return int(self.price.shape[0])
57
58
59 @dataclass(frozen=True)
60 class Attachment2:
61     """附件 2 的逐日实际负载/光伏功率 (365 天 × 144 个 10 分钟点, 单位 kW)。"""
62
63     load_kw: np.ndarray
64     pv_kw: np.ndarray
65     dates: list[str]
66     time_labels: list[str]
67     load_md5: str

```

```

68     pv_md5: str
69     md5: str
70
71     @property
72     def days(self) -> int:
73         return int(self.load_kw.shape[0])
74
75     @property
76     def periods_per_day(self) -> int:
77         return int(self.load_kw.shape[1])
78
79
80 def md5_of(path: Path) -> str:
81     digest = hashlib.md5()
82     with path.open("rb") as handle:
83         for chunk in iter(lambda: handle.read(1 << 20), b''):
84             digest.update(chunk)
85     return digest.hexdigest()
86
87
88 def _clock_label(value: Any) -> str:
89     """把附件时间戳格式化为 ``H:MM`` (datetime/time 走 strftime, 字符串原样返回)。"""
90     if value is None:
91         return ""
92     if hasattr(value, "strftime"):
93         return value.strftime("%H:%M")
94     return str(value).strip()
95
96
97 def _date_label(value: Any) -> str:
98     if value is None:
99         return ""
100     if hasattr(value, "strftime"):
101         return value.strftime("%Y-%m-%d")
102     return str(value).strip()
103
104
105 def read_attachment1(path: Path, expected_rows: int = PERIODS_PER_DAY) -> Attachment1:
106     """读取附件 1 第 1 个工作表; 只保留电价列与时间标签。"""
107     if not path.is_file():
108         raise InputValidationError(f"附件 1 不存在: {path}")
109     workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
110     try:
111         sheet = workbook.worksheets[0]
112         rows = [row for row in sheet.iter_rows(min_row=1, max_col=4, values_only=True)]
113     finally:
114         workbook.close()
115     if len(rows) < 2:
116         raise InputValidationError("附件 1 没有数据行")
117     header = [str(item).strip() for item in rows[0]]
118     if tuple(header) != EXPECTED_HEADER_A1:
119         raise InputValidationError(f"附件 1 表头不匹配: {header} != {list(EXPECTED_HEADER_A1)}")
120     body = rows[1:]
121     if len(body) != expected_rows:
122         raise InputValidationError(f"附件 1 数据行数 {len(body)} != 期望 {expected_rows}")
123     labels = [_clock_label(row[0]) for row in body]
124     try:
125         price = np.array([float(row[1]) for row in body], dtype=float)
126     except (TypeError, ValueError) as exc:
127         raise InputValidationError(f"附件 1 电价列存在非数值单元格: {exc}") from exc
128     if not np.all(np.isfinite(price)):
129         raise InputValidationError("附件 1 电价列含 NaN/Inf")
130     if np.any(price <= 0):
131         raise InputValidationError("附件 1 存在非正电价: 定理 T1 与 LP 对偶前提被破坏")
132     return Attachment1(price=price, time_labels=labels, header=header, md5=md5_of(path))
133
134
135 def _read_power_sheet(
136     sheet: Any, *, name: str, expected_days: int, expected_periods: int
137 ) -> tuple[list[str], list[str], np.ndarray]:
138     rows = [row for row in sheet.iter_rows(min_row=1, max_col=expected_periods + 1, values_only=True)]
139     if len(rows) != expected_days + 1:
140         raise InputValidationError(f"附件 2 [{name}] 行数 {len(rows)} != 表头 + {expected_days}")
141     header = [" " if item is None else str(item) for item in rows[0]]
142     if header[0].strip() != A2_DATE_HEADER:
143         raise InputValidationError(f"附件 2 [{name}] 首列表头不是 {A2_DATE_HEADER}: {header[0]!r}")
144     if len(header) != expected_periods + 1:
145         raise InputValidationError(f"附件 2 [{name}] 列数 {len(header)} != {expected_periods + 1}")
146     body = rows[1:]
147     dates = [_date_label(row[0]) for row in body]
148     try:

```

```

149     values = np.array([float(item) for item in row[1:] for row in body], dtype=float)
150 except (TypeError, ValueError) as exc:
151     raise InputValidationError(f"附件 2 [{name}] 存在非数值单元格: {exc}") from exc
152 if values.shape != (expected_days, expected_periods):
153     raise InputValidationError(f"附件 2 [{name}] 矩阵形状 {values.shape} 不符")
154 if not np.all(np.isfinite(values)):
155     raise InputValidationError(f"附件 2 [{name}] 含 NaN/Inf")
156 if np.any(values < 0):
157     raise InputValidationError(f"附件 2 [{name}] 存在负功率")
158 return header, dates, values
159
160
161 def read_attachment2(
162     path: Path, expected_days: int = 365, expected_periods: int = PERIODS_PER_DAY
163 ) -> Attachment2:
164     """读取附件 2 的『小区负载』与『光伏发电实际功率』（均为实际值）。"""
165     if not path.is_file():
166         raise InputValidationError(f"附件 2 不存在: {path}")
167     workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
168     try:
169         titles = list(workbook.sheetnames)
170         for name in (LOAD_SHEET, PV_SHEET):
171             if name not in titles:
172                 raise InputValidationError(f"附件 2 缺少工作表 [{name}]: {titles}")
173         load_header, load_dates, load = _read_power_sheet(
174             workbook[LOAD_SHEET], name=LOAD_SHEET, expected_days=expected_days, expected_periods=expected_periods
175         )
176         pv_header, pv_dates, pv = _read_power_sheet(
177             workbook[PV_SHEET], name=PV_SHEET, expected_days=expected_days, expected_periods=expected_periods
178         )
179     finally:
180         workbook.close()
181     if load_dates != pv_dates:
182         raise InputValidationError("附件 2 两个工作表的日期序列不一致")
183     _validate_dates(load_dates)
184     if load_header[1:] != pv_header[1:]:
185         raise InputValidationError("附件 2 两个工作表的时间标签不一致")
186     return Attachment2(
187         load_kw=load,
188         pv_kw=pv,
189         dates=load_dates,
190         time_labels=[str(item) for item in load_header[1:]],
191         load_md5=md5_of(path),
192         pv_md5=md5_of(path),
193         md5=md5_of(path),
194     )
195
196
197 def _validate_dates(dates: Sequence[str]) -> None:
198     try:
199         parsed = [datetime.strptime(item, "%Y-%m-%d").date() for item in dates]
200     except ValueError as exc:
201         raise InputValidationError(f"附件 2 日期列无法解析: {exc}") from exc
202     for index in range(1, len(parsed)):
203         if (parsed[index] - parsed[index - 1]).days != 1:
204             raise InputValidationError(f"附件 2 日期不连续: {dates[index - 1]} -> {dates[index]}")
205     if parsed[0].isoformat() != "2025-01-01":
206         raise InputValidationError(f"附件 2 首日不是 2025-01-01: {dates[0]}")
207
208
209 def inspect_template(path: Path) -> dict[str, Any]:
210     """读取 ``result2.xlsx`` 模板的逐行标签/表头, 用于 AS01/AS12 对齐校验 (不写入)。"""
211     if not path.is_file():
212         raise InputValidationError(f"模板不存在: {path}")
213     workbook = openpyxl.load_workbook(path, data_only=True)
214     try:
215         titles = list(workbook.sheetnames)
216         for name in (PLAN_SHEET, STORAGE_SHEET, EMERGENCY_SHEET):
217             if name not in titles:
218                 raise InputValidationError(f"模板缺少工作表 [{name}]: {titles}")
219         plan = workbook[PLAN_SHEET]
220         plan_header = [cell.value for cell in plan[1]]
221         if len(plan_header) < PERIODS_PER_DAY + 3:
222             raise InputValidationError(f"模板 [{PLAN_SHEET}] 列数 {len(plan_header)} 少于 147")
223         if str(plan_header[0]).strip() != PLAN_HEADER_FIRST:
224             raise InputValidationError(f"模板 [{PLAN_SHEET}] 首列表头异常: {plan_header[0]!r}")
225         if str(plan_header[PERIODS_PER_DAY + 1]).strip() != PLAN_HEADER_ENERGY:
226             raise InputValidationError(f"模板 [{PLAN_SHEET}] 第 146 列不是 [{PLAN_HEADER_ENERGY}]")
227         if str(plan_header[PERIODS_PER_DAY + 2]).strip() != PLAN_HEADER_COST:
228             raise InputValidationError(f"模板 [{PLAN_SHEET}] 第 147 列不是 [{PLAN_HEADER_COST}]")
229         plan_labels = [str(item) for item in plan_header[1 : PERIODS_PER_DAY + 1]]

```

```

230     plan_dates = [_date_label(plan.cell(row=row, column=1).value) for row in range(2, plan.max_row + 1)]
231
232     storage = workbook[STORAGE_SHEET]
233     storage_header = [" " if cell.value is None else str(cell.value) for cell in storage[1]]
234     if tuple(storage_header[: len(STORAGE_HEADER)]) != STORAGE_HEADER:
235         raise InputValidationError(f"模板 [{STORAGE_SHEET}] 表头异常: {storage_header}")
236     storage_blocks = [str(storage.cell(row=2 + k, column=2).value) for k in range(BLOCKS_PER_DAY)]
237     storage_max_row = int(storage.max_row)
238
239     emergency = workbook[EMERGENCY_SHEET]
240     emergency_header = [" " if cell.value is None else str(cell.value) for cell in emergency[1]]
241     if tuple(emergency_header[: len(EMERGENCY_HEADER)]) != EMERGENCY_HEADER:
242         raise InputValidationError(f"模板 [{EMERGENCY_SHEET}] 表头异常: {emergency_header}")
243     emergency_max_row = emergency.max_row
244 finally:
245     workbook.close()
246 return {
247     "plan_header_len": len(plan_header),
248     "plan_labels": plan_labels,
249     "plan_dates": plan_dates,
250     "plan_rows": len(plan_dates),
251     "storage_header": storage_header,
252     "storage_blocks": storage_blocks,
253     "storage_max_row": storage_max_row,
254     "emergency_header": emergency_header,
255     "emergency_max_row": emergency_max_row,
256     "md5": md5_of(path),
257 }
258
259
260 def _storage_max_row(path: Path) -> int:
261     workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
262     try:
263         return int(workbook[STORAGE_SHEET].max_row)
264     finally:
265         workbook.close()
266
267
268 def _assign(sheet: Any, *, row: int, column: int, value: Any) -> None:
269     """显式写单元格, ``value=None`` 也**真正清空**。
270
271     缺陷登记 (sanity act-ca831716e72246eb): ``Worksheet.cell(row, column, value=None)`` 在
272     ``value is None`` 时**不赋值**，是静默 no-op，无法清空模板残留值（曾导致交付的
273     ``result2.xlsx`` 残留 ``充放电!A15=2025-12-31``、``!E16='24:00'``）。本函数统一改为
274     直接设置 ``cell.value``，禁止再用 ``cell(..., value=None)`` 表达清空。
275     """
276     sheet.cell(row=row, column=column).value = value
277
278
279 def _clear_rows(sheet: Any, *, first_row: int, last_row: int, last_column: int) -> None:
280     for row in range(first_row, last_row + 1):
281         for column in range(1, last_column + 1):
282             _assign(sheet, row=row, column=column, value=None)
283
284
285 def _residual_report(*, workbook: Any, days: int, emergency_rows: int) -> list[str]:
286     """复核填报后的工作簿：枚举**应空却仍有值**的单元格（防静默 no-op 复发）。
287
288     口径同 AS12 + 团队勘误 R3/R4: ``充放电`` 日期只写块首行、``时刻``/``储电量`` 只写每日前两行；
289     ``计划购电量``/``紧急购电量`` 超出应有行数后不得留值；全簿不得残留模板 ```` 压缩行。
290     """
291     residuals: list[str] = []
292     plan = workbook[PLAN_SHEET]
293     for row in range(days + 2, int(plan.max_row) + 1):
294         for column in range(1, PERIODS_PER_DAY + 4):
295             if plan.cell(row=row, column=column).value is not None:
296                 residuals.append(f"计划购电量!{get_column_letter(column)}{row}")
297             break
298     storage = workbook[STORAGE_SHEET]
299     for index in range(days):
300         for block in range(BLOCKS_PER_DAY):
301             row = 2 + index * BLOCKS_PER_DAY + block
302             if block != 0 and storage.cell(row=row, column=1).value is not None:
303                 residuals.append(f"充放电!A{row} (日期只写块首行)")
304             if block >= 2 and storage.cell(row=row, column=5).value is not None:
305                 residuals.append(f"充放电!E{row} (时刻只写每日前两行)")
306             if block >= 2 and storage.cell(row=row, column=6).value is not None:
307                 residuals.append(f"充放电!F{row} (储电量只写每日前两行)")
308     expected_storage_rows = 1 + days * BLOCKS_PER_DAY
309     for row in range(expected_storage_rows + 1, int(storage.max_row) + 1):
310         for column in range(1, len(STORAGE_HEADER) + 1):

```

```

311         if storage.cell(row=row, column=column).value is not None:
312             residuals.append(f"充放电!{get_column_letter(column)}{row} (超出行数)")
313             break
314     emergency = workbook[EMERGENCY_SHEET]
315     for row in range(1 + emergency_rows + 1, int(emergency.max_row) + 1):
316         for column in range(1, len(EMERGENCY_HEADER) + 1):
317             if emergency.cell(row=row, column=column).value is not None:
318                 residuals.append(f"紧急购电!{get_column_letter(column)}{row} (超出行数)")
319                 break
320     for name in workbook.sheetnames:
321         for row in workbook[name].iter_rows(values_only=True):
322             for value in row:
323                 if isinstance(value, str) and " " in value:
324                     residuals.append(f"{name}!不得残留模板 压缩行")
325     total = len(residuals)
326     if total > 200:
327         residuals = residuals[:200] + [f"... (共 {total} 处, 已截断)"]
328     return residuals
329
330
331 def fill_result2_workbook(
332     *,
333     template: Path,
334     dest: Path,
335     dates: Sequence[str],
336     purchase_kwh: np.ndarray,
337     charge_kwh: np.ndarray,
338     discharge_kwh: np.ndarray,
339     state_start_kwh: np.ndarray,
340     state_end_kwh: np.ndarray,
341     day_energy_kwh: np.ndarray,
342     day_cost_yuan: np.ndarray,
343     emergency_intervals: Sequence[Sequence[tuple[str, float]]],
344 ) -> dict[str, Any]:
345     """把交付期解填入 ``result2.xlsx`` 三个工作表 (保留模板标签与样式, 只写数值单元格)。
346
347     口径 (AS12 + 勘误 R3/R4):
348     * ``计划购电量``: 表头 + 334 天 × 144 个 10 分钟购电量 + ``全天购电量`` + ``全天购电费``;
349     * ``充放电``: 334 天 × 6 个 4 小时块, 日期只写在该块首行; ``时刻``/``储电量`` 写在每日前两行
350       (第 1 行 ``00:00:00`` 载 0:00 储电量、第 2 行 ``24:00`` 载 24:00 储电量);
351     * ``紧急购电量``: 每日期至少 1 行; ``J_d = 0`` 的日期写 ``时间段 = 一``、``购电量 = 0``。
352
353     所有写入都经 :func:`_assign` (含 ``None`` 也真正清空), 并在保存前用 :func:`_residual_report`
354     复核「应空却仍有值」的单元格; 复核结果返回值的 ``format_residuals`` 交给调用方做硬门禁。
355     """
356     days = len(dates)
357     purchase = np.asarray(purchase_kwh, dtype=float)
358     charge = np.asarray(charge_kwh, dtype=float)
359     discharge = np.asarray(discharge_kwh, dtype=float)
360     if purchase.shape != (days, PERIODS_PER_DAY):
361         raise InputValidationError(f"purchase 形状 {purchase.shape} != ({days}, {PERIODS_PER_DAY})")
362     if charge.shape != purchase.shape or discharge.shape != purchase.shape:
363         raise InputValidationError("charge/discharge 形状与 purchase 不一致")
364     if len(emergency_intervals) != days:
365         raise InputValidationError("emergency_intervals 天数与 dates 不一致")
366     if not template.is_file():
367         raise InputValidationError(f"模板不存在: {template}")
368
369     workbook = openpyxl.load_workbook(template)
370     try:
371         titles = list(workbook.sheetnames)
372         for name in (PLAN_SHEET, STORAGE_SHEET, EMERGENCY_SHEET):
373             if name not in titles:
374                 raise InputValidationError(f"模板缺少工作表 [{name}]: {titles}")
375
376         plan = workbook[PLAN_SHEET]
377         if plan.max_row < days + 1:
378             raise InputValidationError(f"模板 [{PLAN_SHEET}] 只有 {plan.max_row - 1} 行, 少于 {days} 天")
379         for index, date_text in enumerate(dates):
380             row = 2 + index
381             actual = _date_label(plan.cell(row=row, column=1).value)
382             if actual != date_text:
383                 raise InputValidationError(f"模板 [{PLAN_SHEET}] 第 {row} 行日期 {actual!r} != {date_text!r}")
384             for period in range(PERIODS_PER_DAY):
385                 _assign(plan, row=row, column=2 + period, value=float(purchase[index, period]))
386                 _assign(plan, row=row, column=PERIODS_PER_DAY + 2, value=float(day_energy_kwh[index]))
387                 _assign(plan, row=row, column=PERIODS_PER_DAY + 3, value=float(day_cost_yuan[index]))
388             if plan.max_row > days + 1:
389                 _clear_rows(plan, first_row=days + 2, last_row=plan.max_row, last_column=PERIODS_PER_DAY + 3)
390
391         storage = workbook[STORAGE_SHEET]

```

```

392     storage_blocks = [str(storage.cell(row=2 + k, column=2).value) for k in range(BLOCKS_PER_DAY)]
393     blocks_filled = 0
394     for index, date_text in enumerate(dates):
395         date_value = datetime.strptime(date_text, "%Y-%m-%d")
396         for block in range(BLOCKS_PER_DAY):
397             row = 2 + index * BLOCKS_PER_DAY + block
398             start = block * BLOCK_ROWS
399             stop = start + BLOCK_ROWS
400             _assign(storage, row=row, column=1, value=date_value if block == 0 else None)
401             _assign(storage, row=row, column=2, value=storage_blocks[block])
402             _assign(storage, row=row, column=3, value=float(np.sum(charge[index, start:stop])))
403             _assign(storage, row=row, column=4, value=float(np.sum(discharge[index, start:stop])))
404             _assign(
405                 storage,
406                 row=row,
407                 column=5,
408                 value="00:00:00" if block == 0 else ("24:00" if block == 1 else None),
409             )
410             _assign(
411                 storage,
412                 row=row,
413                 column=6,
414                 value=float(state_start_kwh[index]) if block == 0 else (
415                     float(state_end_kwh[index]) if block == 1 else None
416                 ),
417             )
418             blocks_filled += 1
419     expected_storage_rows = 1 + days * BLOCKS_PER_DAY
420     if storage.max_row > expected_storage_rows:
421         _clear_rows(storage, first_row=expected_storage_rows + 1, last_row=storage.max_row, last_column=6)
422
423     emergency = workbook[EMERGENCY_SHEET]
424     row = 2
425     interval_rows = 0
426     for index, date_text in enumerate(dates):
427         date_value = datetime.strptime(date_text, "%Y-%m-%d")
428         intervals = list(emergency_intervals[index])
429         if not intervals:
430             intervals = [(ZERO_INTERVAL_TEXT, 0.0)]
431         for order, (slot, kwh) in enumerate(intervals):
432             _assign(emergency, row=row, column=1, value=date_value if order == 0 else None)
433             _assign(emergency, row=row, column=2, value=slot)
434             _assign(emergency, row=row, column=3, value=float(kwh))
435             row += 1
436             interval_rows += 1
437     expected_emergency_rows = row - 1
438     if emergency.max_row > expected_emergency_rows:
439         _clear_rows(emergency, first_row=expected_emergency_rows + 1, last_row=emergency.max_row, last_column=3)
440
441     dest.parent.mkdir(parents=True, exist_ok=True)
442     residuals = _residual_report(workbook=workbook, days=days, emergency_rows=interval_rows)
443     workbook.save(dest)
444 finally:
445     workbook.close()
446     return {
447         "plan_rows_filled": days,
448         "plan_periods_per_row": PERIODS_PER_DAY,
449         "storage_blocks_filled": blocks_filled,
450         "emergency_rows_filled": interval_rows,
451         "storage_blocks": storage_blocks,
452         "format_residuals": residuals,
453     }
454
455
456 def write_json(path: Path, payload: Any) -> None:
457     """写 JSON; 显式拒绝 NaN/Inf, 避免把非有限值伪装成合法结果。"""
458     path.parent.mkdir(parents=True, exist_ok=True)
459     text = json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False)
460     path.write_text(text + "\n", encoding="utf-8")
461
462
463 def delivery_dates(all_dates: Sequence[str], start_day_index: int) -> list[str]:
464     """交付期日期列表 (默认 2025-02-01 ... 2025-12-31, 334 天)。"""
465     return [str(item) for item in all_dates[start_day_index:]]
466
467
468 def day_offset(dates: Sequence[str], target: str) -> int:
469     """在附件 2 的日期序列中定位目标的 0 基索引; 找不到时严格失败。"""
470     for index, item in enumerate(dates):
471         if item == target:
472             return index

```

```

473     raise InputValidationError(f"附件 2 日期序列中找不到 {target}")
474
475
476 __all__ = [
477     "Attachment1",
478     "Attachment2",
479     "InputValidationError",
480     "day_offset",
481     "delivery_dates",
482     "fill_result2_workbook",
483     "inspect_template",
484     "md5_of",
485     "read_attachment1",
486     "read_attachment2",
487     "write_json",
488 ]

```

B.5 问题二：模型构造模块

```

1  # -*- coding: utf-8 -*-
2  """prob02 计划购电 LP (M1 主口径) 的建模、求解与结构自检。
3
4  模型 (P2) 严格对应 ``formulation_v001`` / ``assumption_v001``, 并按团队裁定 B0 只实现**主口径 M1**:
5  ``365 天 * 144 时段 = 52,560`` 时段的单一确定性 LP (连续变量 315,360、等式约束 105,120、非零元 473,039)。
6
7  口径 (不得返问更改):
8  ``Delta = 1/6 h``, ``eta_ch = eta_dis = 0.9``, ``c = 833.3333``, ``q_dis = 750.0000`` (D10 口径丙 / 勘误 E1)、
9  ``E = [1200, 10800]``, ``E_0 = 6000``, ``E_T`` 自由 (AS04/D2 备选 A)、``eta_em = 5``,
10 目标 ``min E (p * b + 5 * p * q_em)`` (元, **不乘 Delta**, 团队裁定 D9)。
11 求解器: CPU HiGHS (``scipy.optimize.linprog``); 本问不使用 GPU、不依赖随机源 (``seed = null``)。
12
13 模型间对照 (M2~M7) 按团队裁定 B1/B5 由 ``ablation`` 阶段在 ``ablations/code/`` 下另建, 本模块只提供 M1。
14 """
15 from __future__ import annotations
16
17 from dataclasses import dataclass
18 from typing import Any
19
20 import numpy as np
21 from scipy.optimize import linprog
22 from scipy.sparse import coo_matrix, csr_matrix
23
24 DELTA_T = 1.0 / 6.0
25 ETA_CH = 0.9
26 ETA_DIS = 0.9
27 ETA_ROUND_TRIP = ETA_CH * ETA_DIS
28 E_INIT = 6000.0
29 E_MIN = 1200.0
30 E_MAX = 10800.0
31 E_CAP_MAX = 12000.0
32 P_MAX = 5000.0
33 C_CAP = P_MAX * DELTA_T # 833.3333 kWh (并网点侧为紧侧)
34 Q_CAP = P_MAX * DELTA_T * ETA_DIS # 750.0000 kWh (电池侧为紧侧, D10 口径丙)
35 ALPHA_EM = 5.0
36 PERIODS_PER_DAY = 144
37 DAYS_FULL = 365
38 DAILY_DELIVERY_START = 31 # 0 基日索引: 2025-02-01 (1 月为预热期)
39 TOL = 1e-6
40 KINDS = ("b", "q_em", "c", "q_dis", "s", "E")
41
42 # 解析界 (formulation_v001 §3.10; 全年)
43 LOWER_BOUND_FULL_YUAN = 7_525_691.1311
44 UPPER_NO_STORAGE_FULL_YUAN = 18_298_592.3663
45 UPPER_NO_STORAGE_DELIVERY_YUAN = 16_407_319.6320
46 DELIVERY_COST_SCALE_FLOOR_YUAN = 1.0e7 # 交付期总费用的量级自检下限 (10^7 元)
47 SPEC_DATES: tuple[tuple[str, int], ...] = (
48     ("2025-03-20", 78),
49     ("2025-06-21", 171),
50     ("2025-09-23", 265),
51     ("2025-12-21", 354),
52 )
53 TABLE1_SLOTS: tuple[tuple[str, int], ...] = (
54     ("10:00-10:10", 60),
55     ("12:00-12:10", 72),
56     ("14:00-14:10", 84),
57     ("16:00-16:10", 96),
58     ("18:00-18:10", 108),
59     ("20:00-20:10", 120),
60 )

```

```

61 BLOCK_ROWS = 24
62 BLOCKS_PER_DAY = 6
63
64
65 @dataclass(frozen=True)
66 class LpData:
67     """按 ``days`` 展开的 LP 输入（探针模式可短于 365 天）。"""
68
69     price: np.ndarray          # (T,)，附件 1 电价逐日重复
70     load_energy: np.ndarray    # (T,)，负载功率 × Δt
71     pv_energy: np.ndarray      # (T,)，光伏功率 × Δt
72     days: int
73
74     @property
75     def periods(self) -> int:
76         return int(self.days * PERIODS_PER_DAY)
77
78     @property
79     def net_load_energy(self) -> float:
80         return float(np.sum(self.load_energy - self.pv_energy))
81
82     @property
83     def total_load_energy(self) -> float:
84         return float(np.sum(self.load_energy))
85
86     @property
87     def total_pv_energy(self) -> float:
88         return float(np.sum(self.pv_energy))
89
90
91 @dataclass
92 class LpSolution:
93     data: LpData
94     status: int
95     message: str
96     iterations: int | None
97     objective: float
98     x: np.ndarray
99     equality_residual_max: float
100     bound_violation_max: float
101
102     def column(self, kind: str) -> np.ndarray:
103         if kind not in KINDS:
104             raise ValueError(f"未知变量种类: {kind}")
105         periods = self.data.periods
106         offset = KINDS.index(kind) * periods
107         return self.x[offset : offset + periods]
108
109     def daily(self, kind: str) -> np.ndarray:
110         """把某一类变量按 ``(days, 144)`` 展开。"""
111         return self.column(kind).reshape(self.data.days, PERIODS_PER_DAY)
112
113
114 def _index(kind: str, period: int, periods: int) -> int:
115     return KINDS.index(kind) * periods + period
116
117
118 def build_lp(data: LpData) -> tuple[np.ndarray, csr_matrix, np.ndarray, np.ndarray, np.ndarray]:
119     """构造 ``min c^T x s.t. A_eq x = b_eq, l <= x <= u``（与 formulation_v001 §3.9 逐项对应）。
120
121     返回 ``(objective, A_eq, b_eq, lower, upper)``；``A_eq`` 为 CSR 稀疏矩阵
122     （``2T × 6T``，非零元 ``9T - 1``）。
123
124     """
125     periods = data.periods
126     size = len(KINDS) * periods
127     if data.load_energy.shape != (periods,) or data.pv_energy.shape != (periods,):
128         raise ValueError("load/pv 长度与 days × 144 不一致")
129
130     objective = np.zeros(size)
131     objective[0:periods] = data.price
132     objective[periods : 2 * periods] = ALPHA_EM * data.price
133
134     lower = np.zeros(size)
135     upper = np.empty(size)
136     lower[0:periods] = 0.0                                # b >= 0（无购电上限，AS10）
137     lower[periods : 2 * periods] = 0.0                    # q_em >= 0（保留变量，AS07）
138     upper[periods : 2 * periods] = np.inf
139     lower[2 * periods : 3 * periods] = 0.0                # 0 <= c <= 833.3333
140     upper[2 * periods : 3 * periods] = C_CAP
141     lower[3 * periods : 4 * periods] = 0.0                # 0 <= q_dis <= 750.0000

```

```

142 upper[3 * periods : 4 * periods] = Q_CAP
143 lower[4 * periods : 5 * periods] = 0.0 # 0 <= s <= PV * Δt
144 upper[4 * periods : 5 * periods] = data.pv_energy
145 lower[5 * periods : 6 * periods] = E_MIN # 1200 <= E <= 10800
146 upper[5 * periods : 6 * periods] = E_MAX
147
148 tau = np.arange(periods, dtype=np.int64)
149 rows: list[np.ndarray] = []
150 cols: list[np.ndarray] = []
151 vals: list[np.ndarray] = []
152 # (R1) 逐时段电量平衡:  $b + q_{em} + q_{dis} - c - s = L \cdot \Delta t - PV \cdot \Delta t$ 
153 rows.append(tau)
154 cols.append(_index("b", 0, periods) + tau)
155 vals.append(np.ones(periods))
156 rows.append(tau)
157 cols.append(_index("q_em", 0, periods) + tau)
158 vals.append(np.ones(periods))
159 rows.append(tau)
160 cols.append(_index("q_dis", 0, periods) + tau)
161 vals.append(np.ones(periods))
162 rows.append(tau)
163 cols.append(_index("c", 0, periods) + tau)
164 vals.append(-np.ones(periods))
165 rows.append(tau)
166 cols.append(_index("s", 0, periods) + tau)
167 vals.append(-np.ones(periods))
168 # (R2) SOC 动态:  $E_t - E_{t-1} - \eta_{ch} \cdot c_t + q_{dis_t} / \eta_{dis} = 0$ ,  $t = 1$  时 RHS =  $E_0$ 
169 rows.append(periods + tau)
170 cols.append(_index("E", 0, periods) + tau)
171 vals.append(np.ones(periods))
172 rows.append(periods + tau[1:])
173 cols.append(_index("E", 0, periods) + tau[1:] - 1)
174 vals.append(-np.ones(periods - 1))
175 rows.append(periods + tau)
176 cols.append(_index("c", 0, periods) + tau)
177 vals.append(-ETA_CH * np.ones(periods))
178 rows.append(periods + tau)
179 cols.append(_index("q_dis", 0, periods) + tau)
180 vals.append((1.0 / ETA_DIS) * np.ones(periods))
181
182 a_eq = coo_matrix(
183     (np.concatenate(vals), (np.concatenate(rows), np.concatenate(cols))),
184     shape=(2 * periods, size),
185 ).tocsr()
186 b_eq = np.concatenate(
187     [data.load_energy - data.pv_energy, np.concatenate([[E_INIT], np.zeros(periods - 1)])])
188 )
189 return objective, a_eq, b_eq, lower, upper
190
191
192 def solve_lp(data: LpData, *, time_limit_seconds: float = 300.0) -> LpSolution:
193     """用 CPU HiGHS 求解 LP, 并返回含约束残差的解。"""
194     objective, a_eq, b_eq, lower, upper = build_lp(data)
195     # scipy 的 linprog 只接受 (n, 2) 的界序列/数组; 这里用 column_stack 保持 0(n) 内存。
196     bounds = np.column_stack((lower, upper))
197     result = linprog(
198         objective,
199         A_eq=a_eq,
200         b_eq=b_eq,
201         bounds=bounds,
202         method="highs",
203         options={"time_limit": float(time_limit_seconds), "presolve": True},
204     )
205     if result.x is None:
206         x = np.full(objective.shape[0], np.nan)
207         equality_residual = float("nan")
208         bound_violation = float("nan")
209     else:
210         x = np.asarray(result.x, dtype=float)
211         equality_residual = float(np.max(np.abs(a_eq @ x - b_eq)))
212         violation_low = float(np.max(lower - x))
213         finite_upper = np.where(np.isfinite(upper), x - upper, -np.inf)
214         violation_up = float(np.max(finite_upper))
215         bound_violation = max(violation_low, violation_up, 0.0)
216     return LpSolution(
217         data=data,
218         status=int(result.status),
219         message=str(result.message),
220         iterations=int(result.nit) if result.nit is not None else None,
221         objective=float(result.fun) if result.fun is not None else float("nan"),
222         x=x,

```

```

223     equality_residual_max=equality_residual,
224     bound_violation_max=bound_violation,
225 )
226
227
228 def day_slice(days: int, day: int) -> slice:
229     return slice(day * PERIODS_PER_DAY, (day + 1) * PERIODS_PER_DAY)
230
231
232 def state_series(solution: LpSolution) -> np.ndarray:
233     """返回长度 ``T + 1`` 的储电量序列 ``[E_0, E_1, ..., E_T]`` (``E_0 = E_INIT``)。"""
234     storage = solution.column("E")
235     return np.concatenate([[E_INIT], storage])
236
237
238 def merge_interval_labels(mask: np.ndarray) -> list[tuple[int, int]]:
239     """把布尔掩码的连续 True 段合并为 ``(start, stop)`` 半开区间 (0 基时段下标)。"""
240     runs: list[tuple[int, int]] = []
241     start: int | None = None
242     for index, flag in enumerate(mask):
243         if flag and start is None:
244             start = index
245         elif not flag and start is not None:
246             runs.append((start, index))
247             start = None
248     if start is not None:
249         runs.append((start, len(mask)))
250     return runs
251
252
253 def _format_minutes(total_minutes: int) -> str:
254     """把「左端点分钟数」格式化为 ``H:MM``: 1440 时按 AS01 写 ``+1``。"""
255     if total_minutes < 1440:
256         return f"{total_minutes // 60}:{total_minutes % 60:02d}"
257     shifted = total_minutes - 1440
258     return f"{shifted // 60}:{shifted % 60:02d}+1"
259
260
261 def interval_label(labels: list[str], start: int, stop: int) -> str:
262     """按 AS12 生成 ``[_i, _j + 10min)`` 的区间标签 (左端点口径)。"""
263     start_label = labels[start]
264     end_minutes = 10 * (stop + 1)
265     return f"{start_label}-{_format_minutes(end_minutes)}"
266
267
268 def emergency_intervals(
269     q_em_day: np.ndarray, labels: list[str], *, tol: float = TOL
270 ) -> list[dict[str, Any]]:
271     """逐日紧急购电区间 (连续 10 分钟时段合并, AS12/A18)。"""
272     mask = np.asarray(q_em_day, dtype=float) > tol
273     out: list[dict[str, Any]] = []
274     for start, stop in merge_interval_labels(mask):
275         out.append(
276             {
277                 "slot": interval_label(labels, start, stop),
278                 "start_period": int(start + 1),
279                 "stop_period": int(stop),
280                 "energy_kwh": round(float(np.sum(q_em_day[start:stop])), 6),
281             }
282         )
283     return out
284
285
286 def _round(value: float, digits: int = 6) -> float:
287     return round(float(value), digits)
288
289
290 def evaluate(solution: LpSolution, *, full_horizon: bool = True) -> dict[str, Any]:
291     """计算交付指标、结构恒等式 (I1)/(I2) (全期与交付期切片) 与全部硬约束检查。"""
292     data = solution.data
293     days = data.days
294     periods = data.periods
295     purchase = solution.column("b")
296     q_em = solution.column("q_em")
297     charge = solution.column("c")
298     discharge = solution.column("q_dis")
299     spill = solution.column("s")
300     state = state_series(solution)
301
302     checks: list[dict[str, Any]] = []
303

```

```

304 def record(name: str, value: float, threshold: float, passed: bool, note: str = "") -> None:
305     checks.append(
306         {"name": name, "value": float(value), "threshold": float(threshold), "passed": bool(passed), "note": note}
307     )
308
309     total_purchase = float(np.sum(purchase))
310     total_q_em = float(np.sum(q_em))
311     total_charge = float(np.sum(charge))
312     total_discharge = float(np.sum(discharge))
313     total_spill = float(np.sum(spill))
314     cost_plan = float(np.sum(data.price * purchase))
315     cost_em = float(np.sum(ALPHA_EM * data.price * q_em))
316     cost = cost_plan + cost_em
317     net_load = data.net_load_energy
318     storage_final = float(state[periods])
319
320     delivery_start = min(DAILY_DELIVERY_START, days)
321     delivery = np.zeros(periods, dtype=bool)
322     delivery[delivery_start * PERIODS_PER_DAY :] = True
323     if delivery.any():
324         state_at_delivery_start = float(state[delivery_start * PERIODS_PER_DAY])
325         net_load_delivery = float(np.sum(data.load_energy[delivery] - data.pv_energy[delivery]))
326         cost_delivery = float(
327             np.sum(data.price[delivery] * purchase[delivery])
328             + np.sum(ALPHA_EM * data.price[delivery] * q_em[delivery])
329         )
330         identity_i1_delivery = float(
331             np.sum(discharge[delivery])
332             - (ETA_ROUND_TRIP * np.sum(charge[delivery]) - ETA_DIS * (storage_final - state_at_delivery_start))
333         )
334         identity_i2_delivery = float(
335             np.sum(purchase[delivery])
336             - (
337                 net_load_delivery
338                 + float(np.sum(spill[delivery]))
339                 + (1.0 - ETA_ROUND_TRIP) * float(np.sum(charge[delivery]))
340                 + ETA_DIS * (storage_final - state_at_delivery_start)
341                 - float(np.sum(q_em[delivery]))
342             )
343         )
344         warmup = slice(0, delivery_start * PERIODS_PER_DAY)
345         january_cost = float(
346             np.sum(data.price[warmup] * purchase[warmup]) + np.sum(ALPHA_EM * data.price[warmup] * q_em[warmup])
347         )
348     else:
349         net_load_delivery = 0.0
350         cost_delivery = 0.0
351         january_cost = 0.0
352         identity_i1_delivery = 0.0
353         identity_i2_delivery = 0.0
354
355     identity_i1 = total_discharge - (ETA_ROUND_TRIP * total_charge - ETA_DIS * (storage_final - E_INIT))
356     identity_i2 = total_purchase - (
357         net_load
358         + total_spill
359         + (1.0 - ETA_ROUND_TRIP) * total_charge
360         + ETA_DIS * (storage_final - E_INIT)
361         - total_q_em
362     )
363
364     day_start = state[:periods:PERIODS_PER_DAY]
365     day_end = state[PERIODS_PER_DAY :: PERIODS_PER_DAY]
366     continuity_gap = float(np.max(np.abs(day_start[1:] - day_end[:-1]))) if days > 1 else 0.0
367
368     worst_bound = solution.bound_violation_max
369     storage_upper_gap = max(float(np.max(state[1:])) - E_MAX, 0.0)
370     storage_lower_gap = max(E_MIN - float(np.min(state[1:])), 0.0)
371     charge_gap = max(float(np.max(charge)) - C_CAP, 0.0)
372     discharge_gap = max(float(np.max(discharge)) - Q_CAP, 0.0)
373     spill_gap = max(float(np.max(spill - data.pv_energy)), 0.0)
374     purchase_gap = max(-float(np.min(purchase)), 0.0)
375     q_em_gap = max(-float(np.min(q_em)), 0.0)
376     complementarity = float(np.sum(charge * discharge))
377
378     charge_power_ac = charge / DELTA_T
379     charge_power_dc = ETA_CH * charge / DELTA_T
380     discharge_power_ac = discharge / DELTA_T
381     discharge_power_dc = discharge / (ETA_DIS * DELTA_T)
382     worst_power = float(
383         max(
384             np.max(charge_power_ac),

```

```

385         np.max(charge_power_dc),
386         np.max(discharge_power_ac),
387         np.max(discharge_power_dc),
388     )
389 )
390
391 record("equality_residual_max", solution.equality_residual_max, TOL,
392       solution.equality_residual_max <= TOL, "(R1)/(R2) 等式残差")
393 record("bound_violation_max", worst_bound, TOL, worst_bound <= TOL, "变量界越界量")
394 record("storage_upper_violation", storage_upper_gap, TOL,
395       float(np.max(state[1:])) <= E_MAX + TOL, "E_ <= 10800")
396 record("storage_lower_violation", storage_lower_gap, TOL,
397       float(np.min(state[1:])) >= E_MIN - TOL, "E_ >= 1200")
398 record("charge_cap_violation", charge_gap, TOL, float(np.max(charge)) <= C_CAP + TOL,
399       "c_ <= 833.3333 (并网侧侧紧)")
400 record("discharge_cap_violation", discharge_gap, TOL, float(np.max(discharge)) <= Q_CAP + TOL,
401       "q_dis_ <= 750.0000 (电池侧侧紧, D10 口径丙 / E1)")
402 record("spill_bound_violation", spill_gap, TOL, bool(np.all(spill <= data.pv_energy + TOL)),
403       "0 <= s_ <= PV_ * Δt")
404 record("purchase_nonneg_violation", purchase_gap, TOL, float(np.min(purchase)) >= -TOL, "b_ >= 0")
405 record("q_em_nonneg_violation", q_em_gap, TOL, float(np.min(q_em)) >= -TOL, "q_em_ >= 0")
406 record("cross_day_continuity", continuity_gap, TOL, continuity_gap <= TOL, "|E_{d,0} - E_{d-1,144}|")
407 record("q_em_zero_total", total_q_em, TOL, total_q_em <= TOL,
408       "定理 T1: 完全信息 + 无购电上限  $\sum q_{em} = 0$  (AS07/AS10)")
409 record("complementarity_sum", complementarity, TOL, complementarity <= TOL,
410       "AS07 同时充放电残差 (引理 L1)")
411 record("max_side_power_kw", worst_power, P_MAX, worst_power <= P_MAX + 1e-3,
412       "并网侧侧与电池侧侧换算功率均 <= 5000 kW")
413 record("identity_I1_residual", identity_i1, TOL, abs(identity_i1) <= TOL,
414       "Eq_dis = E_c - (E_T - E_0)")
415 record("identity_I2_residual", identity_i2, TOL, abs(identity_i2) <= TOL,
416       "Eb = N + Es + 0.19Ec + (E_T - E_0) - Eq_em")
417 if delivery_start < days:
418     record("identity_I1_delivery_residual", identity_i1_delivery, TOL, abs(identity_i1_delivery) <= TOL,
419           "交付期切片 (E_{2/1,0} → E_T)")
420     record("identity_I2_delivery_residual", identity_i2_delivery, TOL, abs(identity_i2_delivery) <= TOL,
421           "交付期切片 (N_req)")
422
423 if full_horizon:
424     lower_bound = float(np.min(data.price) * (net_load + ETA_DIS * (E_MIN - E_INIT)))
425     no_storage_cost = float(np.sum(data.price * np.maximum(data.load_energy - data.pv_energy, 0.0)))
426     record("analytic_lower_bound", cost, lower_bound, cost >= lower_bound - 1e-6,
427           "C >= min(p) * [N + (E_min - E_0)]")
428     record("analytic_upper_bound", no_storage_cost, cost, cost <= no_storage_cost + 1e-6,
429           "无储能可行解的购电费上界")
430     record("magnitude_delivery_10m", cost_delivery, DELIVERY_COST_SCALE_FLOOR_YUAN,
431           cost_delivery >= DELIVERY_COST_SCALE_FLOOR_YUAN,
432           "B2 第 5 条: 交付期总费用须为 10-7 元量级 (防 D9 的 Δt 误乘)")
433
434 purchase_d = solution.daily("b")
435 charge_d = solution.daily("c")
436 discharge_d = solution.daily("q_dis")
437 spill_d = solution.daily("s")
438 q_em_d = solution.daily("q_em")
439 price_d = data.price.reshape(days, PERIODS_PER_DAY)
440 cost_plan_d = np.sum(price_d * purchase_d, axis=1)
441 cost_em_d = np.sum(ALPHA_EM * price_d * q_em_d, axis=1)
442
443 series = {
444     "period_index": list(range(1, periods + 1)),
445     "price_yuan_per_kwh": [float(v) for v in data.price],
446     "load_energy_kwh": [float(v) for v in data.load_energy],
447     "pv_energy_kwh": [float(v) for v in data.pv_energy],
448     "purchase_kwh": [float(v) for v in purchase],
449     "q_em_kwh": [float(v) for v in q_em],
450     "charge_kwh": [float(v) for v in charge],
451     "discharge_kwh": [float(v) for v in discharge],
452     "spill_kwh": [float(v) for v in spill],
453     "storage_kwh": [float(v) for v in solution.column("E")],
454 }
455
456 return {
457     "objective_yuan": _round(cost),
458     "objective_plan_yuan": _round(cost_plan),
459     "objective_emergency_yuan": _round(cost_em),
460     "delivery_cost_yuan": _round(cost_delivery),
461     "january_cost_yuan": _round(january_cost),
462     "totals": {
463         "total_purchase_kwh": _round(total_purchase),
464         "total_q_em_kwh": _round(total_q_em),
465         "total_charge_kwh": _round(total_charge),
466         "total_discharge_kwh": _round(total_discharge),

```

```

466         "total_spill_kwh": _round(total_spill),
467         "net_load_kwh": _round(net_load),
468         "net_load_delivery_kwh": _round(net_load_delivery),
469         "total_load_kwh": _round(data.total_load_energy),
470         "total_pv_kwh": _round(data.total_pv_energy),
471         "max_charge_kwh": _round(float(np.max(charge))),
472         "max_discharge_kwh": _round(float(np.max(discharge))),
473         "max_purchase_kwh": _round(float(np.max(purchase))),
474         "max_purchase_power_kw": _round(float(np.max(purchase)) / DELTA_T, 6),
475         "max_side_power_kw": _round(worst_power),
476         "storage_initial_kwh": _round(E_INIT),
477         "storage_final_kwh": _round(storage_final),
478         "storage_range_kwh": [_round(float(np.min(state[1:])), _round(float(np.max(state[1:])))],
479         "days_with_emergency": int(np.sum(np.any(q_em_d > TOL, axis=1))),
480         "simultaneous_charge_discharge_periods": int(np.sum((charge > TOL & (discharge > TOL))),
481         "day_start_unique_levels": int(len(np.unique(np.round(day_start, 6)))),
482     },
483     "identities": {
484         "I1_residual": float(identity_i1),
485         "I2_residual": float(identity_i2),
486         "I1_delivery_residual": float(identity_i1_delivery),
487         "I2_delivery_residual": float(identity_i2_delivery),
488         "continuity_residual_max": float(continuity_gap),
489     },
490     "daily": {
491         "day_index": list(range(days)),
492         "purchase_kwh": [_round(v) for v in np.sum(purchase_d, axis=1)],
493         "q_em_kwh": [_round(v) for v in np.sum(q_em_d, axis=1)],
494         "charge_kwh": [_round(v) for v in np.sum(charge_d, axis=1)],
495         "discharge_kwh": [_round(v) for v in np.sum(discharge_d, axis=1)],
496         "spill_kwh": [_round(v) for v in np.sum(spill_d, axis=1)],
497         "cost_plan_yuan": [_round(v) for v in cost_plan_d],
498         "cost_emergency_yuan": [_round(v) for v in cost_em_d],
499         "cost_total_yuan": [_round(v) for v in (cost_plan_d + cost_em_d)],
500         "state_start_kwh": [_round(v) for v in day_start],
501         "state_end_kwh": [_round(v) for v in day_end],
502     },
503     "checks": checks,
504     "checks_failed": [item["name"] for item in checks if not item["passed"]],
505     "series": series,
506 }
507
508
509 __all__ = [
510     "ALPHA_EM",
511     "BLOCKS_PER_DAY",
512     "BLOCK_ROWS",
513     "C_CAP",
514     "DAILY_DELIVERY_START",
515     "DAYS_FULL",
516     "DELTA_T",
517     "E_CAP_MAX",
518     "E_INIT",
519     "E_MAX",
520     "E_MIN",
521     "ETA_CH",
522     "ETA_DIS",
523     "ETA_ROUND_TRIP",
524     "LpData",
525     "LpSolution",
526     "P_MAX",
527     "PERIODS_PER_DAY",
528     "Q_CAP",
529     "SPEC_DATES",
530     "TABLE1_SLOTS",
531     "TOL",
532     "build_lp",
533     "day_slice",
534     "emergency_intervals",
535     "evaluate",
536     "interval_label",
537     "merge_interval_labels",
538     "solve_lp",
539     "state_series",
540 ]

```

B.6 问题二：求解主程序

```
1 # -*- coding: utf-8 -*-
```

```

2 """prob02 计算入口：读取附件 1/2，求解 M1 长时域 LP，写出 ``result2.xlsx`` 与全部追踪产物。
3
4 用法（正式任务，由 computation 阶段的 resource-manager 通过 ``scripts/compute_dispatcher.py`` 提交）：
5
6 <venv>/python run_prob02.py --data data/附件1.xlsx --data2 data/附件2.xlsx \
7     --template data/附件5/result2.xlsx \
8     --output problems/microgrid_2025/prob02/versions/assumption_v001/results/<run> \
9     --days 365 --time-limit 300
10
11 探针模式（``--days`` 小于 365）只用于接口自检：不写 ``result2.xlsx``，改写 ``probe_result2.xlsx``，
12 并跳过解析界与量级检查。完整计算必须由 Runner/tasks.py 创建隔离 task (supervised worker) 执行。
13 """
14 from __future__ import annotations
15
16 import argparse
17 import hashlib
18 import os
19 import platform
20 import sys
21 from datetime import datetime, timezone
22 from pathlib import Path
23 from typing import Any
24
25 import numpy as np
26 from prob02_io import (
27     PLAN_SHEET,
28     Attachment1,
29     InputValidationError,
30     delivery_dates,
31     fill_result2_workbook,
32     inspect_template,
33     read_attachment1,
34     read_attachment2,
35     write_json,
36 )
37 from prob02_model import (
38     ALPHA_EM,
39     BLOCKS_PER_DAY,
40     C_CAP,
41     DAILY_DELIVERY_START,
42     DAYS_FULL,
43     DELTA_T,
44     E_INIT,
45     E_MAX,
46     E_MIN,
47     ETA_CH,
48     ETA_DIS,
49     ETA_ROUND_TRIP,
50     P_MAX,
51     PERIODS_PER_DAY,
52     Q_CAP,
53     SPEC_DATES,
54     TABLE1_SLOTS,
55     LpData,
56     LpSolution,
57     emergency_intervals,
58     evaluate,
59     solve_lp,
60     state_series,
61 )
62
63 PROJECT_MARKERS = ("pyproject.toml", "AGENTS.md")
64 DELIVERY_LAST_DAY = 364 # 0 基日索引: 2025-12-31
65 EXIT_OK = 0
66 EXIT_SOLVER_NOT_OPTIMAL = 2
67 EXIT_INPUT_INVALID = 3
68 EXIT_HARD_CHECK_FAILED = 4
69
70
71 def project_root() -> Path:
72     current = Path(__file__).resolve()
73     for candidate in (current, *current.parents):
74         if all((candidate / marker).exists() for marker in PROJECT_MARKERS):
75             return candidate
76     raise RuntimeError("无法从脚本位置定位项目根目录")
77
78
79 def resolve(path_text: str) -> Path:
80     candidate = Path(path_text)
81     return candidate if candidate.is_absolute() else (project_root() / candidate)
82

```

```

83
84 def code_fingerprint(code_dir: Path) -> dict[str, str]:
85     digest = hashlib.sha256()
86     files: dict[str, str] = {}
87     for path in sorted(code_dir.glob("*.py")):
88         payload = path.read_bytes()
89         files[path.name] = hashlib.sha256(payload).hexdigest()
90         digest.update(path.name.encode("utf-8"))
91         digest.update(payload)
92     files["__code_dir__"] = digest.hexdigest()
93     return files
94
95
96 def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
97     parser = argparse.ArgumentParser(
98         description="prob02 计划购电 LP 求解器 (assumption_v001 / formulation_v001, M1 主口径)"
99     )
100     parser.add_argument("--data", default="data/附件1.xlsx", help="附件 1 路径 (只读; 只用电价列)")
101     parser.add_argument("--data2", default="data/附件2.xlsx", help="附件 2 路径 (只读; 实际负载/光伏)")
102     parser.add_argument("--template", default="data/附件5/result2.xlsx", help="result2.xlsx 模板路径 (只读)")
103     parser.add_argument("--output", required=True, help="输出目录 (必须在 assumption_v001 版本目录内)")
104     parser.add_argument("--days", type=int, default=DAYS_FULL, help="优化天数; < 365 为探针模式")
105     parser.add_argument("--time-limit", type=float, default=300.0, help="HiGHS 求解时限 (秒)")
106     return parser.parse_args(argv)
107
108
109 def build_tables(
110     solution: LpSolution,
111     metrics: dict[str, Any],
112     attachment1: Attachment1,
113     template_info: dict[str, Any],
114     *,
115     probe_mode: bool,
116 ) -> dict[str, Any]:
117     """构造表 1/表 2/表 3 (4 个指定日期) 并按 AS01 校验位置口径。
118
119     AS01 要求**两个**标签同时在位: 附件 1 第 i 行时间戳 = 区间左端点 (如 ``10:00``);
120     ``result2.xlsx`` 第 i 个时段表头 = 完整区间 (如 ``10:00-10:10``)。
121     """
122     labels = attachment1.time_labels
123     template_labels = template_info["plan_labels"]
124     days = solution.data.days
125     purchase_d = solution.daily("b")
126     charge_d = solution.daily("c")
127     discharge_d = solution.daily("q_dis")
128     q_em_d = solution.daily("q_em")
129     daily = metrics["daily"]
130
131     table1: dict[str, Any] = {}
132     table2: dict[str, Any] = {}
133     table3: dict[str, Any] = {}
134     covered: list[str] = []
135     label_ok = True
136     for date_text, day_index in SPEC_DATES:
137         if day_index >= days:
138             continue
139         covered.append(date_text)
140         slots: list[dict[str, Any]] = []
141         for slot_name, position in TABLE1_SLOTS:
142             attachment_label = labels[position - 1]
143             template_label = template_labels[position - 1]
144             slot_start = slot_name.split("-")[0]
145             matches = attachment_label == slot_start and template_label == slot_name
146             label_ok = label_ok and matches
147             slots.append(
148                 {
149                     "slot": slot_name,
150                     "position": position,
151                     "attachment_timestamp": attachment_label,
152                     "template_label": template_label,
153                     "label_matches": bool(matches),
154                     "purchase_kwh": round(float(purchase_d[day_index, position - 1]), 6),
155                 }
156             )
157         table1[date_text] = {
158             "slots": slots,
159             "all_day_energy_kwh": daily["purchase_kwh"][day_index],
160             "all_day_cost_yuan": daily["cost_total_yuan"][day_index],
161             "all_day_plan_cost_yuan": daily["cost_plan_yuan"][day_index],
162             "all_day_emergency_cost_yuan": daily["cost_emergency_yuan"][day_index],
163         }

```

```

164     table2[date_text] = {
165         "block_charge_kwh": [
166             round(float(np.sum(charge_d[day_index, k * 24 : (k + 1) * 24])), 6)
167             for k in range(BLOCKS_PER_DAY)
168         ],
169         "block_discharge_kwh": [
170             round(float(np.sum(discharge_d[day_index, k * 24 : (k + 1) * 24])), 6)
171             for k in range(BLOCKS_PER_DAY)
172         ],
173         "storage_0_00_kwh": daily["state_start_kwh"][day_index],
174         "storage_24_00_kwh": daily["state_end_kwh"][day_index],
175     }
176     intervals = emergency_intervals(q_em_d[day_index], labels)
177     table3[date_text] = {
178         "intervals": intervals,
179         "total_kwh": round(float(np.sum(q_em_d[day_index])), 6),
180         "paper_text": "无 (0 kWh) " if not intervals else "",
181     }
182     return {
183         "spec_dates": covered,
184         "table1": table1,
185         "table2": table2,
186         "table3": table3,
187         "table1_label_checks_passed": bool(label_ok),
188         "table3_note": (
189             "题面表 4 的示例日期 2025/3/1 与 3 个区间及其数值是**格式示例**，不是本题答案；"
190             "本问主口径下 Eq_em = 0 (定理 T1)，表 3 为「无 (0 kWh)」"。"
191         ),
192         "probe_mode": probe_mode,
193     }
194
195
196 def main(argv: list[str] | None = None) -> int:
197     args = parse_args(argv)
198     data_path = resolve(args.data)
199     data2_path = resolve(args.data2)
200     template_path = resolve(args.template)
201     output_dir = resolve(args.output)
202     code_dir = Path(__file__).resolve().parent
203     probe_mode = args.days < DAYS_FULL
204
205     if not 2 <= args.days <= DAYS_FULL:
206         print(f"[prob02] --days 必须在 [2, {DAYS_FULL}]，收到 {args.days}", file=sys.stderr)
207         return EXIT_INPUT_INVALID
208
209     try:
210         attachment1 = read_attachment1(data_path, expected_rows=PERIODS_PER_DAY)
211         attachment2 = read_attachment2(data2_path, expected_days=DAYS_FULL, expected_periods=PERIODS_PER_DAY)
212         template_info = inspect_template(template_path)
213     except InputValidationError as exc:
214         print(f"[prob02] 输入校验失败: {exc}", file=sys.stderr)
215         return EXIT_INPUT_INVALID
216
217     if template_info["plan_rows"] < DELIVERY_LAST_DAY + 1 - DAILY_DELIVERY_START:
218         print(
219             f"[prob02] 模板 [{PLAN_SHEET}] 只有 {template_info['plan_rows']} 天，"
220             f"少于交付期 {DELIVERY_LAST_DAY + 1 - DAILY_DELIVERY_START} 天",
221             file=sys.stderr,
222         )
223         return EXIT_INPUT_INVALID
224
225     days = args.days
226     periods = days * PERIODS_PER_DAY
227     lp_data = LpData(
228         price=np.tile(attachment1.price, days),
229         load_energy=attachment2.load_kw[:days].reshape(-1) * DELTA_T,
230         pv_energy=attachment2.pv_kw[:days].reshape(-1) * DELTA_T,
231         days=days,
232     )
233
234     solution = solve_lp(lp_data, time_limit_seconds=args.time_limit)
235
236     run_meta = {
237         "problem_id": "microgrid_2025",
238         "question_id": "prob02",
239         "assumption_version": "assumption_v001",
240         "formulation_version": "formulation_v001",
241         "stage": "computation",
242         "model": "M1",
243         "probe_mode": probe_mode,
244         "days": days,
245         "periods": periods,
246         "created_at": datetime.now(timezone.utc).isoformat(),
247         "task_id": os.environ.get("AUTOMM_TASK_ID"),
248         "output_directory": str(output_dir),

```

```

245     "device": "cpu",
246     "gpu_required": False,
247     "seed": None,
248     "deterministic": True,
249     "solver": "scipy.optimize.linprog(method='highs')",
250     "time_limit_seconds": args.time_limit,
251     "input": {
252         "attachment1": str(data_path),
253         "attachment1_md5": attachment1.md5,
254         "attachment1_periods": attachment1.periods,
255         "attachment2": str(data2_path),
256         "attachment2_md5": attachment2.md5,
257         "attachment2_days": attachment2.days,
258         "template": str(template_path),
259         "template_md5": template_info["md5"],
260         "template_plan_rows": template_info["plan_rows"],
261     },
262     "parameters": {
263         "delta_t_h": DELTA_T,
264         "eta_ch": ETA_CH,
265         "eta_dis": ETA_DIS,
266         "eta_round_trip": ETA_ROUND_TRIP,
267         "alpha_em": 5.0,
268         "e_init_kwh": E_INIT,
269         "e_min_kwh": E_MIN,
270         "e_max_kwh": E_MAX,
271         "e_terminal": None,
272         "p_max_kw": P_MAX,
273         "c_cap_kwh": C_CAP,
274         "q_dis_cap_kwh": Q_CAP,
275         "b_upper_kwh": None,
276     },
277     "code_sha256": code_fingerprint(code_dir),
278     "environment": {
279         "python": sys.version.split()[0],
280         "numpy": np.__version__,
281         "platform": platform.platform(),
282     },
283 }
284
285 if solution.status != 0:
286     write_json(output_dir / "solver_status.json", {
287         "status": solution.status,
288         "message": solution.message,
289         "feasible_incumbent": False,
290         "objective_yuan": None,
291         "iterations": solution.iterations,
292         "device": "cpu",
293         "gpu_required": False,
294         "seed": None,
295     })
296     write_json(output_dir / "run_manifest.json", {**run_meta, "outcome": "solver_not_optimal"})
297     print(f"[prob02] 求解未达最优: status={solution.status} {solution.message}", file=sys.stderr)
298     return EXIT_SOLVER_NOT_OPTIMAL
299
300 metrics = evaluate(solution, full_horizon=not probe_mode)
301 tables = build_tables(solution, metrics, attachment1, template_info, probe_mode=probe_mode)
302 matrix_scale = {
303     "variables": 6 * periods,
304     "equality_constraints": 2 * periods,
305     "variable_bounds": 6 * periods,
306     "nonzero": 9 * periods - 1,
307     "integer_variables": 0,
308 }
309
310 write_json(output_dir / "solver_status.json", {
311     "status": solution.status,
312     "message": solution.message,
313     "solver": "highs",
314     "iterations": solution.iterations,
315     "objective_yuan": metrics["objective_yuan"],
316     "delivery_cost_yuan": metrics["delivery_cost_yuan"],
317     "feasible_incumbent": True,
318     "mip_gap": None,
319     "device": "cpu",
320     "gpu_required": False,
321     "seed": None,
322     "equality_residual_max": solution.equality_residual_max,
323     "bound_violation_max": solution.bound_violation_max,
324     "deterministic": True,
325 })

```

```

326 write_json(output_dir / "solution.json", {
327     "objective_yuan": metrics["objective_yuan"],
328     "objective_plan_yuan": metrics["objective_plan_yuan"],
329     "objective_emergency_yuan": metrics["objective_emergency_yuan"],
330     "delivery_cost_yuan": metrics["delivery_cost_yuan"],
331     "january_cost_yuan": metrics["january_cost_yuan"],
332     "totals": metrics["totals"],
333     "identities": metrics["identities"],
334     "daily": metrics["daily"],
335     "checks": metrics["checks"],
336     "checks_failed": metrics["checks_failed"],
337     "series": metrics["series"],
338 })
339 write_json(output_dir / "tables.json", tables)
340
341 delivery_dates_list = delivery_dates(attachment2.dates, DAILY_DELIVERY_START)[: max(days - DAILY_DELIVERY_START, 0)]
342 workbook_name = "probe_result2.xlsx" if probe_mode else "result2.xlsx"
343 workbook_info: dict[str, Any] | None = None
344 if delivery_dates_list:
345     selected = slice(DAILY_DELIVERY_START, days)
346     purchase_d = solution.daily("b")
347     charge_d = solution.daily("c")
348     discharge_d = solution.daily("q_dis")
349     q_em_d = solution.daily("q_em")
350     price_d = lp_data.price.reshape(days, PERIODS_PER_DAY)
351     state = state_series(solution)
352     day_cost = np.sum(price_d * purchase_d, axis=1) + np.sum(ALPHA_EM * price_d * q_em_d, axis=1)
353     workbook_info = fill_result2_workbook(
354         template=template_path,
355         dest=output_dir / workbook_name,
356         dates=delivery_dates_list,
357         purchase_kwh=purchase_d[selected],
358         charge_kwh=charge_d[selected],
359         discharge_kwh=discharge_d[selected],
360         state_start_kwh=state[:periods:PERIODS_PER_DAY][selected],
361         state_end_kwh=state[PERIODS_PER_DAY::PERIODS_PER_DAY][selected],
362         day_energy_kwh=np.sum(purchase_d, axis=1)[selected],
363         day_cost_yuan=day_cost[selected],
364         emergency_intervals=[
365             [
366                 (item["slot"], item["energy_kwh"])
367                 for item in emergency_intervals(q_em_d[index], attachment1.time_labels)
368             ]
369             for index in range(DAILY_DELIVERY_START, days)
370         ],
371     )
372
373 failed = list(metrics["checks_failed"])
374 if not tables["table1_label_checks_passed"]:
375     failed.append("table1_label_checks")
376 if workbook_info is not None and workbook_info.get("format_residuals"):
377     failed.append("result2_format_residuals")
378 write_json(output_dir / "run_manifest.json", {
379     **run_meta,
380     "outcome": "hard_check_failed" if failed else "completed",
381     "matrix_scale": matrix_scale,
382     "workbook": workbook_name if delivery_dates_list else None,
383     "workbook_info": workbook_info,
384     "delivery_days": len(delivery_dates_list),
385     "checks_failed": failed,
386     "objective_yuan": metrics["objective_yuan"],
387     "delivery_cost_yuan": metrics["delivery_cost_yuan"],
388     "total_purchase_kwh": metrics["totals"]["total_purchase_kwh"],
389     "total_q_em_kwh": metrics["totals"]["total_q_em_kwh"],
390     "total_spill_kwh": metrics["totals"]["total_spill_kwh"],
391     "storage_final_kwh": metrics["totals"]["storage_final_kwh"],
392 })
393
394 print(
395     f"[prob02] {'探针' if probe_mode else '正式'}运行完成: days={days} periods={periods} "
396     f"C_total={metrics['objective_yuan']:.4f} 元, 交付期={metrics['delivery_cost_yuan']:.4f} 元, "
397     f"Eb={metrics['totals']['total_purchase_kwh']:.4f} kWh, Eq_em={metrics['totals']['total_q_em_kwh']:.6f} kWh, "
398     f"Es={metrics['totals']['total_spill_kwh']:.4f} kWh, 失败检查={failed}"
399 )
400 if failed:
401     print(f"[prob02] 硬检查未通过: {failed}", file=sys.stderr)
402     return EXIT_HARD_CHECK_FAILED
403 return EXIT_OK
404
405
406 if __name__ == "__main__":

```

```
407 |         raise SystemExit(main())
```

B.7 问题三：输入输出模块

```
1  # -*- coding: utf-8 -*-
2  """prob03 输入读取与 ``result3.xlsx`` 填报。
3
4  只读 ``data/附件1.xlsx`` (电价列)、``data/附件2.xlsx`` (实际负载/光伏)、``data/附件3.xlsx`` (整点预报)
5  与模板 ``data/附件5/result3.xlsx``; 不修改任何原始数据。
6
7  口径来源: prob03 ``assumption_v001`` (AS01/AS02/AS05/AS16 + 团队裁定 B0-B7) 与 ``formulation_v001`` §3.2/§7。
8
9  纪律 (AS02 / B5):
10 * 附件 1 **只使用电价列** (第 2 列); 其负载/光伏预测列、附件 4、2026 年数据均不进入模型;
11 * 附件 3 的日期列只在每日 ``0:00`` 行出现 (其余为空白字符串), 实现按**前向填充**解析并把
12   ``2025-1-1`` 规范为 ``2025-01-01``;
13 * 附件 3 的整点预报按 AS05 的「整点数值 + 整点锚定线性插值」降尺度 (在 ``prob03_model`` 实现)。
14 """
15 from __future__ import annotations
16
17 import hashlib
18 import json
19 from dataclasses import dataclass
20 from datetime import datetime
21 from pathlib import Path
22 from typing import Any, Sequence
23
24 import numpy as np
25 import openpyxl
26 from openpyxl.utils import get_column_letter
27
28 EXPECTED_HEADER_A1 = ("时间", "电价", "小区负载", "光伏发电预测功率")
29 LOAD_SHEET = "小区负载"
30 PV_SHEET = "光伏发电实际功率"
31 PLAN_SHEET = "计划购电量"
32 ADJUST_SHEET = "调整购电量"
33 STORAGE_SHEET = "充电电量"
34 EMERGENCY_SHEET = "紧急购电量"
35 PLAN_HEADER_FIRST = "日期\\时间"
36 PLAN_HEADER_ENERGY = "全天购电量"
37 PLAN_HEADER_COST = "全天购电费"
38 STORAGE_HEADER = ("日期", "时间段", "充电量", "放电量", "时刻", "储电量")
39 EMERGENCY_HEADER = ("日期", "购电时间段", "购电量")
40 BLOCK_ROWS = 24
41 BLOCKS_PER_DAY = 6
42 PERIODS_PER_DAY = 144
43 ZERO_INTERVAL_TEXT = "---"
44 A2_DATE_HEADER = "日期\\时间"
45 DECISION_HOURS: tuple[int, ...] = (0, 6, 12, 18)
46 FC_HORIZON = 24
47
48
49 class InputValidationError(RuntimeError):
50     """输入结构与 accepted 假设不一致时严格失败。"""
51
52
53 @dataclass(frozen=True)
54 class Attachment1:
55     """附件 1 的 144 个时段电价 (元/kWh) 与时间标签 (只含电价列)。"""
56
57     price: np.ndarray
58     time_labels: list[str]
59     header: list[str]
60     md5: str
61
62     @property
63     def periods(self) -> int:
64         return int(self.price.shape[0])
65
66
67 @dataclass(frozen=True)
68 class Attachment2:
69     """附件 2 的逐日实际负载/光伏功率 (365 天 × 144 个 10 分钟点, 单位 kW)。"""
70
71     load_kw: np.ndarray
72     pv_kw: np.ndarray
73     dates: list[str]
74     time_labels: list[str]
75     md5: str
```

```

76
77 @property
78 def days(self) -> int:
79     return int(self.load_kw.shape[0])
80
81 @property
82 def periods_per_day(self) -> int:
83     return int(self.load_kw.shape[1])
84
85
86 @dataclass(frozen=True)
87 class Attachment3:
88     """附件 3 的整点预报表 ``A[d, s, k]`` (kW; ``s`` 为 ``DECISION_HOURS`` 下标, ``k = 1..24``)。"""
89
90     fc_kw: np.ndarray          # (days, 4, 24)
91     dates: list[str]
92     slots: list[str]
93     md5: str
94
95     @property
96     def days(self) -> int:
97         return int(self.fc_kw.shape[0])
98
99     def row(self, day: int, hour: int) -> np.ndarray:
100         """返回第 ``day`` 天、发布时刻 ``hour`` 的 24 维整点预报向量。"""
101         if hour not in DECISION_HOURS:
102             raise ValueError(f"未知发布时刻: {hour}")
103         return self.fc_kw[day, DECISION_HOURS.index(hour), :]
104
105
106 def md5_of(path: Path) -> str:
107     digest = hashlib.md5()
108     with path.open("rb") as handle:
109         for chunk in iter(lambda: handle.read(1 << 20), b""):
110             digest.update(chunk)
111     return digest.hexdigest()
112
113
114 def _clock_label(value: Any) -> str:
115     """把附件时间戳格式化为 ``H:MM`` (datetime/time 走 strftime, 字符串原样返回)。"""
116     if value is None:
117         return ""
118     if hasattr(value, "strftime"):
119         return value.strftime("%H:%M")
120     return str(value).strip()
121
122
123 def _date_label(value: Any) -> str:
124     if value is None:
125         return ""
126     if hasattr(value, "strftime"):
127         return value.strftime("%Y-%m-%d")
128     return str(value).strip()
129
130
131 def canonical_date(value: Any) -> str:
132     """把附件 3 的日期 (``2025-1-1`` 字符串或 datetime) 规范为 ``YYYY-MM-DD``; 空白返回空串。"""
133     if value is None:
134         return ""
135     if hasattr(value, "strftime"):
136         return value.strftime("%Y-%m-%d")
137     text = str(value).strip()
138     if not text:
139         return ""
140     parts = text.split("-") if "-" in text else text.split("/")
141     if len(parts) == 3:
142         try:
143             return f"{int(parts[0]):04d}-{int(parts[1]):02d}-{int(parts[2]):02d}"
144         except ValueError:
145             return text
146     return text
147
148
149 def _slot_hour(value: Any) -> int:
150     """把附件 3 的 ``预报时刻`` 解析为整点小时数 (``0:00`` + 0)。"""
151     if value is None:
152         raise InputValidationError("附件 3 存在空的预报时刻")
153     if hasattr(value, "strftime"):
154         return int(value.strftime("%H"))
155     text = str(value).strip()
156     if not text:

```

```

157         raise InputValidationError("附件 3 存在空的预报时刻")
158     head = text.split(":")[0].strip()
159     try:
160         return int(head)
161     except ValueError as exc:
162         raise InputValidationError(f"附件 3 预报时刻无法解析: {value!r}") from exc
163
164
165 def read_attachment1(path: Path, expected_rows: int = PERIODS_PER_DAY) -> Attachment1:
166     """读取附件 1 第 1 个工作表: 只保留电价列与时间标签。"""
167     if not path.is_file():
168         raise InputValidationError(f"附件 1 不存在: {path}")
169     workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
170     try:
171         sheet = workbook.worksheets[0]
172         rows = [row for row in sheet.iter_rows(min_row=1, max_col=4, values_only=True)]
173     finally:
174         workbook.close()
175     if len(rows) < 2:
176         raise InputValidationError("附件 1 没有数据行")
177     header = [str(item).strip() for item in rows[0]]
178     if tuple(header) != EXPECTED_HEADER_A1:
179         raise InputValidationError(f"附件 1 表头不匹配: {header} != {list(EXPECTED_HEADER_A1)}")
180     body = rows[1:]
181     if len(body) != expected_rows:
182         raise InputValidationError(f"附件 1 数据行数 {len(body)} != 期望 {expected_rows}")
183     labels = [_clock_label(row[0]) for row in body]
184     try:
185         price = np.array([float(row[1]) for row in body], dtype=float)
186     except (TypeError, ValueError) as exc:
187         raise InputValidationError(f"附件 1 电价列存在非数值单元格: {exc}") from exc
188     if not np.all(np.isfinite(price)):
189         raise InputValidationError("附件 1 电价列含 NaN/Inf")
190     if np.any(price <= 0):
191         raise InputValidationError("附件 1 存在非正电价: 目标函数与对偶前提被破坏")
192     return Attachment1(price=price, time_labels=labels, header=header, md5=md5_of(path))
193
194
195 def _read_power_sheet(
196     sheet: Any, *, name: str, expected_days: int, expected_periods: int
197 ) -> tuple[list[str], list[str], np.ndarray]:
198     rows = [row for row in sheet.iter_rows(min_row=1, max_col=expected_periods + 1, values_only=True)]
199     if len(rows) != expected_days + 1:
200         raise InputValidationError(f"附件 2 [{name}] 行数 {len(rows)} != 表头 + {expected_days}")
201     header = [" " if item is None else str(item) for item in rows[0]]
202     if header[0].strip() != A2_DATE_HEADER:
203         raise InputValidationError(f"附件 2 [{name}] 首列表头不是 {A2_DATE_HEADER}: {header[0]!r}")
204     if len(header) != expected_periods + 1:
205         raise InputValidationError(f"附件 2 [{name}] 列数 {len(header)} != {expected_periods + 1}")
206     body = rows[1:]
207     dates = [_date_label(row[0]) for row in body]
208     try:
209         values = np.array([float(item) for item in row[1:] for row in body], dtype=float)
210     except (TypeError, ValueError) as exc:
211         raise InputValidationError(f"附件 2 [{name}] 存在非数值单元格: {exc}") from exc
212     if values.shape != (expected_days, expected_periods):
213         raise InputValidationError(f"附件 2 [{name}] 矩阵形状 {values.shape} 不符")
214     if not np.all(np.isfinite(values)):
215         raise InputValidationError(f"附件 2 [{name}] 含 NaN/Inf")
216     if np.any(values < 0):
217         raise InputValidationError(f"附件 2 [{name}] 存在负功率")
218     return header, dates, values
219
220
221 def read_attachment2(
222     path: Path, expected_days: int = 365, expected_periods: int = PERIODS_PER_DAY
223 ) -> Attachment2:
224     """读取附件 2 的『小区负载』与『光伏发电实际功率』(均为实际值)。"""
225     if not path.is_file():
226         raise InputValidationError(f"附件 2 不存在: {path}")
227     workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
228     try:
229         titles = list(workbook.sheetnames)
230         for name in (LOAD_SHEET, PV_SHEET):
231             if name not in titles:
232                 raise InputValidationError(f"附件 2 缺少工作表 [{name}]: {titles}")
233         load_header, load_dates, load = _read_power_sheet(
234             workbook[LOAD_SHEET], name=LOAD_SHEET, expected_days=expected_days, expected_periods=expected_periods
235         )
236         pv_header, pv_dates, pv = _read_power_sheet(
237             workbook[PV_SHEET], name=PV_SHEET, expected_days=expected_days, expected_periods=expected_periods

```

```

238         )
239     finally:
240         workbook.close()
241     if load_dates != pv_dates:
242         raise InputValidationError("附件 2 两个工作表的日期序列不一致")
243     _validate_dates(load_dates)
244     if load_header[1:] != pv_header[1:]:
245         raise InputValidationError("附件 2 两个工作表的时间标签不一致")
246     return Attachment2(
247         load_kw=load,
248         pv_kw=pv,
249         dates=load_dates,
250         time_labels=[str(item) for item in load_header[1:]],
251         md5=md5_of(path),
252     )
253
254
255 def _validate_dates(dates: Sequence[str]) -> None:
256     try:
257         parsed = [datetime.strptime(item, "%Y-%m-%d").date() for item in dates]
258     except ValueError as exc:
259         raise InputValidationError(f"附件 2 日期列无法解析: {exc}") from exc
260     for index in range(1, len(parsed)):
261         if (parsed[index] - parsed[index - 1]).days != 1:
262             raise InputValidationError(f"附件 2 日期不连续: {dates[index - 1]} -> {dates[index]}")
263     if parsed[0].isoformat() != "2025-01-01":
264         raise InputValidationError(f"附件 2 首日不是 2025-01-01: {dates[0]}")
265
266
267 def read_attachment3(
268     path: Path, *, expected_days: int = 365, hours: Sequence[int] = DECISION_HOURS
269 ) -> Attachment3:
270     """读取附件 3 的整点预报表，返回 ``(days, 4, 24)`` 张量与规范日期/时刻序列。
271
272     日期列只在每日首个发布时刻行出现（其余为空白），实现按前向填充解析：
273     每日必须恰有 ``len(hours)`` 行且发布时刻按 ``0:00 → 6:00 → 12:00 → 18:00`` 升序。
274     """
275     if not path.is_file():
276         raise InputValidationError(f"附件 3 不存在: {path}")
277     workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
278     try:
279         sheet = workbook.worksheets[0]
280         rows = [row for row in sheet.iter_rows(min_row=1, max_col=2 + FC_HORIZON, values_only=True)]
281     finally:
282         workbook.close()
283     if len(rows) < 2:
284         raise InputValidationError("附件 3 没有数据行")
285     header = [str(item).strip() if item is not None else "" for item in rows[0]]
286     if header[:2] != ["日期", "预报时刻"]:
287         raise InputValidationError(f"附件 3 表头前两列异常: {header[:2]}")
288     body = rows[1:]
289     expected_rows = expected_days * len(hours)
290     if len(body) != expected_rows:
291         raise InputValidationError(f"附件 3 数据行数 {len(body)} != 期望 {expected_rows}")
292
293     current = ""
294     filled: list[str] = []
295     for row in body:
296         text = canonical_date(row[0])
297         if text:
298             current = text
299         filled.append(current)
300
301     dates: list[str] = []
302     slot_texts: list[str] = []
303     fc = np.empty((expected_days, len(hours), FC_HORIZON), dtype=float)
304     for day in range(expected_days):
305         day_dates = set()
306         for slot_index, hour in enumerate(hours):
307             offset = day * len(hours) + slot_index
308             day_dates.add(filled[offset])
309             got_hour = _slot_hour(body[offset][1])
310             if got_hour != int(hour):
311                 raise InputValidationError(
312                     f"附件 3 第 {offset + 2} 行预报时刻 {body[offset][1]!r} != 期望 {hour}:00"
313                 )
314             try:
315                 values = np.array([float(item) for item in body[offset][2:]], dtype=float)
316             except (TypeError, ValueError) as exc:
317                 raise InputValidationError(f"附件 3 第 {offset + 2} 行存在非数值预报: {exc}") from exc
318             fc[day, slot_index, :] = values

```

```

319         if len(day_dates) != 1 or not day_dates.pop():
320             raise InputValidationError(f"附件 3 第 {day + 1} 天的日期前向填充不唯一")
321         dates.append(filled[day * len(hours)])
322         slot_texts.append(f"{int(hours[0]):00}")
323     if not np.all(np.isfinite(fc)):
324         raise InputValidationError("附件 3 预报含 NaN/Inf")
325     if np.any(fc < 0):
326         raise InputValidationError("附件 3 存在负预报")
327     for index in range(1, len(dates)):
328         if dates[index] == dates[index - 1]:
329             raise InputValidationError(f"附件 3 日期未推进: {dates[index]}")
330     return Attachment3(fc_kw=fc, dates=dates, slots=slot_texts, md5=md5_of(path))
331
332
333 def inspect_template(path: Path) -> dict[str, Any]:
334     """读取 ``result3.xlsx`` 模板的逐行标签/表头, 用于 AS01/AS16 对齐校验 (不写入)。"""
335     if not path.is_file():
336         raise InputValidationError(f"模板不存在: {path}")
337     workbook = openpyxl.load_workbook(path, data_only=True)
338     try:
339         titles = list(workbook.sheetnames)
340         for name in (PLAN_SHEET, ADJUST_SHEET, STORAGE_SHEET, EMERGENCY_SHEET):
341             if name not in titles:
342                 raise InputValidationError(f"模板缺少工作表 [{name}]: {titles}")
343         detail: dict[str, Any] = {}
344         for name in (PLAN_SHEET, ADJUST_SHEET):
345             sheet = workbook[name]
346             header = [cell.value for cell in sheet[1]]
347             if len(header) != PERIODS_PER_DAY + 3:
348                 raise InputValidationError(f"模板 [{name}] 列数 {len(header)} != 147")
349             if str(header[0]).strip() != PLAN_HEADER_FIRST:
350                 raise InputValidationError(f"模板 [{name}] 首列表头异常: {header[0]!r}")
351             if str(header[PERIODS_PER_DAY + 1]).strip() != PLAN_HEADER_ENERGY:
352                 raise InputValidationError(f"模板 [{name}] 第 146 列不是 [{PLAN_HEADER_ENERGY}]")
353             if str(header[PERIODS_PER_DAY + 2]).strip() != PLAN_HEADER_COST:
354                 raise InputValidationError(f"模板 [{name}] 第 147 列不是 [{PLAN_HEADER_COST}]")
355             labels = [str(item) for item in header[1 : PERIODS_PER_DAY + 1]]
356             dates = [date_label(sheet.cell(row=row, column=1).value) for row in range(2, sheet.max_row + 1)]
357             detail[name] = {"labels": labels, "dates": dates, "rows": len(dates)}
358
359         storage = workbook[STORAGE_SHEET]
360         storage_header = [" " if cell.value is None else str(cell.value) for cell in storage[1]]
361         if tuple(storage_header[: len(STORAGE_HEADER)]) != STORAGE_HEADER:
362             raise InputValidationError(f"模板 [{STORAGE_SHEET}] 表头异常: {storage_header}")
363         storage_blocks = [
364             str(storage.cell(row=2 + k, column=2).value) for k in range(BLOCKS_PER_DAY)
365         ]
366
367         emergency = workbook[EMERGENCY_SHEET]
368         emergency_header = [" " if cell.value is None else str(cell.value) for cell in emergency[1]]
369         if tuple(emergency_header[: len(EMERGENCY_HEADER)]) != EMERGENCY_HEADER:
370             raise InputValidationError(f"模板 [{EMERGENCY_SHEET}] 表头异常: {emergency_header}")
371
372         merged = {name: [str(rng) for rng in workbook[name].merged_cells.ranges] for name in titles}
373     finally:
374         workbook.close()
375     plan = detail[PLAN_SHEET]
376     adjust = detail[ADJUST_SHEET]
377     if plan["labels"] != adjust["labels"]:
378         raise InputValidationError("模板 [计划购电量] 与 [调整购电量] 列标签不一致")
379     return {
380         "plan_labels": plan["labels"],
381         "plan_dates": plan["dates"],
382         "plan_rows": plan["rows"],
383         "adjust_labels": adjust["labels"],
384         "adjust_rows": adjust["rows"],
385         "storage_header": storage_header,
386         "storage_blocks": storage_blocks,
387         "emergency_header": emergency_header,
388         "merged": merged,
389         "md5": md5_of(path),
390     }
391
392
393 def _assign(sheet: Any, *, row: int, column: int, value: Any) -> None:
394     """显式写单元格, ``value=None`` 也**真正清空** (防 openpyxl 静默 no-op)。"""
395     sheet.cell(row=row, column=column).value = value
396
397
398 # 交付数值的有效位 (团队勘误 R10, 2026-09-11): 求解器返回解带 1e-10 级噪声,
399 # 若不规整就会出现 ``1199.999999997668`` 这类**表现越界** (题面下界 1200) 与 19 位小数,

```

```

400 # 评阅人会当成数据错误。交付 xlsx 一律按 6 位小数写入（绝对误差 5e-7kWh，远小于任何报告量级）。
401 DELIVERY_DECIMALS = 6
402 # 附录 1 的运行区间（题面硬参数；此处只用于**交付投影**，不参与建模）。
403 STATE_MIN_KWH = 1200.0
404 STATE_MAX_KWH = 10800.0
405
406
407 def _num(value: Any) -> float:
408     """交付数值规整：保留 ``DELIVERY_DECIMALS`` 位小数。"""
409     return float(round(float(value), DELIVERY_DECIMALS))
410
411
412 def _state(value: Any, *, clip_log: list[float]) -> float:
413     """储电量交付值：先规整，再投影到 ``[1200, 10800]``，并把最大修正量记入 ``clip_log``。"""
414     raw = float(value)
415     rounded = round(raw, DELIVERY_DECIMALS)
416     projected = min(max(rounded, STATE_MIN_KWH), STATE_MAX_KWH)
417     clip_log.append(abs(projected - raw))
418     return float(projected)
419
420
421 def _clear_rows(sheet: Any, *, first_row: int, last_row: int, last_column: int) -> None:
422     for row in range(first_row, last_row + 1):
423         for column in range(1, last_column + 1):
424             _assign(sheet, row=row, column=column, value=None)
425
426
427 def _residual_report(*, workbook: Any, days: int, emergency_rows: int) -> list[str]:
428     """复核填报后的工作簿：枚举**应空却仍有值**的单元格（防静默 no-op 复发）。"""
429     residuals: list[str] = []
430     for name in (PLAN_SHEET, ADJUST_SHEET):
431         sheet = workbook[name]
432         for row in range(days + 2, int(sheet.max_row) + 1):
433             for column in range(1, PERIODS_PER_DAY + 4):
434                 if sheet.cell(row=row, column=column).value is not None:
435                     residuals.append(f"{name}!{get_column_letter(column)}{row}（超出行数）")
436                     break
437     storage = workbook[STORAGE_SHEET]
438     for index in range(days):
439         for block in range(BLOCKS_PER_DAY):
440             row = 2 + index * BLOCKS_PER_DAY + block
441             if block != 0 and storage.cell(row=row, column=1).value is not None:
442                 residuals.append(f"充放电!A{row}（日期只写块首行）")
443             if block >= 2 and storage.cell(row=row, column=5).value is not None:
444                 residuals.append(f"充放电!E{row}（时刻只写每日前两行）")
445             if block >= 2 and storage.cell(row=row, column=6).value is not None:
446                 residuals.append(f"充放电!F{row}（储电量只写每日前两行）")
447     expected_storage_rows = 1 + days * BLOCKS_PER_DAY
448     for row in range(expected_storage_rows + 1, int(storage.max_row) + 1):
449         for column in range(1, len(STORAGE_HEADER) + 1):
450             if storage.cell(row=row, column=column).value is not None:
451                 residuals.append(f"充放电!{get_column_letter(column)}{row}（超出行数）")
452                 break
453     emergency = workbook[EMERGENCY_SHEET]
454     for row in range(1 + emergency_rows + 1, int(emergency.max_row) + 1):
455         for column in range(1, len(EMERGENCY_HEADER) + 1):
456             if emergency.cell(row=row, column=column).value is not None:
457                 residuals.append(f"紧急购电量!{get_column_letter(column)}{row}（超出行数）")
458                 break
459     for name in workbook.sheetnames:
460         for row in workbook[name].iter_rows(values_only=True):
461             for value in row:
462                 if isinstance(value, str) and " " in value:
463                     residuals.append(f"{name}!不得残留模板 压缩行")
464     total = len(residuals)
465     if total > 200:
466         residuals = residuals[:200] + [f"...（共 {total} 处，已截断）"]
467     return residuals
468
469
470 def fill_result3_workbook(
471     *,
472     template: Path,
473     dest: Path,
474     dates: Sequence[str],
475     planned_kwh: np.ndarray,
476     adjusted_kwh: np.ndarray,
477     charge_kwh: np.ndarray,
478     discharge_kwh: np.ndarray,
479     state_start_kwh: np.ndarray,
480     state_end_kwh: np.ndarray,

```

```

481 day_energy_planned_kwh: np.ndarray,
482 day_energy_adjusted_kwh: np.ndarray,
483 day_cost_yuan: np.ndarray,
484 emergency_intervals: Sequence[Sequence[tuple[str, float]]],
485 ) -> dict[str, Any]:
486     """把交付期解填入 ``result3.xlsx`` 四个工作表（保留模板标签与样式，只写数值单元格）。
487
488     口径（AS16 + 团队裁定 D4-A/D8-A/D11-A）：
489     * ``计划购电量``：``b``；末两列 ``全天购电量 = Eb``、``全天购电费 = C.total``；
490     * ``调整购电量``：最终量 ``q``（替代量）；末两列 ``Eq``、``C_total``；
491     * ``充放电量``：334 天 × 6 个 4 小时块，日期只写块首行；``时刻``/``储电量`` 写在每日前两行；
492     * ``紧急购电量``：每日期至少 1 行；``J_d = 0`` 的日期写 ``时间段 = 一``、``购电量 = 0``。
493
494     """
495     days = len(dates)
496     planned = np.asarray(planned_kwh, dtype=float)
497     adjusted = np.asarray(adjusted_kwh, dtype=float)
498     charge = np.asarray(charge_kwh, dtype=float)
499     discharge = np.asarray(discharge_kwh, dtype=float)
500     clip_log: list[float] = []
501     if planned.shape != (days, PERIODS_PER_DAY):
502         raise InputValidationError(f"planned 形状 {planned.shape} != ({days}, {PERIODS_PER_DAY})")
503     for label, array in (("adjusted", adjusted), ("charge", charge), ("discharge", discharge)):
504         if array.shape != planned.shape:
505             raise InputValidationError(f"{label} 形状与 planned 不一致")
506     if len(emergency_intervals) != days:
507         raise InputValidationError("emergency_intervals 天数与 dates 不一致")
508     if not template.is_file():
509         raise InputValidationError(f"模板不存在: {template}")
510
511     workbook = openpyxl.load_workbook(template)
512     try:
513         titles = list(workbook.sheetnames)
514         for name in (PLAN_SHEET, ADJUST_SHEET, STORAGE_SHEET, EMERGENCY_SHEET):
515             if name not in titles:
516                 raise InputValidationError(f"模板缺少工作表 [{name}]: {titles}")
517
518         for name, values, energy, cost_source in (
519             (PLAN_SHEET, planned, day_energy_planned_kwh, day_cost_yuan),
520             (ADJUST_SHEET, adjusted, day_energy_adjusted_kwh, day_cost_yuan),
521         ):
522             sheet = workbook[name]
523             if sheet.max_row < days + 1:
524                 raise InputValidationError(f"模板 [{name}] 只有 {sheet.max_row - 1} 行, 少于 {days} 天")
525             for index, date_text in enumerate(dates):
526                 row = 2 + index
527                 actual = _date_label(sheet.cell(row=row, column=1).value)
528                 if actual != date_text:
529                     raise InputValidationError(f"模板 [{name}] 第 {row} 行日期 {actual!r} != {date_text!r}")
530             for period in range(PERIODS_PER_DAY):
531                 _assign(sheet, row=row, column=2 + period, value=_num(values[index, period]))
532                 _assign(sheet, row=row, column=PERIODS_PER_DAY + 2, value=_num(energy[index]))
533                 _assign(sheet, row=row, column=PERIODS_PER_DAY + 3, value=_num(cost_source[index]))
534             if sheet.max_row > days + 1:
535                 _clear_rows(sheet, first_row=days + 2, last_row=sheet.max_row, last_column=PERIODS_PER_DAY + 3)
536
537     storage = workbook[STORAGE_SHEET]
538     storage_blocks = [str(storage.cell(row=2 + k, column=2).value) for k in range(BLOCKS_PER_DAY)]
539     blocks_filled = 0
540     for index, date_text in enumerate(dates):
541         date_value = datetime.strptime(date_text, "%Y-%m-%d")
542         for block in range(BLOCKS_PER_DAY):
543             row = 2 + index * BLOCKS_PER_DAY + block
544             start = block * BLOCK_ROWS
545             stop = start + BLOCK_ROWS
546             _assign(storage, row=row, column=1, value=date_value if block == 0 else None)
547             _assign(storage, row=row, column=2, value=storage_blocks[block])
548             _assign(storage, row=row, column=3, value=_num(np.sum(charge[index, start:stop])))
549             _assign(storage, row=row, column=4, value=_num(np.sum(discharge[index, start:stop])))
550             _assign(
551                 storage,
552                 row=row,
553                 column=5,
554                 value="00:00:00" if block == 0 else ("24:00" if block == 1 else None),
555             )
556             _assign(
557                 storage,
558                 row=row,
559                 column=6,
560                 value=_state(state_start_kwh[index], clip_log=clip_log) if block == 0 else (
561                     _state(state_end_kwh[index], clip_log=clip_log) if block == 1 else None
562                 ),
563             )

```

```

562         )
563         blocks_filled += 1
564         expected_storage_rows = 1 + days * BLOCKS_PER_DAY
565         if storage.max_row > expected_storage_rows:
566             _clear_rows(storage, first_row=expected_storage_rows + 1, last_row=storage.max_row, last_column=6)
567
568         emergency = workbook[EMERGENCY_SHEET]
569         row = 2
570         interval_rows = 0
571         for index, date_text in enumerate(dates):
572             date_value = datetime.strptime(date_text, "%Y-%m-%d")
573             intervals = list(emergency_intervals[index])
574             if not intervals:
575                 intervals = [(ZERO_INTERVAL_TEXT, 0.0)]
576             for order, (slot, kwh) in enumerate(intervals):
577                 _assign(emergency, row=row, column=1, value=date_value if order == 0 else None)
578                 _assign(emergency, row=row, column=2, value=slot)
579                 _assign(emergency, row=row, column=3, value=_num(kwh))
580                 row += 1
581                 interval_rows += 1
582             expected_emergency_rows = row - 1
583             if emergency.max_row > expected_emergency_rows:
584                 _clear_rows(emergency, first_row=expected_emergency_rows + 1, last_row=emergency.max_row, last_column=3)
585
586             dest.parent.mkdir(parents=True, exist_ok=True)
587             residuals = _residual_report(workbook=workbook, days=days, emergency_rows=interval_rows)
588             workbook.save(dest)
589         finally:
590             workbook.close()
591         return {
592             "plan_rows_filled": days,
593             "plan_periods_per_row": PERIODS_PER_DAY,
594             "adjust_rows_filled": days,
595             "storage_blocks_filled": blocks_filled,
596             "emergency_rows_filled": interval_rows,
597             "storage_blocks": storage_blocks,
598             "format_residuals": residuals,
599             "delivery_decimals": DELIVERY_DECIMALS,
600             "storage_projection": {
601                 "bounds_kwh": [STATE_MIN_KWH, STATE_MAX_KWH],
602                 "max_abs_correction_kwh": max(clip_log) if clip_log else 0.0,
603                 "cells": len(clip_log),
604             },
605         }
606
607
608 def write_json(path: Path, payload: Any) -> None:
609     """写 JSON; 显式拒绝 NaN/Inf, 避免把非有限值伪装成合法结果。"""
610     path.parent.mkdir(parents=True, exist_ok=True)
611     text = json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False)
612     path.write_text(text + "\n", encoding="utf-8")
613
614
615 def delivery_dates(all_dates: Sequence[str], start_day_index: int) -> list[str]:
616     """交付期日期列表 (默认 2025-02-01 ... 2025-12-31, 334 天)。"""
617     return [str(item) for item in all_dates[start_day_index:]]
618
619
620 def day_offset(dates: Sequence[str], target: str) -> int:
621     """在附件 2 的日期序列中定位目标的 0 基索引; 找不到时严格失败。"""
622     for index, item in enumerate(dates):
623         if item == target:
624             return index
625     raise InputValidationError(f"附件 2 日期序列中找不到 {target}")
626
627
628 __all__ = [
629     "ADJUST_SHEET",
630     "Attachment1",
631     "Attachment2",
632     "Attachment3",
633     "BLOCKS_PER_DAY",
634     "BLOCK_ROWS",
635     "DECISION_HOURS",
636     "EMERGENCY_SHEET",
637     "FC_HORIZON",
638     "InputValidationError",
639     "PERIODS_PER_DAY",
640     "PLAN_SHEET",
641     "STORAGE_SHEET",
642     "ZERO_INTERVAL_TEXT",

```

```

643     "canonical_date",
644     "day_offset",
645     "delivery_dates",
646     "fill_result3_workbook",
647     "inspect_template",
648     "md5_of",
649     "read_attachment1",
650     "read_attachment2",
651     "read_attachment3",
652     "write_json",
653 ]

```

B.8 问题三：三层模型模块

```

1  # -*- coding: utf-8 -*-
2  """prob03 三层顺序向前递推 LP (主口径 ``M1``) 的建模、求解与结构自检。
3
4  严格对应 ``problems/microgrid_2025/prob03/versions/assumption_v001`` (AS01-AS18 + 团队裁定 B0-B7)
5  与 ``formulations/formulation_v001`` (§3.2 降尺度、§3.3 计划层、§3.4 调整层、§3.5 结算层、§3.10 算法):
6
7      for d in D_full:
8          计划层 (PL_d) : 0:00 预报  $\Pi_0$ , min  $\sum p \cdot b$ , 提交  $D_0 = i = 1..36$ 
9          调整层 (AD_{d,m}): m {6,12,18}, 用  $\Pi_m$  顺序重优化  $R_m$ , 只提交  $D_m$ 
10         结算层 (ST_d) : 用附件 2 **实际**光伏闭式求  $s^i$  与  $q_{em}$  (唯一 PV act 入口, §0.4 F2)
11
12     口径 (不得返问更改): ``At = 1/6``, ``ch = _dis = 0.9``, ``c 833.3333``, ``q_dis 750.0000``
13     (D10 口径丙 / 勘误 E1)、``E [1200, 10800]``, ``E_{1/1,0} = 6000``, 终端自由、``_em = 5``,
14     ``_def = 0.5``, ``_over = 1.5``; 目标 ``C_total = C_plan + C_em + C_adj`` (元, **不乘  $\Delta t$ **, 团队 D9)。
15
16  求解器: CPU HiGHS (``scipy.optimize.linprog(method="highs")``); 本问**不使用 GPU**、不依赖随机源。
17  ``M2``-``M8`` 按团队裁定 B0/B2 由 ``ablation`` 阶段在 ``ablations/code`` 下另建, 本模块只实现 ``M1``。
18
19  团队裁定 **T7** (统一冻结 tie-breaking, 效力高于 B4-``C5`` 初版措辞, 2026-09-11):
20
21  * **T7-1**:: 所有层、所有模型 (``M1``-``M8``) 一律使用 ``min [该层主目标] +  $\sum_t (c_{f,d,t} + q_{dis}\{d,t\})$ ``;
22    本模块把 实现为**共享算子** :func:`tiebreak_epsilon` / :func:`throughput_coefficients`,
23    ``M2``-``M8`` 必须调用同一算子 (``T7_EPS_RELATIVE``、次目标系数与不变性阈值完全一致, T7-5)。
24  * **T7-2**:: 逐层落盘「加入次目标前后主目标相对变化 ``T7_INVARIANCE_TOL``」的验证
25    (:class:`TiebreakRecord`), 不满足时按 :data:`T7_SHRINK_FACTOR` 缩小 重跑, 有界于 :data:`T7_MAX_SHRINKS`。
26  * **T7-3**:: 报告该层退化维度 (活跃约束数 - 变量数) 与影响的上游量 (层末状态 ``E``、与基线解的轨迹差),
27    并在 T7 最优面上最大化吞吐量以判定是否仍存在多重最优。
28  """
29  from __future__ import annotations
30
31  import time
32  from dataclasses import dataclass, field
33  from typing import Any
34
35  import numpy as np
36  from scipy.optimize import linprog
37  from scipy.sparse import coo_matrix, csr_matrix, vstack
38
39  DELTA_T = 1.0 / 6.0
40  ETA_CH = 0.9
41  ETA_DIS = 0.9
42  ETA_ROUND_TRIP = ETA_CH * ETA_DIS
43  E_INIT = 6000.0
44  E_MIN = 1200.0
45  E_MAX = 10800.0
46  E_CAP_MAX = 12000.0
47  P_MAX = 5000.0
48  C_CAP = P_MAX * DELTA_T # 833.3333 kWh (并网点侧为紧侧)
49  Q_CAP = P_MAX * DELTA_T * ETA_DIS # 750.0000 kWh (电池侧为紧侧, D10 口径丙 / E1)
50  ALPHA_EM = 5.0
51  BETA_DEF = 0.5
52  BETA_OVER = 1.5
53  PERIODS_PER_DAY = 144
54  DAYS_FULL = 365
55  DAILY_DELIVERY_START = 31 # 0 基日索引: 2025-02-01 (1 月为预热期)
56  TOL = 1e-6
57  # 层内 LP 的**求解器内部**可行性残差门槛 (**不是** formulation §4.3 的「各层余额残差」)。
58  # 本模型只覆盖 HiGHS 的 ``time_limit``/``presolve`` (见 ``_solve``), 其默认
59  # ``primal_feasibility_tolerance`` 为 1e-7, 因此任何层都不可能按要求给出优于 -1e-7 的
60  # ``||A_eq x - b_eq||_w`` 或 ``max(A_ub x - b_ub)``。旧值 1e-8 严格强于求解器自身的可行性保证:
61  # 365 天键在 day 243/hour 18 与 day 41/hour 6 两层分别以 8.79e-8、3.53e-8 的纯数值噪声被判
62  # ``hard_check_failed`` (task 1b57b1cb92e0d5095518, act-ed12930b86a544d59 诊断)。
63  # 现改为「绝对门槛与 AS 的【等式残差 1e-6】一致 + 尺度感知相对门槛」双判据;
64  # formulation §4.3 的「各层余额残差 1e-8」仍由**提交解重算**的 ``layer_spill`` 承担 (BALANCE_TOL)。
65  LAYER_TOL = 1e-6 # 求解器内部残差绝对门槛 (=TOL; HiGHS 可行性容差 1e-7)

```

```

66 LAYER_TOL_RELATIVE = 1e-8          # 求解器内部残差相对门槛:  $\|Ax-b\|_\infty / (\|A\|_\infty \cdot \|x\|_\infty + \|b\|_\infty)$ 
67 BALANCE_TOL = 1e-8                # 提交解重算的层内余额残差 (formulation §4.3 C 判据, 沿用 1e-8)
68 DECISION_HOURS: tuple[int, ...] = (0, 6, 12, 18)
69 BLOCK_ROWS = 24
70 BLOCKS_PER_DAY = 6
71 SPEC_DATES: tuple[tuple[str, int], ...] = (
72     ("2025-03-20", 78),
73     ("2025-06-21", 171),
74     ("2025-09-23", 265),
75     ("2025-12-21", 354),
76 )
77 TABLE1_SLOTS: tuple[tuple[str, int], ...] = (
78     ("10:00-10:10", 60),
79     ("12:00-12:10", 72),
80     ("14:00-14:10", 84),
81     ("16:00-16:10", 96),
82     ("18:00-18:10", 108),
83     ("20:00-20:10", 120),
84 )
85 CROSS_YEAR_DROPPED: dict[int, tuple[int, int]] = {6: (19, 24), 12: (13, 24), 18: (7, 24)}
86 DELIVERY_MAGNITUDE_BAND_YUAN = (3.0e6, 5.0e7)
87 DAILY_MAGNITUDE_BAND_YUAN = (5.0e3, 2.0e5)
88
89 # --- 团队裁定 T7: 统一冻结 tie-breaking (所有层、所有模型 M1-M8) -----
90 # T7-1 原文: ``min [该层主目标] +  $\cdot \Sigma_t(c_{\{d,t\}}+q_{dis_{\{d,t\}}})$ `, `` = 1e-6  $\times$  主目标尺度``。
91 # 团队同句要求「取足够小以保证不改变主目标最优值」。字面取 `` = 1e-6  $\cdot P$ `` 会使次目标总量
92 # ``  $\cdot \Sigma(c+q_{dis})$ `` (上界 ``T_ref = n  $\cdot$  (C_CAP+Q_CAP)``) 与主目标**同量级**, 与「足够小」自相矛盾。
93 # 本实现把该式**相对化**为无量纲权重 (差异已在 implementation.md §1.1 显式登记):
94 #      = T7_EPS_RELATIVE  $\times$  max(|P|, 1) / T_ref
95 # 于是次目标总量恒 1e-6  $\cdot$  max(|P|, 1); 且因次目标是线性的、主目标最优面非空,
96 # 加权的最优解必落在主目标最优面上 (T7-2 的逐层验证即检验这一点)。
97 T7_EPS_RELATIVE = 1e-6
98 T7_INVARIANCE_TOL = 1e-9           # T7-2: 主目标相对变化上限
99 T7_SHRINK_FACTOR = 0.1             # T7-2: 不满足时 缩小倍数
100 T7_MAX_SHRINKS = 8                # T7-2: 缩小次数上限 (有界、确定性)
101 T7_FACE_TOL_RELATIVE = 1e-9        # T7-3: T7 最优面判定的相对容差
102 T7_FACE_TOL_ABSOLUTE = 1e-9        # T7-3: T7 最优面判定的绝对容差
103 # 字典序提交解的主目标最优面带宽容差占 T7_INVARIANCE_TOL 的比例: 留出让求解器数值噪声余量,
104 # 使「约束面上取到边界」的解仍满足 T7-2 的 1e-9 判定 (否则刚好等于 1e-9 会因 5e-16 噪声判负)。
105 T7_FACE_PRIMARY_TOL_FRACTION = 0.5
106 T7_TIEBREAK_RULE = (
107     "min [该层主目标] +  $\cdot \Sigma_t(c_{\{d,t\}}+q_{dis_{\{d,t\}}})$ ,"
108     " = 1e-6  $\cdot$  max(|该层主目标最优值|, 1) / [n  $\cdot$  (c_cap+q_dis_cap)]; "
109     "所有层、所有模型 M1-M8 必须完全一致 (团队 T7-1/T7-5) "
110 )
111
112
113 class LayerFailure(RuntimeError):
114     """某个决策层 LP 未达最优: 由调用方按 ``solver_not_optimal`` 路由 (不得伪报最优)。"""
115
116     def __init__(self, message: str, *, day: int, hour: int, status: int) -> None:
117         super().__init__(message)
118         self.day = day
119         self.hour = hour
120         self.status = status
121
122
123 class BudgetExceeded(RuntimeError):
124     """墙钟预算耗尽: 按 ``solver_not_optimal`` 路由并保留已完成层的证据, 不得伪报完成。"""
125
126
127 @dataclass(frozen=True)
128 class ChainInputs:
129     """整条 ``M1`` 链的输入 (按天展开: ``fc`` 为 ``hour  $\rightarrow$  (days, 144)`` 降尺度预报)。"""
130
131     days: int
132     price: np.ndarray          # (days, 144) 元/kWh
133     load_energy: np.ndarray    # (days, 144) kWh = L  $\cdot$   $\Delta t$ 
134     pv_act_energy: np.ndarray  # (days, 144) kWh = PVact  $\cdot$   $\Delta t$ 
135     forecast_kw: dict[int, np.ndarray]  # hour  $\rightarrow$  (days, 144) kW (未支配时段为 NaN)
136
137     @property
138     def periods(self) -> int:
139         return int(self.days * PERIODS_PER_DAY)
140
141     def forecast_energy(self, hour: int) -> np.ndarray:
142         return self.forecast_kw[hour] * DELTA_T
143
144
145 @dataclass
146 class LayerRecord:

```

```

147     day: int
148     hour: int
149     status: int
150     message: str
151     iterations: int | None
152     objective: float
153     equality_residual_max: float
154     equality_residual_relative_max: float
155     inequality_residual_max: float
156     inequality_residual_relative_max: float
157     bound_violation_max: float
158     seconds: float
159     variables: int
160     equality_rows: int
161     inequality_rows: int
162     nonzero: int
163
164     def as_dict(self) -> dict[str, Any]:
165         return {
166             "day_index": self.day,
167             "hour": self.hour,
168             "status": self.status,
169             "message": self.message,
170             "iterations": self.iterations,
171             "objective_yuan": self.objective,
172             "equality_residual_max": self.equality_residual_max,
173             "equality_residual_relative_max": self.equality_residual_relative_max,
174             "inequality_residual_max": self.inequality_residual_max,
175             "inequality_residual_relative_max": self.inequality_residual_relative_max,
176             "bound_violation_max": self.bound_violation_max,
177             "seconds": self.seconds,
178             "variables": self.variables,
179             "equality_rows": self.equality_rows,
180             "inequality_rows": self.inequality_rows,
181             "nonzero": self.nonzero,
182         }
183
184
185     def _finite_or(value: float, fallback: float = 1e30) -> float:
186         """把非有限值替换为有限哨兵，保证 ``allow_nan=False`` 的 JSON 落盘不失败。"""
187         number = float(value)
188         return number if np.isfinite(number) else float(fallback)
189
190
191 @dataclass
192 class TiebreakRecord:
193     """团队 T7 的逐层审计记录（T7-1 规则 / T7-2 不变性 / T7-3 退化维度与上游量）。"""
194
195     layer: str # "plan" | "adjustment"
196     day: int
197     hour: int
198     primary_before_yuan: float # 未加次目标的主目标最优值 P*
199     primary_after_yuan: float # 加次目标后的主目标值 P'
200     primary_relative_change: float
201     epsilon: float
202     shrinks: int
203     invariance_passed: bool
204     throughput_kwh: float #  $\Sigma(c+q_{dis})$ : 加次目标后的选择
205     throughput_before_kwh: float #  $\Sigma(c+q_{dis})$ : 纯主目标基线解
206     throughput_reduced_kwh: float
207     baseline_state_change_max_kwh: float #  $|x_{tiebreak} - x_{baseline}|_m$  (轨迹差)
208     degeneracy_degree: int # 活跃约束数 - 变量数 (退化维度代理)
209     active_constraints: int
210     variables: int
211     weighted_primary_relative_change: float # T7-1 层面加权式的相对变化 (诊断)
212     weighted_throughput_kwh: float
213     weighted_sum_effective: bool # 加权式是否达到字典序最小值
214     committed_solution: str # "lexicographic" | "weighted_sum_fallback" | "baseline_fallback"
215     throughput_upper_on_optimal_face_kwh: float | None
216     throughput_unique: bool | None
217     probe_status: int | None
218     boundary_state_kwh: float # 影响的上游量: 该层未状态 E
219     baseline_seconds: float
220     solver_seconds: float
221     note: str = ""
222
223     def as_dict(self) -> dict[str, Any]:
224         return {
225             "layer": self.layer,
226             "day_index": int(self.day),
227             "hour": int(self.hour),

```

```

228         "primary_before_yuan": _finite_or(self.primary_before_yuan),
229         "primary_after_yuan": _finite_or(self.primary_after_yuan),
230         "primary_relative_change": _finite_or(self.primary_relative_change),
231         "epsilon": _finite_or(self.epsilon),
232         "shrinks": int(self.shrinks),
233         "invariance_passed": bool(self.invariance_passed),
234         "throughput_kwh": _finite_or(self.throughput_kwh),
235         "throughput_before_kwh": _finite_or(self.throughput_before_kwh),
236         "throughput_reduced_kwh": _finite_or(self.throughput_reduced_kwh),
237         "baseline_state_change_max_kwh": _finite_or(self.baseline_state_change_max_kwh),
238         "degeneracy_degree": int(self.degeneracy_degree),
239         "active_constraints": int(self.active_constraints),
240         "variables": int(self.variables),
241         "weighted_primary_relative_change": _finite_or(self.weighted_primary_relative_change),
242         "weighted_throughput_kwh": _finite_or(self.weighted_throughput_kwh),
243         "weighted_sum_effective": bool(self.weighted_sum_effective),
244         "committed_solution": self.committed_solution,
245         "throughput_upper_on_optimal_face_kwh": (
246             None if self.throughput_upper_on_optimal_face_kwh is None
247             else _finite_or(self.throughput_upper_on_optimal_face_kwh)
248         ),
249         "throughput_unique": (
250             None if self.throughput_unique is None else bool(self.throughput_unique)
251         ),
252         "probe_status": None if self.probe_status is None else int(self.probe_status),
253         "boundary_state_kwh": _finite_or(self.boundary_state_kwh),
254         "baseline_seconds": _finite_or(self.baseline_seconds),
255         "solver_seconds": _finite_or(self.solver_seconds),
256         "note": self.note,
257     }
258
259
260 @dataclass
261 class MIResult:
262     data: ChainInputs
263     plan_b: np.ndarray          # (days, 144) 计划购电量
264     plan_c: np.ndarray          # (days, 144) 计划层轨迹 (仅 D_0 提交, 其余为计划轨迹)
265     plan_q_dis: np.ndarray
266     plan_s: np.ndarray
267     plan_E: np.ndarray
268     q: np.ndarray              # (days, 144) 最终购电量 (替代量)
269     c: np.ndarray
270     q_dis: np.ndarray
271     E: np.ndarray
272     q_em: np.ndarray           # (days, 144) 结算层
273     s_settle: np.ndarray        # (days, 144) 结算层弃光 s'
274     layer_records: list[LayerRecord] = field(default_factory=list)
275     tiebreak_records: list[TiebreakRecord] = field(default_factory=list)
276
277     @property
278     def days(self) -> int:
279         return self.data.days
280
281
282 def dominance_map(periods: int = PERIODS_PER_DAY) -> np.ndarray:
283     """支配时刻 ``i``: ``i = 1..144`` + ``0/6/12/18`` (每层 36 个时段)。"""
284     index = np.arange(1, periods + 1)
285     return 6 * ((index - 1) // 36)
286
287
288 def downscale(pv_row: np.ndarray, hour: int, *, periods: int = PERIODS_PER_DAY) -> np.ndarray:
289     """AS05 降尺度算子 ``Π_m`` (整点值 + 整点锚定线性插值)。
290
291     ``pv_row[k-1] = A_{m,k}`` (``(m+k):00`` 的功率点值, ``k = 1..24``)。时段 ``i`` 的块
292     ``b = i/6``、块内位置 ``j = i - 6(b-1)``、时效 ``k = b - m``:
293
294     * ``k = 1``: 前向保持 ``A_{m,1}``;
295     * ``2 ≤ k ≤ 24``: ``(1 - j/6) * A_{m,k-1} + (j/6) * A_{m,k}``;
296     * ``k < 1`` (早于发布时刻) 或 ``b = m + k > 24`` (跨年项): **未定义 (NaN)** , 不消费。
297     """
298     row = np.asarray(pv_row, dtype=float).reshape(-1)
299     if row.shape[0] != 24:
300         raise ValueError(f"预报向量长度 {row.shape[0]} != 24")
301     if hour not in DECISION_HOURS:
302         raise ValueError(f"未知发布时刻: {hour}")
303     out = np.full(periods, np.nan, dtype=float)
304     for index in range(1, periods + 1):
305         block = (index - 1) // 6 + 1
306         position = index - 6 * (block - 1)
307         lead = block - hour
308         if lead < 1 or lead > 24:

```

```

309         continue
310     if lead == 1:
311         out[index - 1] = row[0]
312     else:
313         out[index - 1] = (1.0 - position / 6.0) * row[lead - 2] + (position / 6.0) * row[lead - 1]
314     return out
315
316
317 def cross_year_dropped_count(hour: int) -> int:
318     """跨年项个数: ``m = 6/12/18`` 分别丢弃 6/12/18 项 (合计 36, AS05 第 4 条 / B7-5)。"""
319     if hour == 0:
320         return 0
321     low, high = CROSS_YEAR_DROPPED[hour]
322     return int(high - low + 1)
323
324
325 def _solve(
326     objective: np.ndarray,
327     a_eq: csr_matrix,
328     b_eq: np.ndarray,
329     a_ub: csr_matrix | None,
330     b_ub: np.ndarray | None,
331     bounds: np.ndarray,
332     *,
333     time_limit_seconds: float,
334 ) -> tuple[Any, float]:
335     started = time.perf_counter()
336     result = linprog(
337         objective,
338         A_eq=a_eq,
339         b_eq=b_eq,
340         A_ub=a_ub,
341         b_ub=b_ub,
342         bounds=bounds,
343         method="highs",
344         options={"time_limit": float(time_limit_seconds), "presolve": True},
345     )
346     return result, time.perf_counter() - started
347
348
349 def _inf_norm(matrix: csr_matrix) -> float:
350     """稀疏矩阵的  $\infty$ -范数 (最大绝对行和), 用于尺度感知的相对残差分母。"""
351     if matrix.shape[0] == 0:
352         return 0.0
353     return float(np.max(np.abs(matrix).sum(axis=1)))
354
355
356 def _relative_residual(residual: float, matrix: csr_matrix, rhs: np.ndarray, x: np.ndarray) -> float:
357     """标准尺度感知相对残差: ``residual / max(1, ||A|| $\infty$  * ||x|| $\infty$  + ||b|| $\infty$ )``。
358
359     分母取 ``||A|| $\infty$  * ||x|| $\infty$  + ||b|| $\infty$ `` (经典 scaled residual), 下限 1 以避免小尺度层的除零放大。
360     """
361     denominator = max(
362         1.0,
363         _inf_norm(matrix) * float(np.max(np.abs(x))) + float(np.max(np.abs(rhs))),
364     )
365     return max(float(residual), 0.0) / denominator
366
367
368 def _residuals(
369     result: Any,
370     a_eq: csr_matrix,
371     b_eq: np.ndarray,
372     a_ub: csr_matrix | None,
373     b_ub: np.ndarray | None,
374     bounds: np.ndarray,
375 ) -> tuple[float, float, float, float, float]:
376     """返回 ``(等式残差, 不等式残差, 界限界量, 等式相对残差, 不等式相对残差)``。"""
377     if result.x is None:
378         return float("nan"), float("nan"), float("nan"), float("nan"), float("nan")
379     x = np.asarray(result.x, dtype=float)
380     equality = float(np.max(np.abs(a_eq @ x - b_eq)))
381     equality_relative = _relative_residual(equality, a_eq, b_eq, x)
382     if a_ub is None or a_ub.shape[0] == 0:
383         inequality, inequality_relative = 0.0, 0.0
384     else:
385         inequality = float(np.max(a_ub @ x - b_ub))
386         inequality_relative = _relative_residual(inequality, a_ub, b_ub, x)
387     lower = bounds[:, 0]
388     upper = bounds[:, 1]
389     violation_low = float(np.max(lower - x))

```

```

390     finite = np.isfinite(upper)
391     violation_up = float(np.max((x - upper)[finite])) if finite.any() else -np.inf
392     bound = max(violation_low, violation_up, 0.0)
393     return equality, inequality, bound, equality_relative, inequality_relative
394
395
396 def throughput_scale(periods: int) -> float:
397     """该层吞吐量  $\Sigma_t(c_{\{d,t\}}+q_{dis_{\{d,t\}}}$  的紧上界 (T7-1 的 相对化分母)。"""
398     return float(periods) * (C_CAP + Q_CAP)
399
400
401 def throughput_coefficients(size: int, c_offset: int, q_dis_offset: int, periods: int) -> np.ndarray:
402     """T7-1 的 次目标系数向量:  $c$  与  $q_{dis}$  位置为 1, 其余为 0 (共享算子, M1-M8 一致)。"""
403     coefficients = np.zeros(size)
404     coefficients[c_offset : c_offset + periods] = 1.0
405     coefficients[q_dis_offset : q_dis_offset + periods] = 1.0
406     return coefficients
407
408
409 def tiebreak_epsilon(primary_reference: float, scale: float) -> float:
410     """T7-1 的  $\epsilon$ :  $1e-6 \times \max(|\text{主目标尺度}|, 1) / \text{吞吐量尺度}$  (无量纲、确定性)。"""
411     return T7_EPS_RELATIVE * max(abs(float(primary_reference)), 1.0) / max(float(scale), 1.0)
412
413
414 def _degeneracy_proxy(
415     x: np.ndarray,
416     n_variables: int,
417     a_eq: csr_matrix,
418     a_ub: csr_matrix | None,
419     b_ub: np.ndarray | None,
420     bounds: np.ndarray,
421     *,
422     tol: float,
423 ) -> tuple[int, int]:
424     """T7-3: 退化维度代理 = 活跃约束数 (等式 + 紧不等式 + 贴界变量) - 变量数。"""
425     active = int(a_eq.shape[0])
426     if a_ub is not None and a_ub.shape[0]:
427         slack = np.asarray(a_ub @ x - b_ub, dtype=float).reshape(-1)
428         active += int(np.sum(slack >= -tol))
429     lower = bounds[:, 0]
430     upper = bounds[:, 1]
431     at_lower = x <= lower + tol
432     at_upper = np.isfinite(upper) & (x >= upper - tol)
433     active += int(np.sum(at_lower | at_upper))
434     return active - int(n_variables), active
435
436
437 def _layer_record(
438     result: Any,
439     seconds: float,
440     *,
441     day: int,
442     hour: int,
443     a_eq: csr_matrix,
444     b_eq: np.ndarray,
445     a_ub: csr_matrix | None,
446     b_ub: np.ndarray | None,
447     bounds: np.ndarray,
448     objective: float | None = None,
449 ) -> LayerRecord:
450     equality, inequality, bound, equality_relative, inequality_relative = _residuals(
451         result, a_eq, b_eq, a_ub, b_ub, bounds
452     )
453     return LayerRecord(
454         day=day,
455         hour=hour,
456         status=int(result.status),
457         message=str(result.message),
458         iterations=int(result.nit) if result.nit is not None else None,
459         objective=float(objective) if objective is not None else (
460             float(result.fun) if result.fun is not None else float("nan")
461         ),
462         equality_residual_max=equality,
463         equality_residual_relative_max=equality_relative,
464         inequality_residual_max=inequality,
465         inequality_residual_relative_max=inequality_relative,
466         bound_violation_max=bound,
467         seconds=seconds,
468         variables=int(a_eq.shape[1]),
469         equality_rows=int(a_eq.shape[0]),
470         inequality_rows=int(a_ub.shape[0]) if a_ub is not None else 0,

```

```

471         nonzero=int(a_eq.nnz + (a_ub.nnz if a_ub is not None else 0)),
472     )
473
474
475 def _solve_with_tiebreak(
476     *,
477     primary_objective: np.ndarray,
478     throughput_objective: np.ndarray,
479     scale: float,
480     a_eq: csr_matrix,
481     b_eq: np.ndarray,
482     a_ub: csr_matrix | None,
483     b_ub: np.ndarray | None,
484     bounds: np.ndarray,
485     layer: str,
486     day: int,
487     hour: int,
488     boundary_index: int,
489     time_limit_seconds: float,
490 ) -> tuple[Any, LayerRecord, TiebreakRecord]:
491     """团队 T7 的共享求解器（`M1` 计划/调整层；`M2`-`M8` 必须复用，T7-5）。
492
493     四段式，全部确定性、无随机源：
494
495     **基线**：纯主目标 LP + `P`（T7-2 的「加入次目标前」）。
496     **T7-1 子面式**：`min [主目标] + E(c+q_dis)`；按 T7-2 逐层验证，不满足则按
497     `T7_SHRINK_FACTOR` 缩小后重跑（有界）。调整层的 `P` 是偏差结算费用，可小到与 HIGHS
498     对偶容差同量级，故该式在数值上可能被求解器忽略（次目标不起作用）——如实记录于
499     `weighted_sum_effective`，**不**作为提交解。
500     **字典序提交解**（T7-1 优先序语义在 -0 的精确、可解析实现）：在主目标最优面
501     `{primary P + 1e-9 * max(|P|,1)}` 上**最小化**吞吐量，得到吞吐量最小的最优轨迹。
502     **T7-3 退化探测**：在同一主目标最优面上**最大化**吞吐量；若上界显著大于 的最小值，
503     说明该层仍存在多个最优轨迹，必须报告退化维度与影响的上游量，不得静默取一个解。
504
505     """
506     n_variables = int(primary_objective.shape[0])
507     total_seconds = 0.0
508     primary_row = np.asarray(primary_objective, dtype=float).reshape(1, -1)
509
510     # 基线：纯主目标 LP
511     baseline_result, baseline_seconds = _solve(
512         primary_objective, a_eq, b_eq, a_ub, b_ub, bounds, time_limit_seconds=time_limit_seconds
513     )
514     total_seconds += baseline_seconds
515     if baseline_result.x is None:
516         record = _layer_record(
517             baseline_result, baseline_seconds, day=day, hour=hour,
518             a_eq=a_eq, b_eq=b_eq, a_ub=a_ub, b_ub=b_ub, bounds=bounds,
519         )
520         return baseline_result, record, TiebreakRecord(
521             layer=layer, day=day, hour=hour,
522             primary_before_yuan=float("nan"), primary_after_yuan=float("nan"),
523             primary_relative_change=float("inf"), epsilon=float("nan"), shrinks=0,
524             invariance_passed=False, throughput_kwh=float("nan"),
525             throughput_before_kwh=float("nan"), throughput_reduced_kwh=float("nan"),
526             baseline_state_change_max_kwh=float("nan"), degeneracy_degree=-1,
527             active_constraints=1, variables=n_variables,
528             weighted_primary_relative_change=float("inf"), weighted_throughput_kwh=float("nan"),
529             weighted_sum_effective=False, committed_solution="none",
530             throughput_upper_on_optimal_face_kwh=None, throughput_unique=None,
531             probe_status=None, boundary_state_kwh=float("nan"),
532             baseline_seconds=baseline_seconds, solver_seconds=total_seconds,
533             note="baseline_not_optimal",
534         )
535
536     x_baseline = np.asarray(baseline_result.x, dtype=float)
537     primary_before = float(np.dot(primary_objective, x_baseline))
538     throughput_before = float(np.dot(throughput_objective, x_baseline))
539     tol_primary = T7_FACE_PRIMARY_TOL_FRACTION * T7_INVARIANCE_TOL * max(abs(primary_before), 1.0)
540
541     # T7-1 子面式（诊断 + T7-2 的「加入次目标后」）+ 收缩
542     epsilon = tiebreak_epsilon(primary_before, scale)
543     shrinks = 0
544     weighted_result = baseline_result
545     weighted_seconds = baseline_seconds
546     weighted_primary = primary_before
547     weighted_relative = 0.0
548     for attempt in range(T7_MAX_SHRINKS + 1):
549         combined = primary_objective + epsilon * throughput_objective
550         weighted_result, weighted_seconds = _solve(
551             combined, a_eq, b_eq, a_ub, b_ub, bounds, time_limit_seconds=time_limit_seconds

```

```

552         total_seconds += weighted_seconds
553         if weighted_result.x is None:
554             break
555         x_weighted = np.asarray(weighted_result.x, dtype=float)
556         weighted_primary = float(np.dot(primary_objective, x_weighted))
557         weighted_relative = abs(weighted_primary - primary_before) / max(abs(primary_before), 1.0)
558         if weighted_relative <= T7_INVARIANCE_TOL or attempt >= T7_MAX_SHRINKS:
559             break
560         epsilon *= T7_SHRINK_FACTOR
561         shrinks += 1
562     weighted_ok = weighted_result.x is not None
563     weighted_throughput = (
564         float(np.dot(throughput_objective, np.asarray(weighted_result.x, dtype=float))) if weighted_ok
565         else float("nan")
566     )
567
568     # 字典序提交解：主目标最优面上最小化吞吐量
569     if a_ub is not None and a_ub.shape[0]:
570         face_a_ub = vstack([a_ub, csr_matrix(primary_row)]).tocsr()
571         face_b_ub = np.concatenate([np.asarray(b_ub, dtype=float), [primary_before + tol_primary]])
572     else:
573         face_a_ub = csr_matrix(primary_row)
574         face_b_ub = np.array([primary_before + tol_primary])
575     lex_result, lex_seconds = _solve(
576         throughput_objective, a_eq, b_eq, face_a_ub, face_b_ub, bounds,
577         time_limit_seconds=time_limit_seconds,
578     )
579     total_seconds += lex_seconds
580     committed_solution = "lexicographic"
581     if lex_result.x is not None and int(lex_result.status) == 0:
582         result = lex_result
583     elif weighted_ok:
584         result = weighted_result
585         committed_solution = "weighted_sum_fallback"
586     else:
587         result = baseline_result
588         committed_solution = "baseline_fallback"
589     x = np.asarray(result.x, dtype=float)
590     primary_after = float(np.dot(primary_objective, x))
591     relative = abs(primary_after - primary_before) / max(abs(primary_before), 1.0)
592     throughput = float(np.dot(throughput_objective, x))
593     invariance_passed = bool(relative <= T7_INVARIANCE_TOL)
594     weighted_sum_effective = bool(
595         weighted_ok
596         and weighted_relative <= T7_INVARIANCE_TOL
597         and (weighted_throughput - throughput) <= max(T7_FACE_TOL_ABSOLUTE, T7_FACE_TOL_RELATIVE * abs(throughput))
598     )
599     boundary_state = float(x[boundary_index])
600     state_change = float(np.max(np.abs(x - x_baseline)))
601     degeneracy_degree, active_constraints = _degeneracy_proxy(
602         x, n_variables, a_eq, a_ub, b_ub, bounds, tol=TOL
603     )
604
605     # T7-3: 同一主目标最优面上最大化吞吐量，判定是否仍有多重最优
606     probe_result, probe_seconds = _solve(
607         -throughput_objective, a_eq, b_eq, face_a_ub, face_b_ub, bounds,
608         time_limit_seconds=time_limit_seconds,
609     )
610     total_seconds += probe_seconds
611     if probe_result.x is not None and int(probe_result.status) == 0:
612         throughput_upper = -float(probe_result.fun)
613         throughput_unique = bool(
614             (throughput_upper - throughput) <= max(T7_FACE_TOL_ABSOLUTE, T7_FACE_TOL_RELATIVE * abs(throughput))
615         )
616         probe_status: int | None = int(probe_result.status)
617     else:
618         throughput_upper = None
619         throughput_unique = None
620         probe_status = int(probe_result.status)
621
622     note = ""
623     if not invariance_passed:
624         note = "primary_invariance_violated"
625     elif not weighted_sum_effective:
626         note = "weighted_sum_below_solver_resolution_lexicographic_committed"
627     if committed_solution != "lexicographic":
628         note = (note + ";" if note else "") + f"committed={committed_solution}"
629
630     if committed_solution == "lexicographic":
631         committed_seconds = lex_seconds
632     elif committed_solution == "weighted_sum_fallback":

```

```

633         committed_seconds = weighted_seconds
634     else:
635         committed_seconds = baseline_seconds
636     record = _layer_record(
637         result, committed_seconds, day=day, hour=hour,
638         a_eq=a_eq, b_eq=b_eq, a_ub=a_ub, b_ub=b_ub, bounds=bounds,
639         objective=primary_after,
640     )
641     tiebreak = TiebreakRecord(
642         layer=layer,
643         day=day,
644         hour=hour,
645         primary_before_yuan=primary_before,
646         primary_after_yuan=primary_after,
647         primary_relative_change=relative,
648         epsilon=epsilon,
649         shrinks=shrinks,
650         invariance_passed=invariance_passed,
651         throughput_kwh=throughput,
652         throughput_before_kwh=throughput_before,
653         throughput_reduced_kwh=throughput_before - throughput,
654         baseline_state_change_max_kwh=state_change,
655         degeneracy_degree=degeneracy_degree,
656         active_constraints=active_constraints,
657         variables=n_variables,
658         weighted_primary_relative_change=weighted_relative,
659         weighted_throughput_kwh=weighted_throughput,
660         weighted_sum_effective=weighted_sum_effective,
661         committed_solution=committed_solution,
662         throughput_upper_on_optimal_face_kwh=throughput_upper,
663         throughput_unique=throughput_unique,
664         probe_status=probe_status,
665         boundary_state_kwh=boundary_state,
666         baseline_seconds=baseline_seconds,
667         solver_seconds=total_seconds,
668         note=note,
669     )
670     return result, record, tiebreak
671
672
673 def solve_plan_layer(
674     *,
675     price: np.ndarray,
676     load_energy: np.ndarray,
677     pv_fc_energy: np.ndarray,
678     e_start: float,
679     time_limit_seconds: float,
680     day: int = -1,
681 ) -> tuple[dict[str, np.ndarray], LayerRecord, TiebreakRecord]:
682     """计划层 ``PL_d``:  $\min \sum p_i \cdot b_i + \sum E(c+q_{dis})$  (团队 T1 主口径 + T7-1 tie-breaking) 。"""
683     n = PERIODS_PER_DAY
684     size = 5 * n
685     objective = np.zeros(size)
686     objective[0:n] = price
687
688     bounds = np.empty((size, 2), dtype=float)
689     bounds[0:n, 0] = 0.0
690     bounds[0:n, 1] = np.inf
691     bounds[n : 2 * n, 0] = 0.0
692     bounds[n : 2 * n, 1] = C_CAP
693     bounds[2 * n : 3 * n, 0] = 0.0
694     bounds[2 * n : 3 * n, 1] = Q_CAP
695     bounds[3 * n : 4 * n, 0] = 0.0
696     bounds[3 * n : 4 * n, 1] = pv_fc_energy
697     bounds[4 * n : 5 * n, 0] = E_MIN
698     bounds[4 * n : 5 * n, 1] = E_MAX
699
700     tau = np.arange(n, dtype=np.int64)
701     rows: list[np.ndarray] = []
702     cols: list[np.ndarray] = []
703     vals: list[np.ndarray] = []
704     rows += [tau, tau, tau, tau]
705     cols += [tau, 2 * n + tau, n + tau, 3 * n + tau]
706     vals += [np.ones(n), np.ones(n), -np.ones(n), -np.ones(n)]
707     rows += [n + tau, n + tau, n + tau[1:], n + tau]
708     cols += [4 * n + tau, n + tau, 4 * n + tau[1:] - 1, 2 * n + tau]
709     vals += [np.ones(n), -ETA_CH * np.ones(n), -np.ones(n - 1), (1.0 / ETA_DIS) * np.ones(n)]
710     a_eq = coo_matrix(
711         (np.concatenate(vals), (np.concatenate(rows), np.concatenate(cols))),
712         shape=(2 * n, size),
713     ).tocsr()

```

```

714 b_eq = np.concatenate([load_energy - pv_fc_energy, np.concatenate([[e_start], np.zeros(n - 1)])])
715
716 throughput = throughput_coefficients(size, n, 2 * n, n)
717 result, record, tiebreak = _solve_with_tiebreak(
718     primary_objective=objective,
719     throughput_objective=throughput,
720     scale=throughput_scale(n),
721     a_eq=a_eq,
722     b_eq=b_eq,
723     a_ub=None,
724     b_ub=None,
725     bounds=bounds,
726     layer="plan",
727     day=day,
728     hour=0,
729     boundary_index=size - 1,
730     time_limit_seconds=time_limit_seconds,
731 )
732 if result.x is None:
733     return {}, record, tiebreak
734 x = np.asarray(result.x, dtype=float)
735 solution = {
736     "b": x[0:n],
737     "c": x[n : 2 * n],
738     "q_dis": x[2 * n : 3 * n],
739     "s": x[3 * n : 4 * n],
740     "E": x[4 * n : 5 * n],
741 }
742 return solution, record, tiebreak
743
744 def solve_adjustment_layer(
745     *,
746     hour: int,
747     price: np.ndarray,
748     load_energy: np.ndarray,
749     pv_fc_energy: np.ndarray,
750     plan_b: np.ndarray,
751     e_start: float,
752     time_limit_seconds: float,
753     day: int = -1,
754 ) -> tuple[dict[str, np.ndarray], LayerRecord, TiebreakRecord]:
755     """调整层 `` (AD_{d,m}) ``: ``min E_{i R_m} [ _def * p * u + _over * p * u ] + * E(c+q_dis)`` (AD-3/AD-5 + T7-1) 。
756
757     仅返回该层解: 调用方只提交 ``D_m`` (其余为前视, 会被后继层重优化)。
758     """
759     global_index = np.arange(6 * hour + 1, PERIODS_PER_DAY + 1)
760     n = int(global_index.shape[0])
761     size = 7 * n
762     objective = np.zeros(size)
763     objective[5 * n : 6 * n] = BETA_DEF * price[global_index - 1]
764     objective[6 * n : 7 * n] = BETA_OVER * price[global_index - 1]
765
766     bounds = np.empty((size, 2), dtype=float)
767     bounds[0:n, 0] = 0.0
768     bounds[0:n, 1] = np.inf
769     bounds[n : 2 * n, 0] = 0.0
770     bounds[n : 2 * n, 1] = C_CAP
771     bounds[2 * n : 3 * n, 0] = 0.0
772     bounds[2 * n : 3 * n, 1] = Q_CAP
773     bounds[3 * n : 4 * n, 0] = 0.0
774     bounds[3 * n : 4 * n, 1] = pv_fc_energy[global_index - 1]
775     bounds[4 * n : 5 * n, 0] = E_MIN
776     bounds[4 * n : 5 * n, 1] = E_MAX
777     bounds[5 * n : 6 * n, 0] = 0.0
778     bounds[5 * n : 6 * n, 1] = np.inf
779
780     tau = np.arange(n, dtype=np.int64)
781     rows: list[np.ndarray] = []
782     cols: list[np.ndarray] = []
783     vals: list[np.ndarray] = []
784     # 层内平衡: q + q_dis - c - s = L * Δt - Π_m * Δt
785     rows += [tau, tau, tau, tau]
786     cols += [tau, 2 * n + tau, n + tau, 3 * n + tau]
787     vals += [np.ones(n), np.ones(n), -np.ones(n), -np.ones(n)]
788     # 状态转移: E_j - E_{j-1} - c_j + q_dis_j / = 0, j = 1 时 RHS = E_{6m}
789     rows += [n + tau, n + tau, n + tau[1:]]
790     cols += [4 * n + tau, n + tau, 4 * n + tau[1:]]
791     vals += [np.ones(n), -ETA_CH * np.ones(n), -np.ones(n - 1), (1.0 / ETA_DIS) * np.ones(n)]
792     a_eq = coo_matrix(
793         (np.concatenate(vals), (np.concatenate(rows), np.concatenate(cols))),

```

```

795     shape=(2 * n, size),
796 ).tocsr()
797 b_eq = np.concatenate(
798     [load_energy[global_index - 1] - pv_fc_energy[global_index - 1], np.concatenate([[e_start], np.zeros(n - 1)])])
799 )
800
801 # 分段线性化 (AD-5, LP 无损): u   b - q (第 j 行); u   q - b (第 n+j 行)
802 plan_slice = plan_b[global_index - 1]
803 ub_rows: list[np.ndarray] = []
804 ub_cols: list[np.ndarray] = []
805 ub_vals: list[np.ndarray] = []
806 ub_rows += [tau, tau]
807 ub_cols += [tau, 5 * n + tau]
808 ub_vals += [-np.ones(n), -np.ones(n)]
809 ub_rows += [n + tau, n + tau]
810 ub_cols += [tau, 6 * n + tau]
811 ub_vals += [np.ones(n), -np.ones(n)]
812 a_ub = coo_matrix(
813     (np.concatenate(ub_vals), (np.concatenate(ub_rows), np.concatenate(ub_cols))),
814     shape=(2 * n, size),
815 ).tocsr()
816 b_ub = np.concatenate([-plan_slice, plan_slice])
817
818 throughput = throughput_coefficients(size, n, 2 * n, n)
819 result, record, tiebreak = _solve_with_tiebreak(
820     primary_objective=objective,
821     throughput_objective=throughput,
822     scale=throughput_scale(n),
823     a_eq=a_eq,
824     b_eq=b_eq,
825     a_ub=a_ub,
826     b_ub=b_ub,
827     bounds=bounds,
828     layer="adjustment",
829     day=day,
830     hour=hour,
831     boundary_index=5 * n - 1,
832     time_limit_seconds=time_limit_seconds,
833 )
834 if result.x is None:
835     return {}, record, tiebreak
836 x = np.asarray(result.x, dtype=float)
837 solution = {
838     "q": x[0:n],
839     "c": x[n : 2 * n],
840     "q_dis": x[2 * n : 3 * n],
841     "s": x[3 * n : 4 * n],
842     "E": x[4 * n : 5 * n],
843     "u_plus": x[5 * n : 6 * n],
844     "u_minus": x[6 * n : 7 * n],
845 }
846 return solution, record, tiebreak
847
848
849 def run_m1(data: ChainInputs, *, time_limit_seconds: float = 60.0, deadline: float | None = None) -> MiResult:
850     """按 ``formulation.md`` §3.10 执行 ``M1``: 逐日 4 次 LP + 闭式结算。
851
852     ``deadline`` 为 ``time.perf_counter()`` 口径的墙钟上限; 超限抛 :class:`BudgetExceeded`,
853     已完成的层证据由调用方写入 solver 日志 (不伪报完成)。
854     """
855     days = data.days
856     shape = (days, PERIODS_PER_DAY)
857     plan_b = np.zeros(shape)
858     plan_c = np.zeros(shape)
859     plan_q_dis = np.zeros(shape)
860     plan_s = np.zeros(shape)
861     plan_E = np.zeros(shape)
862     q = np.zeros(shape)
863     c = np.zeros(shape)
864     q_dis = np.zeros(shape)
865     state = np.zeros(shape)
866     q_em = np.zeros(shape)
867     s_settle = np.zeros(shape)
868     records: list[LayerRecord] = []
869     tiebreaks: list[TiebreakRecord] = []
870
871     pv0 = data.forecast_energy(0)
872     e_prev = E_INIT
873     for day in range(days):
874         if deadline is not None and time.perf_counter() > deadline:
875             raise BudgetExceeded(f"墙钟预算耗尽: 已完成 {day}/{days} 天")

```

```

876 plan, record, tiebreak = solve_plan_layer(
877     price=data.price[day],
878     load_energy=data.load_energy[day],
879     pv_fc_energy=pv0[day],
880     e_start=e_prev,
881     time_limit_seconds=time_limit_seconds,
882     day=day,
883 )
884 records.append(record)
885 tiebreaks.append(tiebreak)
886 if record.status != 0 or not plan:
887     raise LayerFailure(
888         f"计划层 LP 未达最优 (day={day}, status={record.status}): {record.message}",
889         day=day,
890         hour=0,
891         status=record.status,
892     )
893 plan_b[day] = plan["b"]
894 plan_c[day] = plan["c"]
895 plan_q_dis[day] = plan["q_dis"]
896 plan_s[day] = plan["s"]
897 plan_E[day] = plan["E"]
898 q[day, 0:36] = plan["b"][0:36]
899 c[day, 0:36] = plan["c"][0:36]
900 q_dis[day, 0:36] = plan["q_dis"][0:36]
901 state[day, 0:36] = plan["E"][0:36]
902
903 for hour in (6, 12, 18):
904     if deadline is not None and time.perf_counter() > deadline:
905         raise BudgetExceeded(f"墙钟预算耗尽: day={day}, hour={hour}")
906     pv_m = data.forecast_energy(hour)[day]
907     layer, record, tiebreak = solve_adjustment_layer(
908         hour=hour,
909         price=data.price[day],
910         load_energy=data.load_energy[day],
911         pv_fc_energy=pv_m,
912         plan_b=plan_b[day],
913         e_start=float(state[day, 6 * hour - 1]),
914         time_limit_seconds=time_limit_seconds,
915         day=day,
916     )
917     records.append(record)
918     tiebreaks.append(tiebreak)
919     if record.status != 0 or not layer:
920         raise LayerFailure(
921             f"调整层 LP 未达最优 (day={day}, hour={hour}, status={record.status}): {record.message}",
922             day=day,
923             hour=hour,
924             status=record.status,
925         )
926     commit = slice(6 * hour, 6 * hour + 36)
927     q[day, commit] = layer["q"][0:36]
928     c[day, commit] = layer["c"][0:36]
929     q_dis[day, commit] = layer["q_dis"][0:36]
930     state[day, commit] = layer["E"][0:36]
931
932 residual = q[day] + q_dis[day] + data.pv_act_energy[day] - data.load_energy[day] - c[day]
933 s_settle[day] = np.maximum(residual, 0.0)
934 q_em[day] = np.maximum(-residual, 0.0)
935 e_prev = float(state[day, -1])
936
937 return MIResult(
938     data=data,
939     plan_b=plan_b,
940     plan_c=plan_c,
941     plan_q_dis=plan_q_dis,
942     plan_s=plan_s,
943     plan_E=plan_E,
944     q=q,
945     c=c,
946     q_dis=q_dis,
947     E=state,
948     q_em=q_em,
949     s_settle=s_settle,
950     layer_records=records,
951     tiebreak_records=tiebreaks,
952 )
953
954
955 def state_starts(result: MIResult) -> np.ndarray:
956     """每日 0:00 储电量 (``E_{d,0}``: ``E_{1/1,0} = E_INIT``) 。"""

```

```

957     days = result.days
958     out = np.empty(days)
959     out[0] = E_INIT
960     if days > 1:
961         out[1:] = result.E[:-1, -1]
962     return out
963
964
965 def merge_interval_labels(mask: np.ndarray) -> list[tuple[int, int]]:
966     """把布尔掩码的连续 True 段合并为 ``(start, stop)`` 半开区间 (0 基时段下标)。"""
967     runs: list[tuple[int, int]] = []
968     start: int | None = None
969     for index, flag in enumerate(mask):
970         if flag and start is None:
971             start = index
972         elif not flag and start is not None:
973             runs.append((start, index))
974             start = None
975     if start is not None:
976         runs.append((start, len(mask)))
977     return runs
978
979
980 def _format_minutes(total_minutes: int) -> str:
981     """把「左端点分钟数」格式化为 ``H:MM``: 1440 时按 AS01 写 ``+1``。"""
982     if total_minutes < 1440:
983         return f"{total_minutes // 60}:{total_minutes % 60:02d}"
984     shifted = total_minutes - 1440
985     return f"{shifted // 60}:{shifted % 60:02d}+1"
986
987
988 def interval_label(labels: list[str], start: int, stop: int) -> str:
989     """按 AS01 生成 ``[_start+1, _stop+1 + 10min)`` 的区间标签 (左端点口径)。
990
991     团队勘误 R9 (2026-09-11): 两端**都用** ``_format_minutes`` 的 ``H:MM`` 风格 (小时不补零),
992     与附件 5 模板列标签风格 (如 ``9:50-10:00``) 一致。旧实现起点取自 ``labels[start]``,
993     而该标签由 ``strftime("%H:%M")`` 生成,**小时补零**, 导致同一单元格出现 ``06:10-9:20`` 这种
994     两侧风格不一致的写法。``labels`` 参数保留仅为签名兼容 (不再用于拼接)。
995     """
996     del labels # 仅签名兼容: 时间一律由分钟数推导, 避免两套风格混用
997     return f" {_format_minutes(10 * (start + 1))} - { _format_minutes(10 * (stop + 1))}"
998
999
1000 def emergency_intervals(
1001     q_em_day: np.ndarray, labels: list[str], *, tol: float = TOL
1002 ) -> list[dict[str, Any]]:
1003     """逐日紧急购电区间 (连续 10 分钟时段合并, D11-A/A18)。"""
1004     mask = np.asarray(q_em_day, dtype=float) > tol
1005     out: list[dict[str, Any]] = []
1006     for start, stop in merge_interval_labels(mask):
1007         out.append(
1008             {
1009                 "slot": interval_label(labels, start, stop),
1010                 "start_period": int(start + 1),
1011                 "stop_period": int(stop),
1012                 "energy_kwh": round(float(np.sum(q_em_day[start:stop])), 6),
1013             }
1014         )
1015     return out
1016
1017
1018 def _round(value: float, digits: int = 6) -> float:
1019     return round(float(value), digits)
1020
1021
1022 def evaluate(result: MiResult, *, full_horizon: bool = True) -> dict[str, Any]:
1023     """计算指标、结构恒等式 (I1d) - (I6d) (逐日 + 全期 + 交付期切片) 与全部硬约束检查。"""
1024     data = result.data
1025     days = result.days
1026     b = result.plan_b
1027     q = result.q
1028     c = result.c
1029     q_dis = result.q_dis
1030     q_em = result.q_em
1031     s_settle = result.s_settle
1032     pv_act = data.pv_act_energy
1033     load = data.load_energy
1034     price = data.price
1035     starts = state_starts(result)
1036     ends = result.E[:, -1]
1037

```

```

1038 dev_plus = np.maximum(b - q, 0.0)
1039 dev_minus = np.maximum(q - b, 0.0)
1040 cost_plan_d = np.sum(price * b, axis=1)
1041 cost_adj_d = np.sum(price * (BETA_DEF * dev_plus + BETA_OVER * dev_minus), axis=1)
1042 cost_em_d = np.sum(ALPHA_EM * price * q_em, axis=1)
1043 cost_total_d = cost_plan_d + cost_adj_d + cost_em_d
1044
1045 checks: list[dict[str, Any]] = []
1046
1047 def record(name: str, value: float, threshold: float, passed: bool, note: str = "") -> None:
1048     checks.append(
1049         {"name": name, "value": float(value), "threshold": float(threshold), "passed": bool(passed), "note": note}
1050     )
1051
1052 net_day = np.sum(load - pv_act, axis=1)
1053 i1_day = np.sum(q_dis, axis=1) - (ETA_ROUND_TRIP * np.sum(c, axis=1) - ETA_DIS * (ends - starts))
1054 i2_day = np.sum(q, axis=1) - (
1055     net_day
1056     + np.sum(s_settle, axis=1)
1057     + (1.0 - ETA_ROUND_TRIP) * np.sum(c, axis=1)
1058     + ETA_DIS * (ends - starts)
1059     - np.sum(q_em, axis=1)
1060 )
1061 transition = np.abs(
1062     np.diff(np.concatenate([starts[:, None], result.E], axis=1), axis=1)
1063     - ETA_CH * c
1064     + q_dis / ETA_DIS
1065 )
1066 settlement = np.abs(q + q_em + pv_act + q_dis - load - c - s_settle)
1067 continuity = np.abs(starts[1:] - ends[:-1]) if days > 1 else np.zeros(0)
1068
1069 # 层内余额 (提交链 vs 支配层预报, L3):  $s^i(i) = q + \Pi \Delta t + q_{dis} - L\Delta t - c$ 
1070 nu = dominance_map()
1071 forecast_energy_by_hour = {hour: data.forecast_energy(hour) for hour in DECISION_HOURS}
1072 layer_spill = np.empty((days, PERIODS_PER_DAY))
1073 for hour in DECISION_HOURS:
1074     cols = np.where(nu == hour)[0]
1075     layer_spill[:, cols] = (
1076         q[:, cols]
1077         + forecast_energy_by_hour[hour][:, cols]
1078         + q_dis[:, cols]
1079         - load[:, cols]
1080         - c[:, cols]
1081     )
1082 fc_upper = np.empty((days, PERIODS_PER_DAY))
1083 for hour in DECISION_HOURS:
1084     cols = np.where(nu == hour)[0]
1085     fc_upper[:, cols] = forecast_energy_by_hour[hour][:, cols]
1086 spill_upper_violation = float(np.max(layer_spill - fc_upper))
1087 spill_lower_violation = float(np.max(-layer_spill))
1088
1089 if result.layer_records:
1090     eq_max = max(item.equality_residual_max for item in result.layer_records)
1091     eq_rel_max = max(item.equality_residual_relative_max for item in result.layer_records)
1092     ineq_max = max(item.inequality_residual_max for item in result.layer_records)
1093     ineq_rel_max = max(item.inequality_residual_relative_max for item in result.layer_records)
1094     bound_max = max(item.bound_violation_max for item in result.layer_records)
1095     worst_layer = max(item.status for item in result.layer_records)
1096 else:
1097     eq_max, eq_rel_max, ineq_max, ineq_rel_max, bound_max, worst_layer = 0.0, 0.0, 0.0, 0.0, 0.0, 0
1098
1099 charge_power = np.maximum.reduce(
1100     [c / DELTA_T, ETA_CH * c / DELTA_T, q_dis / DELTA_T, q_dis / (ETA_DIS * DELTA_T)]
1101 )
1102 complementarity = float(np.sum(c * q_dis))
1103 same_period = int(np.sum((c > TOL) & (q_dis > TOL)))
1104 q_em_with_charge = int(np.sum((q_em > TOL) & (c > TOL)))
1105
1106 record("layer_status_max", float(worst_layer), 0.0, worst_layer == 0, "所有层 HiGHS status = 0 (Optimal)")
1107 record("layer_equality_residual_max", eq_max, LAYER_TOL, eq_max <= LAYER_TOL,
1108     "各层等式残差 (求解器内部绝对; AS『等式残差 1e-6』, HiGHS 可行性容差 1e-7)")
1109 record("layer_equality_residual_relative_max", eq_rel_max, LAYER_TOL_RELATIVE,
1110     eq_rel_max <= LAYER_TOL_RELATIVE,
1111     "各层等式残差 (尺度感知相对:  $\|A_{eq} x - b_{eq}\|_0 / (\|A_{eq}\|_0 \|x\|_0 + \|b_{eq}\|_0)$ )")
1112 record("layer_inequality_residual_max", ineq_max, LAYER_TOL, ineq_max <= LAYER_TOL,
1113     "各层不等式残差 (求解器内部绝对; HiGHS 可行性容差 1e-7)")
1114 record("layer_inequality_residual_relative_max", ineq_rel_max, LAYER_TOL_RELATIVE,
1115     ineq_rel_max <= LAYER_TOL_RELATIVE,
1116     "各层不等式残差 (尺度感知相对:  $\max(A_{ub} x - b_{ub}, 0) / (\|A_{ub}\|_0 \|x\|_0 + \|b_{ub}\|_0)$ )")
1117 record("layer_bound_violation_max", bound_max, TOL, bound_max <= TOL, "各层变量界越界量")
1118 record("cross_day_continuity", float(np.max(continuity)) if continuity.size else 0.0, TOL,

```

```

1119         (not continuity.size) or float(np.max(continuity)) <= TOL, "|E_{d,0} - E_{d-1,144}| (I4d)")
1120     record("state_transition_residual", float(np.max(transition)), TOL, float(np.max(transition)) <= TOL,
1121           "|E_i - E_{i-1} - 0.9c_i + q_dis_i/0.9| (I5d)")
1122     record("settlement_balance_residual", float(np.max(settlement)), TOL, float(np.max(settlement)) <= TOL,
1123           "(I6d) 结算层等式平衡")
1124     record("layer_spill_upper_violation", spill_upper_violation, BALANCE_TOL,
1125           spill_upper_violation <= BALANCE_TOL,
1126           "提交链层内弃光  $s_{\Pi}(i) \cdot \Delta t$  (L3; formulation $4.3 各层余额残差  $1e-8$ )")
1127     record("layer_spill_lower_violation", spill_lower_violation, BALANCE_TOL,
1128           spill_lower_violation <= BALANCE_TOL,
1129           "提交链层内弃光  $s_0$  (formulation $4.3 各层余额残差  $1e-8$ )")
1130     record("storage_upper_violation", max(float(np.max(result.E)) - E_MAX, 0.0), TOL,
1131           float(np.max(result.E)) <= E_MAX + TOL, "E 10800")
1132     record("storage_lower_violation", max(E_MIN - float(np.min(result.E)), 0.0), TOL,
1133           float(np.min(result.E)) >= E_MIN - TOL, "E 1200")
1134     record("charge_cap_violation", max(float(np.max(c)) - C_CAP, 0.0), TOL, float(np.max(c)) <= C_CAP + TOL,
1135           "c 833.3333 (并网点侧紧)")
1136     record("discharge_cap_violation", max(float(np.max(q_dis)) - Q_CAP, 0.0), TOL,
1137           float(np.max(q_dis)) <= Q_CAP + TOL, "q_dis 750.0000 (电池侧紧, D10 口径丙 / E1)")
1138     record("settle_spill_bound_violation", max(float(np.max(s_settle - pv_act)), 0.0), TOL,
1139           bool(np.all(s_settle <= pv_act + TOL)), "0 s' PV^act \cdot \Delta t (L3)")
1140     record("plan_nonneg_violation", max(-float(np.min(b)), 0.0), TOL, float(np.min(b)) >= -TOL, "b 0")
1141     record("purchase_nonneg_violation", max(-float(np.min(q)), 0.0), TOL, float(np.min(q)) >= -TOL, "q 0")
1142     record("q_em_nonneg_violation", max(-float(np.min(q_em)), 0.0), TOL, float(np.min(q_em)) >= -TOL, "q_em 0")
1143     record("q_eq_b_first_block", float(np.max(np.abs(q[:, 0:36] - b[:, 0:36]))), TOL,
1144           float(np.max(np.abs(q[:, 0:36] - b[:, 0:36])) <= TOL, "i 36 时 q = b (AD-1 替代量口径)")
1145     record("max_side_power_kw", float(np.max(charge_power)), P_MAX, float(np.max(charge_power)) <= P_MAX + 1e-3,
1146           "并网点侧与电池侧换算功率均 5000 kW")
1147     record("identity_I1_residual", float(np.max(np.abs(i1_day))), TOL, float(np.max(np.abs(i1_day))) <= TOL,
1148           "(I1d) Eq_dis = 'Ec - (E_T - E_0)")
1149     record("identity_I2_residual", float(np.max(np.abs(i2_day))), TOL, float(np.max(np.abs(i2_day))) <= TOL,
1150           "(I2d) Eq = N + Es' + 0.19Ec + \Delta E - Eq_em")
1151
1152     # ---- 团队 T7: 统一 tie-breaking 的逐层不变性与退化审计 (T7-1/T7-2/T7-3) ----
1153     t7_records = list(result.tiebreak_records)
1154     if t7_records:
1155         t7_rel_max = _finite_or(max(item.primary_relative_change for item in t7_records))
1156         t7_all_passed = bool(all(item.invariance_passed for item in t7_records))
1157         t7_shrinks = int(sum(item.shrinks for item in t7_records))
1158         t7_degenerate_layers = int(sum(1 for item in t7_records if item.degeneracy_degree > 0))
1159         t7_remaining_multiplicity = int(sum(1 for item in t7_records if item.throughput_unique is False))
1160         t7_probe_unknown = int(sum(1 for item in t7_records if item.throughput_unique is None))
1161         t7_throughput = _finite_or(float(np.sum([item.throughput_kwh for item in t7_records])))
1162         t7_throughput_before = _finite_or(float(np.sum([item.throughput_before_kwh for item in t7_records])))
1163         t7_state_change_max = _finite_or(
1164             float(np.max([item.baseline_state_change_max_kwh for item in t7_records]))
1165         )
1166         t7_epsilon_min = _finite_or(float(np.min([item.epsilon for item in t7_records])))
1167         t7_epsilon_max = _finite_or(float(np.max([item.epsilon for item in t7_records])))
1168         t7_max_degeneracy = int(max(item.degeneracy_degree for item in t7_records))
1169         t7_weighted_rel_max = _finite_or(
1170             max(item.weighted_primary_relative_change for item in t7_records)
1171         )
1172         t7_weighted_effective = int(sum(1 for item in t7_records if item.weighted_sum_effective))
1173         t7_lexicographic = int(sum(1 for item in t7_records if item.committed_solution == "lexicographic"))
1174         t7_fallbacks = int(sum(1 for item in t7_records if item.committed_solution != "lexicographic"))
1175     else:
1176         t7_rel_max, t7_all_passed, t7_shrinks = 0.0, True, 0
1177         t7_degenerate_layers = t7_remaining_multiplicity = t7_probe_unknown = 0
1178         t7_throughput = t7_throughput_before = t7_state_change_max = 0.0
1179         t7_epsilon_min = t7_epsilon_max = 0.0
1180         t7_max_degeneracy = 0
1181         t7_weighted_rel_max = 0.0
1182         t7_weighted_effective = t7_lexicographic = t7_fallbacks = 0
1183
1184     record("t7_primary_invariance_max_relative_change", t7_rel_max, T7_INVARIANCE_TOL,
1185           t7_rel_max <= T7_INVARIANCE_TOL,
1186           "T7-2: 加入次目标前后主目标相对变化 (逐层最大值, 须  $1e-9$ )")
1187     record("t7_invariance_all_layers_passed", float(int(not t7_all_passed)), 0.0, t7_all_passed,
1188           "T7-2: 全部逐层记录 primary_relative_change  $1e-9$  且 invariance_passed=true")
1189     record("t7_epsilon_below_primary_scale", t7_epsilon_max, 1.0, t7_epsilon_max < 1.0,
1190           "T7-1: 为无量纲相对权重 (1); 次目标总量  $1e-6 \cdot$  主目标尺度")
1191
1192     delivery = np.zeros(days, dtype=bool)
1193     delivery_start = min(DAILY_DELIVERY_START, days)
1194     delivery[delivery_start:] = True
1195     if delivery.any():
1196         q_d = q[delivery]
1197         c_d = c[delivery]
1198         q_em_d = q_em[delivery]
1199         s_d = s_settle[delivery]

```

```

1200     pv_d = pv_act[delivery]
1201     load_d = load[delivery]
1202     starts_d = starts[delivery_start]
1203     ends_d = float(ends[-1])
1204     net_d = float(np.sum(load_d - pv_d))
1205     i1_delivery = float(np.sum(q_dis[delivery]) - (ETA_ROUND_TRIP * np.sum(c_d) - ETA_DIS * (ends_d - starts_d)))
1206     i2_delivery = float(
1207         np.sum(q_d)
1208         - (
1209             net_d
1210             + float(np.sum(s_d))
1211             + (1.0 - ETA_ROUND_TRIP) * float(np.sum(c_d))
1212             + ETA_DIS * (ends_d - starts_d)
1213             - float(np.sum(q_em_d))
1214         )
1215     )
1216     record("identity_I1_delivery_residual", i1_delivery, TOL, abs(i1_delivery) <= TOL,
1217           "交付期切片 (I1)")
1218     record("identity_I2_delivery_residual", i2_delivery, TOL, abs(i2_delivery) <= TOL,
1219           "交付期切片 (I2), N_req + E_{2/1,0} - E_T")
1220 else:
1221     net_d = 0.0
1222     i1_delivery = 0.0
1223     i2_delivery = 0.0
1224
1225     cost_plan = float(np.sum(cost_plan_d))
1226     cost_adj = float(np.sum(cost_adj_d))
1227     cost_em = float(np.sum(cost_em_d))
1228     cost_total = cost_plan + cost_em + cost_adj
1229     cost_plan_delivery = float(np.sum(cost_plan_d[delivery])) if delivery.any() else 0.0
1230     cost_adj_delivery = float(np.sum(cost_adj_d[delivery])) if delivery.any() else 0.0
1231     cost_em_delivery = float(np.sum(cost_em_d[delivery])) if delivery.any() else 0.0
1232     cost_total_delivery = cost_plan_delivery + cost_adj_delivery + cost_em_delivery
1233     record("cost_decomposition_identity", cost_total - (cost_plan + cost_em + cost_adj), 1e-6,
1234           abs(cost_total - (cost_plan + cost_em + cost_adj)) <= 1e-6, "(R6d) 费用分解")
1235
1236     bounds: dict[str, Any] = {}
1237     if full_horizon and delivery.any():
1238         p_min = float(np.min(price))
1239         lower = p_min * (
1240             net_d
1241             + (1.0 - ETA_ROUND_TRIP) * float(np.sum(c[delivery]))
1242             + ETA_DIS * (ends_d - starts_d)
1243             - float(np.sum(q_em[delivery]))
1244         )
1245         # 无储能可行策略 (c = q_dis = 0) 的构造上界:
1246         # C_plan(opt)  E p * b_policy; 各调整层目标 该层 R_m 上的策略偏差费用 (策略对任意初态可行),
1247         # 故 C_adj  E_m policy_cost(R_m) (i 72 / 73..108 / 109..144 分别计入 3/2/1 次, 宽松但严格成立);
1248         # C_em  E 5p * q_em_policy.
1249         q_policy = np.empty_like(q)
1250         for hour in DECISION_HOURS:
1251             cols = np.where(nu == hour)[0]
1252             q_policy[:, cols] = np.maximum(load[:, cols] - forecast_energy_by_hour[hour][:, cols], 0.0)
1253             b_policy = np.maximum(load - data.forecast_energy(0), 0.0)
1254             r_policy = q_policy + pv_act - load
1255             q_em_policy = np.maximum(-r_policy, 0.0)
1256             dev_plus_policy = np.maximum(b_policy - q_policy, 0.0)
1257             dev_minus_policy = np.maximum(q_policy - b_policy, 0.0)
1258             policy_dev_cost = price * (BETA_DEF * dev_plus_policy + BETA_OVER * dev_minus_policy)
1259             plan_upper = float(np.sum(price[delivery] * b_policy[delivery]))
1260             adj_upper = 0.0
1261             for hour in (6, 12, 18):
1262                 rows = slice(6 * hour, PERIODS_PER_DAY)
1263                 adj_upper += float(np.sum(policy_dev_cost[delivery, rows]))
1264             em_upper = float(np.sum(ALPHA_EM * price[delivery] * q_em_policy[delivery]))
1265             upper = plan_upper + adj_upper + em_upper
1266             bounds = {
1267                 "delivery_lower_bound_yuan": _round(lower),
1268                 "delivery_lower_bound_formula": "p_min * [N_req + 0.19Ec + (E_T - E_{2/1,0}) - E_q_em]",
1269                 "delivery_constructed_upper_bound_yuan": _round(upper),
1270                 "delivery_upper_bound_parts_yuan": {
1271                     "plan": _round(plan_upper),
1272                     "adjustment": _round(adj_upper),
1273                     "emergency": _round(em_upper),
1274                 },
1275                 "delivery_upper_bound_formula": (
1276                     "无储能可行策略 (c = q_dis = 0, b/q 取该层预报净负荷): "
1277                     "E p * b_policy + E_m E_{R_m} 偏差费用 + E 5p * q_em_policy (C_adj 部分对重叠时段重复计入, 宽松但严格)"
1278                 ),
1279                 "delivery_formulation_reference_no_storage_actual_yuan": _round(
1280                     float(np.sum(price[delivery] * np.maximum((load - pv_act)[delivery], 0.0)))

```

```

1281         ),
1282         "mean_daily_delivery_cost_yuan": _round(cost_total_delivery / max(int(delivery.sum()), 1)),
1283     }
1284     record("analytic_lower_bound", cost_total_delivery, lower, cost_total_delivery >= lower - 1e-6,
1285           "交付期 C_total p_min * [N_req + 0.19Ec + ΔE - Σq_em]")
1286     record("analytic_upper_bound", cost_total_delivery, upper, cost_total_delivery <= upper + 1e-6,
1287           "交付期 C_total 无储能可行策略的构造上界 (C_adj 部分为重叠计的宽松界)")
1288     record("magnitude_delivery_band", cost_total_delivery, DELIVERY_MAGNITUDE_BAND_YUAN[1],
1289           DELIVERY_MAGNITUDE_BAND_YUAN[0] <= cost_total_delivery <= DELIVERY_MAGNITUDE_BAND_YUAN[1],
1290           "B3-5: 交付期费用须落在 106~108 元 (107 量级), 防 D9 的 Δt 误乘")
1291     mean_daily = cost_total_delivery / max(int(delivery.sum()), 1)
1292     record("magnitude_daily_band", mean_daily, DAILY_MAGNITUDE_BAND_YUAN[1],
1293           DAILY_MAGNITUDE_BAND_YUAN[0] <= mean_daily <= DAILY_MAGNITUDE_BAND_YUAN[1],
1294           "B3-5: 日均费用须为 104 元量级; 落入 103 元即 Δt 硬错误")
1295
1296     simultaneous_mask = (c > TOL) & (q_dis > TOL)
1297     statistics = {
1298         "complementarity_sum_kwh2": _round(complementarity),
1299         "simultaneous_charge_discharge_periods": same_period,
1300         "simultaneous_charge_kwh": _round(float(np.sum(c[simultaneous_mask]))),
1301         "simultaneous_discharge_kwh": _round(float(np.sum(q_dis[simultaneous_mask]))),
1302         "periods_with_q_em_and_charge": q_em_with_charge,
1303         "days_with_emergency": int(np.sum(np.any(q_em > TOL, axis=1))),
1304         "periods_with_emergency": int(np.sum(q_em > TOL)),
1305         "days_with_deviation": int(np.sum(np.any((dev_plus > TOL) | (dev_minus > TOL), axis=1))),
1306         "note": (
1307             "同充放属调整层目标 (仅偏差结算、无吞吐成本) 导致的 LP 退化, 不是硬约束违反;"
1308             "§5.1 的硬约束清单不含该项, §6.11 的【期望 0】是设计期望; 费用与全部恒等式不受影响。"
1309         ),
1310     }
1311
1312     series = {
1313         "period_index": list(range(1, days * PERIODS_PER_DAY + 1)),
1314         "day_index": [int(v) for v in np.repeat(np.arange(days), PERIODS_PER_DAY)],
1315         "price_yuan_per_kwh": [float(v) for v in price.reshape(-1)],
1316         "load_energy_kwh": [float(v) for v in load.reshape(-1)],
1317         "pv_actual_energy_kwh": [float(v) for v in pv_act.reshape(-1)],
1318         "plan_purchase_kwh": [float(v) for v in b.reshape(-1)],
1319         "final_purchase_kwh": [float(v) for v in q.reshape(-1)],
1320         "q_em_kwh": [float(v) for v in q_em.reshape(-1)],
1321         "charge_kwh": [float(v) for v in c.reshape(-1)],
1322         "discharge_kwh": [float(v) for v in q_dis.reshape(-1)],
1323         "spill_settlement_kwh": [float(v) for v in s_settle.reshape(-1)],
1324         "storage_kwh": [float(v) for v in result.E.reshape(-1)],
1325     }
1326
1327     return {
1328         "objective_yuan": _round(cost_total),
1329         "cost_plan_yuan": _round(cost_plan),
1330         "cost_adj_yuan": _round(cost_adj),
1331         "cost_em_yuan": _round(cost_em),
1332         "delivery": {
1333             "cost_total_yuan": _round(cost_total_delivery),
1334             "cost_plan_yuan": _round(cost_plan_delivery),
1335             "cost_adj_yuan": _round(cost_adj_delivery),
1336             "cost_em_yuan": _round(cost_em_delivery),
1337             "total_plan_kwh": _round(float(np.sum(b[delivery]))) if delivery.any() else 0.0,
1338             "total_purchase_kwh": _round(float(np.sum(q[delivery]))) if delivery.any() else 0.0,
1339             "total_q_em_kwh": _round(float(np.sum(q_em[delivery]))) if delivery.any() else 0.0,
1340             "total_charge_kwh": _round(float(np.sum(c[delivery]))) if delivery.any() else 0.0,
1341             "total_discharge_kwh": _round(float(np.sum(q_dis[delivery]))) if delivery.any() else 0.0,
1342             "total_spill_kwh": _round(float(np.sum(s_settle[delivery]))) if delivery.any() else 0.0,
1343             "net_load_kwh": _round(net_d),
1344             "storage_start_kwh": _round(starts[delivery_start]) if delivery.any() else 0.0,
1345             "storage_final_kwh": _round(ends_d) if delivery.any() else 0.0,
1346             "days": int(delivery.sum()),
1347         },
1348         "totals": {
1349             "total_plan_kwh": _round(float(np.sum(b))),
1350             "total_purchase_kwh": _round(float(np.sum(q))),
1351             "total_q_em_kwh": _round(float(np.sum(q_em))),
1352             "total_charge_kwh": _round(float(np.sum(c))),
1353             "total_discharge_kwh": _round(float(np.sum(q_dis))),
1354             "total_spill_kwh": _round(float(np.sum(s_settle))),
1355             "net_load_kwh": _round(float(np.sum(net_day))),
1356             "max_charge_kwh": _round(float(np.max(c))),
1357             "max_discharge_kwh": _round(float(np.max(q_dis))),
1358             "max_plan_purchase_kwh": _round(float(np.max(b))),
1359             "max_final_purchase_kwh": _round(float(np.max(q))),
1360             "max_side_power_kw": _round(float(np.max(charge_power))),
1361             "storage_initial_kwh": _round(E_INIT),
1362             "storage_final_kwh": _round(float(ends[-1])),
1363             "storage_range_kwh": [_round(float(np.min(result.E))), _round(float(np.max(result.E)))],

```

```

1362         "days_with_emergency": int(np.sum(np.any(q_em > TOL, axis=1))),
1363         "periods_with_emergency": int(np.sum(q_em > TOL)),
1364         "days_with_deviation": int(np.sum(np.any((dev_plus > TOL) | (dev_minus > TOL), axis=1))),
1365         "simultaneous_charge_discharge_periods": same_period,
1366         "periods_with_q_em_and_charge": q_em_with_charge,
1367     },
1368     "identities": {
1369         "I1_daily_max_abs": float(np.max(np.abs(i1_day))),
1370         "I2_daily_max_abs": float(np.max(np.abs(i2_day))),
1371         "I1_residual": float(np.sum(i1_day)),
1372         "I2_residual": float(np.sum(i2_day)),
1373         "I1_delivery_residual": float(i1_delivery),
1374         "I2_delivery_residual": float(i2_delivery),
1375         "continuity_residual_max": float(np.max(continuity)) if continuity.size else 0.0,
1376         "transition_residual_max": float(np.max(transition)),
1377         "settlement_residual_max": float(np.max(settlement)),
1378     },
1379     "bounds": bounds,
1380     "statistics": statistics,
1381     "t7_tiebreak": {
1382         "rule": T7_TIEBREAK_RULE,
1383         "eps_relative": T7_EPS_RELATIVE,
1384         "invariance_tol": T7_INVARIANCE_TOL,
1385         "shrink_factor": T7_SHRINK_FACTOR,
1386         "max_shrinks": T7_MAX_SHRINKS,
1387         "layers": len(t7_records),
1388         "max_primary_relative_change": t7_rel_max,
1389         "all_layers_invariance_passed": t7_all_passed,
1390         "total_shrinks": t7_shrinks,
1391         "epsilon_min": t7_epsilon_min,
1392         "epsilon_max": t7_epsilon_max,
1393         "weighted_sum_max_primary_relative_change": t7_weighted_rel_max,
1394         "weighted_sum_effective_layers": t7_weighted_effective,
1395         "lexicographic_committed_layers": t7_lexicographic,
1396         "fallback_committed_layers": t7_fallbacks,
1397         "throughput_primary_only_kwh": t7_throughput_before,
1398         "throughput_tiebreak_kwh": t7_throughput,
1399         "throughput_reduced_kwh": t7_throughput_before - t7_throughput,
1400         "max_baseline_trajectory_change_kwh": t7_state_change_max,
1401         "degenerate_layers": t7_degenerate_layers,
1402         "max_degeneracy_degree": t7_max_degeneracy,
1403         "layers_with_remaining_multiplicity": t7_remaining_multiplicity,
1404         "layers_with_unknown_multiplicity": t7_probe_unknown,
1405         "note": (
1406             "T7-1 次目标  $\Sigma(c+q_{dis})$  的 为无量纲相对权重; 提交解为 主目标最优面上的吞吐量最小解"
1407             " (T7-1 优先序在  $-0$  的精确实现), 的字面加权式作为诊断对照逐层落盘。"
1408             "T7-3 的 degeneracy_degree = 活跃约束数 - 变量数, layers_with_remaining_multiplicity 为 "
1409             "「主目标最优面上吞吐量上界 > 提交解吞吐量」的层数 (即仍有多重最优)。"
1410         ),
1411     },
1412     "daily": {
1413         "dates_index": list(range(days)),
1414         "plan_kwh": [_round(v) for v in np.sum(b, axis=1)],
1415         "purchase_kwh": [_round(v) for v in np.sum(q, axis=1)],
1416         "q_em_kwh": [_round(v) for v in np.sum(q_em, axis=1)],
1417         "charge_kwh": [_round(v) for v in np.sum(c, axis=1)],
1418         "discharge_kwh": [_round(v) for v in np.sum(q_dis, axis=1)],
1419         "spill_kwh": [_round(v) for v in np.sum(s_settle, axis=1)],
1420         "cost_plan_yuan": [_round(v) for v in cost_plan_d],
1421         "cost_adj_yuan": [_round(v) for v in cost_adj_d],
1422         "cost_em_yuan": [_round(v) for v in cost_em_d],
1423         "cost_total_yuan": [_round(v) for v in cost_total_d],
1424         "state_start_kwh": [_round(v) for v in starts],
1425         "state_end_kwh": [_round(v) for v in ends],
1426         "I1_residual": [_round(v) for v in i1_day],
1427         "I2_residual": [_round(v) for v in i2_day],
1428     },
1429     "checks": checks,
1430     "checks_failed": [item["name"] for item in checks if not item["passed"]],
1431     "series": series,
1432 }
1433
1434
1435 __all__ = [
1436     "ALPHA_EM",
1437     "BALANCE_TOL",
1438     "BETA_DEF",
1439     "BETA_OVER",
1440     "BLOCKS_PER_DAY",
1441     "BLOCK_ROWS",
1442     "BudgetExceeded",

```

```

1443     "C_CAP",
1444     "CROSS_YEAR_DROPPED",
1445     "ChainInputs",
1446     "DAILY_DELIVERY_START",
1447     "DAYS_FULL",
1448     "DECISION_HOURS",
1449     "DELTA_T",
1450     "E_CAP_MAX",
1451     "E_INIT",
1452     "E_MAX",
1453     "E_MIN",
1454     "ETA_CH",
1455     "ETA_DIS",
1456     "ETA_ROUND_TRIP",
1457     "LAYER_TOL",
1458     "LAYER_TOL_RELATIVE",
1459     "LayerFailure",
1460     "LayerRecord",
1461     "M1Result",
1462     "P_MAX",
1463     "PERIODS_PER_DAY",
1464     "Q_CAP",
1465     "SPEC_DATES",
1466     "TABLE1_SLOTS",
1467     "TOL",
1468     "T7_EPS_RELATIVE",
1469     "T7_INVARIANCE_TOL",
1470     "T7_MAX_SHRINKS",
1471     "T7_SHRINK_FACTOR",
1472     "T7_TIEBREAK_RULE",
1473     "TiebreakRecord",
1474     "cross_year_dropped_count",
1475     "dominance_map",
1476     "downscale",
1477     "emergency_intervals",
1478     "evaluate",
1479     "interval_label",
1480     "merge_interval_labels",
1481     "run_m1",
1482     "solve_adjustment_layer",
1483     "solve_plan_layer",
1484     "state_starts",
1485     "tiebreak_epsilon",
1486     "throughput_coefficients",
1487     "throughput_scale",
1488 ]

```

B.9 问题三：求解主程序

```

1  # -*- coding: utf-8 -*-
2  """prob03 计算入口：读取附件 1/2/3，执行 ``M1`` 三层顺序向前递推，写出 ``result3.xlsx`` 与追踪产物。
3
4  用法（正式任务，由 computation 阶段的 resource-manager 通过 ``scripts/compute_dispatcher.py`` 提交）：
5
6      <venv>/python run_prob03.py --data data/附件1.xlsx --data2 data/附件2.xlsx --data3 data/附件3.xlsx \
7          --template data/附件5/result3.xlsx \
8          --output problems/microgrid_2025/prob03/versions/assumption_v001/results/<run> \
9          --days 365 --time-limit 60
10
11  探针模式（``--days`` 小于 365）只用于接口自检：不写 ``result3.xlsx``，改写 ``probe_result3.xlsx``，
12  并跳过解析界与量级检查。完整计算必须由 Runner/tasks.py 创建隔离 task (supervised worker) 执行。
13
14  口径（不得逐问更改）：主口径 ``M1``（团队裁定 B0）；目标 ``C_total = C_plan + C_em + C_adj``（元，不带  $\Delta t$ , D9）；
15  ``c 833.3333``、``q_dis 750.0000``（D10 口径丙 / E1）、``E [1200,10800]``、``E_{1/1,0} = 6000``、终端自由；
16  统一 tie-breaking（团队 T7-1）：所有层一律 ``min [层主目标] + \cdot E_t(c+q_dis)``，逐层落盘 T7-2 不变性验证与
17  T7-3 退化/多重最优审计（``t7_tiebreak.json``）。
18  """
19  from __future__ import annotations
20
21  import argparse
22  import hashlib
23  import os
24  import platform
25  import sys
26  import time
27  from datetime import datetime, timezone
28  from pathlib import Path
29  from typing import Any
30

```

```

31 import numpy as np
32 from prob03_io import (
33     DECISION_HOURS,
34     PLAN_SHEET,
35     Attachment1,
36     InputValidationError,
37     delivery_dates,
38     fill_result3_workbook,
39     inspect_template,
40     read_attachment1,
41     read_attachment2,
42     read_attachment3,
43     write_json,
44 )
45 from prob03_model import (
46     ALPHA_EM,
47     BETA_DEF,
48     BETA_OVER,
49     BLOCKS_PER_DAY,
50     C_CAP,
51     DAILY_DELIVERY_START,
52     DAYS_FULL,
53     DELTA_T,
54     E_INIT,
55     E_MAX,
56     E_MIN,
57     ETA_CH,
58     ETA_DIS,
59     ETA_ROUND_TRIP,
60     P_MAX,
61     PERIODS_PER_DAY,
62     Q_CAP,
63     SPEC_DATES,
64     T7_EPS_RELATIVE,
65     T7_INVARIANCE_TOL,
66     T7_MAX_SHRINKS,
67     T7_SHRINK_FACTOR,
68     T7_TIEBREAK_RULE,
69     TABLE1_SLOTS,
70     BudgetExceeded,
71     ChainInputs,
72     LayerFailure,
73     M1Result,
74     cross_year_dropped_count,
75     downscale,
76     emergency_intervals,
77     evaluate,
78     run_m1,
79     state_starts,
80 )
81
82 PROJECT_MARKERS = ("pyproject.toml", "AGENTS.md")
83 DELIVERY_LAST_DAY = 364 # 0 基日索引: 2025-12-31
84 EXIT_OK = 0
85 EXIT_SOLVER_NOT_OPTIMAL = 2
86 EXIT_INPUT_INVALID = 3
87 EXIT_HARD_CHECK_FAILED = 4
88
89
90 def project_root() -> Path:
91     current = Path(__file__).resolve()
92     for candidate in (current, *current.parents):
93         if all((candidate / marker).exists() for marker in PROJECT_MARKERS):
94             return candidate
95     raise RuntimeError("无法从脚本位置定位项目根目录")
96
97
98 def resolve(path_text: str) -> Path:
99     candidate = Path(path_text)
100     return candidate if candidate.is_absolute() else (project_root() / candidate)
101
102
103 def code_fingerprint(code_dir: Path) -> dict[str, str]:
104     digest = hashlib.sha256()
105     files: dict[str, str] = {}
106     for path in sorted(code_dir.glob("*.py")):
107         payload = path.read_bytes()
108         files[path.name] = hashlib.sha256(payload).hexdigest()
109         digest.update(path.name.encode("utf-8"))
110         digest.update(payload)
111     files["__code_dir__"] = digest.hexdigest()

```

```

112     return files
113
114
115 def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
116     parser = argparse.ArgumentParser(
117         description=(
118             "prob03 计划/调整/结算三层顺序 LP 求解器 (assumption_v001 / formulation_v001, M1 主口径; "
119             "统一 tie-breaking = 团队 T7) "
120         )
121     )
122     parser.add_argument("--data", default="data/附件1.xlsx", help="附件 1 路径 (只读; 只用电价列)")
123     parser.add_argument("--data2", default="data/附件2.xlsx", help="附件 2 路径 (只读; 实际负载/光伏)")
124     parser.add_argument("--data3", default="data/附件3.xlsx", help="附件 3 路径 (只读; 整点光伏预报)")
125     parser.add_argument("--template", default="data/附件5/result3.xlsx", help="result3.xlsx 模板路径 (只读)")
126     parser.add_argument("--output", required=True, help="输出目录 (必须在 assumption_v001 版本目录内)")
127     parser.add_argument("--days", type=int, default=DAYS_FULL, help="优化天数: < 365 为探针模式")
128     parser.add_argument("--time-limit", type=float, default=60.0, help="**单层** LP 的 HiGS 时限 (秒)")
129     parser.add_argument("--max-wall-seconds", type=float, default=3300.0, help="整条链的墙钟预算 (秒)")
130     return parser.parse_args(argv)
131
132
133 def build_layer_inputs(attachment3: Any, days: int) -> tuple[dict[int, np.ndarray], dict[str, Any]]:
134     """按 AS05 生成各决策时刻的降尺度预报矩阵, 并记录层输入指纹与支配/跨年结构。"""
135     forecast: dict[int, np.ndarray] = {}
136     info: dict[str, Any] = {}
137     for hour in DECISION_HOURS:
138         matrix = np.vstack([downscale(attachment3.row(day, hour), hour) for day in range(days)])
139         forecast[hour] = matrix
140         finite = int(np.count_nonzero(np.isfinite(matrix)))
141         digest = hashlib.sha256(np.round(matrix[np.isfinite(matrix)], 6).tobytes()).hexdigest()
142         info[f"hour_{hour}"] = {
143             "dominated_periods_per_day": PERIODS_PER_DAY - 6 * hour,
144             "finite_cells": finite,
145             "nan_cells": int(matrix.size - finite),
146             "expected_nan_cells": 6 * hour * days,
147             "cross_year_dropped_items_per_day": cross_year_dropped_count(hour),
148             "forecast_matrix_sha256": digest,
149             "uses_actual_pv": False,
150         }
151     info["dominance_rule"] = (
152         "nu(i) = 6*floor((i-1)/36); D_0/D_6/D_12/D_18 各 36 时段 (i<=36 / 37..72 / 73..108 / 109..144) "
153     )
154     info["downscale_rule"] = (
155         "k=1 前向保持 A_{m,1}; k>=2 取 (1-j/6)A_{m,k-1} + (j/6)A_{m,k}; "
156         "块能量 (2.5A_{k-1}+3.5A_k)/6 (不保块内能量, T3) "
157     )
158     info["cross_year_dropped_total_per_day"] = sum(cross_year_dropped_count(h) for h in DECISION_HOURS)
159     info["actual_pv_scope"] = "settlement_layer_only ($0.4 F2: 决策层输入不含 PV^act) "
160     return forecast, info
161
162
163 def build_tables(
164     result: MlResult,
165     metrics: dict[str, Any],
166     attachment1: Attachment1,
167     template_info: dict[str, Any],
168     *,
169     probe_mode: bool,
170 ) -> dict[str, Any]:
171     """构造表 1/表 2/表 3 (4 个指定日期) 并按 AS01 校验位置口径。"""
172     labels = attachment1.time_labels
173     template_labels = template_info["plan_labels"]
174     days = result.days
175     q = result.q
176     c = result.c
177     q_dis = result.q_dis
178     q_em = result.q_em
179     daily = metrics["daily"]
180
181     table1: dict[str, Any] = {}
182     table2: dict[str, Any] = {}
183     table3: dict[str, Any] = {}
184     covered: list[str] = []
185     label_ok = True
186     for date_text, day_index in SPEC_DATES:
187         if day_index >= days:
188             continue
189         covered.append(date_text)
190         slots: list[dict[str, Any]] = []
191         for slot_name, position in TABLE1_SLOTS:
192             attachment_label = labels[position - 1]

```

```

193     template_label = template_labels[position - 1]
194     matches = attachment_label == slot_name.split("-")[0] and template_label == slot_name
195     label_ok = label_ok and matches
196     slots.append(
197         {
198             "slot": slot_name,
199             "position": position,
200             "attachment_timestamp": attachment_label,
201             "template_label": template_label,
202             "label_matches": bool(matches),
203             "final_purchase_kwh": round(float(q[day_index, position - 1]), 6),
204         }
205     )
206     table1[date_text] = {
207         "slots": slots,
208         "all_day_energy_kwh": daily["purchase_kwh"][day_index],
209         "all_day_cost_yuan": daily["cost_total_yuan"][day_index],
210         "all_day_plan_cost_yuan": daily["cost_plan_yuan"][day_index],
211         "all_day_adjust_cost_yuan": daily["cost_adj_yuan"][day_index],
212         "all_day_emergency_cost_yuan": daily["cost_em_yuan"][day_index],
213     }
214     table2[date_text] = {
215         "block_charge_kwh": [
216             round(float(np.sum(c[day_index, k * 24 : (k + 1) * 24])), 6) for k in range(BLOCKS_PER_DAY)
217         ],
218         "block_discharge_kwh": [
219             round(float(np.sum(q_dis[day_index, k * 24 : (k + 1) * 24])), 6) for k in range(BLOCKS_PER_DAY)
220         ],
221         "storage_0_00_kwh": daily["state_start_kwh"][day_index],
222         "storage_24_00_kwh": daily["state_end_kwh"][day_index],
223     }
224     intervals = emergency_intervals(q_em[day_index], labels)
225     table3[date_text] = {
226         "intervals": intervals,
227         "total_kwh": round(float(np.sum(q_em[day_index])), 6),
228         "paper_text": "无 (0 kWh) " if not intervals else "",
229     }
230     return {
231         "spec_dates": covered,
232         "table1": table1,
233         "table2": table2,
234         "table3": table3,
235         "table1_label_checks_passed": bool(label_ok),
236         "table3_note": (
237             "题目表 4 的示例日期 2025/3/1 与 3 个区间及其数值是**格式示例**，不是本题答案；"
238             "本问紧急购电由预报—实际光伏误差驱动（结算层），表 3 按 D11-A 合并连续时段并以左端点标签填报。"
239         ),
240         "table2_note": (
241             "表 2 的 0:00/24:00 为计划窗首/末状态（AS01 左端点口径使计划窗为 0:10-24:10，模板固有相位）；"
242             "prob03 为多日滚动切片，E_{d,0} = E_{d-1,144} 随日变化（团队勘误 R1），与 prob01 单日周期端点恒 6000 不同。"
243         ),
244         "probe_mode": probe_mode,
245     }
246
247
248 def build_t7_audit(result: M1Result, metrics: dict[str, Any], *, days: int) -> dict[str, Any]:
249     """团队 T7 的落盘审计：逐层不变性验证（T7-2）+ 退化维度与多重最优（T7-3）。"""
250     records = [item.as_dict() for item in result.tiebreak_records]
251     checks = [item for item in metrics["checks"] if item["name"].startswith("t7_")]
252     return {
253         "policy": "T7（团队裁定：统一冻结 tie-breaking，效力高于 B4-C5 初版措辞）",
254         "rule": T7_TIEBREAK_RULE,
255         "epsilon_definition": (
256             " = T7_EPS_RELATIVE * max(|该层主目标最优值|, 1) / [n * (c_cap + q_dis_cap)]"
257         ),
258         "eps_relative": T7_EPS_RELATIVE,
259         "invariance_tol": T7_INVARIANCE_TOL,
260         "shrink_factor": T7_SHRINK_FACTOR,
261         "max_shrinks": T7_MAX_SHRINKS,
262         "days": days,
263         "layers": len(records),
264         "plan_layers": int(sum(1 for item in result.tiebreak_records if item.layer == "plan")),
265         "adjustment_layers": int(sum(1 for item in result.tiebreak_records if item.layer == "adjustment")),
266         "summary": metrics["t7_tiebreak"],
267         "checks": checks,
268         "checks_failed": [item["name"] for item in checks if not item["passed"]],
269         "per_layer": records,
270         "m8_fairness_note": (
271             "T7-5: ablation 的 M2-M8（尤其 M8 独立第二实现）必须复用 prob03_model.solve_with_tiebreak / "
272             "tiebreak_epsilon / throughput_coefficients, tie-breaking 规则须与 M1 完全一致，"
273             "仅允许在 HiGHS 设置（presolve、消元顺序）上不同。"

```

```

274     ),
275 }
276
277
278 def main(argv: list[str] | None = None) -> int:
279     args = parse_args(argv)
280     data_path = resolve(args.data)
281     data2_path = resolve(args.data2)
282     data3_path = resolve(args.data3)
283     template_path = resolve(args.template)
284     output_dir = resolve(args.output)
285     code_dir = Path(__file__).resolve().parent
286     probe_mode = args.days < DAYS_FULL
287
288     if not 2 <= args.days <= DAYS_FULL:
289         print(f"[prob03] --days 必须在 [2, {DAYS_FULL}], 收到 {args.days}", file=sys.stderr)
290         return EXIT_INPUT_INVALID
291     days = args.days
292
293     try:
294         attachment1 = read_attachment1(data_path, expected_rows=PERIODS_PER_DAY)
295         attachment2 = read_attachment2(data2_path, expected_days=DAYS_FULL, expected_periods=PERIODS_PER_DAY)
296         attachment3 = read_attachment3(data3_path, expected_days=DAYS_FULL)
297         template_info = inspect_template(template_path)
298     except InputValidationError as exc:
299         print(f"[prob03] 输入校验失败: {exc}", file=sys.stderr)
300         return EXIT_INPUT_INVALID
301
302     if attachment3.dates != attachment2.dates:
303         print(f"[prob03] 附件 3 与附件 2 的日期序列不一致", file=sys.stderr)
304         return EXIT_INPUT_INVALID
305     delivery_days_expected = max(DELIVERY_LAST_DAY + 1 - DAILY_DELIVERY_START, 0)
306     if template_info["plan_rows"] < delivery_days_expected:
307         print(
308             f"[prob03] 模板 [{PLAN_SHEET}] 只有 {template_info['plan_rows']} 天, "
309             f"少于交付期 {delivery_days_expected} 天",
310             file=sys.stderr,
311         )
312         return EXIT_INPUT_INVALID
313
314     forecast, layer_inputs = build_layer_inputs(attachment3, days)
315     for hour in DECISION_HOURS:
316         info = layer_inputs[f"hour_{hour}"]
317         if info["nan_cells"] != info["expected_nan_cells"]:
318             print(f"[prob03] 降尺度覆盖域异常: hour={hour} nan={info['nan_cells']}", file=sys.stderr)
319             return EXIT_INPUT_INVALID
320         dominated = np.where(np.isfinite(forecast[hour][0]))[0]
321         if not np.isfinite(forecast[hour][:, dominated]).all():
322             print(f"[prob03] 降尺度预报在支配域含 NaN: hour={hour}", file=sys.stderr)
323             return EXIT_INPUT_INVALID
324
325     chain = ChainInputs(
326         days=days,
327         price=np.tile(attachment1.price, (days, 1)),
328         load_energy=attachment2.load_kw[:days] * DELTA_T,
329         pv_act_energy=attachment2.pv_kw[:days] * DELTA_T,
330         forecast_kw=forecast,
331     )
332     run_meta = {
333         "problem_id": "microgrid_2025",
334         "question_id": "prob03",
335         "assumption_version": "assumption_v001",
336         "formulation_version": "formulation_v001",
337         "stage": "computation",
338         "model": "M1",
339         "probe_mode": probe_mode,
340         "days": days,
341         "periods": days * PERIODS_PER_DAY,
342         "created_at": datetime.now(timezone.utc).isoformat(),
343         "task_id": os.environ.get("AUTOMM_TASK_ID"),
344         "output_directory": str(output_dir),
345         "device": "cpu",
346         "gpu_required": False,
347         "seed": None,
348         "deterministic": True,
349         "solver": "scipy.optimize.linprog(method='highs')",
350         "layer_time_limit_seconds": args.time_limit,
351         "max_wall_seconds": args.max_wall_seconds,
352         "input": {
353             "attachment1": str(data_path),
354             "attachment1_md5": attachment1.md5,

```

```

355         "attachment1_periods": attachment1.periods,
356         "attachment2": str(data2_path),
357         "attachment2_md5": attachment2.md5,
358         "attachment2_days": attachment2.days,
359         "attachment3": str(data3_path),
360         "attachment3_md5": attachment3.md5,
361         "attachment3_days": attachment3.days,
362         "template": str(template_path),
363         "template_md5": template_info["md5"],
364         "template_plan_rows": template_info["plan_rows"],
365     },
366     "parameters": {
367         "delta_t_h": DELTA_T,
368         "eta_ch": ETA_CH,
369         "eta_dis": ETA_DIS,
370         "eta_round_trip": ETA_ROUND_TRIP,
371         "alpha_em": ALPHA_EM,
372         "beta_def": BETA_DEF,
373         "beta_over": BETA_OVER,
374         "e_init_kwh": E_INIT,
375         "e_min_kwh": E_MIN,
376         "e_max_kwh": E_MAX,
377         "e_terminal": None,
378         "p_max_kw": P_MAX,
379         "c_cap_kwh": C_CAP,
380         "q_dis_cap_kwh": Q_CAP,
381         "b_upper_kwh": None,
382         "decision_hours": list(DECISION_HOURS),
383     },
384     "layer_inputs": layer_inputs,
385     "code_sha256": code_fingerprint(code_dir),
386     "environment": {
387         "python": sys.version.split()[0],
388         "numpy": np.__version__,
389         "platform": platform.platform(),
390     },
391 }
392 # 每层名义 LP 次数 (T7 四段式, 与 implementation.md §2.1 / task_spec.yaml 的 5,840 一致):
393 # 纯主目标基线 (T7-2 的「加入前」) T7-1 字面加权式 字典序提交解 (主目标最优面上最小吞吐量)
394 # T7-3 最优面退化探测; T7-2 每触发一次 收缩另加 1 次 (有界于 T7_MAX_SHRINKS=8)。
395 solver_calls_per_layer = 4
396 matrix_scale = {
397     "plan_layer": {"variables": 720, "equality_constraints": 288, "inequalities": 0,
398                   "solver_calls_nominal": solver_calls_per_layer},
399     "adjustment_layer_m6": {"variables": 756, "equality_constraints": 216, "inequalities": 216,
400                             "solver_calls_nominal": solver_calls_per_layer},
401     "adjustment_layer_m12": {"variables": 504, "equality_constraints": 144, "inequalities": 144,
402                              "solver_calls_nominal": solver_calls_per_layer},
403     "adjustment_layer_m18": {"variables": 252, "equality_constraints": 72, "inequalities": 72,
404                              "solver_calls_nominal": solver_calls_per_layer},
405     "per_day": {"variables": 2232, "equality_constraints": 720, "inequalities": 432,
406                "layer_solves": 4, "solver_calls_nominal": 4 * solver_calls_per_layer},
407     "lp_calls_total": 4 * days,
408     "lp_solver_invocations_nominal": 4 * days * solver_calls_per_layer,
409     "tiebreak_solver_calls_per_layer_nominal": solver_calls_per_layer,
410     "tiebreak_extra_calls_note": (
411         "T7-2 不变性不满足时每个缩小一次 会额外增加一次 LP (有界于 T7_MAX_SHRINKS); "
412         "实际调用次数见 run_manifest.t7_tiebreak.total_shrinks。"
413     ),
414     "integer_variables_main": 0,
415 }
416
417 started = time.perf_counter()
418 try:
419     result = run_m1(chain, time_limit_seconds=args.time_limit, deadline=started + args.max_wall_seconds)
420 except (LayerFailure, BudgetExceeded) as exc:
421     status = getattr(exc, "status", -1)
422     write_json(
423         output_dir / "solver_status.json",
424         {
425             "status": int(status),
426             "message": str(exc),
427             "feasible_incumbent": False,
428             "model": "M1",
429             "device": "cpu",
430             "gpu_required": False,
431             "seed": None,
432             "seconds": time.perf_counter() - started,
433         },
434     )
435 write_json(

```

```

436         output_dir / "run_manifest.json",
437         {**run_meta, "outcome": "solver_not_optimal", "matrix_scale": matrix_scale},
438     )
439     print(f"[prob03] 求解未达最优: {exc}", file=sys.stderr)
440     return EXIT_SOLVER_NOT_OPTIMAL
441     elapsed = time.perf_counter() - started
442
443     metrics = evaluate(result, full_horizon=not probe_mode)
444     tables = build_tables(result, metrics, attachment1, template_info, probe_mode=probe_mode)
445     t7_audit = build_t7_audit(result, metrics, days=days)
446
447     write_json(
448         output_dir / "solver_status.json",
449         {
450             "status": 0,
451             "message": "all layers optimal",
452             "solver": "highs",
453             "model": "M1",
454             "objective_yuan": metrics["objective_yuan"],
455             "delivery_cost_yuan": metrics["delivery"]["cost_total_yuan"],
456             "feasible_incumbent": True,
457             "mip_gap": None,
458             "device": "cpu",
459             "gpu_required": False,
460             "seed": None,
461             "lp_calls": len(result.layer_records),
462             "lp_solver_invocations_nominal": matrix_scale["lp_solver_invocations_nominal"],
463             "t7_tiebreak": t7_audit["summary"],
464             "layer_status_max": max((item.status for item in result.layer_records), default=0),
465             "layer_equality_residual_max": max(
466                 (item.equality_residual_max for item in result.layer_records), default=0.0
467             ),
468             "layer_equality_residual_relative_max": max(
469                 (item.equality_residual_relative_max for item in result.layer_records), default=0.0
470             ),
471             "layer_inequality_residual_max": max(
472                 (item.inequality_residual_max for item in result.layer_records), default=0.0
473             ),
474             "layer_inequality_residual_relative_max": max(
475                 (item.inequality_residual_relative_max for item in result.layer_records), default=0.0
476             ),
477             "layer_bound_violation_max": max(
478                 (item.bound_violation_max for item in result.layer_records), default=0.0
479             ),
480             "layer_seconds_total": (
481                 float(np.sum([item.seconds for item in result.layer_records])) if result.layer_records else 0.0
482             ),
483             "wall_seconds": elapsed,
484             "deterministic": True,
485         },
486     )
487     write_json(
488         output_dir / "solution.json",
489         {
490             "objective_yuan": metrics["objective_yuan"],
491             "cost_plan_yuan": metrics["cost_plan_yuan"],
492             "cost_adj_yuan": metrics["cost_adj_yuan"],
493             "cost_em_yuan": metrics["cost_em_yuan"],
494             "delivery": metrics["delivery"],
495             "totals": metrics["totals"],
496             "identities": metrics["identities"],
497             "bounds": metrics["bounds"],
498             "statistics": metrics["statistics"],
499             "daily": metrics["daily"],
500             "checks": metrics["checks"],
501             "checks_failed": metrics["checks_failed"],
502             "t7_tiebreak": t7_audit["summary"],
503             "layer_summary": [item.as_dict() for item in result.layer_records],
504             "series": metrics["series"],
505         },
506     )
507     write_json(output_dir / "tables.json", tables)
508     write_json(output_dir / "t7_tiebreak.json", t7_audit)
509
510     delivery_dates_list = delivery_dates(attachment2.dates, DAILY_DELIVERY_START)[max(days - DAILY_DELIVERY_START, 0)]
511     workbook_name = "probe_result3.xlsx" if probe_mode else "result3.xlsx"
512     workbook_info: dict[str, Any] | None = None
513     if delivery_dates_list:
514         selected = slice(DAILY_DELIVERY_START, days)
515         b = result.plan_b[selected]
516         q = result.q[selected]

```

```

517     c = result.c[selected]
518     q_dis = result.q_dis[selected]
519     q_em_d = result.q_em[selected]
520     starts = state_starts(result)[selected]
521     ends = result.E[:, -1][selected]
522     price_raw = chain.price[selected]
523     dev_plus = np.maximum(b - q, 0.0)
524     dev_minus = np.maximum(q - b, 0.0)
525     cost_total_d = (
526         np.sum(price_raw * b, axis=1)
527         + np.sum(price_raw * (BETA_DEF * dev_plus + BETA_OVER * dev_minus), axis=1)
528         + np.sum(ALPHA_EM * price_raw * q_em_d, axis=1)
529     )
530     workbook_info = fill_result3_workbook(
531         template=template_path,
532         dest=output_dir / workbook_name,
533         dates=delivery_dates_list,
534         planned_kwh=b,
535         adjusted_kwh=q,
536         charge_kwh=c,
537         discharge_kwh=q_dis,
538         state_start_kwh=starts,
539         state_end_kwh=ends,
540         day_energy_planned_kwh=np.sum(b, axis=1),
541         day_energy_adjusted_kwh=np.sum(q, axis=1),
542         day_cost_yuan=cost_total_d,
543         emergency_intervals=[
544             [
545                 (item["slot"], item["energy_kwh"])
546                 for item in emergency_intervals(q_em_d[index], attachment1.time_labels)
547             ]
548             for index in range(len(delivery_dates_list))
549         ],
550     )
551
552     failed = list(metrics["checks_failed"])
553     if not tables["table1_label_checks_passed"]:
554         failed.append("table1_label_checks")
555     if workbook_info is not None and workbook_info.get("format_residuals"):
556         failed.append("result3_format_residuals")
557     write_json(
558         output_dir / "run_manifest.json",
559         {
560             **run_meta,
561             "outcome": "hard_check_failed" if failed else "completed",
562             "matrix_scale": matrix_scale,
563             "lp_calls": len(result.layer_records),
564             "lp_solver_invocations_nominal": matrix_scale["lp_solver_invocations_nominal"],
565             "t7_tiebreak": t7_audit["summary"],
566             "t7_tiebreak_artifact": "t7_tiebreak.json",
567             "wall_seconds": elapsed,
568             "workbook": workbook_name if delivery_dates_list else None,
569             "workbook_info": workbook_info,
570             "delivery_days": len(delivery_dates_list),
571             "checks_failed": failed,
572             "statistics": metrics["statistics"],
573             "objective_yuan": metrics["objective_yuan"],
574             "cost_plan_yuan": metrics["cost_plan_yuan"],
575             "cost_adj_yuan": metrics["cost_adj_yuan"],
576             "cost_em_yuan": metrics["cost_em_yuan"],
577             "delivery_cost_yuan": metrics["delivery"]["cost_total_yuan"],
578             "total_purchase_kwh": metrics["delivery"]["total_purchase_kwh"],
579             "total_q_em_kwh": metrics["delivery"]["total_q_em_kwh"],
580             "total_spill_kwh": metrics["delivery"]["total_spill_kwh"],
581             "storage_final_kwh": metrics["totals"]["storage_final_kwh"],
582         },
583     )
584
585     print(
586         f"[prob03] {'探针' if probe_mode else '正式'}运行完成: days={days} periods={days * PERIODS_PER_DAY} "
587         f"LP={len(result.layer_records)} C_total={metrics['objective_yuan']:.4f} 元, "
588         f"交付期={metrics['delivery']['cost_total_yuan']:.4f} 元, "
589         f"Db={metrics['totals']['total_plan_kwh']:.4f} kWh, Eq={metrics['totals']['total_purchase_kwh']:.4f} kWh, "
590         f"Eq_em={metrics['totals']['total_q_em_kwh']:.6f} kWh, Es={metrics['totals']['total_spill_kwh']:.4f} kWh, "
591         f"T7主目标最大相对变化={metrics['t7_tiebreak']['max_primary_relative_change']:.3e}"
592         f" (阈值 {T7_INVARIANCE_TOL:.0e}, 全部通过={metrics['t7_tiebreak']['all_layers_invariance_passed']}) , "
593         f"吞吐量 {metrics['t7_tiebreak']['throughput_primary_only_kwh']:.4f} kWh"
594         f"{metrics['t7_tiebreak']['throughput_tiebreak_kwh']:.4f} kWh, "
595         f"失败检查={failed}"
596     )
597     if failed:

```

```

598         print(f"[prob03] 硬检查未通过: {failed}", file=sys.stderr)
599         return EXIT_HARD_CHECK_FAILED
600     return EXIT_OK
601
602
603 if __name__ == "__main__":
604     raise SystemExit(main())

```

B.10 问题四：输入输出模块

```

1  # -*- coding: utf-8 -*-
2  """prob04 输入读取与 ``result4-2.xlsx`` / ``result4-3.xlsx`` 填报（两链共用）。
3
4  只读 ``data/附件2.xlsx``（实际负载/光伏）、``data/附件3.xlsx``（整点预报，仅 4-3）、
5  ``data/附件4.xlsx``（实时电价，两链唯一价格来源）与模板 ``data/附件5/result4-2.xlsx`` /
6  ``data/附件5/result4-3.xlsx``；不修改任何原始数据。
7
8  口径来源：prob04 ``assumption_v001``（AS01-AS23 + 团队 ``A8-(b)`` 六条派生）与
9  ``formulations/formulation_v001``（$1.1-$1.5、$8.1-$8.3）。
10
11  纪律（AS02 / $8.7 黑名单）：
12  * **附件 1 不使用**（其电价被附件 4 取代，其负载/光伏预测列属 prob01 语义）；
13  * 决策层价格一律来自「历史实际价 + 预测器」（``\hat{p}``，见 ``prob04_predict``），**不是**当天实际价；
14    本模块只负责读取与填报，不构造任何决策价；
15  * 附件 3 的 ``日期`` 列只在每日 ``0:00`` 行出现（1095/1460 行为空），实现按**前向填充**解析，
16    并把 ``2025-1-1`` 规范为 ``2025-01-01``；
17  * 两个交付工作簿**必须新写**入独立 ``output_directory``；模板只读打开。
18  """
19  from __future__ import annotations
20
21  import hashlib
22  import json
23  from dataclasses import dataclass
24  from datetime import datetime
25  from pathlib import Path
26  from typing import Any, Sequence
27
28  import numpy as np
29  import openpyxl
30  from openpyxl.utils import get_column_letter
31
32  LOAD_SHEET = "小区负载"
33  PV_SHEET = "光伏发电实际功率"
34  PLAN_SHEET = "计划购电量"
35  ADJUST_SHEET = "调整购电量"
36  STORAGE_SHEET = "充放电"
37  EMERGENCY_SHEET = "紧急购电量"
38  PLAN_HEADER_FIRST = "日期\\时间"
39  PLAN_HEADER_ENERGY = "全天购电量"
40  PLAN_HEADER_COST = "全天购电费"
41  STORAGE_HEADER = ("日期", "时间段", "充电量", "放电量", "时刻", "储电量")
42  EMERGENCY_HEADER = ("日期", "购电时间段", "购电量")
43  BLOCK_ROWS = 24
44  BLOCKS_PER_DAY = 6
45  PERIODS_PER_DAY = 144
46  DAYS_FULL = 365
47  ZERO_INTERVAL_TEXT = "--"
48  POWER_DATE_HEADER = "日期\\时间"
49  PRICE_SHEET = "Sheet1"
50  DECISION_HOURS: tuple[int, ...] = (0, 6, 12, 18)
51  FC_HORIZON = 24
52  # 交付数值的有效位（承 prob03 勘误 R10）：求解器解带 1e-10 级噪声，交付一律按 6 位小数写入。
53  DELIVERY_DECIMALS = 6
54  STATE_MIN_KWH = 1200.0
55  STATE_MAX_KWH = 10800.0
56
57
58  class InputValidationError(RuntimeError):
59      """输入结构与 accepted 假设不一致时严格失败。"""
60
61
62  @dataclass(frozen=True)
63  class Attachment2:
64      """附件 2 的逐日实际负载/光伏功率（365 天 × 144 个 10 分钟点，单位 kW）。"""
65
66      load_kw: np.ndarray
67      pv_kw: np.ndarray
68      dates: list[str]
69      time_labels: list[str]

```

```

70     md5: str
71
72     @property
73     def days(self) -> int:
74         return int(self.load_kw.shape[0])
75
76     @property
77     def periods_per_day(self) -> int:
78         return int(self.load_kw.shape[1])
79
80
81 @dataclass(frozen=True)
82 class Attachment3:
83     """附件 3 的整点预报表 ``A[d, s, k]`` (kW; ``s`` 为 ``DECISION_HOURS`` 下标, ``k = 1..24``) 。"""
84
85     fc_kw: np.ndarray          # (days, 4, 24)
86     dates: list[str]
87     slots: list[str]
88     md5: str
89
90     @property
91     def days(self) -> int:
92         return int(self.fc_kw.shape[0])
93
94     def row(self, day: int, hour: int) -> np.ndarray:
95         """返回第 ``day`` 天、发布时刻 ``hour`` 的 24 维整点预报向量。"""
96         if hour not in DECISION_HOURS:
97             raise ValueError(f"未知发布时刻: {hour}")
98         return self.fc_kw[day, DECISION_HOURS.index(hour), :]
99
100
101 @dataclass(frozen=True)
102 class Attachment4:
103     """附件 4 的逐日逐时段**实际**实时电价 (365 天 × 144 个 10 分钟点, 元/kWh) 。”"""
104
105     price: np.ndarray
106     dates: list[str]
107     time_labels: list[str]
108     header: list[str]
109     md5: str
110
111     @property
112     def days(self) -> int:
113         return int(self.price.shape[0])
114
115     @property
116     def periods_per_day(self) -> int:
117         return int(self.price.shape[1])
118
119
120 def md5_of(path: Path) -> str:
121     digest = hashlib.md5()
122     with path.open("rb") as handle:
123         for chunk in iter(lambda: handle.read(1 << 20), b""):
124             digest.update(chunk)
125     return digest.hexdigest()
126
127
128 def _clock_label(value: Any) -> str:
129     """把附件时间戳格式化为 ``H:MM`` (小时不补零, 与 AS01 左端点口径/模板列标签风格一致) 。
130
131     ``openpyxl`` 把附件 2/附件 4 的表头时刻读成 ``datetime.time`` , 其 ``str()`` 会带秒
132     (``datetime.time(10, 0)`` + ``"10:00:00"``) , 与 ``AS01`` 的 ``H:MM`` (``"10:00"``) 不同;
133     本函数统一为 ``H:MM`` (字符串原样 strip 返回, 保留 ``"0:00+1"`` 一类模板写法) 。
134     """
135     if value is None:
136         return ""
137     if hasattr(value, "strftime"):
138         return f"{int(value.hour)}:{int(value.minute):02d}"
139     return str(value).strip()
140
141
142 def _date_label(value: Any) -> str:
143     if value is None:
144         return ""
145     if hasattr(value, "strftime"):
146         return value.strftime("%Y-%m-%d")
147     return str(value).strip()
148
149
150 def canonical_date(value: Any) -> str:

```

```

151     """把附件 3 的日期 (``2025-1-1`` 字符串或 datetime) 规范为 ``YYYY-MM-DD``; 空白返回空串。"""
152     if value is None:
153         return ""
154     if hasattr(value, "strftime"):
155         return value.strftime("%Y-%m-%d")
156     text = str(value).strip()
157     if not text:
158         return ""
159     parts = text.split("-") if "-" in text else text.split("/")
160     if len(parts) == 3:
161         try:
162             return f"{int(parts[0]):04d}-{int(parts[1]):02d}-{int(parts[2]):02d}"
163         except ValueError:
164             return text
165     return text
166
167
168 def _slot_hour(value: Any) -> int:
169     """把附件 3 的 ``预报时刻`` 解析为整点小时数 (``0:00`` + 0)。"""
170     if value is None:
171         raise InputValidationError("附件 3 存在空的预报时刻")
172     if hasattr(value, "strftime"):
173         return int(value.strftime("%H"))
174     text = str(value).strip()
175     if not text:
176         raise InputValidationError("附件 3 存在空的预报时刻")
177     head = text.split(":")[0].strip()
178     try:
179         return int(head)
180     except ValueError as exc:
181         raise InputValidationError(f"附件 3 预报时刻无法解析: {value!r}") from exc
182
183
184 def _validate_dates(dates: Sequence[str], *, source: str) -> None:
185     try:
186         parsed = [datetime.strptime(item, "%Y-%m-%d").date() for item in dates]
187     except ValueError as exc:
188         raise InputValidationError(f"{source}日期列无法解析: {exc}") from exc
189     for index in range(1, len(parsed)):
190         if (parsed[index] - parsed[index - 1]).days != 1:
191             raise InputValidationError(f"{source}日期不连续: {dates[index - 1]} -> {dates[index]}")
192     if parsed[0].isoformat() != "2025-01-01":
193         raise InputValidationError(f"{source}首日不是 2025-01-01: {dates[0]}")
194
195
196 def _read_power_sheet(
197     sheet: Any, *, name: str, expected_days: int, expected_periods: int
198 ) -> tuple[list[str], list[str], np.ndarray]:
199     rows = [row for row in sheet.iter_rows(min_row=1, max_col=expected_periods + 1, values_only=True)]
200     if len(rows) != expected_days + 1:
201         raise InputValidationError(f"附件 2 [{name}] 行数 {len(rows)} != 表头 + {expected_days}")
202     # 表头时刻列经 ``_clock_label`` 规范为 ``H:MM`` (openpyxl 的 ``datetime.time`` 直接 str 会带秒,
203     # 导致表 1 的 [attachment_timestamp 为区间左端点] 硬检查失败; 2026-09-11 修复)。
204     header = [" " if item is None else _clock_label(item) for item in rows[0]]
205     if header[0].strip() != POWER_DATE_HEADER:
206         raise InputValidationError(f"附件 2 [{name}] 首列表头不是 {POWER_DATE_HEADER}: {header[0]!r}")
207     if len(header) != expected_periods + 1:
208         raise InputValidationError(f"附件 2 [{name}] 列数 {len(header)} != {expected_periods + 1}")
209     body = rows[1:]
210     dates = [_date_label(row[0]) for row in body]
211     try:
212         values = np.array([[float(item) for item in row[1:]] for row in body], dtype=float)
213     except (TypeError, ValueError) as exc:
214         raise InputValidationError(f"附件 2 [{name}] 存在非数值单元格: {exc}") from exc
215     if values.shape != (expected_days, expected_periods):
216         raise InputValidationError(f"附件 2 [{name}] 矩阵形状 {values.shape} 不符")
217     if not np.all(np.isfinite(values)):
218         raise InputValidationError(f"附件 2 [{name}] 含 NaN/Inf")
219     if np.any(values < 0):
220         raise InputValidationError(f"附件 2 [{name}] 存在负功率")
221     return header, dates, values
222
223
224 def read_attachment2(
225     path: Path, expected_days: int = DAYS_FULL, expected_periods: int = PERIODS_PER_DAY
226 ) -> Attachment2:
227     """读取附件 2 的 [小区负载] 与 [光伏发电实际功率] (均为实际值)。"""
228     if not path.is_file():
229         raise InputValidationError(f"附件 2 不存在: {path}")
230     workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
231     try:

```

```

232     titles = list(workbook.sheetnames)
233     for name in (LOAD_SHEET, PV_SHEET):
234         if name not in titles:
235             raise InputValidationError(f"附件 2 缺少工作表 [{name}]: {titles}")
236     load_header, load_dates, load = _read_power_sheet(
237         workbook[LOAD_SHEET], name=LOAD_SHEET, expected_days=expected_days, expected_periods=expected_periods
238     )
239     pv_header, pv_dates, pv = _read_power_sheet(
240         workbook[PV_SHEET], name=PV_SHEET, expected_days=expected_days, expected_periods=expected_periods
241     )
242     finally:
243         workbook.close()
244     if load_dates != pv_dates:
245         raise InputValidationError("附件 2 两个工作表的日期序列不一致")
246     _validate_dates(load_dates, source="附件 2 ")
247     if load_header[1:] != pv_header[1:]:
248         raise InputValidationError("附件 2 两个工作表的时间标签不一致")
249     return Attachment2(
250         load_kw=load,
251         pv_kw=pv,
252         dates=load_dates,
253         time_labels=[str(item) for item in load_header[1:]],
254         md5=md5_of(path),
255     )
256
257
258 def read_attachment3(
259     path: Path, *, expected_days: int = DAYS_FULL, hours: Sequence[int] = DECISION_HOURS
260 ) -> Attachment3:
261     """读取附件 3 的整点预报表, 返回 ``(days, 4, 24)`` 张量与规范日期/时刻序列。
262
263     ``日期`` 列只在每日首个发布时刻行出现 (其余为空白), 实现按**前向填充**解析 (团队勘误 prob03 E1):
264     每日必须恰有 ``len(hours)`` 行且发布时刻按 ``0:00 → 6:00 → 12:00 → 18:00`` 升序。
265     """
266     if not path.is_file():
267         raise InputValidationError(f"附件 3 不存在: {path}")
268     workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
269     try:
270         sheet = workbook.worksheets[0]
271         rows = [row for row in sheet.iter_rows(min_row=1, max_col=2 + FC_HORIZON, values_only=True)]
272     finally:
273         workbook.close()
274     if len(rows) < 2:
275         raise InputValidationError("附件 3 没有数据行")
276     header = [str(item).strip() if item is not None else "" for item in rows[0]]
277     if header[:2] != ["日期", "预报时刻"]:
278         raise InputValidationError(f"附件 3 表头前两列异常: {header[:2]}")
279     body = rows[1:]
280     expected_rows = expected_days * len(hours)
281     if len(body) != expected_rows:
282         raise InputValidationError(f"附件 3 数据行数 {len(body)} != 期望 {expected_rows}")
283
284     current = ""
285     filled: list[str] = []
286     for row in body:
287         text = canonical_date(row[0])
288         if text:
289             current = text
290             filled.append(current)
291
292     dates: list[str] = []
293     slot_texts: list[str] = []
294     fc = np.empty((expected_days, len(hours), FC_HORIZON), dtype=float)
295     for day in range(expected_days):
296         day_dates = set()
297         for slot_index, hour in enumerate(hours):
298             offset = day * len(hours) + slot_index
299             day_dates.add(filled[offset])
300             got_hour = _slot_hour(body[offset][1])
301             if got_hour != int(hour):
302                 raise InputValidationError(
303                     f"附件 3 第 {offset + 2} 行预报时刻 {body[offset][1]!r} != 期望 {hour}:00"
304                 )
305             try:
306                 values = np.array([float(item) for item in body[offset][2:]], dtype=float)
307             except (TypeError, ValueError) as exc:
308                 raise InputValidationError(f"附件 3 第 {offset + 2} 行存在非数值预报: {exc}") from exc
309             fc[day, slot_index, :] = values
310         if len(day_dates) != 1 or not day_dates.pop():
311             raise InputValidationError(f"附件 3 第 {day + 1} 天的日期前向填充不唯一")
312     dates.append(filled[day * len(hours)])

```

```

313         slot_texts.append(f"{int(hours[0]):00}")
314     if not np.all(np.isfinite(fc)):
315         raise InputValidationError("附件 3 预报含 NaN/Inf")
316     if np.any(fc < 0):
317         raise InputValidationError("附件 3 存在负预报")
318     for index in range(1, len(dates)):
319         if dates[index] == dates[index - 1]:
320             raise InputValidationError(f"附件 3 日期未推进: {dates[index]}")
321     return Attachment3(fc_kw=fc, dates=dates, slots=slot_texts, md5=md5_of(path))
322
323
324 def read_attachment4_price(
325     path: Path, expected_days: int = DAYS_FULL, expected_periods: int = PERIODS_PER_DAY
326 ) -> Attachment4:
327     """读取附件 4 的 ``Sheet1`` (366 行 × 145 列) + ``(365, 144)`` 实际价矩阵。
328
329     表头 ``日期\\时间`` + 144 个 10 分钟列, 与附件 2 **同相位** (列数与标签逐列一致由调用方核对)。
330     """
331     if not path.is_file():
332         raise InputValidationError(f"附件 4 不存在: {path}")
333     workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)
334     try:
335         titles = list(workbook.sheetnames)
336         if PRICE_SHEET not in titles:
337             raise InputValidationError(f"附件 4 缺少工作表 [{PRICE_SHEET}]: {titles}")
338         sheet = workbook[PRICE_SHEET]
339         rows = [row for row in sheet.iter_rows(min_row=1, max_col=expected_periods + 1, values_only=True)]
340     finally:
341         workbook.close()
342     if len(rows) != expected_days + 1:
343         raise InputValidationError(f"附件 4 行数 {len(rows)} != 表头 + {expected_days}")
344     # 同 ``_read_power_sheet``: 时刻列表头统一规范为 ``H:MM``, 使附件 4 与附件 2 的
345     # ``time_labels`` 逐列一致 (相位核对) 且与 ``AS01`` 左端点口径一致。
346     header = [" " if item is None else _clock_label(item) for item in rows[0]]
347     if header[0].strip() != POWER_DATE_HEADER:
348         raise InputValidationError(f"附件 4 首列表头不是 {POWER_DATE_HEADER}: {header[0]!r}")
349     if len(header) != expected_periods + 1:
350         raise InputValidationError(f"附件 4 列数 {len(header)} != {expected_periods + 1}")
351     body = rows[1:]
352     dates = [_date_label(row[0]) for row in body]
353     try:
354         price = np.array([[float(item) for item in row[1:]] for row in body], dtype=float)
355     except (TypeError, ValueError) as exc:
356         raise InputValidationError(f"附件 4 存在非数值单元格: {exc}") from exc
357     if price.shape != (expected_days, expected_periods):
358         raise InputValidationError(f"附件 4 矩阵形状 {price.shape} 不符")
359     if not np.all(np.isfinite(price)):
360         raise InputValidationError("附件 4 含 NaN/Inf")
361     if np.any(price < 0):
362         raise InputValidationError("附件 4 存在负电价")
363     _validate_dates(dates, source="附件 4 ")
364     return Attachment4(
365         price=price,
366         dates=dates,
367         time_labels=[str(item) for item in header[1:]],
368         header=header,
369         md5=md5_of(path),
370     )
371
372
373 def _template_header(header: list[Any], sheet_name: str, *, periods: int = PERIODS_PER_DAY) -> list[str]:
374     if len(header) != periods + 3:
375         raise InputValidationError(f"模板 [{sheet_name}] 列数 {len(header)} != {periods + 3}")
376     if str(header[0]).strip() != PLAN_HEADER_FIRST:
377         raise InputValidationError(f"模板 [{sheet_name}] 首列表头异常: {header[0]!r}")
378     if str(header[periods + 1]).strip() != PLAN_HEADER_ENERGY:
379         raise InputValidationError(f"模板 [{sheet_name}] 第 {periods + 2} 列不是 [{PLAN_HEADER_ENERGY}] ")
380     if str(header[periods + 2]).strip() != PLAN_HEADER_COST:
381         raise InputValidationError(f"模板 [{sheet_name}] 第 {periods + 3} 列不是 [{PLAN_HEADER_COST}] ")
382     return [str(item) for item in header[1 : periods + 1]]
383
384
385 def _inspect_template(
386     path: Path,
387     *,
388     required_sheets: Sequence[str],
389     purchase_sheets: Sequence[str],
390     label_columns: int,
391 ) -> dict[str, Any]:
392     """只读打开模板并**实例**工作表集合、行列数、列标签、六块与 ``时刻`` 前两行、```` 压缩行。"""
393     if not path.is_file():

```

```

394         raise InputValidationError(f"模板不存在: {path}")
395     measured: dict[str, Any] = {"sheets": [], "purchase": {}, "storage": {}, "emergency": {}}
396     workbook = openpyxl.load_workbook(path, data_only=True)
397     try:
398         titles = list(workbook.sheetnames)
399         measured["sheets"] = titles
400         for name in required_sheets:
401             if name not in titles:
402                 raise InputValidationError(f"模板缺少工作表 [{name}]: {titles}")
403         for name in purchase_sheets:
404             sheet = workbook[name]
405             labels = _template_header([cell.value for cell in sheet[1]], name)
406             dates = [_date_label(sheet.cell(row=row, column=1).value) for row in range(2, sheet.max_row + 1)]
407             measured["purchase"][name] = {
408                 "labels": labels,
409                 "label_count": len(labels),
410                 "rows": len(dates),
411                 "first_date": dates[0] if dates else None,
412                 "last_date": dates[-1] if dates else None,
413                 "max_column": int(sheet.max_column),
414             }
415         storage = workbook[STORAGE_SHEET]
416         storage_header = [" " if cell.value is None else str(cell.value) for cell in storage[1]]
417         if tuple(storage_header[: len(STORAGE_HEADER)]) != STORAGE_HEADER:
418             raise InputValidationError(f"模板 [{STORAGE_SHEET}] 表头异常: {storage_header}")
419         block_rows = min(BLOCKS_PER_DAY, max(int(storage.max_row) - 1, 0))
420         measured["storage"] = {
421             "header": storage_header,
422             "max_row": int(storage.max_row),
423             "blocks": [str(storage.cell(row=2 + k, column=2).value) for k in range(block_rows)],
424             "time_first_two_rows": [
425                 " " if storage.cell(row=2 + k, column=5).value is None else str(storage.cell(row=2 + k, column=5).value)
426                 for k in range(min(2, block_rows))
427             ],
428             "label_columns": label_columns,
429         }
430         emergency = workbook[EMERGENCY_SHEET]
431         emergency_header = [" " if cell.value is None else str(cell.value) for cell in emergency[1]]
432         if tuple(emergency_header[: len(EMERGENCY_HEADER)]) != EMERGENCY_HEADER:
433             raise InputValidationError(f"模板 [{EMERGENCY_SHEET}] 表头异常: {emergency_header}")
434         measured["emergency"] = {
435             "header": emergency_header,
436             "max_row": int(emergency.max_row),
437             "has_compact_row": any(
438                 isinstance(cell, str) and " " in cell
439                 for row in emergency.iter_rows(values_only=True)
440                 for cell in row
441             ),
442         }
443         measured["merged"] = {name: [str(rng) for rng in workbook[name].merged_cells.ranges] for name in titles}
444     finally:
445         workbook.close()
446     measured["md5"] = md5_of(path)
447     return measured
448
449
450 def _geometry_differences(
451     measured: dict[str, Any], expected: dict[str, Any], *, path: Path
452 ) -> list[dict[str, Any]]:
453     """把「实测 vs formulation §8.3 登记值」的差异逐条列出（实测以模板为准，差异只登记不修改）。"""
454     diffs: list[dict[str, Any]] = []
455     for key, want in expected.items():
456         if key == "sheets":
457             got = measured["sheets"]
458         elif key.endswith(".rows"):
459             got = measured["purchase"][key.split(".")[0]]["rows"]
460         elif key.endswith(".labels_last"):
461             got = measured["purchase"][key.split(".")[0]]["labels"][-1]
462         elif key.endswith(".labels_first"):
463             got = measured["purchase"][key.split(".")[0]]["labels"][0]
464         elif key == "storage.max_row":
465             got = measured["storage"]["max_row"]
466         elif key == "emergency.max_row":
467             got = measured["emergency"]["max_row"]
468         else:
469             continue
470         if got != want:
471             diffs.append({"field": key, "expected": want, "measured": got, "template": str(path)})
472     return diffs
473
474

```

```

475 def inspect_template42(path: Path) -> dict[str, Any]:
476     """只读复核 ``result4-2.xlsx`` (计划购电量/充放电量/紧急购电量), 实测值以模板为准。"""
477     measured = _inspect_template(
478         path,
479         required_sheets=(PLAN_SHEET, STORAGE_SHEET, EMERGENCY_SHEET),
480         purchase_sheets=(PLAN_SHEET,),
481         label_columns=PERIODS_PER_DAY,
482     )
483     expected = {
484         "sheets": [PLAN_SHEET, STORAGE_SHEET, EMERGENCY_SHEET],
485         f"{PLAN_SHEET}.rows": 334,
486         f"{PLAN_SHEET}.labels_first": "0:10-0:20",
487         f"{PLAN_SHEET}.labels_last": "0:00-0:10+1",
488         "storage.max_row": 20,
489         "emergency.max_row": 11,
490     }
491     differences = _geometry_differences(measured, expected, path=path)
492     return {
493         "template": str(path),
494         "measured": measured,
495         "formulation_registered": expected,
496         "differences": differences,
497         "difference_note": (
498             "formulation $8.3 的行数 (充放电量 max_row / 紧急购电量 max_row) 来自 prob02 模板示例, "
499             "本问以**只读实测**为准并逐条登记; 差异不代表模板损坏, 也不修改模板。"
500         ),
501     }
502
503
504 def inspect_template43(path: Path) -> dict[str, Any]:
505     """只读复核 ``result4-3.xlsx`` (计划购电量/调整购电量/充放电量/紧急购电量), 实测值以模板为准。"""
506     measured = _inspect_template(
507         path,
508         required_sheets=(PLAN_SHEET, ADJUST_SHEET, STORAGE_SHEET, EMERGENCY_SHEET),
509         purchase_sheets=(PLAN_SHEET, ADJUST_SHEET),
510         label_columns=PERIODS_PER_DAY,
511     )
512     plan = measured["purchase"][PLAN_SHEET]
513     adjust = measured["purchase"][ADJUST_SHEET]
514     label_consistency = plan["labels"] == adjust["labels"]
515     expected = {
516         "sheets": [PLAN_SHEET, ADJUST_SHEET, STORAGE_SHEET, EMERGENCY_SHEET],
517         f"{PLAN_SHEET}.rows": 334,
518         f"{ADJUST_SHEET}.rows": 334,
519         f"{PLAN_SHEET}.labels_first": "0:10-0:20",
520         f"{PLAN_SHEET}.labels_last": "0:00-0:10+1",
521         "storage.max_row": 26,
522         "emergency.max_row": 11,
523     }
524     differences = _geometry_differences(measured, expected, path=path)
525     if not label_consistency:
526         differences.append(
527             {"field": "plan_vs_adjust_labels", "expected": "identical", "measured": "different", "template": str(path)}
528         )
529     return {
530         "template": str(path),
531         "measured": measured,
532         "formulation_registered": expected,
533         "plan_adjust_labels_identical": bool(label_consistency),
534         "differences": differences,
535         "difference_note": (
536             "formulation $8.3 的行数来自 prob03 模板示例, 本问以**只读实测**为准并逐条登记; "
537             "`计划购电量`与`调整购电量`的列标签须逐列相同(替代量口径)。"
538         ),
539     }
540
541
542 def _assign(sheet: Any, *, row: int, column: int, value: Any) -> None:
543     """显式写单元格, ``value=None`` 也**真正清空** (防 openpyxl 静默 no-op)。"""
544     sheet.cell(row=row, column=column).value = value
545
546
547 def _num(value: Any) -> float:
548     """交付数值规整: 保留 ``DELIVERY_DECIMALS`` 位小数。"""
549     return float(round(float(value), DELIVERY_DECIMALS))
550
551
552 def _state(value: Any, *, clip_log: list[float]) -> float:
553     """储电量交付值: 先规整, 再投影到 ``[1200, 10800]``, 并把最大修正量记入 ``clip_log``。"""
554     raw = float(value)
555     rounded = round(raw, DELIVERY_DECIMALS)

```

```

556     projected = min(max(rounded, STATE_MIN_KWH), STATE_MAX_KWH)
557     clip_log.append(abs(projected - raw))
558     return float(projected)
559
560
561 def _clear_rows(sheet: Any, *, first_row: int, last_row: int, last_column: int) -> None:
562     for row in range(first_row, last_row + 1):
563         for column in range(1, last_column + 1):
564             _assign(sheet, row=row, column=column, value=None)
565
566
567 def _residual_report(
568     *, workbook: Any, purchase_sheets: Sequence[str], days: int, emergency_rows: int
569 ) -> list[str]:
570     """复核填报后的工作簿：枚举**应空却仍有值**的单元格（防静默 no-op 复发）。"""
571     residuals: list[str] = []
572     for name in purchase_sheets:
573         sheet = workbook[name]
574         for row in range(days + 2, int(sheet.max_row) + 1):
575             for column in range(1, PERIODS_PER_DAY + 4):
576                 if sheet.cell(row=row, column=column).value is not None:
577                     residuals.append(f"{name}!{get_column_letter(column)}{row} (超出行数)")
578                     break
579     storage = workbook[STORAGE_SHEET]
580     for index in range(days):
581         for block in range(BLOCKS_PER_DAY):
582             row = 2 + index * BLOCKS_PER_DAY + block
583             if block != 0 and storage.cell(row=row, column=1).value is not None:
584                 residuals.append(f"充放电量!A{row} (日期只写块首行)")
585             if block >= 2 and storage.cell(row=row, column=5).value is not None:
586                 residuals.append(f"充放电量!E{row} (时刻只写每日前两行)")
587             if block >= 2 and storage.cell(row=row, column=6).value is not None:
588                 residuals.append(f"充放电量!F{row} (储电量只写每日前两行)")
589     expected_storage_rows = 1 + days * BLOCKS_PER_DAY
590     for row in range(expected_storage_rows + 1, int(storage.max_row) + 1):
591         for column in range(1, len(STORAGE_HEADER) + 1):
592             if storage.cell(row=row, column=column).value is not None:
593                 residuals.append(f"充放电量!{get_column_letter(column)}{row} (超出行数)")
594                 break
595     emergency = workbook[EMERGENCY_SHEET]
596     for row in range(1 + emergency_rows + 1, int(emergency.max_row) + 1):
597         for column in range(1, len(EMERGENCY_HEADER) + 1):
598             if emergency.cell(row=row, column=column).value is not None:
599                 residuals.append(f"紧急购电量!{get_column_letter(column)}{row} (超出行数)")
600                 break
601     for name in workbook.sheetnames:
602         for row in workbook[name].iter_rows(values_only=True):
603             for value in row:
604                 if isinstance(value, str) and " " in value:
605                     residuals.append(f"{name}!不得残留模板 压缩行")
606     total = len(residuals)
607     if total > 200:
608         residuals = residuals[:200] + [f"... (共 {total} 处, 已截断)"]
609     return residuals
610
611
612 def _fill_purchase_sheet(
613     sheet: Any,
614     *,
615     name: str,
616     dates: Sequence[str],
617     values: np.ndarray,
618     day_energy_kwh: np.ndarray,
619     day_cost_yuan: np.ndarray,
620 ) -> None:
621     days = len(dates)
622     if sheet.max_row < days + 1:
623         raise InputValidationError(f"模板 [{name}] 只有 {sheet.max_row - 1} 行, 少于 {days} 天")
624     for index, date_text in enumerate(dates):
625         row = 2 + index
626         actual = _date_label(sheet.cell(row=row, column=1).value)
627         if actual != date_text:
628             raise InputValidationError(f"模板 [{name}] 第 {row} 行日期 {actual!r} != {date_text!r}")
629     for period in range(PERIODS_PER_DAY):
630         _assign(sheet, row=row, column=2 + period, value=_num(values[index, period]))
631         _assign(sheet, row=row, column=PERIODS_PER_DAY + 2, value=_num(day_energy_kwh[index]))
632         _assign(sheet, row=row, column=PERIODS_PER_DAY + 3, value=_num(day_cost_yuan[index]))
633     if sheet.max_row > days + 1:
634         _clear_rows(sheet, first_row=days + 2, last_row=sheet.max_row, last_column=PERIODS_PER_DAY + 3)
635
636

```

```

637 def _fill_storage_sheet(
638     storage: Any,
639     *,
640     dates: Sequence[str],
641     charge: np.ndarray,
642     discharge: np.ndarray,
643     state_start_kwh: np.ndarray,
644     state_end_kwh: np.ndarray,
645     clip_log: list[float],
646 ) -> tuple[int, list[str]]:
647     blocks = [str(storage.cell(row=2 + k, column=2).value) for k in range(BLOCKS_PER_DAY)]
648     filled = 0
649     for index, date_text in enumerate(dates):
650         date_value = datetime.strptime(date_text, "%Y-%m-%d")
651         for block in range(BLOCKS_PER_DAY):
652             row = 2 + index * BLOCKS_PER_DAY + block
653             start = block * BLOCK_ROWS
654             stop = start + BLOCK_ROWS
655             _assign(storage, row=row, column=1, value=date_value if block == 0 else None)
656             _assign(storage, row=row, column=2, value=blocks[block])
657             _assign(storage, row=row, column=3, value=_num(np.sum(charge[index, start:stop])))
658             _assign(storage, row=row, column=4, value=_num(np.sum(discharge[index, start:stop])))
659             _assign(
660                 storage,
661                 row=row,
662                 column=5,
663                 value="00:00:00" if block == 0 else ("24:00" if block == 1 else None),
664             )
665             _assign(
666                 storage,
667                 row=row,
668                 column=6,
669                 value=_state(state_start_kwh[index], clip_log=clip_log) if block == 0 else (
670                     _state(state_end_kwh[index], clip_log=clip_log) if block == 1 else None
671                 ),
672             )
673             filled += 1
674     expected_rows = 1 + len(dates) * BLOCKS_PER_DAY
675     if storage.max_row > expected_rows:
676         _clear_rows(storage, first_row=expected_rows + 1, last_row=storage.max_row, last_column=6)
677     return filled, blocks
678
679 def _fill_emergency_sheet(
680     emergency: Any,
681     *,
682     dates: Sequence[str],
683     emergency_intervals: Sequence[Sequence[tuple[str, float]]],
684 ) -> int:
685     row = 2
686     filled = 0
687     for index, date_text in enumerate(dates):
688         date_value = datetime.strptime(date_text, "%Y-%m-%d")
689         intervals = list(emergency_intervals[index])
690         if not intervals:
691             intervals = [(ZERO_INTERVAL_TEXT, 0.0)]
692         for order, (slot, kwh) in enumerate(intervals):
693             _assign(emergency, row=row, column=1, value=date_value if order == 0 else None)
694             _assign(emergency, row=row, column=2, value=slot)
695             _assign(emergency, row=row, column=3, value=_num(kwh))
696             row += 1
697             filled += 1
698     expected_rows = row - 1
699     if emergency.max_row > expected_rows:
700         _clear_rows(emergency, first_row=expected_rows + 1, last_row=emergency.max_row, last_column=3)
701     return filled
702
703
704
705 def fill_result42_workbook(
706     *,
707     template: Path,
708     dest: Path,
709     dates: Sequence[str],
710     purchase_kwh: np.ndarray,
711     charge_kwh: np.ndarray,
712     discharge_kwh: np.ndarray,
713     state_start_kwh: np.ndarray,
714     state_end_kwh: np.ndarray,
715     day_energy_kwh: np.ndarray,
716     day_cost_yuan: np.ndarray,
717     emergency_intervals: Sequence[Sequence[tuple[str, float]]],

```

```

718 ) -> dict[str, Any]:
719     """把 ``4-2`` 链的交付期解填入 ``result4-2.xlsx`` (三工作表)。
720
721     口径 (AS20 + formulation $8.3):
722     * ``计划购电量``: ``b``; 末两列 ``全天购电量 = Eb``, ``全天购电费 = C_total^{act}`` (``4-2`` 无 ``C_adj``);
723     * ``充放电电量``: 334 天  $\times$  6 个 4 小时块, ``充电量``/``放电量`` **分别列示不冲抵**, 日期只写块首行,
724     ``时刻`` = ``00:00:00``/``24:00`` 与 ``储电量`` = ``E_{fd,0}``/``E_{fd,144}`` 只写每日前两行;
725     * ``紧急购电量``: 每日至少 1 行; ``J_d = 0`` 写 ``-``/``0``。
726
727     """
728     days = len(dates)
729     purchase = np.asarray(purchase_kwh, dtype=float)
730     charge = np.asarray(charge_kwh, dtype=float)
731     discharge = np.asarray(discharge_kwh, dtype=float)
732     clip_log: list[float] = []
733     if purchase.shape != (days, PERIODS_PER_DAY):
734         raise InputValidationError(f"purchase 形状 {purchase.shape} != ({days}, {PERIODS_PER_DAY})")
735     for label, array in (("charge", charge), ("discharge", discharge)):
736         if array.shape != purchase.shape:
737             raise InputValidationError(f"{label} 形状与 purchase 不一致")
738     if len(emergency_intervals) != days:
739         raise InputValidationError("emergency_intervals 天数与 dates 不一致")
740     if not template.is_file():
741         raise InputValidationError(f"模板不存在: {template}")
742
743     workbook = openpyxl.load_workbook(template)
744     try:
745         titles = list(workbook.sheetnames)
746         for name in (PLAN_SHEET, STORAGE_SHEET, EMERGENCY_SHEET):
747             if name not in titles:
748                 raise InputValidationError(f"模板缺少工作表 [{name}]: {titles}")
749         _fill_purchase_sheet(
750             workbook[PLAN_SHEET],
751             name=PLAN_SHEET,
752             dates=dates,
753             values=purchase,
754             day_energy_kwh=day_energy_kwh,
755             day_cost_yuan=day_cost_yuan,
756         )
757         blocks_filled, blocks = _fill_storage_sheet(
758             workbook[STORAGE_SHEET],
759             dates=dates,
760             charge=charge,
761             discharge=discharge,
762             state_start_kwh=state_start_kwh,
763             state_end_kwh=state_end_kwh,
764             clip_log=clip_log,
765         )
766         emergency_rows = _fill_emergency_sheet(
767             workbook[EMERGENCY_SHEET], dates=dates, emergency_intervals=emergency_intervals
768         )
769         dest.parent.mkdir(parents=True, exist_ok=True)
770         residuals = _residual_report(
771             workbook=workbook, purchase_sheets=(PLAN_SHEET,), days=days, emergency_rows=emergency_rows
772         )
773         workbook.save(dest)
774     finally:
775         workbook.close()
776     return {
777         "chain": "4-2",
778         "plan_rows_filled": days,
779         "plan_periods_per_row": PERIODS_PER_DAY,
780         "storage_blocks_filled": blocks_filled,
781         "storage_blocks": blocks,
782         "emergency_rows_filled": emergency_rows,
783         "format_residuals": residuals,
784         "delivery_decimals": DELIVERY_DECIMALS,
785         "storage_projection": {
786             "bounds_kwh": [STATE_MIN_KWH, STATE_MAX_KWH],
787             "max_abs_correction_kwh": max(clip_log) if clip_log else 0.0,
788             "cells": len(clip_log),
789         },
790     }
791
792 def fill_result43_workbook(
793     *,
794     template: Path,
795     dest: Path,
796     dates: Sequence[str],
797     planned_kwh: np.ndarray,
798     adjusted_kwh: np.ndarray,

```

```

799     charge_kwh: np.ndarray,
800     discharge_kwh: np.ndarray,
801     state_start_kwh: np.ndarray,
802     state_end_kwh: np.ndarray,
803     day_energy_planned_kwh: np.ndarray,
804     day_energy_adjusted_kwh: np.ndarray,
805     day_cost_yuan: np.ndarray,
806     emergency_intervals: Sequence[Sequence[tuple[str, float]]],
807 ) -> dict[str, Any]:
808     """把 ``4-3`` 链的交付期解填入 ``result4-3.xlsx`` (四工作表)。
809
810     口径 (AS20 + formulation $8.3) :
811     * ``计划购电量``: ``b``; 末两列 ``Eb``、``C_total^{act}``;
812     * ``调整购电量``: ``最终量`` ``q`` (替代量口径); 末两列 ``Eq``、``C_total^{act}``;
813     * ``充放电量`` / ``紧急购电量``: 与 ``4-2`` 同款几何。
814
815     """
816     days = len(dates)
817     planned = np.asarray(planned_kwh, dtype=float)
818     adjusted = np.asarray(adjusted_kwh, dtype=float)
819     charge = np.asarray(charge_kwh, dtype=float)
820     discharge = np.asarray(discharge_kwh, dtype=float)
821     clip_log: list[float] = []
822     if planned.shape != (days, PERIODS_PER_DAY):
823         raise InputValidationError(f"planned 形状 {planned.shape} != ({days}, {PERIODS_PER_DAY})")
824     for label, array in (("adjusted", adjusted), ("charge", charge), ("discharge", discharge)):
825         if array.shape != planned.shape:
826             raise InputValidationError(f"{label} 形状与 planned 不一致")
827     if len(emergency_intervals) != days:
828         raise InputValidationError("emergency_intervals 天数与 dates 不一致")
829     if not template.is_file():
830         raise InputValidationError(f"模板不存在: {template}")
831
832     workbook = openpyxl.load_workbook(template)
833     try:
834         titles = list(workbook.sheetnames)
835         for name in (PLAN_SHEET, ADJUST_SHEET, STORAGE_SHEET, EMERGENCY_SHEET):
836             if name not in titles:
837                 raise InputValidationError(f"模板缺少工作表 [{name}]: {titles}")
838         for name, values, energy in (
839             (PLAN_SHEET, planned, day_energy_planned_kwh),
840             (ADJUST_SHEET, adjusted, day_energy_adjusted_kwh),
841         ):
842             _fill_purchase_sheet(
843                 workbook[name],
844                 name=name,
845                 dates=dates,
846                 values=values,
847                 day_energy_kwh=energy,
848                 day_cost_yuan=day_cost_yuan,
849             )
850         blocks_filled, blocks = _fill_storage_sheet(
851             workbook[STORAGE_SHEET],
852             dates=dates,
853             charge=charge,
854             discharge=discharge,
855             state_start_kwh=state_start_kwh,
856             state_end_kwh=state_end_kwh,
857             clip_log=clip_log,
858         )
859         emergency_rows = _fill_emergency_sheet(
860             workbook[EMERGENCY_SHEET], dates=dates, emergency_intervals=emergency_intervals
861         )
862         dest.parent.mkdir(parents=True, exist_ok=True)
863         residuals = _residual_report(
864             workbook=workbook,
865             purchase_sheets=(PLAN_SHEET, ADJUST_SHEET),
866             days=days,
867             emergency_rows=emergency_rows,
868         )
869         workbook.save(dest)
870     finally:
871         workbook.close()
872     return {
873         "chain": "4-3",
874         "plan_rows_filled": days,
875         "adjust_rows_filled": days,
876         "plan_periods_per_row": PERIODS_PER_DAY,
877         "storage_blocks_filled": blocks_filled,
878         "storage_blocks": blocks,
879         "emergency_rows_filled": emergency_rows,
880         "format_residuals": residuals,

```

```

880         "delivery_decimals": DELIVERY_DECIMALS,
881         "storage_projection": {
882             "bounds_kwh": [STATE_MIN_KWH, STATE_MAX_KWH],
883             "max_abs_correction_kwh": max(clip_log) if clip_log else 0.0,
884             "cells": len(clip_log),
885         },
886     }
887
888
889 def write_json(path: Path, payload: Any) -> None:
890     """写 JSON；显式拒绝 NaN/Inf，避免把非有限值伪装成合法结果。"""
891     path.parent.mkdir(parents=True, exist_ok=True)
892     text = json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False)
893     path.write_text(text + "\n", encoding="utf-8")
894
895
896 def delivery_dates(all_dates: Sequence[str], start_day_index: int) -> list[str]:
897     """交付期日期列表（默认 2025-02-01 ... 2025-12-31, 334 天）。"""
898     return [str(item) for item in all_dates[start_day_index:]]
899
900
901 def day_offset(dates: Sequence[str], target: str) -> int:
902     """在日期序列中定位目标的 0 基索引；找不到时严格失败。"""
903     for index, item in enumerate(dates):
904         if item == target:
905             return index
906     raise InputValidationError(f"日期序列中找不到 {target}")
907
908
909 def _format_minutes(total_minutes: int) -> str:
910     """把「左端点分钟数」格式化为 ``H:MM``；1440 时按 AS01 写 ``+1``。"""
911     if total_minutes < 1440:
912         return f"{total_minutes // 60}:{total_minutes % 60:02d}"
913     shifted = total_minutes - 1440
914     return f"{shifted // 60}:{shifted % 60:02d}+1"
915
916
917 def interval_label(start: int, stop: int) -> str:
918     """按 AS01 生成 ``[_start+1, _stop+1 + 10min)`` 的左端点区间标签。
919
920     ``start``/``stop`` 为 0 基时段下标（半开区间 ``[start, stop)``）；两端**都**用 ``H:MM`` 风格
921     （小时不补零），与附件 5 模板列标签风格（如 ``9:50-10:00``）一致（承 prob03 勘误 R9）。
922     """
923     return f"({_format_minutes(10 * (start + 1))}-{_format_minutes(10 * (stop + 1))})"
924
925
926 __all__ = [
927     "ADJUST_SHEET",
928     "Attachment2",
929     "Attachment3",
930     "Attachment4",
931     "BLOCKS_PER_DAY",
932     "BLOCK_ROWS",
933     "DAYS_FULL",
934     "DECISION_HOURS",
935     "DELIVERY_DECIMALS",
936     "EMERGENCY_SHEET",
937     "FC_HORIZON",
938     "InputValidationError",
939     "LOAD_SHEET",
940     "PERIODS_PER_DAY",
941     "PLAN_SHEET",
942     "PRICE_SHEET",
943     "PV_SHEET",
944     "STORAGE_SHEET",
945     "ZERO_INTERVAL_TEXT",
946     "canonical_date",
947     "day_offset",
948     "delivery_dates",
949     "fill_result42_workbook",
950     "fill_result43_workbook",
951     "inspect_template42",
952     "inspect_template43",
953     "interval_label",
954     "md5_of",
955     "read_attachment2",
956     "read_attachment3",
957     "read_attachment4_price",
958     "write_json",
959 ]

```

B.11 问题四：两条链的模型模块

```
1  #-*- coding: utf-8 -*-
2  """prob04 两条链（`4-2` 逐日 LP / `4-3` 三层顺序向前递推 LP）的建模、求解与结构自检。
3
4  严格对应 `problems/microgrid_2025/prob04/versions/assumption_v001`（AS01-AS23 + 团队 `A8-(b)` 六条派生）
5  与 `formulations/formulation_v001`（$1.4 降尺度、$2.2-$2.6 链 `4-2`、$3.2-$3.7 链 `4-3`、$4.2 `A2-PRE`）：
6
7  ...
8  链 4-2（M4-2，逐日向前递推）
9  for d in D_full:
10     计划层 (PL2_d):  $\min \sum \hat{p}_{d,i} \cdot b_i + E_{em} \cdot \hat{p}_{d,i} \cdot q_{em,i}$  （ $q_{em}$  变量**必须保留**）
11     提交全天 ( $b, c, q_{dis}, s$ ) 与  $E$  轨迹;  $q := b$  (4-2 无独立  $q$  变量, F-3)
12     结算层 (ST2_d): 闭式  $r = b + q_{dis} + PV^{act} \cdot \Delta t - L \cdot \Delta t - c$ ;  $s' = \max(0, r)$ ;  $q_{em} = \max(0, -r)$ 
13
14  链 4-3（M4-3，三层顺序向前递推）
15  for d in D_full:
16     计划层 (PL_d) :  $m=0$ , 用  $\Pi_0[PV]$  与  $\hat{p}^{(0)}$ ,  $\min \sum \hat{p}^{(0)} \cdot b$ ; **不含  $q_{em}$ **；提交  $D_0 = i = 1..36$ 
17     调整层 (AD_{d,m}):  $m \in \{6, 12, 18\}$ , 用  $\Pi_m[PV]$  与  $\hat{p}^{(m)}$   $= \_m \cdot \hat{p}^{(act)}_{d-1}$  ( $i > 6m$ ),
18          $b$  为参数, 顺序重优化  $R_m$ , 只提交  $D_m = \{i : 6m < i \leq 6m+36\}$ 
19     结算层 (ST_d) : 与 4-2 逐项相同的闭式 ( $PV^{act}$  与  $p^{(act)}$  的唯一入口)
20  ...
21
22  口径（两链不得逐问更改）: ``Delta = 1/6``, ``eta_ch = eta_dis = 0.9``, ``c = 833.3333``, ``q_dis = 750.0000``,
23  ``E = [1200, 10800]``, ``E_{1/1,0} = 6000``, 终端自由、``eta_em = 5``（两链）、``beta_def = 0.5`` / ``beta_over = 1.5``
24  （仅 `4-3`）；费用以**元**计且**不乘** ``Delta``。
25
26  求解器: CPU HiGHS（`scipy.optimize.linprog(method="highs")`）；两条链**都不使用 GPU**、不依赖随机源。
27
28  统一 tie-break (AS21, 承 prob03 团队裁定 T7-1/T9)：所有层、所有模型一律「先 ``min`` 该层主目标、
29  再在主目标最优化面上 ``min E_t (c_t + q_dis_t)``」；字面 加权式只作**逐层诊断**，不作提交解。
30  """
31  from __future__ import annotations
32
33  import time
34  from dataclasses import dataclass, field
35  from typing import Any
36
37  import numpy as np
38  from prob04_predict import D_REQ_START, DECISION_HOURS
39  from scipy.optimize import linprog
40  from scipy.sparse import coo_matrix, csr_matrix, vstack
41
42  DELTA_T = 1.0 / 6.0
43  ETA_CH = 0.9
44  ETA_DIS = 0.9
45  ETA_ROUND_TRIP = ETA_CH * ETA_DIS
46  E_INIT = 6000.0
47  E_MIN = 1200.0
48  E_MAX = 10800.0
49  P_MAX = 5000.0
50  C_CAP = P_MAX * DELTA_T # 833.3333 kWh（并网点侧为紧侧）
51  Q_CAP = P_MAX * DELTA_T * ETA_DIS # 750.0000 kWh（电池侧为紧侧）
52  ALPHA_EM = 5.0
53  BETA_DEF = 0.5
54  BETA_OVER = 1.5
55  PERIODS_PER_DAY = 144
56  DAYS_FULL = 365
57  TOL = 1e-6
58  LAYER_TOL = 1e-6
59  LAYER_TOL_RELATIVE = 1e-8
60  BALANCE_TOL = 1e-8
61  BLOCK_ROWS = 24
62  BLOCKS_PER_DAY = 6
63  SPEC_DATES: tuple[tuple[str, int], ...] = (
64      ("2025-03-20", 78),
65      ("2025-06-21", 171),
66      ("2025-09-23", 265),
67      ("2025-12-21", 354),
68  )
69  TABLE1_SLOTS: tuple[tuple[str, int], ...] = (
70      ("10:00-10:10", 60),
71      ("12:00-12:10", 72),
72      ("14:00-14:10", 84),
73      ("16:00-16:10", 96),
74      ("18:00-18:10", 108),
75      ("20:00-20:10", 120),
76  )
77  CROSS_YEAR_DROPPED: dict[int, tuple[int, int]] = {6: (19, 24), 12: (13, 24), 18: (7, 24)}
78  COMMIT_BLOCK = 36
```

```

79 DELIVERY_MAGNITUDE_BAND_YUAN = (3.0e6, 5.0e7)
80 DAILY_MAGNITUDE_BAND_YUAN = (5.0e3, 2.0e5)
81
82 # --- AS21 (承 prob03 T7-1/T7-2/T7-3/T7-5): 统一冻结 tie-breaking -----
83 T7_EPS_RELATIVE = 1e-6
84 T7_INVARIANCE_TOL = 1e-9
85 T7_SHRINK_FACTOR = 0.1
86 T7_MAX_SHRINKS = 8
87 T7_FACE_TOL_RELATIVE = 1e-9
88 T7_FACE_TOL_ABSOLUTE = 1e-9
89 T7_FACE_PRIMARY_TOL_FRACTION = 0.5
90 T7_TIEBREAK_RULE = (
91     "min [该层主目标] +  $\cdot \Sigma_t(c_{\{d,t\}}+q_{\text{dis}_{\{d,t\}}})$ , "
92     " =  $1e-6 \cdot \max(|\text{该层主目标最优值}|, 1) / [n \cdot (c_{\text{cap}}+q_{\text{dis\_cap}})]$ ; "
93     "所有层、所有模型 (4-2/4-3/A2-PRE/ablation 对照) 必须完全一致 (AS21 承 prob03 T7-1/T7-5) "
94 )
95
96
97 class LayerFailure(RuntimeError):
98     """某个决策层 LP 未达最优: 由调用方按 ``solver_not_optimal`` 路由 (不得伪报最优)。"""
99
100     def __init__(self, message: str, *, day: int, hour: int, status: int) -> None:
101         super().__init__(message)
102         self.day = day
103         self.hour = hour
104         self.status = status
105
106
107 class BudgetExceeded(RuntimeError):
108     """墙钟预算耗尽: 按 ``solver_not_optimal`` 路由并保留已完成层的证据, 不得伪报完成。"""
109
110
111 @dataclass(frozen=True)
112 class ChainInputs:
113     """一条链的输入 (按天展开)。
114
115     ``price_act``: 附件 4 的实际价 (结算用, 两链唯一价格入口);
116     ``decision_price``: 计划层的决策价 ``\hat{p}`` (``4-2`` = (PF-PERSIST); ``4-3`` = ``\hat{p}^{\{0\}}``);
117     ``forecast_kw``: ``4-3`` 的 ``hour + (days, 144)`` 降尺度预报 (未支配时段为 NaN); ``4-2`` 为空字典。
118     """
119
120     days: int
121     price_act: np.ndarray
122     decision_price: np.ndarray
123     load_energy: np.ndarray
124     pv_act_energy: np.ndarray
125     forecast_kw: dict[int, np.ndarray] = field(default_factory=dict)
126
127     @property
128     def periods(self) -> int:
129         return int(self.days * PERIODS_PER_DAY)
130
131     def forecast_energy(self, hour: int) -> np.ndarray:
132         return self.forecast_kw[hour] * DELTA_T
133
134
135 @dataclass
136 class LayerRecord:
137     day: int
138     hour: int
139     layer: str
140     status: int
141     message: str
142     iterations: int | None
143     objective: float
144     equality_residual_max: float
145     equality_residual_relative_max: float
146     inequality_residual_max: float
147     inequality_residual_relative_max: float
148     bound_violation_max: float
149     seconds: float
150     variables: int
151     equality_rows: int
152     inequality_rows: int
153     nonzero: int
154
155     def as_dict(self) -> dict[str, Any]:
156         return {
157             "day_index": self.day,
158             "hour": self.hour,
159             "layer": self.layer,

```

```

160         "status": self.status,
161         "message": self.message,
162         "iterations": self.iterations,
163         "objective_yuan": self.objective,
164         "equality_residual_max": self.equality_residual_max,
165         "equality_residual_relative_max": self.equality_residual_relative_max,
166         "inequality_residual_max": self.inequality_residual_max,
167         "inequality_residual_relative_max": self.inequality_residual_relative_max,
168         "bound_violation_max": self.bound_violation_max,
169         "seconds": self.seconds,
170         "variables": self.variables,
171         "equality_rows": self.equality_rows,
172         "inequality_rows": self.inequality_rows,
173         "nonzero": self.nonzero,
174     }
175
176
177 @dataclass
178 class TiebreakRecord:
179     """AS21 的逐层审计记录（主目标不变性 / 吞吐量 / 退化维度 / 边界状态 / 提交解来源）。"""
180
181     layer: str
182     day: int
183     hour: int
184     primary_before_yuan: float
185     primary_after_yuan: float
186     primary_relative_change: float
187     epsilon: float
188     shrinks: int
189     invariance_passed: bool
190     throughput_kwh: float
191     throughput_before_kwh: float
192     throughput_reduced_kwh: float
193     baseline_state_change_max_kwh: float
194     degeneracy_degree: int
195     active_constraints: int
196     variables: int
197     weighted_primary_relative_change: float
198     weighted_throughput_kwh: float
199     weighted_sum_effective: bool
200     committed_solution: str
201     throughput_upper_on_optimal_face_kwh: float | None
202     throughput_unique: bool | None
203     probe_status: int | None
204     boundary_state_kwh: float
205     baseline_seconds: float
206     solver_seconds: float
207     note: str = ""
208
209     def as_dict(self) -> dict[str, Any]:
210         return {
211             "layer": self.layer,
212             "day_index": int(self.day),
213             "hour": int(self.hour),
214             "primary_before_yuan": _finite_or(self.primary_before_yuan),
215             "primary_after_yuan": _finite_or(self.primary_after_yuan),
216             "primary_relative_change": _finite_or(self.primary_relative_change),
217             "epsilon": _finite_or(self.epsilon),
218             "shrinks": int(self.shrinks),
219             "invariance_passed": bool(self.invariance_passed),
220             "throughput_kwh": _finite_or(self.throughput_kwh),
221             "throughput_before_kwh": _finite_or(self.throughput_before_kwh),
222             "throughput_reduced_kwh": _finite_or(self.throughput_reduced_kwh),
223             "baseline_state_change_max_kwh": _finite_or(self.baseline_state_change_max_kwh),
224             "degeneracy_degree": int(self.degeneracy_degree),
225             "active_constraints": int(self.active_constraints),
226             "variables": int(self.variables),
227             "weighted_primary_relative_change": _finite_or(self.weighted_primary_relative_change),
228             "weighted_throughput_kwh": _finite_or(self.weighted_throughput_kwh),
229             "weighted_sum_effective": bool(self.weighted_sum_effective),
230             "committed_solution": self.committed_solution,
231             "throughput_upper_on_optimal_face_kwh": (
232                 None if self.throughput_upper_on_optimal_face_kwh is None
233                 else _finite_or(self.throughput_upper_on_optimal_face_kwh)
234             ),
235             "throughput_unique": (
236                 None if self.throughput_unique is None else bool(self.throughput_unique)
237             ),
238             "probe_status": None if self.probe_status is None else int(self.probe_status),
239             "boundary_state_kwh": _finite_or(self.boundary_state_kwh),
240             "baseline_seconds": _finite_or(self.baseline_seconds),

```

```

241         "solver_seconds": _finite_or(self.solver_seconds),
242         "note": self.note,
243     }
244
245
246 @dataclass
247 class ChainResult:
248     """一条链的完整解（提交链 + 计划轨迹 + 审计记录；两链共用同一容器）。"""
249
250     chain: str
251     days: int
252     price_act: np.ndarray          # (days, 144) 附件 4 实际价
253     price_fc: np.ndarray          # (days, 144) 决策层信念价  $\hat{p}^{fc}$ 
254     decision_price: np.ndarray     # (days, 144) 计划层决策价  $\hat{p}$  (4-2 与 4-3 同式)
255     load_energy: np.ndarray       # (days, 144) kWh =  $L \cdot \Delta t$  (附件 2 实际)
256     pv_act_energy: np.ndarray     # (days, 144) kWh =  $PV^{act} \cdot \Delta t$  (附件 2 实际)
257     plan_b: np.ndarray
258     q: np.ndarray                # 最终量 (4-2 中  $q := b$ )
259     c: np.ndarray
260     q_dis: np.ndarray
261     E: np.ndarray
262     q_em: np.ndarray
263     s_settle: np.ndarray
264     plan_c: np.ndarray
265     plan_q_dis: np.ndarray
266     plan_s: np.ndarray
267     plan_E: np.ndarray
268     plan_q_em: np.ndarray        # 计划层解中的  $q_{em}$  变量 (4-2 保留变量; 4-3 无该变量, 恒 0)
269     layer_records: list[LayerRecord] = field(default_factory=list)
270     tiebreak_records: list[TiebreakRecord] = field(default_factory=list)
271     kappa_records: list[dict[str, Any]] = field(default_factory=list)
272     fallback_records: list[dict[str, Any]] = field(default_factory=list)
273     # 支配层光伏电量矩阵 (4-2: {0:  $PV^{act} \cdot \Delta t$ ; 4-3: {m:  $\Pi_m[PV] \cdot \Delta t$ }}, 供层内余额核对使用
274     layer_pv_energy: dict[int, np.ndarray] = field(default_factory=dict)
275
276 @property
277 def periods(self) -> int:
278     return int(self.days * PERIODS_PER_DAY)
279
280
281 def _finite_or(value: float, fallback: float = 1e30) -> float:
282     """把非有限值替换为有限哨兵, 保证 ``allow_nan=False`` 的 JSON 落盘不失败。"""
283     number = float(value)
284     return number if np.isfinite(number) else float(fallback)
285
286
287 def dominance_map(periods: int = PERIODS_PER_DAY) -> np.ndarray:
288     """支配时刻 ``i``: ``i = 1..144`` → ``0/6/12/18`` (每层 36 个时段)。"""
289     index = np.arange(1, periods + 1)
290     return 6 * ((index - 1) // COMMIT_BLOCK)
291
292
293 def downscale(pv_row: np.ndarray, hour: int, *, periods: int = PERIODS_PER_DAY) -> np.ndarray:
294     """AS19/formulation §1.4 的降尺度算子 `` $\Pi_m$ `` (整点数值 + 整点锚定线性插值)。
295
296     ``pv_row[k-1] = A_{m,k}`` (``(m+k):00`` 的功率点值, ``k = 1..24``)。时段 ``i`` 的块
297     ``b = i/6``、块内位置 ``j = i - 6(b-1)``、时效 ``k = b - m``:
298
299     * ``k = 1``: 前向保持 ``A_{m,1}`` (发布后第一小时无 ``P(m)``);
300     * ``2 ≤ k ≤ 24``: ``(1 - j/6) * A_{m,k-1} + (j/6) * A_{m,k}``;
301     * ``k < 1`` (早于发布时刻): **未定义 (NaN) **, 不消费。
302
303     跨年项 (``m + k > 24``、即 ``k > 24 - m``) 在本函数中**不产生任何输出** (该时段落在次日、
304     不在当日 144 时段内), 其个数由 :func:`cross_year_dropped_count` 登记。
305     """
306     row = np.asarray(pv_row, dtype=float).reshape(-1)
307     if row.shape[0] != 24:
308         raise ValueError(f"预报向量长度 {row.shape[0]} != 24")
309     if hour not in DECISION_HOURS:
310         raise ValueError(f"未知发布时刻: {hour}")
311     out = np.full(periods, np.nan, dtype=float)
312     for index in range(1, periods + 1):
313         block = (index - 1) // 6 + 1
314         position = index - 6 * (block - 1)
315         lead = block - hour
316         if lead < 1:
317             continue
318         if lead == 1:
319             out[index - 1] = row[0]
320         else:
321             out[index - 1] = (1.0 - position / 6.0) * row[lead - 2] + (position / 6.0) * row[lead - 1]

```

```

322     return out
323
324
325 def cross_year_dropped_count(hour: int) -> int:
326     """跨年项个数: ``m = 6/12/18`` 分别丢弃 6/12/18 项 (合计 36, $1.4 / $8.4)。"""
327     if hour == 0:
328         return 0
329     low, high = CROSS_YEAR_DROPPED[hour]
330     return int(high - low + 1)
331
332
333 def block_energy_bias(pv_row: np.ndarray, hour: int) -> dict[str, float]:
334     """块内能量偏差统计 (**强制披露**, $1.4): 插值 vs 分段常数。
335
336     块 ``b`` 的 6 个时段之和 ``= (2.5 * A_{k-1} + 3.5 * A_k) / 6`` (``k = 2``), 分段常数口径为 ``A_k``;
337     逐块给出二者之差与绝对差之和 (``k = 1`` 的块两口径相同, 差为 0)。
338     """
339     row = np.asarray(pv_row, dtype=float).reshape(-1)
340     total_interp = 0.0
341     total_pieewise = 0.0
342     biases: list[float] = []
343     for block in range(hour + 1, 25):
344         lead = block - hour
345         if lead == 1:
346             interp = row[0]
347         else:
348             interp = (2.5 * row[lead - 2] + 3.5 * row[lead - 1]) / 6.0
349         pieewise = row[lead - 1]
350         biases.append(float(interp - pieewise))
351         total_interp += float(interp)
352         total_pieewise += float(pieewise)
353     return {
354         "blocks": float(len(biases)),
355         "sum_interpolated_kwh": total_interp,
356         "sum_pieewise_constant_kwh": total_pieewise,
357         "net_bias_kwh": total_interp - total_pieewise,
358         "max_abs_block_bias_kwh": max((abs(item) for item in biases), default=0.0),
359         "sum_abs_block_bias_kwh": float(np.sum(np.abs(biases))) if biases else 0.0,
360     }
361
362
363 def _solve(
364     objective: np.ndarray,
365     a_eq: csr_matrix,
366     b_eq: np.ndarray,
367     a_ub: csr_matrix | None,
368     b_ub: np.ndarray | None,
369     bounds: np.ndarray,
370     *,
371     time_limit_seconds: float,
372 ) -> tuple[Any, float]:
373     started = time.perf_counter()
374     result = linprog(
375         objective,
376         A_eq=a_eq,
377         b_eq=b_eq,
378         A_ub=a_ub,
379         b_ub=b_ub,
380         bounds=bounds,
381         method="highs",
382         options={"time_limit": float(time_limit_seconds), "presolve": True},
383     )
384     return result, time.perf_counter() - started
385
386
387 def _inf_norm(matrix: csr_matrix) -> float:
388     """稀疏矩阵的  $\infty$ -范数 (最大绝对行和), 用于尺度感知的相对残差分母。"""
389     if matrix.shape[0] == 0:
390         return 0.0
391     return float(np.max(np.abs(matrix).sum(axis=1)))
392
393
394 def _relative_residual(residual: float, matrix: csr_matrix, rhs: np.ndarray, x: np.ndarray) -> float:
395     """标准尺度感知相对残差: ``residual / max(1, ||A|| $\infty$  * ||x|| $\infty$  + ||b|| $\infty$ )``。"""
396     denominator = max(
397         1.0,
398         _inf_norm(matrix) * float(np.max(np.abs(x))) + float(np.max(np.abs(rhs))),
399     )
400     return max(float(residual), 0.0) / denominator
401
402

```

```

403 def _residuals(
404     result: Any,
405     a_eq: csr_matrix,
406     b_eq: np.ndarray,
407     a_ub: csr_matrix | None,
408     b_ub: np.ndarray | None,
409     bounds: np.ndarray,
410 ) -> tuple[float, float, float, float, float]:
411     """返回 ``(等式残差, 不等式残差, 界越界量, 等式相对残差, 不等式相对残差)``。"""
412     if result.x is None:
413         return float("nan"), float("nan"), float("nan"), float("nan"), float("nan")
414     x = np.asarray(result.x, dtype=float)
415     equality = float(np.max(np.abs(a_eq @ x - b_eq)))
416     equality_relative = _relative_residual(equality, a_eq, b_eq, x)
417     if a_ub is None or a_ub.shape[0] == 0:
418         inequality, inequality_relative = 0.0, 0.0
419     else:
420         inequality = float(np.max(a_ub @ x - b_ub))
421         inequality_relative = _relative_residual(inequality, a_ub, b_ub, x)
422     lower = bounds[:, 0]
423     upper = bounds[:, 1]
424     violation_low = float(np.max(lower - x))
425     finite = np.isfinite(upper)
426     violation_up = float(np.max((x - upper)[finite])) if finite.any() else -np.inf
427     bound = max(violation_low, violation_up, 0.0)
428     return equality, inequality, bound, equality_relative, inequality_relative
429
430
431 def throughput_scale(periods: int) -> float:
432     """该层吞吐量 ``E_t(c_{d,t}+q_{dis_{d,t}})`` 的紧上界 (AS21 的 相对化分母)。"""
433     return float(periods) * (C_CAP + Q_CAP)
434
435
436 def throughput_coefficients(size: int, c_offset: int, q_dis_offset: int, periods: int) -> np.ndarray:
437     """AS21 的次目标系数向量: ``c`` 与 ``q_dis`` 位置为 1, 其余为 0 (共享算子, 所有链/模型一致)。"""
438     coefficients = np.zeros(size)
439     coefficients[c_offset : c_offset + periods] = 1.0
440     coefficients[q_dis_offset : q_dis_offset + periods] = 1.0
441     return coefficients
442
443
444 def tiebreak_epsilon(primary_reference: float, scale: float) -> float:
445     """AS21 的 ``1e-6 * max(|主目标尺度|, 1) / 吞吐量尺度`` (无量纲、确定性)。"""
446     return T7_EPS_RELATIVE * max(abs(float(primary_reference)), 1.0) / max(float(scale), 1.0)
447
448
449 def _degeneracy_proxy(
450     x: np.ndarray,
451     n_variables: int,
452     a_eq: csr_matrix,
453     a_ub: csr_matrix | None,
454     b_ub: np.ndarray | None,
455     bounds: np.ndarray,
456     *,
457     tol: float,
458 ) -> tuple[int, int]:
459     """退化维度代理 = 活跃约束数 (等式 + 紧不等式 + 贴界变量) - 变量数。"""
460     active = int(a_eq.shape[0])
461     if a_ub is not None and a_ub.shape[0]:
462         slack = np.asarray(a_ub @ x - b_ub, dtype=float).reshape(-1)
463         active += int(np.sum(slack >= -tol))
464     lower = bounds[:, 0]
465     upper = bounds[:, 1]
466     at_lower = x <= lower + tol
467     at_upper = np.isfinite(upper) & (x >= upper - tol)
468     active += int(np.sum(at_lower | at_upper))
469     return active - int(n_variables), active
470
471
472 def _layer_record(
473     result: Any,
474     seconds: float,
475     *,
476     day: int,
477     hour: int,
478     layer: str,
479     a_eq: csr_matrix,
480     b_eq: np.ndarray,
481     a_ub: csr_matrix | None,
482     b_ub: np.ndarray | None,
483     bounds: np.ndarray,

```

```

484     objective: float | None = None,
485 ) -> LayerRecord:
486     equality, inequality, bound, equality_relative, inequality_relative = _residuals(
487         result, a_eq, b_eq, a_ub, b_ub, bounds
488     )
489     return LayerRecord(
490         day=day,
491         hour=hour,
492         layer=layer,
493         status=int(result.status),
494         message=str(result.message),
495         iterations=int(result.nit) if result.nit is not None else None,
496         objective=float(objective) if objective is not None else (
497             float(result.fun) if result.fun is not None else float("nan")
498         ),
499         equality_residual_max=equality,
500         equality_residual_relative_max=equality_relative,
501         inequality_residual_max=inequality,
502         inequality_residual_relative_max=inequality_relative,
503         bound_violation_max=bound,
504         seconds=seconds,
505         variables=int(a_eq.shape[1]),
506         equality_rows=int(a_eq.shape[0]),
507         inequality_rows=int(a_ub.shape[0]) if a_ub is not None else 0,
508         nonzero=int(a_eq.nnz + (a_ub.nnz if a_ub is not None else 0)),
509     )
510
511
512 def _solve_with_tiebreak(
513     *,
514     primary_objective: np.ndarray,
515     throughput_objective: np.ndarray,
516     scale: float,
517     a_eq: csr_matrix,
518     b_eq: np.ndarray,
519     a_ub: csr_matrix | None,
520     b_ub: np.ndarray | None,
521     bounds: np.ndarray,
522     layer: str,
523     day: int,
524     hour: int,
525     boundary_index: int,
526     time_limit_seconds: float,
527 ) -> tuple[Any, LayerRecord, TiebreakRecord]:
528     """AS21 的共享求解器（承 prob03 团队 T7 的四段式：两链所有层与所有对照模型必须复用）。
529
530     **基线**：纯主目标 LP + ``P``（不变性判定的「加入次目标前」）；
531     **字典序提交解**（AS21 优先序在 ``-0`` 的精确实现）：在主目标最优面
532     ``T7_SHRINK_FACTOR`` 缩小后重跑（有界）；该式只作**诊断**，不作提交解；
533     **字典序提交解**（AS21 优先序在 ``-0`` 的精确实现）：在主目标最优面
534     ``{primary P* + 1e-9 * max(|P|,1)}`` 上**最小化**吞吐量；
535     **退化探测**：在同一主目标最优面上**最大化**吞吐量，判定是否仍存在多重最优。
536     """
537     n_variables = int(primary_objective.shape[0])
538     total_seconds = 0.0
539     primary_row = np.asarray(primary_objective, dtype=float).reshape(1, -1)
540
541     baseline_result, baseline_seconds = _solve(
542         primary_objective, a_eq, b_eq, a_ub, b_ub, bounds, time_limit_seconds=time_limit_seconds
543     )
544     total_seconds += baseline_seconds
545     if baseline_result.x is None:
546         record = _layer_record(
547             baseline_result, baseline_seconds, day=day, hour=hour, layer=layer,
548             a_eq=a_eq, b_eq=b_eq, a_ub=a_ub, b_ub=b_ub, bounds=bounds,
549         )
550     return baseline_result, record, TiebreakRecord(
551         layer=layer, day=day, hour=hour,
552         primary_before_yuan=float("nan"), primary_after_yuan=float("nan"),
553         primary_relative_change=float("inf"), epsilon=float("nan"), shrinks=0,
554         invariance_passed=False, throughput_kwh=float("nan"),
555         throughput_before_kwh=float("nan"), throughput_reduced_kwh=float("nan"),
556         baseline_state_change_max_kwh=float("nan"), degeneracy_degree=-1,
557         active_constraints=-1, variables=n_variables,
558         weighted_primary_relative_change=float("inf"), weighted_throughput_kwh=float("nan"),
559         weighted_sum_effective=False, committed_solution="none",
560         throughput_upper_on_optimal_face_kwh=None, throughput_unique=None,
561         probe_status=None, boundary_state_kwh=float("nan"),
562         baseline_seconds=baseline_seconds, solver_seconds=total_seconds,
563         note="baseline_not_optimal",
564     )

```

```

565
566 x_baseline = np.asarray(baseline_result.x, dtype=float)
567 primary_before = float(np.dot(primary_objective, x_baseline))
568 throughput_before = float(np.dot(throughput_objective, x_baseline))
569 tol_primary = T7_FACE_PRIMARY_TOL_FRACTION * T7_INVARIANCE_TOL * max(abs(primary_before), 1.0)
570
571 epsilon = tiebreak_epsilon(primary_before, scale)
572 shrinks = 0
573 weighted_result = baseline_result
574 weighted_seconds = baseline_seconds
575 weighted_primary = primary_before
576 weighted_relative = 0.0
577 for attempt in range(T7_MAX_SHRINKS + 1):
578     combined = primary_objective + epsilon * throughput_objective
579     weighted_result, weighted_seconds = _solve(
580         combined, a_eq, b_eq, a_ub, b_ub, bounds, time_limit_seconds=time_limit_seconds
581     )
582     total_seconds += weighted_seconds
583     if weighted_result.x is None:
584         break
585     x_weighted = np.asarray(weighted_result.x, dtype=float)
586     weighted_primary = float(np.dot(primary_objective, x_weighted))
587     weighted_relative = abs(weighted_primary - primary_before) / max(abs(primary_before), 1.0)
588     if weighted_relative <= T7_INVARIANCE_TOL or attempt >= T7_MAX_SHRINKS:
589         break
590     epsilon *= T7_SHRINK_FACTOR
591     shrinks += 1
592 weighted_ok = weighted_result.x is not None
593 weighted_throughput = (
594     float(np.dot(throughput_objective, np.asarray(weighted_result.x, dtype=float))) if weighted_ok
595     else float("nan")
596 )
597
598 if a_ub is not None and a_ub.shape[0]:
599     face_a_ub = vstack([a_ub, csr_matrix(primary_row)]).tocsr()
600     face_b_ub = np.concatenate([np.asarray(b_ub, dtype=float), [primary_before + tol_primary]])
601 else:
602     face_a_ub = csr_matrix(primary_row)
603     face_b_ub = np.array([primary_before + tol_primary])
604 lex_result, lex_seconds = _solve(
605     throughput_objective, a_eq, b_eq, face_a_ub, face_b_ub, bounds,
606     time_limit_seconds=time_limit_seconds,
607 )
608 total_seconds += lex_seconds
609 committed_solution = "lexicographic"
610 if lex_result.x is not None and int(lex_result.status) == 0:
611     result = lex_result
612 elif weighted_ok:
613     result = weighted_result
614     committed_solution = "weighted_sum_fallback"
615 else:
616     result = baseline_result
617     committed_solution = "baseline_fallback"
618 x = np.asarray(result.x, dtype=float)
619 primary_after = float(np.dot(primary_objective, x))
620 relative = abs(primary_after - primary_before) / max(abs(primary_before), 1.0)
621 throughput = float(np.dot(throughput_objective, x))
622 invariance_passed = bool(relative <= T7_INVARIANCE_TOL)
623 weighted_sum_effective = bool(
624     weighted_ok
625     and weighted_relative <= T7_INVARIANCE_TOL
626     and (weighted_throughput - throughput) <= max(T7_FACE_TOL_ABSOLUTE, T7_FACE_TOL_RELATIVE * abs(throughput))
627 )
628 boundary_state = float(x[boundary_index])
629 state_change = float(np.max(np.abs(x - x_baseline)))
630 degeneracy_degree, active_constraints = _degeneracy_proxy(
631     x, n_variables, a_eq, a_ub, b_ub, bounds, tol=TOL
632 )
633
634 probe_result, probe_seconds = _solve(
635     -throughput_objective, a_eq, b_eq, face_a_ub, face_b_ub, bounds,
636     time_limit_seconds=time_limit_seconds,
637 )
638 total_seconds += probe_seconds
639 if probe_result.x is not None and int(probe_result.status) == 0:
640     throughput_upper = -float(probe_result.fun)
641     throughput_unique = bool(
642         (throughput_upper - throughput) <= max(T7_FACE_TOL_ABSOLUTE, T7_FACE_TOL_RELATIVE * abs(throughput))
643     )
644     probe_status: int | None = int(probe_result.status)
645 else:

```

```

646         throughput_upper = None
647         throughput_unique = None
648         probe_status = int(probe_result.status)
649
650     note = ""
651     if not invariance_passed:
652         note = "primary_invariance_violated"
653     elif not weighted_sum_effective:
654         note = "weighted_sum_below_solver_resolution_lexicographic_committed"
655     if committed_solution != "lexicographic":
656         note = (note + "," if note else "") + f"committed={committed_solution}"
657
658     if committed_solution == "lexicographic":
659         committed_seconds = lex_seconds
660     elif committed_solution == "weighted_sum_fallback":
661         committed_seconds = weighted_seconds
662     else:
663         committed_seconds = baseline_seconds
664     record = _layer_record(
665         result, committed_seconds, day=day, hour=hour, layer=layer,
666         a_eq=a_eq, b_eq=b_eq, a_ub=a_ub, b_ub=b_ub, bounds=bounds,
667         objective=primary_after,
668     )
669     tiebreak = TiebreakRecord(
670         layer=layer,
671         day=day,
672         hour=hour,
673         primary_before_yuan=primary_before,
674         primary_after_yuan=primary_after,
675         primary_relative_change=relative,
676         epsilon=epsilon,
677         shrinks=shrinks,
678         invariance_passed=invariance_passed,
679         throughput_kwh=throughput,
680         throughput_before_kwh=throughput_before,
681         throughput_reduced_kwh=throughput_before - throughput,
682         baseline_state_change_max_kwh=state_change,
683         degeneracy_degree=degeneracy_degree,
684         active_constraints=active_constraints,
685         variables=n_variables,
686         weighted_primary_relative_change=weighted_relative,
687         weighted_throughput_kwh=weighted_throughput,
688         weighted_sum_effective=weighted_sum_effective,
689         committed_solution=committed_solution,
690         throughput_upper_on_optimal_face_kwh=throughput_upper,
691         throughput_unique=throughput_unique,
692         probe_status=probe_status,
693         boundary_state_kwh=boundary_state,
694         baseline_seconds=baseline_seconds,
695         solver_seconds=total_seconds,
696         note=note,
697     )
698     return result, record, tiebreak
699
700
701 def _balance_and_state_rows(
702     n: int,
703     *,
704     size: int,
705     pv_energy: np.ndarray,
706     load_energy: np.ndarray,
707     e_start: float,
708     balance_offsets: tuple[int, int, int, int],
709     energy_offset: int,
710 ) -> tuple[csr_matrix, np.ndarray]:
711     """构造「层内平衡 + 状态转移」两组等式（`4-2`/`4-3` 计划层/调整层共用骨架）。
712
713     ``balance_offsets = (purchase_offset, charge_offset, discharge_offset, spill_offset)``:
714     平衡式为 ``purchase + discharge - charge - spill = L * Δt - PV_{层} * Δt``。
715     """
716     purchase_off, charge_off, discharge_off, spill_off = balance_offsets
717     tau = np.arange(n, dtype=np.int64)
718     rows: list[np.ndarray] = []
719     cols: list[np.ndarray] = []
720     vals: list[np.ndarray] = []
721     rows += [tau, tau, tau, tau]
722     cols += [purchase_off + tau, discharge_off + tau, charge_off + tau, spill_off + tau]
723     vals += [np.ones(n), np.ones(n), -np.ones(n), -np.ones(n)]
724     rows += [n + tau, n + tau, n + tau[1:], n + tau]
725     cols += [energy_offset + tau, charge_off + tau, energy_offset + tau[1:] - 1, discharge_off + tau]
726     vals += [np.ones(n), -ETA_CH * np.ones(n), -np.ones(n - 1), (1.0 / ETA_DIS) * np.ones(n)]

```

```

727     a_eq = coo_matrix(
728         (np.concatenate(vals), (np.concatenate(rows), np.concatenate(cols))),
729         shape=(2 * n, size),
730     ).tocsr()
731     b_eq = np.concatenate([load_energy - pv_energy, np.concatenate([e_start, np.zeros(n - 1)])])
732     return a_eq, b_eq
733
734
735 def solve_day_plan_42(
736     *,
737     price: np.ndarray,
738     load_energy: np.ndarray,
739     pv_act_energy: np.ndarray,
740     e_start: float,
741     time_limit_seconds: float,
742     day: int = -1,
743 ) -> tuple[dict[str, np.ndarray], LayerRecord, TiebreakRecord]:
744     """链 4-2 的 单 日 计 划 层 (PL2_d):  $\min E \hat{p} \cdot b + E_{em} \hat{p} \cdot q_{em}$ 。
745
746     变量序:  $b \mid c \mid q_{dis} \mid s \mid E \mid q_{em}$  ( $6 \times 144 = 864$  变量 / 288 等式)。
747      $q_{em}$  变量与目标项**必须保留** (AS15/PL2-7), 不得因预期恒零而删除。
748     """
749     n = PERIODS_PER_DAY
750     size = 6 * n
751     objective = np.zeros(size)
752     objective[0:n] = price
753     objective[5 * n : 6 * n] = ALPHA_EM * price
754
755     spill_upper = np.maximum(0.0, pv_act_energy - load_energy)
756     bounds = np.empty((size, 2), dtype=float)
757     bounds[0:n, 0] = 0.0
758     bounds[0:n, 1] = np.inf
759     bounds[n : 2 * n, 0] = 0.0
760     bounds[n : 2 * n, 1] = C_CAP
761     bounds[2 * n : 3 * n, 0] = 0.0
762     bounds[2 * n : 3 * n, 1] = Q_CAP
763     bounds[3 * n : 4 * n, 0] = 0.0
764     bounds[3 * n : 4 * n, 1] = spill_upper
765     bounds[4 * n : 5 * n, 0] = E_MIN
766     bounds[4 * n : 5 * n, 1] = E_MAX
767     bounds[5 * n : 6 * n, 0] = 0.0
768     bounds[5 * n : 6 * n, 1] = np.inf
769
770     a_eq, b_eq = _balance_and_state_rows(
771         n,
772         size=size,
773         pv_energy=pv_act_energy,
774         load_energy=load_energy,
775         e_start=e_start,
776         balance_offsets=(0, n, 2 * n, 3 * n),
777         energy_offset=4 * n,
778     )
779     #  $q_{em}$  只出现在平衡式 (与 b 同位): 把其系数补进前 n 行
780     extra = coo_matrix(
781         (np.ones(n), (np.arange(n), 5 * n + np.arange(n))), shape=(2 * n, size)
782     ).tocsr()
783     a_eq = (a_eq + extra).tocsr()
784
785     throughput = throughput_coefficients(size, n, 2 * n, n)
786     result, record, tiebreak = _solve_with_tiebreak(
787         primary_objective=objective,
788         throughput_objective=throughput,
789         scale=throughput_scale(n),
790         a_eq=a_eq,
791         b_eq=b_eq,
792         a_ub=None,
793         b_ub=None,
794         bounds=bounds,
795         layer="plan42",
796         day=day,
797         hour=0,
798         boundary_index=size - n - 1,
799         time_limit_seconds=time_limit_seconds,
800     )
801     if result.x is None:
802         return {}, record, tiebreak
803     x = np.asarray(result.x, dtype=float)
804     return (
805         {
806             "b": x[0:n],
807             "c": x[n : 2 * n],

```

```

808         "q_dis": x[2 * n : 3 * n],
809         "s": x[3 * n : 4 * n],
810         "E": x[4 * n : 5 * n],
811         "q_em": x[5 * n : 6 * n],
812     },
813     record,
814     tiebreak,
815 )
816
817
818 def solve_plan_layer_43(
819     *,
820     price: np.ndarray,
821     load_energy: np.ndarray,
822     pv_fc_energy: np.ndarray,
823     e_start: float,
824     time_limit_seconds: float,
825     day: int = -1,
826 ) -> tuple[dict[str, np.ndarray], LayerRecord, TiebreakRecord]:
827     """链 4-3 的计划层 (PL-d):  $\min \sum \hat{p}^{(0)} \cdot b$  ( $5 \times 144 = 720$  变量 / 288 等式)。
828
829     **本层不含 q_em 变量** (承 prob03 (PL-1) / formulation E-F2)。
830     """
831     n = PERIODS_PER_DAY
832     size = 5 * n
833     objective = np.zeros(size)
834     objective[0:n] = price
835
836     spill_upper = np.maximum(0.0, pv_fc_energy - load_energy)
837     bounds = np.empty((size, 2), dtype=float)
838     bounds[0:n, 0] = 0.0
839     bounds[0:n, 1] = np.inf
840     bounds[n : 2 * n, 0] = 0.0
841     bounds[n : 2 * n, 1] = C_CAP
842     bounds[2 * n : 3 * n, 0] = 0.0
843     bounds[2 * n : 3 * n, 1] = Q_CAP
844     bounds[3 * n : 4 * n, 0] = 0.0
845     bounds[3 * n : 4 * n, 1] = spill_upper
846     bounds[4 * n : 5 * n, 0] = E_MIN
847     bounds[4 * n : 5 * n, 1] = E_MAX
848
849     a_eq, b_eq = _balance_and_state_rows(
850         n,
851         size=size,
852         pv_energy=pv_fc_energy,
853         load_energy=load_energy,
854         e_start=e_start,
855         balance_offsets=(0, n, 2 * n, 3 * n),
856         energy_offset=4 * n,
857     )
858     throughput = throughput_coefficients(size, n, 2 * n, n)
859     result, record, tiebreak = _solve_with_tiebreak(
860         primary_objective=objective,
861         throughput_objective=throughput,
862         scale=throughput_scale(n),
863         a_eq=a_eq,
864         b_eq=b_eq,
865         a_ub=None,
866         b_ub=None,
867         bounds=bounds,
868         layer="plan43",
869         day=day,
870         hour=0,
871         boundary_index=size - 1,
872         time_limit_seconds=time_limit_seconds,
873     )
874     if result.x is None:
875         return {}, record, tiebreak
876     x = np.asarray(result.x, dtype=float)
877     return (
878         {
879             "b": x[0:n],
880             "c": x[n : 2 * n],
881             "q_dis": x[2 * n : 3 * n],
882             "s": x[3 * n : 4 * n],
883             "E": x[4 * n : 5 * n],
884         },
885         record,
886         tiebreak,
887     )
888

```

```

889
890 def solve_adjustment_layer_43(
891     *,
892     hour: int,
893     price: np.ndarray,
894     load_energy: np.ndarray,
895     pv_fc_energy: np.ndarray,
896     plan_b: np.ndarray,
897     e_start: float,
898     time_limit_seconds: float,
899     day: int = -1,
900 ) -> tuple[dict[str, np.ndarray], LayerRecord, TiebreakRecord]:
901     """注 ``4-3`` 的调整层 ``(AD_{d,m})``，``m`` 为{6,12,18}"""。
902
903     变量定义在 ``R_m = {i : 6m < i < 144}`` (``n = 144 - 6m``) : ``q | c | q_dis | s | E | u | u``;
904     主目标 ``min E_{R_m}[_def * \hat{p}^{(m)} * u + _over * \hat{p}^{(m)} * u]``; ``b`` 为**参数**；
905     分段线性化 ``u - b - q``，``u - q - b``，``u \ge 0`` (LP 无损，$S-L5)。
906     调用方只提交 ``D_m`` (前 36 个时段)，``R_m`` 的尾部为前瞻。
907     """
908     global_index = np.arange(6 * hour + 1, PERIODS_PER_DAY + 1)
909     n = int(global_index.shape[0])
910     size = 7 * n
911     objective = np.zeros(size)
912     objective[5 * n : 6 * n] = BETA_DEF * price[global_index - 1]
913     objective[6 * n : 7 * n] = BETA_OVER * price[global_index - 1]
914
915     pv_slice = pv_fc_energy[global_index - 1]
916     load_slice = load_energy[global_index - 1]
917     spill_upper = np.maximum(0.0, pv_slice - load_slice)
918     bounds = np.empty((size, 2), dtype=float)
919     bounds[0:n, 0] = 0.0
920     bounds[0:n, 1] = np.inf
921     bounds[n : 2 * n, 0] = 0.0
922     bounds[n : 2 * n, 1] = C_CAP
923     bounds[2 * n : 3 * n, 0] = 0.0
924     bounds[2 * n : 3 * n, 1] = Q_CAP
925     bounds[3 * n : 4 * n, 0] = 0.0
926     bounds[3 * n : 4 * n, 1] = spill_upper
927     bounds[4 * n : 5 * n, 0] = E_MIN
928     bounds[4 * n : 5 * n, 1] = E_MAX
929     bounds[5 * n : 7 * n, 0] = 0.0
930     bounds[5 * n : 7 * n, 1] = np.inf
931
932     a_eq, b_eq = _balance_and_state_rows(
933         n,
934         size=size,
935         pv_energy=pv_slice,
936         load_energy=load_slice,
937         e_start=e_start,
938         balance_offsets=(0, n, 2 * n, 3 * n),
939         energy_offset=4 * n,
940     )
941     tau = np.arange(n, dtype=np.int64)
942     plan_slice = plan_b[global_index - 1]
943     ub_rows = [tau, tau, n + tau, n + tau]
944     ub_cols = [tau, 5 * n + tau, tau, 6 * n + tau]
945     ub_vals = [-np.ones(n), -np.ones(n), np.ones(n), -np.ones(n)]
946     a_ub = coo_matrix(
947         (np.concatenate(ub_vals), (np.concatenate(ub_rows), np.concatenate(ub_cols))),
948         shape=(2 * n, size),
949     ).tocsr()
950     b_ub = np.concatenate([-plan_slice, plan_slice])
951
952     throughput = throughput_coefficients(size, n, 2 * n, n)
953     result, record, tiebreak = _solve_with_tiebreak(
954         primary_objective=objective,
955         throughput_objective=throughput,
956         scale=throughput_scale(n),
957         a_eq=a_eq,
958         b_eq=b_eq,
959         a_ub=a_ub,
960         b_ub=b_ub,
961         bounds=bounds,
962         layer=f"adjustment{hour}",
963         day=day,
964         hour=hour,
965         boundary_index=5 * n - 1,
966         time_limit_seconds=time_limit_seconds,
967     )
968     if result.x is None:
969         return {}, record, tiebreak

```

```

970     x = np.asarray(result.x, dtype=float)
971     return (
972         {
973             "q": x[0:n],
974             "c": x[n : 2 * n],
975             "q_dis": x[2 * n : 3 * n],
976             "s": x[3 * n : 4 * n],
977             "E": x[4 * n : 5 * n],
978             "u_plus": x[5 * n : 6 * n],
979             "u_minus": x[6 * n : 7 * n],
980         },
981         record,
982         tiebreak,
983     )
984
985
986 def _deadline_hit(deadline: float | None, message: str) -> None:
987     if deadline is not None and time.perf_counter() > deadline:
988         raise BudgetExceeded(message)
989
990
991 def run_chain_42(
992     data: ChainInputs, *, time_limit_seconds: float = 60.0, deadline: float | None = None
993 ) -> ChainResult:
994     """按 formulation $2.5 执行链 ``4-2`` (``M4-2``): 逐日 1 次 LP + 闭式结算。"""
995     days = data.days
996     shape = (days, PERIODS_PER_DAY)
997     plan_b = np.zeros(shape)
998     plan_c = np.zeros(shape)
999     plan_q_dis = np.zeros(shape)
1000     plan_s = np.zeros(shape)
1001     plan_E = np.zeros(shape)
1002     plan_q_em = np.zeros(shape)
1003     q_em_settle = np.zeros(shape)
1004     s_settle = np.zeros(shape)
1005     records: list[LayerRecord] = []
1006     tiebreaks: list[TiebreakRecord] = []
1007
1008     e_prev = E_INIT
1009     for day in range(days):
1010         _deadline_hit(deadline, f"墙钟预算耗尽: 已完成 {day}/{days} 天 (4-2)")
1011         plan, record, tiebreak = solve_day_plan_42(
1012             price=data.decision_price[day],
1013             load_energy=data.load_energy[day],
1014             pv_act_energy=data.pv_act_energy[day],
1015             e_start=e_prev,
1016             time_limit_seconds=time_limit_seconds,
1017             day=day,
1018         )
1019         records.append(record)
1020         tiebreaks.append(tiebreak)
1021         if record.status != 0 or not plan:
1022             raise LayerFailure(
1023                 f"4-2 计划层 LP 未达最优 (day={day}, status={record.status}): {record.message}",
1024                 day=day,
1025                 hour=0,
1026                 status=record.status,
1027             )
1028         plan_b[day] = plan["b"]
1029         plan_c[day] = plan["c"]
1030         plan_q_dis[day] = plan["q_dis"]
1031         plan_s[day] = plan["s"]
1032         plan_E[day] = plan["E"]
1033         plan_q_em[day] = plan["q_em"]
1034         residual = plan["b"] + plan["q_dis"] + data.pv_act_energy[day] - data.load_energy[day] - plan["c"]
1035         s_settle[day] = np.maximum(residual, 0.0)
1036         q_em_settle[day] = np.maximum(-residual, 0.0)
1037         e_prev = float(plan["E"][-1])
1038
1039     return ChainResult(
1040         chain="4-2",
1041         days=days,
1042         price_act=data.price_act,
1043         price_fc=data.decision_price,
1044         decision_price=data.decision_price,
1045         load_energy=data.load_energy,
1046         pv_act_energy=data.pv_act_energy,
1047         plan_b=plan_b,
1048         q=plan_b.copy(),
1049         c=plan_c.copy(),
1050         q_dis=plan_q_dis.copy(),

```

```

1051     E=plan_E.copy(),
1052     q_em=q_em_settle,
1053     s_settle=s_settle,
1054     plan_c=plan_c,
1055     plan_q_dis=plan_q_dis,
1056     plan_s=plan_s,
1057     plan_E=plan_E,
1058     plan_q_em=plan_q_em,
1059     layer_records=records,
1060     tiebreak_records=tiebreaks,
1061     layer_pv_energy={0: data.pv_act_energy},
1062 )
1063
1064
1065 def run_chain_43(
1066     data: ChainInputs, *, time_limit_seconds: float = 60.0, deadline: float | None = None
1067 ) -> ChainResult:
1068     """按 formulation §3.6 执行链 ``4-3`` (``M4-3``): 逐日 4 次 LP + 闭式结算。"""
1069     from prob04_predict import belief_price, kappa_schedule, layer_price
1070
1071     days = data.days
1072     shape = (days, PERIODS_PER_DAY)
1073     plan_b = np.zeros(shape)
1074     plan_c = np.zeros(shape)
1075     plan_q_dis = np.zeros(shape)
1076     plan_s = np.zeros(shape)
1077     plan_E = np.zeros(shape)
1078     q = np.zeros(shape)
1079     c = np.zeros(shape)
1080     q_dis = np.zeros(shape)
1081     state = np.zeros(shape)
1082     q_em_settle = np.zeros(shape)
1083     s_settle = np.zeros(shape)
1084     price_fc = np.zeros(shape)
1085     records: list[LayerRecord] = []
1086     tiebreaks: list[TiebreakRecord] = []
1087     kappa_records: list[dict[str, Any]] = []
1088     fallback_records: list[dict[str, Any]] = []
1089
1090     pv0 = data.forecast_energy(0)
1091     e_prev = E_INIT
1092     for day in range(days):
1093         _deadline_hit(deadline, f"墙钟预算耗尽: 已完成 {day}/{days} 天 (4-3) ")
1094         schedule = kappa_schedule(data.price_act, day)
1095         kappas = {int(key): float(value) for key, value in schedule["kappa"].items()}
1096         kappa_records.extend(schedule["records"])
1097         price_fc[day] = belief_price(data.price_act, day, kappas)
1098         if day == 0:
1099             fallback_records.append(
1100                 {
1101                     "day_index": 0,
1102                     "date": "2025-01-01",
1103                     "marker": "fallback_constant_price",
1104                     "scope": "plan_layer_and_adjustment_unrealized_part",
1105                     "value": 1.0,
1106                     "reason": (
1107                         "H_d = 空集 (无前一日), (PF-HIST) 扩张窗均值无定义; 取常量中性价 1.0: "
1108                         "平坦价 无套利激励 储能保持 6000 kWh, 是最少信息的合法选择; "
1109                         "不使用当日未实现实际价、不使用未来价格、不使用全量均值"
1110                     ),
1111                 }
1112             )
1113         plan, record, tiebreak = solve_plan_layer_43(
1114             price=data.decision_price[day],
1115             load_energy=data.load_energy[day],
1116             pv_fc_energy=pv0[day],
1117             e_start=e_prev,
1118             time_limit_seconds=time_limit_seconds,
1119             day=day,
1120         )
1121         records.append(record)
1122         tiebreaks.append(tiebreak)
1123         if record.status != 0 or not plan:
1124             raise LayerFailure(
1125                 f"4-3 计划层 LP 未达最优 (day={day}, status={record.status}): {record.message}",
1126                 day=day,
1127                 hour=0,
1128                 status=record.status,
1129             )
1130     plan_b[day] = plan["b"]
1131     plan_c[day] = plan["c"]

```

```

1132     plan_q_dis[day] = plan["q_dis"]
1133     plan_s[day] = plan["s"]
1134     plan_E[day] = plan["E"]
1135     q[day, 0:COMMIT_BLOCK] = plan["b"][0:COMMIT_BLOCK]
1136     c[day, 0:COMMIT_BLOCK] = plan["c"][0:COMMIT_BLOCK]
1137     q_dis[day, 0:COMMIT_BLOCK] = plan["q_dis"][0:COMMIT_BLOCK]
1138     state[day, 0:COMMIT_BLOCK] = plan["E"][0:COMMIT_BLOCK]
1139
1140     for hour in (6, 12, 18):
1141         _deadline_hit(deadline, f"墙钟预算耗尽: day={day}, hour={hour} (4-3) ")
1142         layer_price_vector = layer_price(data.price_act, day, hour, float(kappas[hour]))
1143         pv_m = data.forecast_energy(hour)[day]
1144         layer, record, tiebreak = solve_adjustment_layer_43(
1145             hour=hour,
1146             price=layer_price_vector,
1147             load_energy=data.load_energy[day],
1148             pv_fc_energy=pv_m,
1149             plan_b=plan_b[day],
1150             e_start=float(state[day, 6 * hour - 1]),
1151             time_limit_seconds=time_limit_seconds,
1152             day=day,
1153         )
1154         records.append(record)
1155         tiebreaks.append(tiebreak)
1156         if record.status != 0 or not layer:
1157             raise LayerFailure(
1158                 f"4-3 调整层 LP 未达最优 (day={day}, hour={hour}, status={record.status}): {record.message}",
1159                 day=day,
1160                 hour=hour,
1161                 status=record.status,
1162             )
1163         commit = slice(6 * hour, 6 * hour + COMMIT_BLOCK)
1164         q[day, commit] = layer["q"][0:COMMIT_BLOCK]
1165         c[day, commit] = layer["c"][0:COMMIT_BLOCK]
1166         q_dis[day, commit] = layer["q_dis"][0:COMMIT_BLOCK]
1167         state[day, commit] = layer["E"][0:COMMIT_BLOCK]
1168
1169     residual = q[day] + q_dis[day] + data.pv_act_energy[day] - data.load_energy[day] - c[day]
1170     s_settle[day] = np.maximum(residual, 0.0)
1171     q_em_settle[day] = np.maximum(-residual, 0.0)
1172     e_prev = float(state[day, -1])
1173
1174     return ChainResult(
1175         chain="4-3",
1176         days=days,
1177         price_act=data.price_act,
1178         price_fc=price_fc,
1179         decision_price=data.decision_price,
1180         load_energy=data.load_energy,
1181         pv_act_energy=data.pv_act_energy,
1182         plan_b=plan_b,
1183         q=q,
1184         c=c,
1185         q_dis=q_dis,
1186         E=state,
1187         q_em=q_em_settle,
1188         s_settle=s_settle,
1189         plan_c=plan_c,
1190         plan_q_dis=plan_q_dis,
1191         plan_s=plan_s,
1192         plan_E=plan_E,
1193         plan_q_em=np.zeros(shape),
1194         layer_records=records,
1195         tiebreak_records=tiebreaks,
1196         kappa_records=kappa_records,
1197         fallback_records=fallback_records,
1198         layer_pv_energy={hour: data.forecast_energy(hour) for hour in DECISION_HOURS},
1199     )
1200
1201
1202 def solve_a2_pre(
1203     data: ChainInputs, *, time_limit_seconds: float = 60.0
1204 ) -> tuple[ChainResult, list[LayerRecord], list[TiebreakRecord]]:
1205     """`A2-PRE` (formulation §4.2 / 决策点 D8-A): 价格完全预知 + 附件 2 实际光伏的单一长时域 LP。
1206
1207     仅用于 ``4-2`` 的**成本下界锚点**与 ``ablation`` 对照 (``C{act}(4-2) C{pre}``, §5-L6);
1208     **不进入主结果**。变量序: ``b | c | q_dis | s | E`` (``5 * days * 144``), 无 ``q_em``、无调整层。
1209     """
1210     days = data.days
1211     n = days * PERIODS_PER_DAY
1212     size = 5 * n

```

```

1213 price_flat = data.price_act.reshape(-1)
1214 load_flat = data.load_energy.reshape(-1)
1215 pv_flat = data.pv_act_energy.reshape(-1)
1216 objective = np.zeros(size)
1217 objective[0:n] = price_flat
1218
1219 spill_upper = np.maximum(0.0, pv_flat - load_flat)
1220 bounds = np.empty((size, 2), dtype=float)
1221 bounds[0:n, 0] = 0.0
1222 bounds[0:n, 1] = np.inf
1223 bounds[n : 2 * n, 0] = 0.0
1224 bounds[n : 2 * n, 1] = C_CAP
1225 bounds[2 * n : 3 * n, 0] = 0.0
1226 bounds[2 * n : 3 * n, 1] = Q_CAP
1227 bounds[3 * n : 4 * n, 0] = 0.0
1228 bounds[3 * n : 4 * n, 1] = spill_upper
1229 bounds[4 * n : 5 * n, 0] = E_MIN
1230 bounds[4 * n : 5 * n, 1] = E_MAX
1231
1232 a_eq, b_eq = _balance_and_state_rows(
1233     n,
1234     size=size,
1235     pv_energy=pv_flat,
1236     load_energy=load_flat,
1237     e_start=E_INIT,
1238     balance_offsets=(0, n, 2 * n, 3 * n),
1239     energy_offset=4 * n,
1240 )
1241 throughput = throughput_coefficients(size, n, 2 * n, n)
1242 result, record, tiebreak = _solve_with_tiebreak(
1243     primary_objective=objective,
1244     throughput_objective=throughput,
1245     scale=throughput_scale(n),
1246     a_eq=a_eq,
1247     b_eq=b_eq,
1248     a_ub=None,
1249     b_ub=None,
1250     bounds=bounds,
1251     layer="a2_pre",
1252     day=-1,
1253     hour=-1,
1254     boundary_index=size - 1,
1255     time_limit_seconds=time_limit_seconds,
1256 )
1257 if result.x is None:
1258     raise LayerFailure(
1259         f"A2-PRE 长时域 LP 未达最优 (status={record.status}): {record.message}",
1260         day=-1,
1261         hour=-1,
1262         status=record.status,
1263     )
1264 x = np.asarray(result.x, dtype=float)
1265 b = x[0:n].reshape(days, PERIODS_PER_DAY)
1266 c = x[n : 2 * n].reshape(days, PERIODS_PER_DAY)
1267 q_dis = x[2 * n : 3 * n].reshape(days, PERIODS_PER_DAY)
1268 s = x[3 * n : 4 * n].reshape(days, PERIODS_PER_DAY)
1269 state = x[4 * n : 5 * n].reshape(days, PERIODS_PER_DAY)
1270 residual = b + q_dis + data.pv_act_energy - data.load_energy - c
1271 chain = ChainResult(
1272     chain="4-2",
1273     days=days,
1274     price_act=data.price_act,
1275     price_fc=data.decision_price,
1276     decision_price=data.decision_price,
1277     load_energy=data.load_energy,
1278     pv_act_energy=data.pv_act_energy,
1279     plan_b=b,
1280     q=b.copy(),
1281     c=c,
1282     q_dis=q_dis,
1283     E=state,
1284     q_em=np.maximum(-residual, 0.0),
1285     s_settle=np.maximum(residual, 0.0),
1286     plan_c=c,
1287     plan_q_dis=q_dis,
1288     plan_s=s,
1289     plan_E=state,
1290     plan_q_em=np.zeros_like(b),
1291     layer_records=[record],
1292     tiebreak_records=[tiebreak],
1293     layer_pv_energy={0: data.pv_act_energy},

```

```

1294     )
1295     return chain, [record], [tiebreak]
1296
1297
1298 def state_starts(result: ChainResult) -> np.ndarray:
1299     """每日 0:00 储电量 (``E_{d,0}``; ``E_{1/1,0} = E_INIT``) 。"""
1300     out = np.empty(result.days)
1301     out[0] = E_INIT
1302     if result.days > 1:
1303         out[1:] = result.E[:-1, -1]
1304     return out
1305
1306
1307 def merge_interval_labels(mask: np.ndarray) -> list[tuple[int, int]]:
1308     """把布尔掩码的连续 True 段合并为 ``(start, stop)`` 半开区间 (0 基时段下标) 。"""
1309     runs: list[tuple[int, int]] = []
1310     start: int | None = None
1311     for index, flag in enumerate(mask):
1312         if flag and start is None:
1313             start = index
1314         elif not flag and start is not None:
1315             runs.append((start, index))
1316             start = None
1317     if start is not None:
1318         runs.append((start, len(mask)))
1319     return runs
1320
1321
1322 def emergency_intervals(q_em_day: np.ndarray, *, tol: float = TOL) -> list[dict[str, Any]]:
1323     """逐日紧急购电区间 (连续 ``q_em > 0`` 时段合并 + 左端点标签 ``[_i, _j+10min]``) 。"""
1324     from prob04_io import interval_label
1325
1326     mask = np.asarray(q_em_day, dtype=float) > tol
1327     out: list[dict[str, Any]] = []
1328     for start, stop in merge_interval_labels(mask):
1329         out.append(
1330             {
1331                 "slot": interval_label(start, stop),
1332                 "start_period": int(start + 1),
1333                 "stop_period": int(stop),
1334                 "energy_kwh": round(float(np.sum(q_em_day[start:stop])), 6),
1335             }
1336         )
1337     return out
1338
1339
1340 def _round(value: float, digits: int = 6) -> float:
1341     return round(float(value), digits)
1342
1343
1344 def cost_breakdown(result: ChainResult, *, use_actual_price: bool) -> dict[str, np.ndarray]:
1345     """按「实际价」或「决策层信念价」逐日计算 ``C_plan``/``C_adj``/``C_em`` (同一组决策量, 只换价格) 。"""
1346     price = result.price_act if use_actual_price else result.price_fc
1347     b = result.plan_b
1348     q = result.q
1349     plan = np.sum(price * b, axis=1)
1350     if result.chain == "4-3":
1351         deviation_plus = np.maximum(b - q, 0.0)
1352         deviation_minus = np.maximum(q - b, 0.0)
1353         adjustment = np.sum(price * (BETA_DEF * deviation_plus + BETA_OVER * deviation_minus), axis=1)
1354     else:
1355         adjustment = np.zeros(result.days)
1356     emergency = np.sum(ALPHA_EM * price * result.q_em, axis=1)
1357     return {
1358         "plan": plan,
1359         "adjustment": adjustment,
1360         "emergency": emergency,
1361         "total": plan + adjustment + emergency,
1362     }
1363
1364
1365 def _layer_pv_and_cap(result: ChainResult) -> tuple[np.ndarray, np.ndarray]:
1366     """提交链的「支配层光伏电量」与「弃光物理上界」矩阵 (逐时段) 。
1367
1368     ``4-2``: 只有一个决策层 (全天 144 时段均由 0:00 计划支配), PV 用附件 2 实际;
1369     ``4-3``: ``(i)`` 支配 ``\Pi_{(i)}[PV] \cdot \Delta t`` (计划层 ``\Pi_0``; 调整层 ``\Pi_6/\Pi_{12}/\Pi_{18}``) 。
1370     """
1371     days = result.days
1372     nu = dominance_map() if result.chain == "4-3" else np.zeros(PERIODS_PER_DAY, dtype=int)
1373     layer_pv = np.full((days, PERIODS_PER_DAY), np.nan)
1374     for hour, matrix in result.layer_pv_energy.items():

```

```

1375     cols = np.where(nu == hour)[0]
1376     layer_pv[:, cols] = matrix[:, cols]
1377     if np.any(~np.isfinite(layer_pv)):
1378         raise ValueError("提交链的支配层光伏存在 NaN: 降尺度覆盖域异常")
1379     cap = np.maximum(0.0, layer_pv - result.load_energy)
1380     return layer_pv, cap
1381
1382
1383 def evaluate(result: ChainResult, *, probe_mode: bool, full_horizon: bool = True) -> dict[str, Any]:
1384     """计算指标、结构恒等式 ``I2-*``/``I3-*`` (逐日 + 交付期切片 + 全期) 与全部硬约束检查。
1385
1386     ``probe_mode=True`` 时**跳过**四项「解析界/量级带」硬检查（探针不构成交付证据），
1387     但仍在 stdout 打印诊断值；其余结构恒等式与边界检查一律执行。
1388     """
1389     chain = result.chain
1390     days = result.days
1391     b = result.plan_b
1392     q = result.q
1393     c = result.c
1394     q_dis = result.q_dis
1395     q_em = result.q_em
1396     s_settle = result.s_settle
1397     price = result.price_act
1398     load = result.load_energy
1399     pv_act = result.pv_act_energy
1400     starts = state_starts(result)
1401     ends = result.E[:, -1]
1402
1403     act = cost_breakdown(result, use_actual_price=True)
1404     fc = cost_breakdown(result, use_actual_price=False)
1405     cost_plan_d, cost_adj_d = act["plan"], act["adjustment"]
1406     cost_em_d, cost_total_d = act["emergency"], act["total"]
1407
1408     checks: list[dict[str, Any]] = []
1409
1410     def record(name: str, value: float, threshold: float, passed: bool, note: str = "") -> None:
1411         checks.append(
1412             {"name": name, "value": float(value), "threshold": float(threshold), "passed": bool(passed), "note": note}
1413         )
1414
1415     # ---- 逐日恒等式（两链同式；4-2 中  $q := b$ ） ----
1416     net_day = np.sum(load - pv_act, axis=1)
1417     i1_day = np.sum(q_dis, axis=1) - (ETA_ROUND_TRIP * np.sum(c, axis=1) - ETA_DIS * (ends - starts))
1418     i2_day = np.sum(q, axis=1) - (
1419         net_day
1420         + np.sum(s_settle, axis=1)
1421         + (1.0 - ETA_ROUND_TRIP) * np.sum(c, axis=1)
1422         + ETA_DIS * (ends - starts)
1423         - np.sum(q_em, axis=1)
1424     )
1425     transition = np.abs(
1426         np.diff(np.concatenate([starts[:, None], result.E], axis=1), axis=1)
1427         - ETA_CH * c
1428         + q_dis / ETA_DIS
1429     )
1430     settlement = np.abs(q + q_em + pv_act + q_dis - load - c - s_settle)
1431     continuity = np.abs(starts[1:] - ends[:-1]) if days > 1 else np.zeros(0)
1432
1433     # ---- 层内余额（提交链 vs 支配层预报，§5-L3/L4）:  $s^{\sim}[(i)] = q + \Pi \Delta t + q_{dis} - L\Delta t - c$  ----
1434     layer_pv, spill_cap = _layer_pv_and_cap(result)
1435     layer_spill = q + layer_pv + q_dis - load - c
1436     spill_upper_violation = float(np.max(layer_spill - spill_cap))
1437     spill_lower_violation = float(np.max(-layer_spill))
1438
1439     if result.layer_records:
1440         eq_max = max(item.equality_residual_max for item in result.layer_records)
1441         eq_rel_max = max(item.equality_residual_relative_max for item in result.layer_records)
1442         ineq_max = max(item.inequality_residual_max for item in result.layer_records)
1443         ineq_rel_max = max(item.inequality_residual_relative_max for item in result.layer_records)
1444         bound_max = max(item.bound_violation_max for item in result.layer_records)
1445         worst_layer = max(item.status for item in result.layer_records)
1446     else:
1447         eq_max = eq_rel_max = ineq_max = ineq_rel_max = bound_max = 0.0
1448         worst_layer = 0
1449
1450     charge_power = np.maximum.reduce(
1451         [c / DELTA_T, ETA_CH * c / DELTA_T, q_dis / DELTA_T, q_dis / (ETA_DIS * DELTA_T)]
1452     )
1453     same_period = int(np.sum((c > TOL) & (q_dis > TOL)))
1454     q_em_with_charge = int(np.sum((q_em > TOL) & (c > TOL)))
1455

```

```

1456 record("layer_status_max", float(worst_layer), 0.0, worst_layer == 0, "所有层 HiGS status = 0 (Optimal)")
1457 record("layer_equality_residual_max", eq_max, LAYER_TOL, eq_max <= LAYER_TOL,
1458 "各层等式残差 (求解器内部绝对; [等式残差 1e-6], HiGS 可行性容差 1e-7) ")
1459 record("layer_equality_residual_relative_max", eq_rel_max, LAYER_TOL_RELATIVE, eq_rel_max <= LAYER_TOL_RELATIVE,
1460 "各层等式残差 (尺度感知相对:  $\|A_{eq} x - b_{eq}\|_0 / (\|A_{eq}\|_0 \|x\|_0 + \|b_{eq}\|_0)$  ")
1461 record("layer_inequality_residual_max", ineq_max, LAYER_TOL, ineq_max <= LAYER_TOL,
1462 "各层不等式残差 (求解器内部绝对) ")
1463 record("layer_inequality_residual_relative_max", ineq_rel_max, LAYER_TOL_RELATIVE,
1464 ineq_rel_max <= LAYER_TOL_RELATIVE, "各层不等式残差 (尺度感知相对) ")
1465 record("layer_bound_violation_max", bound_max, TOL, bound_max <= TOL, "各层变量界越界量")
1466 record("cross_day_continuity", float(np.max(continuity)) if continuity.size else 0.0, TOL,
1467 (not continuity.size) or float(np.max(continuity)) <= TOL, " $|E_{d,0} - E_{d-1,144}|$  (跨日状态闭合) ")
1468 record("state_transition_residual", float(np.max(transition)), TOL, float(np.max(transition)) <= TOL,
1469 " $|E_i - E_{i-1}| - 0.9c_i + q_{dis,i}/0.9|$  (状态转移闭合) ")
1470 record("settlement_balance_residual", float(np.max(settlement)), TOL, float(np.max(settlement)) <= TOL,
1471 "结算层等式平衡  $|q + q_{em} + PV^{act} \Delta t + q_{dis} - L \Delta t - c - s'|$  1e-6")
1472 record("layer_spill_upper_violation", spill_upper_violation, BALANCE_TOL, spill_upper_violation <= BALANCE_TOL,
1473 "提交链层内弃光  $s^+_{\{i\}}$   $\max(0, \Pi_{\{i\}} \Delta t - L \Delta t)$  (层内余额上界) ")
1474 record("layer_spill_lower_violation", spill_lower_violation, BALANCE_TOL, spill_lower_violation <= BALANCE_TOL,
1475 "提交链层内弃光  $s^-_{\{i\}}$  0 (层内余额下界) ")
1476 record("storage_upper_violation", max(float(np.max(result.E)) - E_MAX, 0.0), TOL,
1477 float(np.max(result.E)) <= E_MAX + TOL, "E 10800")
1478 record("storage_lower_violation", max(E_MIN - float(np.min(result.E)), 0.0), TOL,
1479 float(np.min(result.E)) >= E_MIN - TOL, "E 1200")
1480 record("storage_initial_exact", abs(starts[0] - E_INIT), TOL, abs(starts[0] - E_INIT) <= TOL,
1481 " $E_{1/1,0} = 6000$ ")
1482 record("charge_cap_violation", max(float(np.max(c)) - C_CAP, 0.0), TOL, float(np.max(c)) <= C_CAP + TOL,
1483 "c 833.3333 (并网点侧限) ")
1484 record("discharge_cap_violation", max(float(np.max(q_dis)) - Q_CAP, 0.0), TOL,
1485 float(np.max(q_dis)) <= Q_CAP + TOL, "q_dis 750.0000 (电池侧限) ")
1486 record("settle_spill_bound_violation", max(float(np.max(s_settle - pv_act)), 0.0), TOL,
1487 bool(np.all(s_settle <= pv_act + TOL)), "0 s'  $PV^{act} \Delta t$ ")
1488 record("plan_nonneg_violation", max(-float(np.min(b)), 0.0), TOL, float(np.min(b)) >= -TOL, "b 0")
1489 record("purchase_nonneg_violation", max(-float(np.min(q)), 0.0), TOL, float(np.min(q)) >= -TOL, "q 0")
1490 record("q_em_nonneg_violation", max(-float(np.min(q_em)), 0.0), TOL, float(np.min(q_em)) >= -TOL, "q_em 0")
1491 record("max_side_power_kw", float(np.max(charge_power)), P_MAX, float(np.max(charge_power)) <= P_MAX + 1e-3,
1492 "并网点侧与电池侧换算功率 (四路) 均 5000 kW")
1493 record("identity_I1_residual", float(np.max(np.abs(i1_day))), TOL, float(np.max(np.abs(i1_day))) <= TOL,
1494 " $E_{q_{dis}} = E_c - (E_T - E_0)$  (逐日最大残差) ")
1495 record("identity_I2_residual", float(np.max(np.abs(i2_day))), TOL, float(np.max(np.abs(i2_day))) <= TOL,
1496 " $E_q = N + E_s + 0.19E_c + \Delta E - E_{q_{em}}$  (逐日最大残差) ")
1497
1498 if chain == "4-3":
1499 record("q_eq_b_first_block", float(np.max(np.abs(q[:, 0:COMMIT_BLOCK] - b[:, 0:COMMIT_BLOCK])), TOL,
1500 float(np.max(np.abs(q[:, 0:COMMIT_BLOCK] - b[:, 0:COMMIT_BLOCK])) <= TOL,
1501 "i 36 时 q = b (计划层提交口径 AD-1) ")
1502 covered = np.zeros((days, PERIODS_PER_DAY), dtype=int)
1503 for hour in (6, 12, 18):
1504 covered[:, 6 * hour : 6 * hour + COMMIT_BLOCK] += 1
1505 covered[:, 0:COMMIT_BLOCK] += 1
1506 record("commit_partition_exact", float(int(np.sum(covered != 1))), 0.0, bool(np.all(covered == 1)),
1507 "提交时段划分 D_0 D_6 D_12 D_18 恰为 144 个时段且互不重叠 i 6m 只被写入一次、"
1508 "不回溯 (AS18/(AD-1)) 由提交链构造保证")
1509 else:
1510 record("chain42_q_em_expected_zero", float(np.max(result.plan_q_em)), TOL,
1511 float(np.max(result.plan_q_em)) <= TOL,
1512 "4-2 计划层 q_em 0 (§2.3 定理:  $_{em} = 5 > 1$ , 任一最优解把 q_em 换成 b 严格改进) ")
1513 record("chain42_settle_q_em_expected_zero", float(np.max(q_em)), TOL, float(np.max(q_em)) <= TOL,
1514 "4-2 结算层 q_em = max(0, -r) 0 (计划层等式平衡精确闭合 r = s 0) ")
1515
1516 total_act = float(np.sum(cost_total_d))
1517 plan_act = float(np.sum(cost_plan_d))
1518 adj_act = float(np.sum(cost_adj_d))
1519 em_act = float(np.sum(cost_em_d))
1520 record("cost_decomposition_identity", total_act - (plan_act + adj_act + em_act), 1e-6,
1521 abs(total_act - (plan_act + adj_act + em_act)) <= 1e-6,
1522 " $C_{total}^{act} = C_{plan}^{act} + C_{adj}^{act} + C_{em}^{act}$  (4-2 的 C_adj 恒 0) ")
1523 fc_total = float(np.sum(fc["total"]))
1524 fc_plan = float(np.sum(fc["plan"]))
1525 fc_adj = float(np.sum(fc["adjustment"]))
1526 fc_em = float(np.sum(fc["emergency"]))
1527 record("cost_decomposition_identity_fc", fc_total - (fc_plan + fc_adj + fc_em), 1e-6,
1528 abs(fc_total - (fc_plan + fc_adj + fc_em)) <= 1e-6,
1529 " $C_{total}^{fc} = C_{plan}^{fc} + C_{adj}^{fc} + C_{em}^{fc}$  (决策层信念价口径, 同一组决策量) ")
1530 delta_price = price - result.price_fc
1531 delta_plan = float(np.sum(delta_price * b))
1532 delta_em = float(np.sum(ALPHA_EM * delta_price * q_em))
1533 delta_adj = (
1534 float(np.sum(delta_price * (BETA_DEF * np.maximum(b - q, 0.0) + BETA_OVER * np.maximum(q - b, 0.0))))
1535 if chain == "4-3" else 0.0
1536 )

```

```

1537 record("delta_c_price_identity", (total_act - fc_total) - (delta_plan + delta_adj + delta_em),
1538      abs((total_act - fc_total) - (delta_plan + delta_adj + delta_em)) <= 1e-6,
1539      "AC_price = AC_plan + AC_adj + AC_em, 其中 AC_* =  $\Sigma(p^{\text{act}} - \hat{p}^{\text{fc}}) \cdot \text{对应量 (同一组决策量)}$ ")
1540
1541 # ---- AS21 逐层不变性与退化审计 ----
1542 t7 = result.tiebreak_records
1543 if t7:
1544     t7_rel_max = _finite_or(max(item.primary_relative_change for item in t7))
1545     t7_all_passed = bool(all(item.invariance_passed for item in t7))
1546     t7_shrinks = int(sum(item.shrinks for item in t7))
1547     t7_degenerate_layers = int(sum(1 for item in t7 if item.degeneracy_degree > 0))
1548     t7_remaining = int(sum(1 for item in t7 if item.throughput_unique is False))
1549     t7_unknown = int(sum(1 for item in t7 if item.throughput_unique is None))
1550     t7_throughput = _finite_or(float(np.sum([item.throughput_kwh for item in t7])))
1551     t7_throughput_before = _finite_or(float(np.sum([item.throughput_before_kwh for item in t7])))
1552     t7_state_change = _finite_or(float(np.max([item.baseline_state_change_max_kwh for item in t7])))
1553     t7_eps_min = _finite_or(float(np.min([item.epsilon for item in t7])))
1554     t7_eps_max = _finite_or(float(np.max([item.epsilon for item in t7])))
1555     t7_max_degeneracy = int(max(item.degeneracy_degree for item in t7))
1556     t7_weighted_rel_max = _finite_or(max(item.weighted_primary_relative_change for item in t7))
1557     t7_weighted_effective = int(sum(1 for item in t7 if item.weighted_sum_effective))
1558     t7_lexicographic = int(sum(1 for item in t7 if item.committed_solution == "lexicographic"))
1559     t7_fallbacks = int(sum(1 for item in t7 if item.committed_solution != "lexicographic"))
1560 else:
1561     t7_rel_max, t7_all_passed, t7_shrinks = 0.0, True, 0
1562     t7_degenerate_layers = t7_remaining = t7_unknown = 0
1563     t7_throughput = t7_throughput_before = t7_state_change = 0.0
1564     t7_eps_min = t7_eps_max = 0.0
1565     t7_max_degeneracy = t7_weighted_effective = t7_lexicographic = t7_fallbacks = 0
1566     t7_weighted_rel_max = 0.0
1567 record("t7_primary_invariance_max_relative_change", t7_rel_max, T7_INVARIANCE_TOL, t7_rel_max <= T7_INVARIANCE_TOL,
1568      "AS21/T7-2: 加入次目标前后主目标相对变化 (逐层最大值, 须  $1e-9$ )")
1569 record("t7_invariance_all_layers_passed", float(int(not t7_all_passed)), 0.0, t7_all_passed,
1570      "AS21/T7-2: 全部逐层记录 invariance_passed=true")
1571 record("t7_epsilon_below_primary_scale", t7_eps_max, 1.0, t7_eps_max < 1.0,
1572      "AS21/T7-1: 为无量纲相对权重 (1); 次目标总量  $1e-6 \cdot \text{主目标尺度}$ ")
1573
1574 # ---- 交付期切片 ----
1575 delivery = np.zeros(days, dtype=bool)
1576 delivery_start = min(D_REQ_START, days)
1577 delivery[delivery_start:] = True
1578 if delivery.any():
1579     d_start_state = float(starts[delivery_start])
1580     d_end_state = float(ends[-1])
1581     net_d = float(np.sum((load - pv_act)[delivery]))
1582     i1_delivery = float(
1583         np.sum(q.dis[delivery]) - (ETA_ROUND_TRIP * np.sum(c[delivery]) - ETA_DIS * (d_end_state - d_start_state))
1584     )
1585     i2_delivery = float(
1586         np.sum(q[delivery])
1587         - (
1588             net_d
1589             + float(np.sum(s_settle[delivery]))
1590             + (1.0 - ETA_ROUND_TRIP) * float(np.sum(c[delivery]))
1591             + ETA_DIS * (d_end_state - d_start_state)
1592             - float(np.sum(q_em[delivery]))
1593         )
1594     )
1595     record("identity_I1_delivery_residual", i1_delivery, TOL, abs(i1_delivery) <= TOL, "交付期切片 (I1)/(I3-1)")
1596     record("identity_I2_delivery_residual", i2_delivery, TOL, abs(i2_delivery) <= TOL,
1597      "交付期切片 (I2)/(I3-8):  $E_q = N_{\text{req}} + \Sigma s' + 0.19E_c + (E_T - E_{(2/1,0)}) - E_q^{\text{em}}$ ")
1598 else:
1599     net_d = i1_delivery = i2_delivery = 0.0
1600     d_start_state = d_end_state = 0.0
1601
1602 # ---- 解析界与量级带 (正式模式硬检查; 探针模式只打印诊断, 但**仍然计算**这些参考值) ----
1603 bounds: dict[str, Any] = {}
1604 if delivery.any():
1605     p_min = float(np.min(price))
1606     lower = p_min * float(np.sum(b[delivery]))
1607     policy_b = np.maximum(load - pv_act, 0.0)
1608     policy_q = np.empty_like(q)
1609     if chain == "4-3":
1610         nu = dominance_map()
1611         for hour, matrix in result.layer_pv_energy.items():
1612             cols = np.where(nu == hour)[0]
1613             policy_q[:, cols] = np.maximum(load[:, cols] - matrix[:, cols], 0.0)
1614     else:
1615         policy_q = policy_b.copy()
1616     policy_plus = np.maximum(policy_b - policy_q, 0.0)
1617     policy_minus = np.maximum(policy_q - policy_b, 0.0)

```

```

1618 policy_em = np.maximum(load - policy_q - pv_act, 0.0)
1619 policy_plan = float(np.sum(price[delivery] * policy_b[delivery]))
1620 policy_adj = float(
1621     np.sum(price[delivery] * (BETA_DEF * policy_plus[delivery] + BETA_OVER * policy_minus[delivery]))
1622 ) if chain == "4-3" else 0.0
1623 policy_emc = float(np.sum(ALPHA_EM * price[delivery] * policy_em[delivery]))
1624 upper = policy_plan + policy_adj + policy_emc
1625 total_act_delivery = float(np.sum(cost_total_d[delivery]))
1626 mean_daily = total_act_delivery / max(int(delivery.sum()), 1)
1627 bounds = {
1628     "delivery_plan_lower_bound_yuan": _round(lower),
1629     "delivery_plan_lower_bound_formula": "p^act_min * E_{D_req} b (C_total^act C_plan^act p_min * E_b)",
1630     "delivery_constructed_upper_bound_yuan": _round(upper),
1631     "delivery_upper_bound_parts_yuan": {
1632         "plan": _round(policy_plan),
1633         "adjustment": _round(policy_adj),
1634         "emergency": _round(policy_emc),
1635     },
1636     "delivery_upper_bound_formula": (
1637         "无储能可行策略 (c = q_dis = 0、各层 q/b 取该层预报净负荷、s 取该层物理盈余): "
1638         "E^p^act * b_policy + E[0.5p^act(b-q)^+ + 1.5p^act(q-b)^+] + E5p^act * q^em_policy"
1639         " (4-2 无 C_adj 项)"
1640     ),
1641     "mean_daily_delivery_cost_yuan": _round(mean_daily),
1642     "magnitude_band_delivery_yuan": list(DELIVERY_MAGNITUDE_BAND_YUAN),
1643     "magnitude_band_daily_yuan": list(DAILY_MAGNITUDE_BAND_YUAN),
1644     "magnitude_band_delivery_ok": bool(
1645         DELIVERY_MAGNITUDE_BAND_YUAN[0] <= total_act_delivery <= DELIVERY_MAGNITUDE_BAND_YUAN[1]
1646     ),
1647     "magnitude_band_daily_ok": bool(
1648         DAILY_MAGNITUDE_BAND_YUAN[0] <= mean_daily <= DAILY_MAGNITUDE_BAND_YUAN[1]
1649     ),
1650     "delivery_cost_total_act_yuan": _round(total_act_delivery),
1651     "hard_checks_applied": bool(full_horizon and not probe_mode),
1652 }
1653 if not probe_mode:
1654     record("analytic_lower_bound", total_act_delivery, lower, total_act_delivery >= lower - 1e-6,
1655         "交付期 C_total^act p^act_min * E_b (构造性下界)")
1656     record("analytic_upper_bound", total_act_delivery, upper, total_act_delivery <= upper + 1e-6,
1657         "交付期 C_total^act 无储能可行策略的构造上界")
1658     record("magnitude_delivery_band", total_act_delivery, DELIVERY_MAGNITUDE_BAND_YUAN[1],
1659         DELIVERY_MAGNITUDE_BAND_YUAN[0] <= total_act_delivery <= DELIVERY_MAGNITUDE_BAND_YUAN[1],
1660         "交付期费用须落在 [3e6, 5e7] 元 (10^7 量级), 防 Δt 误乘")
1661     record("magnitude_daily_band", mean_daily, DAILY_MAGNITUDE_BAND_YUAN[1],
1662         DAILY_MAGNITUDE_BAND_YUAN[0] <= mean_daily <= DAILY_MAGNITUDE_BAND_YUAN[1],
1663         "日均费用须落在 [5e3, 2e5] 元; 落入 10^3 元即 Δt 硬错误")
1664
1665 # ---- 统计量 (不作硬失败) ----
1666 clip_events = [item for item in result.kappa_records if item.get("clipped")]
1667 statistics = {
1668     "simultaneous_charge_discharge_periods": same_period,
1669     "periods_with_q_em_and_charge": q_em_with_charge,
1670     "periods_with_spill": int(np.sum(s_settle > TOL)),
1671     "days_with_emergency": int(np.sum(np.any(q_em > TOL, axis=1))),
1672     "periods_with_emergency": int(np.sum(q_em > TOL)),
1673     "max_purchase_kwh": _round(float(np.max(q))),
1674     "max_purchase_day_kwh": _round(float(np.max(np.sum(q, axis=1)))),
1675     "kappa_clip_events": len(clip_events),
1676     "kappa_clip_records": clip_events,
1677     "kappa0_identity_holds": bool(
1678         all(item["kappa"] == 1.0 for item in result.kappa_records if int(item["m"]) == 0)
1679     ),
1680     "kappa_records": len(result.kappa_records),
1681     "degenerate_layers": t7_degenerate_layers,
1682     "layers_with_remaining_multiplicity": t7_remaining,
1683     "note": (
1684         "同充放时段数与 q_em>0 且 c>0 时段数属 LP 退化的记录量, 不作硬失败;"
1685         "4-2 的 E_q_em 预期恒 0 (§2.3 定理), 4-3 的 E_q^em 预期 > 0 (预报-实际光伏缺口)。"
1686     ),
1687 }
1688
1689 delivery_slice = delivery if delivery.any() else np.zeros(days, dtype=bool)
1690
1691 def _sum(array: np.ndarray) -> float:
1692     return _round(float(np.sum(array[delivery_slice]))) if delivery_slice.any() else 0.0
1693
1694 belief = {
1695     "price_basis": (
1696         "决策层信念价 \\hat p^{fc}_{d,i} = \\hat p^{((i))}_{d,i}"
1697         " (4-2: \\hat p = PF-PERSIST; 4-3: _0 1 与 _6/_12/_18 分段)"
1698     ),
1699 }

```

```

1699     "cost_plan_fc_yuan": _round(fc_plan),
1700     "cost_adj_fc_yuan": _round(fc_adj),
1701     "cost_em_fc_yuan": _round(fc_em),
1702     "cost_total_fc_yuan": _round(fc_total),
1703     "delta_c_price_yuan": _round(total_act - fc_total),
1704     "delta_c_plan_yuan": _round(plan_act - fc_plan),
1705     "delta_c_adj_yuan": _round(adj_act - fc_adj),
1706     "delta_c_em_yuan": _round(em_act - fc_em),
1707     "same_decisions": True,
1708     "note": "同一组已提交决策量，仅替换计价价格（不重解模型）；AC_price 的符号按实测（P3），不得预设为正。",
1709 }
1710
1711 series = {
1712     "period_index": list(range(1, days * PERIODS_PER_DAY + 1)),
1713     "day_index": [int(v) for v in np.repeat(np.arange(days), PERIODS_PER_DAY)],
1714     "price_actual_yuan_per_kwh": [float(v) for v in price.reshape(-1)],
1715     "price_belief_yuan_per_kwh": [float(v) for v in result.price_fc.reshape(-1)],
1716     "load_energy_kwh": [float(v) for v in load.reshape(-1)],
1717     "pv_actual_energy_kwh": [float(v) for v in pv_act.reshape(-1)],
1718     "plan_purchase_kwh": [float(v) for v in b.reshape(-1)],
1719     "final_purchase_kwh": [float(v) for v in q.reshape(-1)],
1720     "q_em_kwh": [float(v) for v in q_em.reshape(-1)],
1721     "charge_kwh": [float(v) for v in c.reshape(-1)],
1722     "discharge_kwh": [float(v) for v in q_dis.reshape(-1)],
1723     "spill_settlement_kwh": [float(v) for v in s_settle.reshape(-1)],
1724     "storage_kwh": [float(v) for v in result.E.reshape(-1)],
1725 }
1726
1727 return {
1728     "chain": chain,
1729     "probe_mode": bool(probe_mode),
1730     "objective_yuan": _round(total_act),
1731     "cost_plan_yuan": _round(plan_act),
1732     "cost_adj_yuan": _round(adj_act),
1733     "cost_em_yuan": _round(em_act),
1734     "belief": belief,
1735     "delivery": {
1736         "cost_total_yuan": _round(float(np.sum(cost_total_d[delivery_slice]))) if delivery_slice.any() else 0.0,
1737         "cost_plan_yuan": _round(float(np.sum(cost_plan_d[delivery_slice]))) if delivery_slice.any() else 0.0,
1738         "cost_adj_yuan": _round(float(np.sum(cost_adj_d[delivery_slice]))) if delivery_slice.any() else 0.0,
1739         "cost_em_yuan": _round(float(np.sum(cost_em_d[delivery_slice]))) if delivery_slice.any() else 0.0,
1740         "cost_total_fc_yuan": _round(float(np.sum(fc["total"][delivery_slice]))) if delivery_slice.any() else 0.0,
1741         "delta_c_price_yuan": (
1742             _round(float(np.sum((cost_total_d - fc["total"])[delivery_slice]))) if delivery_slice.any() else 0.0
1743         ),
1744         "total_plan_kwh": _sum(b),
1745         "total_purchase_kwh": _sum(q),
1746         "total_q_em_kwh": _sum(q_em),
1747         "total_charge_kwh": _sum(c),
1748         "total_discharge_kwh": _sum(q_dis),
1749         "total_spill_kwh": _sum(s_settle),
1750         "net_load_kwh": _round(net_d),
1751         "storage_start_kwh": _round(d_start_state),
1752         "storage_final_kwh": _round(d_end_state),
1753         "days": int(delivery_slice.sum()),
1754         "first_date_index": int(delivery_start),
1755         "last_date_index": days - 1,
1756     },
1757     "totals": {
1758         "total_plan_kwh": _round(float(np.sum(b))),
1759         "total_purchase_kwh": _round(float(np.sum(q))),
1760         "total_q_em_kwh": _round(float(np.sum(q_em))),
1761         "total_charge_kwh": _round(float(np.sum(c))),
1762         "total_discharge_kwh": _round(float(np.sum(q_dis))),
1763         "total_spill_kwh": _round(float(np.sum(s_settle))),
1764         "net_load_kwh": _round(float(np.sum(net_day))),
1765         "max_charge_kwh": _round(float(np.max(c))),
1766         "max_discharge_kwh": _round(float(np.max(q_dis))),
1767         "max_plan_purchase_kwh": _round(float(np.max(b))),
1768         "max_final_purchase_kwh": _round(float(np.max(q))),
1769         "max_side_power_kw": _round(float(np.max(charge_power))),
1770         "storage_initial_kwh": _round(E_INIT),
1771         "storage_final_kwh": _round(float(ends[-1])),
1772         "storage_range_kwh": [_round(float(np.min(result.E))), _round(float(np.max(result.E)))],
1773         "days_with_emergency": int(np.sum(np.any(q_em > TOL, axis=1))),
1774         "periods_with_emergency": int(np.sum(q_em > TOL)),
1775         "simultaneous_charge_discharge_periods": same_period,
1776         "periods_with_q_em_and_charge": q_em_with_charge,
1777     },
1778     "identities": {
1779         "I1_daily_max_abs": float(np.max(np.abs(i1_day))),

```

```

1780         "I2_daily_max_abs": float(np.max(np.abs(i2_day))),
1781         "I1_delivery_residual": float(i1_delivery),
1782         "I2_delivery_residual": float(i2_delivery),
1783         "continuity_residual_max": float(np.max(continuity)) if continuity.size else 0.0,
1784         "transition_residual_max": float(np.max(transition)),
1785         "settlement_residual_max": float(np.max(settlement)),
1786         "layer_spill_upper_violation": spill_upper_violation,
1787         "layer_spill_lower_violation": spill_lower_violation,
1788     },
1789     "bounds": bounds,
1790     "statistics": statistics,
1791     "t7_tiebreak": {
1792         "rule": T7_TIEBREAK_RULE,
1793         "eps_relative": T7_EPS_RELATIVE,
1794         "invariance_tol": T7_INVARIANCE_TOL,
1795         "shrink_factor": T7_SHRINK_FACTOR,
1796         "max_shrinks": T7_MAX_SHRINKS,
1797         "layers": len(t7),
1798         "max_primary_relative_change": t7_rel_max,
1799         "all_layers_invariance_passed": t7_all_passed,
1800         "total_shrinks": t7_shrinks,
1801         "epsilon_min": t7_eps_min,
1802         "epsilon_max": t7_eps_max,
1803         "weighted_sum_max_primary_relative_change": t7_weighted_rel_max,
1804         "weighted_sum_effective_layers": t7_weighted_effective,
1805         "lexicographic_committed_layers": t7_lexicographic,
1806         "fallback_committed_layers": t7_fallbacks,
1807         "throughput_primary_only_kwh": t7_throughput_before,
1808         "throughput_tiebreak_kwh": t7_throughput,
1809         "throughput_reduced_kwh": t7_throughput_before - t7_throughput,
1810         "max_baseline_trajectory_change_kwh": t7_state_change,
1811         "degenerate_layers": t7_degenerate_layers,
1812         "max_degeneracy_degree": t7_max_degeneracy,
1813         "layers_with_remaining_multiplicity": t7_remaining,
1814         "layers_with_unknown_multiplicity": t7_unknown,
1815         "note": (
1816             "AS21: 提交解为主目标最优面上吞吐量最小的字典序解 (T7-1 优先序在 +0 的精确实现), "
1817             "字面 加权式只作逐层诊断; degeneracy_degree = 活跃约束数 - 变量数。"
1818         ),
1819     },
1820     "daily": {
1821         "dates_index": list(range(days)),
1822         "plan_kwh": [_round(v) for v in np.sum(b, axis=1)],
1823         "purchase_kwh": [_round(v) for v in np.sum(q, axis=1)],
1824         "q_em_kwh": [_round(v) for v in np.sum(q_em, axis=1)],
1825         "charge_kwh": [_round(v) for v in np.sum(c, axis=1)],
1826         "discharge_kwh": [_round(v) for v in np.sum(q_dis, axis=1)],
1827         "spill_kwh": [_round(v) for v in np.sum(s_settle, axis=1)],
1828         "cost_plan_yuan": [_round(v) for v in cost_plan_d],
1829         "cost_adj_yuan": [_round(v) for v in cost_adj_d],
1830         "cost_em_yuan": [_round(v) for v in cost_em_d],
1831         "cost_total_yuan": [_round(v) for v in cost_total_d],
1832         "cost_total_fc_yuan": [_round(v) for v in fc["total"]],
1833         "delta_c_price_yuan": [_round(v) for v in (cost_total_d - fc["total"])],
1834         "state_start_kwh": [_round(v) for v in starts],
1835         "state_end_kwh": [_round(v) for v in ends],
1836         "I1_residual": [_round(v) for v in i1_day],
1837         "I2_residual": [_round(v) for v in i2_day],
1838     },
1839     "checks": checks,
1840     "checks_failed": [item["name"] for item in checks if not item["passed"]],
1841     "series": series,
1842 }
1843
1844
1845 __all__ = [
1846     "ALPHA_EM",
1847     "BALANCE_TOL",
1848     "BETA_DEF",
1849     "BETA_OVER",
1850     "BLOCKS_PER_DAY",
1851     "BLOCK_ROWS",
1852     "BudgetExceeded",
1853     "C_CAP",
1854     "COMMIT_BLOCK",
1855     "CROSS_YEAR_DROPPED",
1856     "ChainInputs",
1857     "ChainResult",
1858     "DAILY_MAGNITUDE_BAND_YUAN",
1859     "DAYS_FULL",
1860     "DECISION_HOURS",

```

```

1861 "DELIVERY_MAGNITUDE_BAND_YUAN",
1862 "DELTA_T",
1863 "D_REQ_START",
1864 "E_INIT",
1865 "E_MAX",
1866 "E_MIN",
1867 "ETA_CH",
1868 "ETA_DIS",
1869 "ETA_ROUND_TRIP",
1870 "LAYER_TOL",
1871 "LAYER_TOL_RELATIVE",
1872 "LayerFailure",
1873 "LayerRecord",
1874 "P_MAX",
1875 "PERIODS_PER_DAY",
1876 "Q_CAP",
1877 "SPEC_DATES",
1878 "TABLE1_SLOTS",
1879 "TOL",
1880 "T7_EPS_RELATIVE",
1881 "T7_INVARIANCE_TOL",
1882 "T7_MAX_SHRINKS",
1883 "T7_SHRINK_FACTOR",
1884 "T7_TIEBREAK_RULE",
1885 "TiebreakRecord",
1886 "block_energy_bias",
1887 "cost_breakdown",
1888 "cross_year_dropped_count",
1889 "dominance_map",
1890 "downscale",
1891 "emergency_intervals",
1892 "evaluate",
1893 "merge_interval_labels",
1894 "run_chain_42",
1895 "run_chain_43",
1896 "solve_a2_pre",
1897 "solve_adjustment_layer_43",
1898 "solve_day_plan_42",
1899 "solve_plan_layer_43",
1900 "state_starts",
1901 "throughput_coefficients",
1902 "throughput_scale",
1903 "tiebreak_epsilon",
1904 ]

```

B.12 问题四：价格预测器

```

1  # -*- coding: utf-8 -*-
2  """prob04 价格预测器、滚动回测与 ``4-3`` 调整层的滚动水平校正 ``_m``。
3
4  严格对应 ``formulations/formulation_v001`` §1.2-§1.5 与 ``assumption_v001`` 的 AS08/AS11/AS22,
5  以及团队 ``A8-(b)`` 派生  :
6
7  ...
8  (PF-PERSIST) \\hat p_{d,i} = p^{act}_{d-1,i} (主口径, 两链共用)
9  (PF-HIST) \\hat p_{d,i} = mean_{d'} H_d p^{act}_{d',i} (强制基准)
10 (PF-DUAL) s_i = mean_i(\\bar p)/mean(\\bar p); _d = mean_{d'} L_d mean_i(p^{act}_{d',i})
11 \\hat p_{d,i} = _d * s_i (强制基准)
12 (PF-AR) \\hat p_{d,i} = a_i + _i * p^{act}_{d-1,i} (ablation 对照)
13 (K-3) _m = clip(A_m/B_m, 0.5, 2.0), A_m/B_m 为当日/前一日 i 6m 均值; _0 1
14 ...
15
16 **信息集纪律 (AS07/AS22) **: ``H_d`` = ``{d' : d' < d}`` 是**显式入参**；本模块的任何函数都只能通过
17 ``history``/``day_index`` 访问价格，不得隐式读取 ``d`` `d`` 的任何价格。禁止出现「全天电价已知」类表述。
18
19 **2025-01-01 回退 (forced, formulation §1.3.4) **: ``H_d`` = ```` 时 (PF-HIST) 的扩张窗均值无定义，
20 故该日决策价取常量中性价 ``\\hat p_{1,i} = 1.0`` (``fallback_constant_price``)：平坦价 无套利激励
21 储能保持 6000 kWh，是最少信息的合法选择；该日**不参与回测** (``D_full`` 计 364 天)。
22 """
23 from __future__ import annotations
24
25 import numpy as np
26
27 DAYS_FULL = 365
28 PERIODS_PER_DAY = 144
29 D_REQ_START = 31 # 0 基日索引：2025-02-01 (1 月为预热期)
30 FALLBACK_CONSTANT_PRICE = 1.0 # 2025-01-01 的常量中性决策价 (元/kWh)
31 DUAL_RECENT_DAYS = 7
32 AR_MIN_PAIRS = 3 # PF-AR 逐时段 OLS 的最少配对样本数 (见 pf_ar 的退化说明)

```

```

33 KAPPA_LOWER = 0.5
34 KAPPA_UPPER = 2.0
35 DECISION_HOURS: tuple[int, ...] = (0, 6, 12, 18)
36 PRIMARY_METHOD = "PF-PERSIST"
37 BASELINE_METHODS: tuple[str, ...] = ("PF-PERSIST", "PF-DUAL", "PF-HIST")
38 ABLATION_METHODS: tuple[str, ...] = ("PF-AR",)
39 ALL_METHODS: tuple[str, ...] = ("PF-PERSIST", "PF-DUAL", "PF-HIST", "PF-AR")
40 # 「更简者」判定用（选择规则 BT-5 的并列裁决）：数值越小越简。
41 SIMPLICITY_RANK: dict[str, int] = {"PF-PERSIST": 0, "PF-HIST": 1, "PF-DUAL": 2, "PF-AR": 3}
42 MAE_TIE_RELATIVE_TOL = 0.01 # MAE 相对差 < 1% 视为并列
43 METHOD_FORMULAS: dict[str, str] = {
44     "PF-PERSIST": "hat p_{d,i} = p^{act}_{d-1,i} (主口径)",
45     "PF-HIST": "hat p_{d,i} = mean_{d' in H_d} p^{act}_{d',i} (扩张窗历史同时段均值)",
46     "PF-DUAL": "hat p_{d,i} = mu_d * s_i; s_i 为归一化历史形状 (mean_i(s_i)=1), mu_d 为最近 7 个可用日的日内均价均值",
47     "PF-AR": (
48         "hat p_{d,i} = a_i + phi_i * p^{act}_{d-1,i} (逐时段 OLS, 系数只用 H_d 估计; "
49         f"配对样本 < {AR_MIN_PAIRS} 组时回退 PF-HIST; ablation 对照)"
50     ),
51 }
52
53
54 def available_history(day_index: int, *, days: int = DAYS_FULL) -> list[int]:
55     """`H_d = { d' : d' < d }` (显式入参, 防止信息集越界; AS22 / (G-5)). """
56     if day_index < 0:
57         raise ValueError("day_index 必须非负")
58     return list(range(min(day_index, days)))
59
60
61 def _history_rows(price: np.ndarray, history: list[int]) -> np.ndarray:
62     if not history:
63         raise ValueError("空历史集合在 (PF-HIST)/(PF-DUAL)/(PF-AR) 中无定义")
64     return np.asarray(price, dtype=float)[np.asarray(history, dtype=int), :]
65
66
67 def fallback_price(periods: int = PERIODS_PER_DAY) -> np.ndarray:
68     """`H_d = `` (仅 2025-01-01) 时的常量中性决策价 ``\hat p_{1.0}``. """
69     return np.full(periods, FALLBACK_CONSTANT_PRICE, dtype=float)
70
71
72 def pf_persist(price: np.ndarray, day_index: int, history: list[int]) -> np.ndarray:
73     """(PF-PERSIST) 前一日同时段; `H_d = `` 时返回常量中性价 (回退规则)`. """
74     if not history:
75         return fallback_price(int(np.asarray(price).shape[1]))
76     return np.array(price[day_index - 1], dtype=float, copy=True)
77
78
79 def pf_hist(price: np.ndarray, day_index: int, history: list[int]) -> np.ndarray:
80     """(PF-HIST) 历史同时段扩张窗均值; `H_d = `` 时返回常量中性价. """
81     del day_index
82     if not history:
83         return fallback_price(int(np.asarray(price).shape[1]))
84     return _history_rows(price, history).mean(axis=0)
85
86
87 def pf_dual(price: np.ndarray, day_index: int, history: list[int]) -> np.ndarray:
88     """(PF-DUAL) 水平 × 时段双因子; `H_d = `` 时返回常量中性价. """
89     del day_index
90     if not history:
91         return fallback_price(int(np.asarray(price).shape[1]))
92     rows = _history_rows(price, history)
93     shape_mean = rows.mean(axis=0)
94     level = float(shape_mean.mean())
95     if level <= 0:
96         return fallback_price(int(np.asarray(price).shape[1]))
97     shape = shape_mean / level
98     recent = history[-DUAL_RECENT_DAYS:]
99     mu = float(np.asarray(price, dtype=float)[np.asarray(recent, dtype=int), :].mean())
100     return mu * shape
101
102
103 def pf_ar(price: np.ndarray, day_index: int, history: list[int]) -> tuple[np.ndarray, bool]:
104     """(PF-AR) 逐时段 AR(1): `a_i + _i * p^{act}_{d-1,i}`, 系数只用 `H_d` 估计。
105
106     返回 `(预测向量, 是否退化回退)`. 当可用配对样本 `(d'-1, d')` 少于 `AR_MIN_PAIRS` 组时,
107     逐时段 OLS 无定义 (2 点样本下斜率由共线残差决定、可无界; 实测 2025-01-03 会给出 134 元/kWh
108     的病态预测), 故回退为 (PF-HIST) 的扩张窗均值并标记 `used_fallback=True` (ablation 口径,
109     不影响主口径选择)。配对样本 `(d'-1, d')` 全部取自 `H_d` 内部 (`d' < d`),
110     回归自变量用 `p^{act}_{d-1,i}`, **不使用** d 当日或未来的任何价格。
111     """
112     matrix = np.asarray(price, dtype=float)
113     if not history:

```

```

114         return fallback_price(matrix.shape[1]), True
115     pairs = [d for d in history if d - 1 >= 0]
116     if len(pairs) < AR_MIN_PAIRS:
117         return _history_rows(price, history).mean(axis=0), True
118     current = matrix[np.asarray(pairs, dtype=int), :] # (n, 144)
119     lagged = matrix[np.asarray([d - 1 for d in pairs], dtype=int), :]
120     x_mean = lagged.mean(axis=0)
121     y_mean = current.mean(axis=0)
122     dx = lagged - x_mean
123     dy = current - y_mean
124     variance = (dx * dx).sum(axis=0)
125     covariance = (dx * dy).sum(axis=0)
126     positive = variance > 1e-12
127     phi = np.where(positive, covariance / np.where(positive, variance, 1.0), 0.0)
128     intercept = y_mean - phi * x_mean
129     return intercept + phi * matrix[day_index - 1], False
130
131
132 def predict(price: np.ndarray, day_index: int, method: str = PRIMARY_METHOD) -> dict[str, object]:
133     """按显式历史集合预测第 ``day_index`` 天的 144 点决策价。
134
135     返回 ``{"price": ndarray, "history_size": int, "fallback": str|None, "method": str}``。
136     ``fallback`` 取值: ``None`` / ``"fallback_constant_price"`` (``H_d = ``) /
137     ``"pf_ar_regression_undefined"`` (配对样本 < 2, 退化为 PF-HIST)。
138     """
139     history = available_history(day_index)
140     fallback: str | None = None
141     used_fallback = False
142     if method == "PF-PERSIST":
143         value = pf_persist(price, day_index, history)
144     elif method == "PF-HIST":
145         value = pf_hist(price, day_index, history)
146     elif method == "PF-DUAL":
147         value = pf_dual(price, day_index, history)
148     elif method == "PF-AR":
149         value, used_fallback = pf_ar(price, day_index, history)
150         if used_fallback:
151             fallback = "pf_ar_regression_undefined"
152     else:
153         raise ValueError(f"未知预测器: {method}")
154     if not history:
155         fallback = "fallback_constant_price"
156     return {
157         "method": method,
158         "price": value,
159         "history_size": len(history),
160         "fallback": fallback,
161     }
162
163
164 def predict_day(price: np.ndarray, day_index: int, *, method: str = PRIMARY_METHOD) -> np.ndarray:
165     """便捷包装: 只取预测价向量 (调用方已承诺 ``day_index`` 与信息集口径)。"""
166     return np.asarray(predict(price, day_index, method)["price"], dtype=float)
167
168
169 def _metrics(actual: np.ndarray, forecast: np.ndarray) -> dict[str, float]:
170     error = np.abs(actual - forecast)
171     with np.errstate(divide="ignore", invalid="ignore"):
172         ratio = np.where(actual > 0, error / np.where(actual > 0, actual, 1.0), 0.0)
173     return {
174         "mae": float(error.mean()),
175         "mape": float(ratio.mean()),
176         "rmse": float(np.sqrt(np.mean(error**2))),
177         "bias": float((forecast - actual).mean()),
178         "p50": float(np.percentile(error, 50)),
179         "p90": float(np.percentile(error, 90)),
180         "p99": float(np.percentile(error, 99)),
181         "max": float(error.max()),
182     }
183
184
185 def roll_backtest(
186     price: np.ndarray,
187     *,
188     days: int = DAYS_FULL,
189     methods: tuple[str, ...] = ALL_METHODS,
190     selection_methods: tuple[str, ...] = BASELINE_METHODS,
191 ) -> dict[str, object]:
192     """滚动回测 (BT-1...BT-6): 只用 ``t`` 日之前的信息, 主判据窗口 ``D_req`` (334 天)。
193
194     ``D_full`` = 剔除 2025-01-01 的 364 天 (诊断窗口); ``D_req`` = 2025-02-01...12-31 (334 天, 主判据)。

```

```

195 ``selection_methods`` 为**参与主口径选择**的预测器集合（默认三个强制基准；
196 ``PF-AR`` 按 ASOS 登记为 ``ablation`` 强制对照，**不参与主口径选择**，其指标仍全量落盘）。
197 """
198 matrix = np.asarray(price, dtype=float)
199 day_rows = list(range(1, min(days, matrix.shape[0])))
200 windows = {
201     "D_req": [d for d in day_rows if d >= D_REQ_START],
202     "D_full": day_rows,
203 }
204 forecasts: dict[str, np.ndarray] = {}
205 fallback_counts: dict[str, dict[str, int]] = {}
206 for method in methods:
207     stack = np.empty((len(day_rows), matrix.shape[1]), dtype=float)
208     counters = {"fallback_constant_price": 0, "pf_ar_regression_undefined": 0}
209     for offset, day in enumerate(day_rows):
210         item = predict(matrix, day, method)
211         stack[offset] = np.asarray(item["price"], dtype=float)
212         if item["fallback"]:
213             counters[str(item["fallback"])] += 1
214     forecasts[method] = stack
215     fallback_counts[method] = counters
216
217 report: dict[str, object] = {
218     "protocol": {
219         "information_set": "H_d = {d' : d' < d}（显式入参；禁止使用 d 当日与未来信息）",
220         "windows": {
221             "D_req": "2025-02-01...2025-12-31（334 天，主判据）",
222             "D_full": "2025-01-02...2025-12-31（364 天，诊断）",
223         },
224         "metrics": "MAE（主判据）、MAPE、RMSE、bias、绝对误差 P50/P90/P99、逐日 MAE 夺冠计数",
225         "selection_rule": "D_req MAE 最小；MAE 相对差 < 1% 视为并列，取更简者（BT-5）",
226         "frozen": True,
227     },
228     "formulas": METHOD_FORMULAS,
229     "days_available": len(day_rows),
230     "fallbacks": fallback_counts,
231     "windows": {},
232     "method_metrics": {},
233 }
234 for window_name, window_days in windows.items():
235     offsets = [day - 1 for day in window_days]
236     actual = matrix[np.asarray(window_days, dtype=int), :]
237     entry: dict[str, object] = {"days": len(window_days), "first_day": window_days[0], "last_day": window_days[-1]}
238     per_method: dict[str, dict[str, float]] = {}
239     daily_mae: dict[str, np.ndarray] = {}
240     for method in methods:
241         stack = forecasts[method][np.asarray(offsets, dtype=int), :]
242         per_method[method] = _metrics(actual, stack)
243         daily_mae[method] = np.abs(actual - stack).mean(axis=1)
244     wins = {method: 0 for method in methods}
245     ties = 0
246     for index in range(len(window_days)):
247         values = np.array([daily_mae[method][index] for method in methods])
248         best = float(values.min())
249         winners = [method for method in methods if daily_mae[method][index] <= best + 1e-12]
250         if len(winners) > 1:
251             ties += 1
252             for method in winners:
253                 wins[method] += 1
254     for method in methods:
255         per_method[method]["daily_mae_win_days"] = float(wins[method])
256         per_method[method]["daily_mae_win_fraction"] = float(wins[method] / max(len(window_days), 1))
257     entry["metrics"] = per_method
258     entry["tie_days"] = ties
259     report["windows"][window_name] = entry # type: ignore[index]
260
261 main_window = report["windows"]["D_req"] # type: ignore[index]
262 mae_table = {
263     method: float(main_window["metrics"][method]["mae"]) # type: ignore[index]
264     for method in selection_methods
265 }
266 ordered = sorted(mae_table.items(), key=lambda item: (item[1], SIMPLICITY_RANK.get(item[0], 99)))
267 best_mae = ordered[0][1]
268 tied = [
269     name for name, value in mae_table.items()
270     if best_mae > 0 and (value - best_mae) / best_mae < MAE_TIE_RELATIVE_TOL
271 ]
272 selected = sorted(tied, key=lambda name: SIMPLICITY_RANK.get(name, 99))[0] if tied else ordered[0][0]
273 report["selection"] = {
274     "window": "D_req",
275     "selection_methods": list(selection_methods),

```

```

276     "mae_by_method": mae_table,
277     "relative_difference_vs_best": {
278         name: (0.0 if best_mae == 0 else (value - best_mae) / best_mae) for name, value in mae_table.items()
279     },
280     "tie_tolerance": MAE_TIE_RELATIVE_TOL,
281     "tied_methods": tied,
282     "selected": selected,
283     "simplicity_rank": SIMPLICITY_RANK,
284     "baseline_methods": list(BASELINE_METHODS),
285     "ablation_methods": list(ABLATION_METHODS),
286     "note": (
287         "PF-AR 只作 ablation 强制对照, 按 AS08 不参与主口径选择; "
288         "所有预测器的指标 (含 PF-AR) 仍全量落盘于 windows 中。"
289     ),
290 }
291 return report
292
293
294 def kappa_m(price: np.ndarray, day_index: int, m: int) -> dict[str, object]:
295     """4-3 调整层的水平校正因子  $\kappa_m = (K-3)/(K-4)$ 。
296
297      $\kappa_m$  = 当日  $\kappa_m$  实际均价;  $\kappa_m$  = **前一日同段**  $\kappa_m$  均价;
298      $\kappa_m = \text{clip}(A_m/B_m, 0.5, 2.0)$ ;  $\kappa_0 = 1$ 。每次截断 (或缺失前一日) **逐个登记**，禁止静默截断。
299
300     matrix = np.asarray(price, dtype=float)
301     if m == 0:
302         return {
303             "day_index": int(day_index),
304             "m": 0,
305             "a_m": None,
306             "b_m": None,
307             "raw": None,
308             "kappa": 1.0,
309             "clipped": False,
310             "reason": "kappa_0_identity",
311         }
312     if m not in (6, 12, 18):
313         raise ValueError(f"m 必须属于 {{0,6,12,18}}, 收到 {m}")
314     cutoff = 6 * m
315     a_m = float(matrix[day_index, :cutoff].mean())
316     if day_index - 1 < 0:
317         return {
318             "day_index": int(day_index),
319             "m": int(m),
320             "a_m": a_m,
321             "b_m": None,
322             "raw": None,
323             "kappa": 1.0,
324             "clipped": False,
325             "reason": "no_previous_day_kappa_neutral",
326         }
327     b_m = float(matrix[day_index - 1, :cutoff].mean())
328     if b_m <= 0:
329         return {
330             "day_index": int(day_index),
331             "m": int(m),
332             "a_m": a_m,
333             "b_m": b_m,
334             "raw": None,
335             "kappa": 1.0,
336             "clipped": False,
337             "reason": "zero_denominator_kappa_neutral",
338         }
339     raw = a_m / b_m
340     clipped_value = min(KAPPA_UPPER, max(KAPPA_LOWER, raw))
341     return {
342         "day_index": int(day_index),
343         "m": int(m),
344         "a_m": a_m,
345         "b_m": b_m,
346         "raw": raw,
347         "kappa": float(clipped_value),
348         "clipped": bool(clipped_value != raw),
349         "reason": "clip_to_bounds" if clipped_value != raw else "ok",
350     }
351
352
353 def kappa_schedule(price: np.ndarray, day_index: int) -> dict[str, object]:
354     """一日的  $\kappa_m$  全表 ( $\kappa_m = 0/6/12/18$ ) 与截断登记。"""
355     records = [kappa_m(price, day_index, m) for m in DECISION_HOURS]
356     clipped = [item for item in records if item["clipped"]]

```

```

357     return {
358         "day_index": int(day_index),
359         "records": records,
360         "kappa": {int(item["m"]): float(item["kappa"]) for item in records},
361         "clip_events": clipped,
362     }
363
364
365 def layer_price(
366     price: np.ndarray,
367     day_index: int,
368     m: int,
369     kappa: float,
370     *,
371     periods: int = PERIODS_PER_DAY,
372 ) -> np.ndarray:
373     """按 (K-2) 构造第 ``m`` 层的决策价  $\hat{p}^{(m)}_{d,i}$ 。
374
375     ``i = 6m`` 用当日**已实现**实际价; ``i > 6m`` 用  $\hat{p}^{(m)}_{d-1,i}$ ;
376     ``m = 0`` 时全天为  $\hat{p}^{(0)}_{d-1,i}$  (``_0 = 1`` 与 (PF-PERSIST) 逐项相同)。
377     无前一日 (``d = 2025-01-01``) 时未实现部分的基项取常量中性价 ``1.0`` (回退登记在调用方)。
378     """
379     matrix = np.asarray(price, dtype=float)
380     out = np.empty(periods, dtype=float)
381     cutoff = 6 * m
382     base = matrix[day_index - 1] if day_index - 1 >= 0 else fallback_price(periods)
383     out[cutoff:] = kappa * base[cutoff:]
384     if m > 0:
385         out[:cutoff] = matrix[day_index, :cutoff]
386     return out
387
388
389 def belief_price(
390     price: np.ndarray, day_index: int, kappas: dict[int, float], *, periods: int = PERIODS_PER_DAY
391 ) -> np.ndarray:
392     """(G-4) 决策层信念价  $\hat{p}^{(fc)}_{d,i} = \hat{p}^{((i))}_{d,i}$ , ``i=6*(i-1)/36``。
393
394     由于 ``i = D_m`` 必有 ``i > 6m`` 本式逐段等价于  $\hat{p}^{(i)}_{d-1,i}$  (``_0 = 1``)。
395     """
396     matrix = np.asarray(price, dtype=float)
397     index = np.arange(1, periods + 1)
398     nu = 6 * ((index - 1) // 36)
399     base = matrix[day_index - 1] if day_index - 1 >= 0 else fallback_price(periods)
400     factors = np.array([float(kappas.get(int(hour), 1.0)) for hour in nu], dtype=float)
401     return factors * base
402
403
404 __all__ = [
405     "ABLATION_METHODS",
406     "ALL_METHODS",
407     "AR_MIN_PAIRS",
408     "BASELINE_METHODS",
409     "DAYS_FULL",
410     "D_REQ_START",
411     "DECISION_HOURS",
412     "DUAL_RECENT_DAYS",
413     "FALLBACK_CONSTANT_PRICE",
414     "KAPPA_LOWER",
415     "KAPPA_UPPER",
416     "MAE_TIE_RELATIVE_TOL",
417     "METHOD_FORMULAS",
418     "PERIODS_PER_DAY",
419     "PRIMARY_METHOD",
420     "SIMPLICITY_RANK",
421     "available_history",
422     "belief_price",
423     "fallback_price",
424     "kappa_m",
425     "kappa_schedule",
426     "layer_price",
427     "pf_ar",
428     "pf_dual",
429     "pf_hist",
430     "pf_persist",
431     "predict",
432     "predict_day",
433     "roll_backtest",
434 ]

```

B.13 问题四：求解主程序

```
1 # -*- coding: utf-8 -*-
2 """prob04 计算入口：链 ``4-2`` ( 重算问题 2 ) 与链 ``4-3`` ( 重算问题 3 ) 的统一 CLI。
3
4 用法 (正式任务, 由 computation 阶段的 resource-manager 通过 ``scripts/compute_dispatcher.py`` 提交) ::
5
6     <venv>/python run_prob04.py --chain 4-2 \\  
7         --data2 data/附件2.xlsx --data4 data/附件4.xlsx \\  
8         --template42 data/附件5/result4-2.xlsx \\  
9         --output problems/microgrid_2025/prob04/versions/assumption_v001/results/<run> \\  
10        --days 365 --time-limit 60 --max-wall-seconds 1500
11     <venv>/python run_prob04.py --chain 4-3 --data3 data/附件3.xlsx --template43 data/附件5/result4-3.xlsx ...
12
13 探针模式 (``--days`` 小于 365) 只用于接口自检: 不写 ``result4-2.xlsx``/``result4-3.xlsx``, 改写
14 ``probe_result4-2.xlsx``/``probe_result4-3.xlsx``, ``run_manifest.probe_mode=true``, 跳过「解析界/量级带」
15 四项硬检查, 并在 stdout 显式说明。完整计算必须由 Runner/tasks.py 创建隔离 task (supervised worker) 执行。
16
17 口径 (不得返问更改): ``A8-(b)`` —— 0:00 **不见**当天实时电价, 决策层一律用 ``\hat p`` (历史实际价
18 + 预测器, 主口径 ``PF-PERSIST``), 结算层一律用附件 4 实际价; ``4-3`` 的调整层按 ``_m`` 滚动更新;
19 目标 ``C_total = C_plan (+ C_adj) + C_em`` (元, **不乘  $\Delta t$ ); ``c 833.3333``、``q_dis 750.0000``、
20 ``E [1200, 10800]``、``E_{1/1,0} = 6000``、终端自由; 统一 tie-break (AS21, 承 prob03 T7-1/T9)。
21
22 **禁止**出现「全天电价已知」类表述 (``A8-(b)`` 派生); 本文件与全部产物均按「决策—结算分离」陈述。
23 """
24 from __future__ import annotations
25
26 import argparse
27 import hashlib
28 import os
29 import platform
30 import sys
31 import time
32 from datetime import datetime, timezone
33 from pathlib import Path
34 from typing import Any
35
36 import numpy as np
37 from prob04_io import (
38     DAYS_FULL,
39     DECISION_HOURS,
40     PLAN_SHEET,
41     Attachment2,
42     Attachment4,
43     InputValidationError,
44     fill_result42_workbook,
45     fill_result43_workbook,
46     inspect_template42,
47     inspect_template43,
48     read_attachment2,
49     read_attachment3,
50     read_attachment4_price,
51     write_json,
52 )
53 from prob04_model import (
54     ALPHA_EM,
55     BETA_DEF,
56     BETA_OVER,
57     BLOCKS_PER_DAY,
58     C_CAP,
59     COMMIT_BLOCK,
60     D_REQ_START,
61     DAILY_MAGNITUDE_BAND_YUAN,
62     DELIVERY_MAGNITUDE_BAND_YUAN,
63     DELTA_T,
64     E_INIT,
65     E_MAX,
66     E_MIN,
67     ETA_CH,
68     ETA_DIS,
69     ETA_ROUND_TRIP,
70     P_MAX,
71     PERIODS_PER_DAY,
72     Q_CAP,
73     SPEC_DATES,
74     T7_EPS_RELATIVE,
75     T7_INVARIANCE_TOL,
76     T7_MAX_SHRINKS,
77     T7_SHRINK_FACTOR,
78     T7_TIEBREAK_RULE,
```

```

79     TABLE1_SLOTS,
80     BudgetExceeded,
81     ChainInputs,
82     ChainResult,
83     LayerFailure,
84     block_energy_bias,
85     cross_year_dropped_count,
86     downscale,
87     emergency_intervals,
88     evaluate,
89     run_chain_42,
90     run_chain_43,
91     solve_a2_pre,
92     state_starts,
93 )
94 from prob04_predict import (
95     ALL_METHODS,
96     DUAL_RECENT_DAYS,
97     FALLBACK_CONSTANT_PRICE,
98     KAPPA_LOWER,
99     KAPPA_UPPER,
100    PRIMARY_METHOD,
101    predict,
102    roll_backtest,
103 )
104
105 PROJECT_MARKERS = ("pyproject.toml", "AGENTS.md")
106 EXIT_OK = 0
107 EXIT_SOLVER_NOT_OPTIMAL = 2
108 EXIT_INPUT_INVALID = 3
109 EXIT_HARD_CHECK_FAILED = 4
110 CHAIN_42 = "4-2"
111 CHAIN_43 = "4-3"
112 # 跨年（不消费）预报项的 k 区间（formulation §1.4 / §8.4）: m = 6/12/18 分别丢弃 6/12/18 项，合计 36。
113 CROSS_YEAR_DROPPED_RANGE: dict[int, tuple[int, int]] = {6: (19, 24), 12: (13, 24), 18: (7, 24)}
114
115
116 def project_root() -> Path:
117     current = Path(__file__).resolve()
118     for candidate in (current, *current.parents):
119         if all((candidate / marker).exists() for marker in PROJECT_MARKERS):
120             return candidate
121     raise RuntimeError("无法从脚本位置定位项目根目录")
122
123
124 def resolve(path_text: str) -> Path:
125     candidate = Path(path_text)
126     return candidate if candidate.is_absolute() else (project_root() / candidate)
127
128
129 def code_fingerprint(code_dir: Path) -> dict[str, str]:
130     digest = hashlib.sha256()
131     files: dict[str, str] = {}
132     for path in sorted(code_dir.glob("*.py")):
133         payload = path.read_bytes()
134         files[path.name] = hashlib.sha256(payload).hexdigest()
135         digest.update(path.name.encode("utf-8"))
136         digest.update(payload)
137     files["__code_dir__"] = digest.hexdigest()
138     return files
139
140
141 def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
142     parser = argparse.ArgumentParser(
143         description=(
144             "prob04 求解器：链 4-2（逐日计划 LP， 重算问题 2）与链 4-3（0:00 计划 + 6:00/12:00/18:00 "
145             "调整 + 0.5×/1.5× 偏差结算， 重算问题 3）； assumption_v001 / formulation_v001； "
146             "决策—结算分离（A8-(b)），主预测器 PF-PERSIST，统一 tie-break = AS21"
147         )
148     )
149     parser.add_argument("--chain", choices=(CHAIN_42, CHAIN_43), required=True, help="交付链：4-2 或 4-3")
150     parser.add_argument("--data2", default="data/附件2.xlsx", help="附件 2 路径（只读；实际负载/光伏）")
151     parser.add_argument("--data4", default="data/附件4.xlsx", help="附件 4 路径（只读；实际实时电价）")
152     parser.add_argument("--data3", default="data/附件3.xlsx", help="附件 3 路径（只读；整点光伏预报，仅 4-3）")
153     parser.add_argument("--template42", default="data/附件5/result4-2.xlsx", help="result4-2.xlsx 模板（只读）")
154     parser.add_argument("--template43", default="data/附件5/result4-3.xlsx", help="result4-3.xlsx 模板（只读）")
155     parser.add_argument("--output", required=True, help="输出目录（必须在 assumption_v001 版本目录内）")
156     parser.add_argument("--days", type=int, default=DAYS_FULL, help="优化天数；< 365 为探针模式")
157     parser.add_argument("--time-limit", type=float, default=60.0, help="***单层** LP 的 HiGHS 时限（秒）")
158     parser.add_argument("--max-wall-seconds", type=float, default=1500.0, help="整条链的墙钟预算（秒）")
159     parser.add_argument("--model", choices=("main", "a2-pre"), default="main",

```

```

160         help="main = 主口径; a2-pre = 4-2 的价格完全预知下界锚点（仅探针/sanity, 不进入主结果）")
161     return parser.parse_args(argv)
162
163
164 def build_decision_prices(price_act: np.ndarray, days: int) -> tuple[np.ndarray, list[dict[str, Any]]]:
165     """构造计划层决策价矩阵（两链共用 ``PF-PERSIST`` 并登记 ``H_d = `` 的常量中性价回退。"""
166     matrix = np.empty((days, PERIODS_PER_DAY))
167     fallbacks: list[dict[str, Any]] = []
168     for day in range(days):
169         item = predict(price_act, day, PRIMARY_METHOD)
170         matrix[day] = np.asarray(item["price"], dtype=float)
171         if item["fallback"]:
172             fallbacks.append(
173                 {
174                     "day_index": day,
175                     "date": "2025-01-01" if day == 0 else None,
176                     "marker": str(item["fallback"]),
177                     "method": PRIMARY_METHOD,
178                     "history_size": int(item["history_size"]),
179                     "value": FALLBACK_CONSTANT_PRICE,
180                     "scope": "plan_layer_decision_price",
181                     "reason": (
182                         "H_d = 空集（无前一日），（PF-HIST）的扩张窗均值无定义；改为常量中性价 "
183                         f"{FALLBACK_CONSTANT_PRICE}：平坦价 无套利激励 储能保持 6000 kWh, "
184                         "是最少信息的合法选择；不使用未来价格、不使用全量均值；该日不参与回测"
185                     ),
186                 }
187             )
188     return matrix, fallbacks
189
190
191 def build_layer_inputs(attachment3: Any, days: int) -> tuple[dict[int, np.ndarray], dict[str, Any]]:
192     """按 formulation §1.4 生成各决策时刻的降尺度预报矩阵，并记录层输入指纹与跨年结构。"""
193     forecast: dict[int, np.ndarray] = {}
194     info: dict[str, Any] = {}
195     biases = [block_energy_bias(attachment3.row(0, hour), hour) for hour in DECISION_HOURS]
196     for hour in DECISION_HOURS:
197         matrix = np.vstack([downscale(attachment3.row(day, hour), hour) for day in range(days)])
198         forecast[hour] = matrix
199         finite = int(np.count_nonzero(np.isfinite(matrix)))
200         digest = hashlib.sha256(np.round(matrix[np.isfinite(matrix)], 6).tobytes()).hexdigest()
201         info[f"hour_{hour}"] = {
202             "dominated_periods_per_day": PERIODS_PER_DAY - 6 * hour,
203             "finite_cells": finite,
204             "nan_cells": int(matrix.size - finite),
205             "expected_nan_cells": 6 * hour * days,
206             "cross_year_dropped_items_per_day": cross_year_dropped_count(hour),
207             "forecast_matrix_sha256": digest,
208             "forecast_source": f"附件 3 的 {hour}:00 发布整点预报（日期列前向填充，团队勘误 prob03 E1）",
209             "locked_periods": f"i {6 * hour}（已提交，本层不再决策）" if hour else "无（计划层支配全天）",
210             "initial_state_source": (
211                 "E_{d-1,144}（前一日已提交链末态）" if hour == 0
212                 else f"E_{{{d},{6 * hour}}}"（上一已提交链末态）"
213             ),
214             "uses_actual_pv": False,
215         }
216     info["dominance_rule"] = (
217         "nu(i) = 6*floor((i-1)/36); D_0/D_6/D_12/D_18 各 36 时段 (i<=36 / 37..72 / 73..108 / 109..144)"
218     )
219     info["downscale_rule"] = (
220         "k=1 前向保持 A_{m,1}; 2<=k<=24 取 (1-j/6)A_{m,k-1} + (j/6)A_{m,k}; "
221         "k<1 (i<=6m) 未定义; k>24-m 的跨年项不消费、不截断、不外推"
222     )
223     info["cross_year_dropped_total_per_day"] = sum(cross_year_dropped_count(h) for h in DECISION_HOURS)
224     info["cross_year_dropped_list"] = {
225         f"m={hour}": (
226             f"k={CROSS_YEAR_DROPPED_RANGE[hour][0]}..{CROSS_YEAR_DROPPED_RANGE[hour][1]}"
227             "（→ 2026-01-01 及以后，不消费、不截断、不外推）"
228         )
229         for hour in (6, 12, 18)
230     }
231     info["block_energy_non_conservation"] = {
232         "disclosure": (
233             "块 b 的 6 个时段之和 = (2.5A_{k-1} + 3.5A_k)/6（kWh, k>=2），一般 != A_k * 1h: "
234             "降尺度**不保块内能量**，必须在论文与图注显式披露"
235         ),
236         "day0_examples": {
237             f"m={hour}": bias for hour, bias in zip(DECISION_HOURS, biases, strict=True)
238         },
239     }
240     info["actual_pv_scope"] = "settlement_layer_only（决策层输入不含 PV`act`）"

```

```

241     return forecast, info
242
243
244 def build_tables(
245     result: ChainResult,
246     metrics: dict[str, Any],
247     time_labels: list[str],
248     template_labels: list[str],
249     *,
250     probe_mode: bool,
251 ) -> dict[str, Any]:
252     """构造表 1/表 2/表 3 (4 个指定日期) 并核对列标签与行位置口径 (formulation §8.3)。"""
253     days = result.days
254     q = result.q
255     c = result.c
256     q_dis = result.q_dis
257     q_em = result.q_em
258     daily = metrics["daily"]
259
260     table1: dict[str, Any] = {}
261     table2: dict[str, Any] = {}
262     table3: dict[str, Any] = {}
263     covered: list[str] = []
264     label_ok = True
265     for date_text, day_index in SPEC_DATES:
266         if day_index >= days:
267             continue
268         covered.append(date_text)
269         slots: list[dict[str, Any]] = []
270         for slot_name, position in TABLE1_SLOTS:
271             attachment_label = time_labels[position - 1]
272             template_label = template_labels[position - 1]
273             matches = attachment_label == slot_name.split("-")[0] and template_label == slot_name
274             label_ok = label_ok and matches
275             slots.append(
276                 {
277                     "slot": slot_name,
278                     "position": position,
279                     "attachment_timestamp": attachment_label,
280                     "template_label": template_label,
281                     "label_matches": bool(matches),
282                     "final_purchase_kwh": round(float(q[day_index, position - 1]), 6),
283                 }
284             )
285         table1[date_text] = {
286             "slots": slots,
287             "quantity_basis": "final_quantity (4-2: b; 4-3: q) ",
288             "all_day_energy_kwh": daily["purchase_kwh"][day_index],
289             "all_day_cost_yuan": daily["cost_total_yuan"][day_index],
290             "all_day_plan_cost_yuan": daily["cost_plan_yuan"][day_index],
291             "all_day_adjust_cost_yuan": daily["cost_adj_yuan"][day_index],
292             "all_day_emergency_cost_yuan": daily["cost_em_yuan"][day_index],
293         }
294         table2[date_text] = {
295             "block_rows": {f"block_{k + 1}": f"{24 * k + 1}..{24 * (k + 1)}" for k in range(BLOCKS_PER_DAY)},
296             "block_charge_kwh": [
297                 round(float(np.sum(c[day_index, k * 24 : (k + 1) * 24])), 6) for k in range(BLOCKS_PER_DAY)
298             ],
299             "block_discharge_kwh": [
300                 round(float(np.sum(q_dis[day_index, k * 24 : (k + 1) * 24])), 6) for k in range(BLOCKS_PER_DAY)
301             ],
302             "storage_0_00_kwh": daily["state_start_kwh"][day_index],
303             "storage_24_00_kwh": daily["state_end_kwh"][day_index],
304             "charge_discharge_not_netted": True,
305         }
306         intervals = emergency_intervals(q_em[day_index])
307         table3[date_text] = {
308             "intervals": intervals,
309             "rows": max(len(intervals), 1),
310             "total_kwh": round(float(np.sum(q_em[day_index])), 6),
311             "paper_text": "无 (0 kWh) " if not intervals else "",
312         }
313     return {
314         "chain": result.chain,
315         "spec_dates": covered,
316         "spec_dates_reference": {name: index for name, index in SPEC_DATES},
317         "table1": table1,
318         "table2": table2,
319         "table3": table3,
320         "table1_slot_positions": [position for _, position in TABLE1_SLOTS],
321         "table1_label_checks_passed": bool(label_ok),

```

```

322     "table1_quantity_basis": "final_quantity (4-2 为 b; 4-3 为最终量 q)",
323     "table1_cost_basis": "全天购电费 = C_total~{act},d (正文另给 C_plan/C_adj/C_em 分项)",
324     "table3_note": (
325         "题面表 4 的示例日期 2025/3/1 与三个区间及其数值是**格式示例**，不是本问答案；"
326         "本表按「连续 q_em>0 时段合并为区间 + 左端点标签 [_i, _j+10min)」填报，"
327         f"每日至少 1 行 (J_d = 0 写 {chr(0x2014)}/0)。"
328     ),
329     "table2_note": (
330         "表 2 的 0:00/24:00 为计划窗首/末状态 (左端点口径使计划窗为 0:10-24:10，模板固有相位)。"
331         "**本题为多日滚动切片：E_{d,0} = E_{d-1,144} 随日变化**，与 prob01 的单日周期口径"
332         " (E_{d,0} = E_{d,144} 6000 kWh) 必须显式区分 (承 prob02 勘误 R1)；"
333         "充电量与放电量分别列示、不冲抵。"
334     ),
335     "probe_mode": bool(probe_mode),
336 }
337
338
339 def build_tiebreak_audit(result: ChainResult, metrics: dict[str, Any], *, days: int) -> dict[str, Any]:
340     """AS21 的落盘审计：逐层不变性验证 + 退化维度与多重最优探测。"""
341     records = [item.as_dict() for item in result.tiebreak_records]
342     checks = [item for item in metrics["checks"] if item["name"].startswith("t7_")]
343     return {
344         "policy": "AS21 (承 prob03 团队裁定 T7-1/T9: 统一冻结 tie-break, 所有层、所有模型一致)",
345         "rule": T7_TIEBREAK_RULE,
346         "epsilon_definition": " = T7_EPS_RELATIVE × max(|该层主目标最优值|, 1) / [n × (c_cap + q_dis_cap)]",
347         "eps_relative": T7_EPS_RELATIVE,
348         "invariance_tol": T7_INVARIANCE_TOL,
349         "shrink_factor": T7_SHRINK_FACTOR,
350         "max_shrinks": T7_MAX_SHRINKS,
351         "chain": result.chain,
352         "days": days,
353         "layers": len(records),
354         "plan_layers": int(sum(1 for item in result.tiebreak_records if item.layer.startswith("plan"))),
355         "adjustment_layers": int(sum(1 for item in result.tiebreak_records if item.layer.startswith("adjustment"))),
356         "summary": metrics["t7_tiebreak"],
357         "checks": checks,
358         "checks_failed": [item["name"] for item in checks if not item["passed"]],
359         "per_layer": records,
360         "reuse_note": (
361             "AS21 承 prob03 T7-5: 本问的 ablation/robustness 对照 (A2-PRE、1、M4-ter 等) 必须复用 "
362             "prob04_model._solve_with_tiebreak / tiebreak_epsilon / throughput_coefficients, "
363             "tie-break 规则须与主口径完全一致，仅允许在 HiGHS 设置上不同。"
364         ),
365     }
366
367
368 def _round(value: float, digits: int = 6) -> float:
369     return round(float(value), digits)
370
371
372 def main(argv: list[str] | None = None) -> int:
373     args = parse_args(argv)
374     output_dir = resolve(args.output)
375     code_dir = Path(__file__).resolve().parent
376     chain = args.chain
377     probe_mode = args.days < DAYS_FULLL
378
379     if not 2 <= args.days <= DAYS_FULLL:
380         print(f"[prob04] --days 必须在 [2, {DAYS_FULLL}]，收到 {args.days}", file=sys.stderr)
381         return EXIT_INPUT_INVALID
382     if args.model == "a2-pre" and chain != CHAIN_42:
383         print(f"[prob04] --model a2-pre 仅适用于 --chain 4-2 (下界锚点，不进入主结果)", file=sys.stderr)
384         return EXIT_INPUT_INVALID
385     if args.model == "a2-pre" and not probe_mode:
386         print(
387             "[prob04] --model a2-pre 只允许在探针模式 (--days < 365) 下运行：它是 ablation/sanity 的"
388             "价格完全预知下界锚点，不得作为主结果",
389             file=sys.stderr,
390         )
391         return EXIT_INPUT_INVALID
392
393     days = args.days
394     data2_path = resolve(args.data2)
395     data4_path = resolve(args.data4)
396     data3_path = resolve(args.data3)
397     template42_path = resolve(args.template42)
398     template43_path = resolve(args.template43)
399     template_path = template42_path if chain == CHAIN_42 else template43_path
400
401     try:
402         attachment2: Attachment2 = read_attachment2(data2_path, expected_days=DAYS_FULLL)

```

```

403     attachment4: Attachment4 = read_attachment4_price(data4_path, expected_days=DAYS_FULL)
404     if attachment4.time_labels != attachment2.time_labels:
405         raise InputValidationError("附件 4 与附件 2 的 144 个时间列标签不一致 (相位不同)")
406     if attachment4.dates != attachment2.dates:
407         raise InputValidationError("附件 4 与附件 2 的日期序列不一致")
408     attachment3 = None
409     if chain == CHAIN_43:
410         attachment3 = read_attachment3(data3_path, expected_days=DAYS_FULL)
411         if attachment3.dates != attachment2.dates:
412             raise InputValidationError("附件 3 与附件 2 的日期序列不一致")
413     template_info = (
414         inspect_template42(template42_path) if chain == CHAIN_42
415         else inspect_template43(template43_path)
416     )
417 except InputValidationError as exc:
418     print(f"[prob04] 输入校验失败: {exc}", file=sys.stderr)
419     return EXIT_INPUT_INVALID
420
421 if template_info["differences"]:
422     print(f"[prob04] 模板实测与 formulation 登记值存在差异 (以实测为准): {template_info['differences']}")
423 template_labels_key = PLAN_SHEET
424 template_labels = template_info["measured"]["purchase"][template_labels_key]["labels"]
425
426 decision_price, fallback_records = build_decision_prices(attachment4.price, days)
427 forecast: dict[int, np.ndarray] = {}
428 layer_inputs: dict[str, Any] = {
429     "plan_layer": {
430         "price_source": "PF-PERSIST: \\hat p_{d,i} = p^{act}_{d-1,i} (仅 d-1 的历史实际价)",
431         "price_information_set": "H_d = {d' : d' < d} (显式入参)",
432         "load_pv_source": (
433             "附件 2 实际负载/光伏 (问题 2 无预报机制, D3-A)" if chain == CHAIN_42
434             else "负载 = 附件 2 实际; 光伏 = 附件 3 的 0:00 发布预报降尺度  $\Pi_0$ "
435         ),
436         "locked_periods": "无 (计划层支配全天 144 时段)" if chain == CHAIN_42 else "无 (提交 D_0 = i 36)",
437         "initial_state_source": "E_{d-1,144} (E_{1/1,0} = 6000)",
438     },
439     "settlement_layer": {
440         "price_source": "附件 4 实际价  $p^{act}_{d,i}$  (唯一价格入口)",
441         "pv_source": "附件 2 实际  $PV^{act}$  (唯一  $PV^{act}$  入口)",
442         "q_em_definition": "q_em = max(0, -r), r = q + q_dis +  $PV^{act} \cdot \Delta t - L \cdot \Delta t - c$ ",
443     },
444     "future_price_used_in_decision": False,
445     "blacklist_note": "全文与全部产物不得出现「全天电价已知」类表述 (A8-(b) 派生)",
446 }
447 if chain == CHAIN_43:
448     forecast, chain_layer_inputs = build_layer_inputs(attachment3, days)
449     layer_inputs.update(chain_layer_inputs)
450     layer_inputs["plan_layer"]["price_source"] = "\\hat p^{(0)}_{d,i} = p^{act}_{d-1,i} (_0 1)"
451     for hour in DECISION_HOURS:
452         info = layer_inputs[f"hour_{hour}"]
453         if info["nan_cells"] != info["expected_nan_cells"]:
454             print(f"[prob04] 降尺度覆盖域异常: hour={hour} nan={info['nan_cells']}", file=sys.stderr)
455             return EXIT_INPUT_INVALID
456         dominated = np.where(np.isfinite(forecast[hour][0]))[0]
457         if not np.isfinite(forecast[hour][:, dominated]).all():
458             print(f"[prob04] 降尺度预报在支配域含 NaN: hour={hour}", file=sys.stderr)
459             return EXIT_INPUT_INVALID
460
461 decision_prices_used = attachment4.price if args.model == "a2-pre" else decision_price
462 build_forecast = forecast if chain == CHAIN_43 else {}
463
464 solver_calls_per_layer = 4
465 if chain == CHAIN_42:
466     matrix_scale = {
467         "plan_layer": {"variables": 6 * PERIODS_PER_DAY, "equality_constraints": 2 * PERIODS_PER_DAY,
468             "inequalities": 0, "solver_calls_nominal": solver_calls_per_layer},
469         "per_day": {"variables": 6 * PERIODS_PER_DAY, "equality_constraints": 2 * PERIODS_PER_DAY,
470             "layer_solves": 1, "solver_calls_nominal": solver_calls_per_layer},
471         "lp_calls_total": days,
472         "lp_solver_invocations_nominal": days * solver_calls_per_layer,
473     }
474 else:
475     matrix_scale = {
476         "plan_layer": {"variables": 5 * PERIODS_PER_DAY, "equality_constraints": 2 * PERIODS_PER_DAY,
477             "inequalities": 0, "solver_calls_nominal": solver_calls_per_layer},
478         "adjustment_layer_m6": {"variables": 7 * 108, "equality_constraints": 216, "inequalities": 216,
479             "solver_calls_nominal": solver_calls_per_layer},
480         "adjustment_layer_m12": {"variables": 7 * 72, "equality_constraints": 144, "inequalities": 144,
481             "solver_calls_nominal": solver_calls_per_layer},
482         "adjustment_layer_m18": {"variables": 7 * 36, "equality_constraints": 72, "inequalities": 72,
483             "solver_calls_nominal": solver_calls_per_layer},

```

```

484         "per_day": {"variables": 2232, "equality_constraints": 720, "inequalities": 432,
485                    "layer_solves": 4, "solver_calls_nominal": 4 * solver_calls_per_layer},
486         "lp_calls_total": 4 * days,
487         "lp_solver_invocations_nominal": 4 * days * solver_calls_per_layer,
488     }
489     matrix_scale["tiebreak_solver_calls_per_layer_nominal"] = solver_calls_per_layer
490     matrix_scale["integer_variables_main"] = 0
491     matrix_scale["note"] = (
492         "AS21/T7 四段式（基线 + 字面加权诊断 + 字典序提交解 + 退化探测）： "
493         "T7-2 每次 收缩另加 1 次（有界于 T7_MAX_SHRINKS）；实际调用次数见 solver_status.lp_calls。"
494     )
495
496     chain_inputs = ChainInputs(
497         days=days,
498         price_act=attachment4.price[:days],
499         decision_price=decision_prices_used[:days],
500         load_energy=attachment2.load_kw[:days] * DELTA_T,
501         pv_act_energy=attachment2.pv_kw[:days] * DELTA_T,
502         forecast_kw=build_forecast,
503     )
504
505     started = time.perf_counter()
506     result: ChainResult
507     try:
508         if args.model == "a2-pre":
509             result, records, tiebreaks = solve_a2_pre(chain_inputs, time_limit_seconds=args.time_limit)
510         elif chain == CHAIN_42:
511             result = run_chain_42(chain_inputs, time_limit_seconds=args.time_limit,
512                                 deadline=started + args.max_wall_seconds)
513         else:
514             result = run_chain_43(chain_inputs, time_limit_seconds=args.time_limit,
515                                 deadline=started + args.max_wall_seconds)
516     except (LayerFailure, BudgetExceeded) as exc:
517         status = getattr(exc, "status", -1)
518         write_json(
519             output_dir / "solver_status.json",
520             {
521                 "status": int(status),
522                 "message": str(exc),
523                 "feasible_incumbent": False,
524                 "chain": chain,
525                 "model": args.model,
526                 "device": "cpu",
527                 "gpu_required": False,
528                 "seed": None,
529                 "days": days,
530                 "probe_mode": probe_mode,
531                 "seconds": time.perf_counter() - started,
532             },
533         )
534         write_json(
535             output_dir / "run_manifest.json",
536             {
537                 "problem_id": "microgrid_2025",
538                 "question_id": "prob04",
539                 "assumption_version": "assumption_v001",
540                 "formulation_version": "formulation_v001",
541                 "stage": "computation",
542                 "chain": chain,
543                 "model": args.model,
544                 "probe_mode": probe_mode,
545                 "days": days,
546                 "outcome": "solver_not_optimal",
547                 "matrix_scale": matrix_scale,
548                 "task_id": os.environ.get("AUTOMM_TASK_ID"),
549             },
550         )
551         print(f"[prob04] 求解未达最优: {exc}", file=sys.stderr)
552         return EXIT_SOLVER_NOT_OPTIMAL
553     elapsed = time.perf_counter() - started
554
555     metrics = evaluate(result, probe_mode=probe_mode, full_horizon=not probe_mode)
556     backtest = roll_backtest(attachment4.price, days=DAYS_FULL, methods=ALL_METHODS)
557     tables = build_tables(result, metrics, attachment2.time_labels, template_labels, probe_mode=probe_mode)
558     tiebreak_audit = build_tiebreak_audit(result, metrics, days=days)
559
560     delivery_index = slice(min(D_REQ_START, days), days)
561     delivery_dates_list = attachment2.dates[delivery_index]
562     starts = state_starts(result)[delivery_index]
563     ends = result.E[:, -1][delivery_index]
564     day_cost = np.asarray(metrics["daily"]["cost_total_yuan"], dtype=float)[delivery_index]

```

```

565 workbook_info: dict[str, Any] | None = None
566 if len(delivery_dates_list):
567     b_delivery = result.plan_b[delivery_index]
568     q_delivery = result.q[delivery_index]
569     c_delivery = result.c[delivery_index]
570     q_dis_delivery = result.q_dis[delivery_index]
571     q_em_delivery = result.q_em[delivery_index]
572     intervals = [
573         [(item["slot"], item["energy_kwh"]) for item in emergency_intervals(q_em_delivery[index])]
574         for index in range(len(delivery_dates_list))
575     ]
576     wb_prefix = "probe_" if probe_mode else ""
577     wb_suffix = "a2-pre_" if args.model == "a2-pre" else ""
578     workbook_name = f"{wb_prefix}{wb_suffix}result{chain}.xlsx"
579     if chain == CHAIN_42:
580         workbook_info = fill_result42_workbook(
581             template=template_path,
582             dest=output_dir / workbook_name,
583             dates=delivery_dates_list,
584             purchase_kwh=b_delivery,
585             charge_kwh=c_delivery,
586             discharge_kwh=q_dis_delivery,
587             state_start_kwh=starts,
588             state_end_kwh=ends,
589             day_energy_kwh=np.sum(b_delivery, axis=1),
590             day_cost_yuan=day_cost,
591             emergency_intervals=intervals,
592         )
593     else:
594         workbook_info = fill_result43_workbook(
595             template=template_path,
596             dest=output_dir / workbook_name,
597             dates=delivery_dates_list,
598             planned_kwh=b_delivery,
599             adjusted_kwh=q_delivery,
600             charge_kwh=c_delivery,
601             discharge_kwh=q_dis_delivery,
602             state_start_kwh=starts,
603             state_end_kwh=ends,
604             day_energy_planned_kwh=np.sum(b_delivery, axis=1),
605             day_energy_adjusted_kwh=np.sum(q_delivery, axis=1),
606             day_cost_yuan=day_cost,
607             emergency_intervals=intervals,
608         )
609     else:
610         workbook_name = None
611
612 failed = list(metrics["checks_failed"])
613 if not tables["table1_label_checks_passed"]:
614     failed.append("table1_label_checks")
615 if workbook_info is not None and workbook_info.get("format_residuals"):
616     failed.append("workbook_format_residuals")
617
618 naming = {
619     "chain_4-2": (
620         "  重算问题 2: 逐日 0:00 一次计划 LP (无调整层、无偏差结算): q b; "
621         "C_total^act = C_plan^act + C_em^act (q_em 0, §2.3 定理)"
622     ),
623     "chain_4-3": (
624         "  重算问题 3: 0:00 计划 + 6:00/12:00/18:00 调整 ( _m 滚动水平校正) + 0.5×/1.5× 双向偏差结算; "
625         "C_total^act = C_plan^act + C_adj^act + C_em^act; 结算用 PV^act 与 p^act 的唯一入口"
626     ),
627     "price_basis": "决策层用 \\hat p (PF-PERSIST / _m 滚动更新), 结算层一律用附件 4 实际价 p^act",
628     "quantity_basis": "run_manifest.total_purchase_kwh 装 Eq (4-2 中 q b); stdout 摘要为 D_full 口径",
629 }
630
631 run_meta: dict[str, Any] = {
632     "problem_id": "microgrid_2025",
633     "question_id": "prob04",
634     "assumption_version": "assumption_v001",
635     "formulation_version": "formulation_v001",
636     "stage": "computation",
637     "chain": chain,
638     "model": args.model,
639     "probe_mode": probe_mode,
640     "days": days,
641     "periods": days * PERIODS_PER_DAY,
642     "created_at": datetime.now(timezone.utc).isoformat(),
643     "task_id": os.environ.get("AUTOMM_TASK_ID") or None,
644     "output_directory": str(output_dir),
645     "device": "cpu",

```

```

646     "gpu_required": False,
647     "seed": None,
648     "deterministic": True,
649     "solver": "scipy.optimize.linprog(method='highs')",
650     "layer_time_limit_seconds": args.time_limit,
651     "max_wall_seconds": args.max_wall_seconds,
652     "naming": naming,
653     "input": {
654         "attachment2": str(data2_path),
655         "attachment2_md5": attachment2.md5,
656         "attachment2_days": attachment2.days,
657         "attachment3": str(data3_path) if chain == CHAIN_43 else None,
658         "attachment3_md5": attachment3.md5 if attachment3 is not None else None,
659         "attachment4": str(data4_path),
660         "attachment4_md5": attachment4.md5,
661         "attachment4_days": attachment4.days,
662         "attachment1_used": False,
663         "template": str(template_path),
664         "template_md5": template_info["measured"]["md5"],
665         "template_measured": template_info["measured"],
666         "template_formulation_registered": template_info["formulation_registered"],
667         "template_differences": template_info["differences"],
668     },
669     "parameters": {
670         "delta_t_h": DELTA_T,
671         "eta_ch": ETA_CH,
672         "eta_dis": ETA_DIS,
673         "eta_round_trip": ETA_ROUND_TRIP,
674         "alpha_em": ALPHA_EM,
675         "beta_def": BETA_DEF if chain == CHAIN_43 else None,
676         "beta_over": BETA_OVER if chain == CHAIN_43 else None,
677         "e_init_kwh": E_INIT,
678         "e_min_kwh": E_MIN,
679         "e_max_kwh": E_MAX,
680         "e_terminal": None,
681         "p_max_kw": P_MAX,
682         "c_cap_kwh": C_CAP,
683         "q_dis_cap_kwh": Q_CAP,
684         "b_upper_kwh": None,
685         "decision_hours": list(DECISION_HOURS) if chain == CHAIN_43 else [0],
686         "kappa_bounds": [KAPPA_LOWER, KAPPA_UPPER] if chain == CHAIN_43 else None,
687         "forecast_primary": PRIMARY_METHOD,
688         "forecast_dual_recent_days": DUAL_RECENT_DAYS,
689         "forecast_fallback_constant_price": FALLBACK_CONSTANT_PRICE,
690         "delivery_start_index": D_REQ_START,
691         "commit_block_periods": COMMIT_BLOCK if chain == CHAIN_43 else PERIODS_PER_DAY,
692     },
693     "layer_inputs": layer_inputs,
694     "forecast_fallbacks": fallback_records,
695     "code_sha256": code_fingerprint(code_dir),
696     "environment": {
697         "python": sys.version.split()[0],
698         "numpy": np.__version__,
699         "platform": platform.platform(),
700     },
701 }
702
703 write_json(
704     output_dir / "solver_status.json",
705     {
706         "status": 0,
707         "message": "all layers optimal",
708         "solver": "highs",
709         "chain": chain,
710         "model": args.model,
711         "objective_yuan": metrics["objective_yuan"],
712         "delivery_cost_yuan": metrics["delivery"]["cost_total_yuan"],
713         "feasible_incumbent": True,
714         "mip_gap": None,
715         "device": "cpu",
716         "gpu_required": False,
717         "seed": None,
718         "days": days,
719         "probe_mode": probe_mode,
720         "lp_calls": len(result.layer_records),
721         "lp_solver_invocations_nominal": matrix_scale["lp_solver_invocations_nominal"],
722         "layer_status_max": max((item.status for item in result.layer_records), default=0),
723         "layer_equality_residual_max": max(
724             (item.equality_residual_max for item in result.layer_records), default=0.0
725         ),
726         "layer_equality_residual_relative_max": max(

```

```

727         (item.equality_residual_relative_max for item in result.layer_records), default=0.0
728     ),
729     "layer_inequality_residual_max": max(
730         (item.inequality_residual_max for item in result.layer_records), default=0.0
731     ),
732     "layer_bound_violation_max": max(
733         (item.bound_violation_max for item in result.layer_records), default=0.0
734     ),
735     "layer_seconds_total": float(np.sum([item.seconds for item in result.layer_records]))
736     if result.layer_records else 0.0,
737     "wall_seconds": elapsed,
738     "deterministic": True,
739 },
740 )
741 write_json(
742     output_dir / "solution.json",
743     {
744         "chain": chain,
745         "model": args.model,
746         "probe_mode": probe_mode,
747         "days": days,
748         "objective_yuan": metrics["objective_yuan"],
749         "cost_plan_yuan": metrics["cost_plan_yuan"],
750         "cost_adj_yuan": metrics["cost_adj_yuan"],
751         "cost_em_yuan": metrics["cost_em_yuan"],
752         "belief": metrics["belief"],
753         "delivery": metrics["delivery"],
754         "totals": metrics["totals"],
755         "identities": metrics["identities"],
756         "bounds": metrics["bounds"],
757         "statistics": metrics["statistics"],
758         "daily": metrics["daily"],
759         "checks": metrics["checks"],
760         "checks_failed": metrics["checks_failed"],
761         "t7_tiebreak": metrics["t7_tiebreak"],
762         "forecast_fallbacks": fallback_records + list(result.fallback_records),
763         "layer_summary": [item.as_dict() for item in result.layer_records],
764         "series": metrics["series"],
765     },
766 )
767 write_json(output_dir / "tables.json", tables)
768 write_json(output_dir / "forecast_backtest.json", backtest)
769 write_json(output_dir / "tiebreak_audit.json", tiebreak_audit)
770 write_json(
771     output_dir / "run_manifest.json",
772     {
773         **run_meta,
774         "outcome": "hard_check_failed" if failed else "completed",
775         "matrix_scale": matrix_scale,
776         "lp_calls": len(result.layer_records),
777         "lp_solver_invocations_nominal": matrix_scale["lp_solver_invocations_nominal"],
778         "t7_tiebreak": metrics["t7_tiebreak"],
779         "tiebreak_artifact": "tiebreak_audit.json",
780         "wall_seconds": elapsed,
781         "workbook": workbook_name,
782         "workbook_info": workbook_info,
783         "format_residuals": (workbook_info or {}).get("format_residuals", []),
784         "delivery_days": len(delivery_dates_list),
785         "delivery": metrics["delivery"],
786         "statistics": metrics["statistics"],
787         "total_plan_kwh": metrics["delivery"]["total_plan_kwh"],
788         "total_purchase_kwh": metrics["delivery"]["total_purchase_kwh"],
789         "total_q_em_kwh": metrics["delivery"]["total_q_em_kwh"],
790         "total_charge_kwh": metrics["delivery"]["total_charge_kwh"],
791         "total_discharge_kwh": metrics["delivery"]["total_discharge_kwh"],
792         "total_spill_kwh": metrics["delivery"]["total_spill_kwh"],
793         "storage_final_kwh": metrics["totals"]["storage_final_kwh"],
794         "checks_failed": failed,
795     },
796 )
797
798 if probe_mode:
799     print(
800         "[prob04] **探针模式** (--days < 365)：不写 result{0}.xlsx, 改写 probe_result{0}.xlsx; "
801         "已跳过「解析界/量级带」四项硬检查；本产物**不是**交付结果。".format(chain)
802     )
803     print(
804         f"[prob04] 链 {chain} (model={args.model}) 运行完成: days={days} periods={days * PERIODS_PER_DAY} "
805         f"LP={len(result.layer_records)} (名义调用 {matrix_scale['lp_solver_invocations_nominal']}) "
806     )
807     print(

```

```

808     f"[prob04] D_full 口径: C_total`act={metrics['objective_yuan']:.4f} 元 "
809     f"C_plan={metrics['cost_plan_yuan']:.4f}, C_adj={metrics['cost_adj_yuan']:.4f}, "
810     f"C_em={metrics['cost_em_yuan']:.4f}); Eb={metrics['totals']['total_plan_kwh']:.4f} kWh, "
811     f"Eq={metrics['totals']['total_purchase_kwh']:.4f} kWh, Eq`em={metrics['totals']['total_q_em_kwh']:.6f} kWh, "
812     f"Es={metrics['totals']['total_spill_kwh']:.4f} kWh, Ec={metrics['totals']['total_charge_kwh']:.4f} kWh, "
813     f"Eq_dis={metrics['totals']['total_discharge_kwh']:.4f} kWh, "
814     f"E_0={metrics['totals']['storage_initial_kwh']:.4f} → E_T={metrics['totals']['storage_final_kwh']:.4f} kWh"
815 )
816 print(
817     f"[prob04] D_req 口径 (交付期 {metrics['delivery']['days']} 天): "
818     f"C_total`act={metrics['delivery']['cost_total_yuan']:.4f} 元"
819     f" (C_plan={metrics['delivery']['cost_plan_yuan']:.4f}, "
820     f"C_adj={metrics['delivery']['cost_adj_yuan']:.4f}, "
821     f"C_em={metrics['delivery']['cost_em_yuan']:.4f}); "
822     f"C_total`fc={metrics['delivery']['cost_total_fc_yuan']:.4f} 元; "
823     f"ΔC_price={metrics['delivery']['delta_c_price_yuan']:.4f} 元"
824 )
825 print(
826     f"[prob04] 恒等式残差: I1={metrics['identities']['I1_daily_max_abs']:.3e}, "
827     f"I2={metrics['identities']['I2_daily_max_abs']:.3e}, "
828     f"跨日={metrics['identities']['continuity_residual_max']:.3e}, "
829     f"状态转移={metrics['identities']['transition_residual_max']:.3e}, "
830     f"结算平衡={metrics['identities']['settlement_residual_max']:.3e}, "
831     f"层内余额上界={metrics['identities']['layer_spill_upper_violation']:.3e}"
832 )
833 print(
834     f"[prob04] AS21 tie-break: 主目标最大相对变化={metrics['t7_tiebreak']['max_primary_relative_change']:.3e}"
835     f" (阈值 {T7_INVARIANCE_TOL:.0e}, 全部通过={metrics['t7_tiebreak']['all_layers_invariance_passed']}) , "
836     f"吞吐量 {metrics['t7_tiebreak']['throughput_primary_only_kwh']:.4f}→"
837     f"{metrics['t7_tiebreak']['throughput_tiebreak_kwh']:.4f} kWh, "
838     f"退化层数={metrics['t7_tiebreak']['degenerate_layers']}/"
839     f"{metrics['t7_tiebreak']['layers']}, 待提交解来源={metrics['t7_tiebreak']['lexicographic_committed_layers']}"
840     f"lexicographic / {metrics['t7_tiebreak']['fallback_committed_layers']} fallback"
841 )
842 if probe_mode:
843     print(
844         "[prob04] 探针诊断 (**跳过硬检查**, 诊断值仅供参考, 不构成交付证据): "
845         f"C_total`act={metrics['delivery']['cost_total_yuan']} 元"
846         f" (量级带内={metrics['bounds'].get('magnitude_band_delivery_ok')}) , "
847         f"日均={metrics['bounds'].get('mean_daily_delivery_cost_yuan')} 元"
848         f" (量级带内={metrics['bounds'].get('magnitude_band_daily_ok')}) , "
849         f"解析下界={metrics['bounds'].get('delivery_plan_lower_bound_yuan')} 元, "
850         f"构造上界={metrics['bounds'].get('delivery_constructed_upper_bound_yuan')} 元; "
851         f"正式量级带 = 交付期 {DELIVERY_MAGNITUDE_BAND_YUAN} 元 / 日均 {DAILY_MAGNITUDE_BAND_YUAN} 元"
852     )
853 print(
854     "[prob04] 预测器回测 (D_req 334 天, 主判据 MAE): "
855     + "; ".join(
856         f"{name} MAE={backtest['windows']['D_req']['metrics'][name]['mae']:.6f} "
857         f"MAPE={backtest['windows']['D_req']['metrics'][name]['mape']:.4f} "
858         f"夺冠={int(backtest['windows']['D_req']['metrics'][name]['daily_mae_win_days'])}"
859         for name in ALL_METHODS
860     )
861     + f" 选定 {backtest['selection']['selected']}"
862 )
863 print(f"[prob04] 输出目录: {output_dir}; 工作簿={workbook_name}; 失败检查={failed}")
864 if failed:
865     print(f"[prob04] 硬检查未通过: {failed}", file=sys.stderr)
866     return EXIT_HARD_CHECK_FAILED
867 return EXIT_OK
868
869
870 if __name__ == "__main__":
871     raise SystemExit(main())

```